

ANEXO III – PLANTILLA DOCUMENTO DE DISEÑO

1	Diseño y arquitectura de vitalXAI	2
1.1	Organización arquitectónica de la solución	4
1.1.1	Capa de presentación y acceso desde el navegador	4
1.1.2	Capa HTTP y coordinación de los subsistemas funcionales	5
1.1.3	Servicios de aplicación y motores de inteligencia artificial	5
1.1.4	Persistencia relacional y ejecución asíncrona	6
1.1.5	Vista global de la arquitectura	7
1.2	Condiciones de calidad, normativa y restricciones del diseño	9
1.2.1	Requisitos no funcionales y respuesta arquitectónica	9
1.2.2	Estándares y normas aplicables	14
1.2.3	Restricciones técnicas y límites del entorno	15
1.3	Subsistemas de Diseño	16
1.3.1	SD-001: Acceso, identidad y gestión de sesiones	17
1.3.2	SD-002: Diagnóstico asistido y generación de resultados	18
1.3.3	SD-003: Historial y gestión de consultas	20
1.3.4	SD-004: Laboratorio de experimentación MLOps	21
1.3.5	SD-005: Supervisión y administración	23
1.3.6	SD-006: Cola de trabajos y capacidades transversales	24
1.3.7	Relaciones de integración entre los subsistemas de diseño	26
1.4	Requisitos de Operación y seguridad	28
2	ARQUITECTURA DE SOPORTE	28
2.1	Subsistemas de Soporte	29
2.2	Mecanismos Diseño	29
3	MODELO FÍSICO DE DATOS	29
3.1	Diseño del Modelo Físico de Datos	30
3.2	Acceso a los Datos	30
4	DISEÑO DE CASOS DE USO	31
4.1	Subsistema de Análisis S1	32
4.1.1	Diagrama de Casos de Uso	32
4.1.2	Casos de Uso Reales	32
4.1.3	Diagrama de Interacción entre Objetos	32
4.2	Subsistema de Análisis S2	32
5	DISEÑO DE CLASES	32
5.1	Subsistema de Diseño S1	33
5.1.1	Modelo de Clases	33
5.1.2	Definición de Clases	33
5.2	Subsistema de Diseño S2	34
6	DISEÑO DE INTERFACES	34
6.1	Subsistema de Diseño S1	35
6.1.1	Navegación	35
6.1.2	Descripción de las interfaces	35

6.1.3 Descripción de los Informes	35
6.2 Subsistema de Diseño S2	36
7 ESPECIFICACIONES DE CONSTRUCCIÓN	37
7.1 Entorno de Construcción	37
7.2 Subsistemas de Construcción y Componentes	37
7.3 Elaboración de Especificaciones de Construcción	37
7.4 Elaboración de Especificaciones del Modelo Físico de Datos	37
8 CARGA INICIAL DE DATOS O MIGRACIÓN	37
8.1 Entorno de Carga Inicial o Migración	38
8.2 Procedimientos de Carga Inicial o Migración	38
9 PLAN DE PRUEBAS	38
9.1 Entornos de Pruebas	39
9.2 Definición de Niveles de Prueba	39
10 REQUISITOS DE IMPLANTACIÓN	39
10.1 Requisitos de Documentación	40
10.2 Requisitos de Implantación	40

1 Diseño y arquitectura de vitalXAI

El presente capítulo inicia la parte de diseño de vitalXAI y transforma las decisiones funcionales establecidas durante el análisis en una organización técnica concreta. Mientras que los capítulos anteriores describen qué debe hacer la plataforma, qué actores intervienen y qué requisitos debe satisfacer, el diseño determina cómo se estructuran los componentes que permiten ofrecer esas capacidades. Esta transición se realiza manteniendo la trazabilidad con los objetivos, requisitos, casos de uso y subsistemas definidos en los capítulos 11 a 14.

El diseño se apoya en una aplicación web de servidor desarrollada con Python y FastAPI. La solución proporciona al usuario una interfaz accesible desde el navegador, expone los servicios funcionales mediante rutas HTTP, persiste la información operativa en MySQL y delega los trabajos de mayor duración en un worker interno de ejecución asíncrona. Sobre esta estructura se integran el motor de inferencia de imágenes, los módulos de explicabilidad, el laboratorio de experimentación MLOps, el asistente conversacional conectado con Groq y los generadores de informes PDF.

La organización técnica no reproduce de forma literal la distribución de los subsistemas de análisis. Estos subsistemas continúan siendo la referencia funcional para garantizar la trazabilidad, pero en el diseño se agrupan los elementos según sus responsabilidades de ejecución: presentación, entrada HTTP, lógica de aplicación y aprendizaje automático, persistencia y procesamiento asíncrono. Esta separación permite que la interfaz no tenga que conocer los detalles del motor de inferencia o de la base de datos, y que las tareas computacionalmente costosas puedan continuar sin mantener bloqueada la petición que las originó (Larman, 2004).

La arquitectura descrita en este capítulo corresponde a la implementación actual del sistema. Por este motivo, se documentan FastAPI, las plantillas Jinja2, los scripts JavaScript, MySQL, el pool de conexiones, la cola persistida en la base de datos y el worker basado en `asyncio`, sin sustituirlos por tecnologías previstas en versiones anteriores de la documentación o por componentes que no forman parte del código ejecutable. La correspondencia entre esta solución y los subsistemas de análisis se desarrolla en los apartados posteriores del documento de diseño.

1.1 Organización arquitectónica de la solución

La solución adopta una arquitectura cliente-servidor organizada en capas lógicas y servicios internos especializados. El cliente es el navegador del usuario, que recibe las páginas HTML renderizadas mediante Jinja2 y ejecuta los recursos JavaScript y CSS de la interfaz. El servidor concentra la lógica de negocio, la validación de las peticiones, la gestión de la identidad, el acceso a los datos y la coordinación de los trabajos de inteligencia artificial. La base de datos MySQL proporciona la persistencia estructurada, mientras que la cola de trabajos y el worker interno desacoplan las operaciones de larga duración del ciclo de petición HTTP.

Esta decisión responde a dos características esenciales de vitalXAI. La primera es que la complejidad computacional debe permanecer en el servidor: el navegador envía una imagen, una configuración o una solicitud de consulta, pero no carga los modelos de aprendizaje profundo ni ejecuta los entrenamientos. La segunda es que algunas operaciones no pueden resolverse de manera fiable dentro del tiempo de una petición ordinaria. La inferencia, la generación de mapas de explicabilidad, el entrenamiento y la validación externa pueden requerir un tiempo considerable; por ello, se registran como trabajos en `job_queue` y se procesan posteriormente mediante el worker de la aplicación.

1.1.1 Capa de presentación y acceso desde el navegador

La capa de presentación está formada por las plantillas HTML del directorio `templates`, los recursos estáticos del directorio `static` y el navegador del usuario. La aplicación no utiliza una SPA independiente: las páginas se generan en el servidor mediante Jinja2 y se sirven como respuestas HTML desde las rutas correspondientes. Entre las vistas principales se encuentran las páginas de inicio de sesión y registro, el panel de diagnóstico, el laboratorio de entrenamiento y la interfaz de administración.

Los scripts JavaScript asociados a las páginas complementan este renderizado con interacciones asíncronas. Por medio de `fetch`, el navegador puede enviar formularios, cargar imágenes, consultar estados de trabajos, recuperar resultados y actualizar partes de la interfaz sin tener que reconstruir toda la página en cada operación. Los recursos `dashboard.js`, `training.js`, `admin.js` e `i18n.js` concentran la lógica de interacción propia de las vistas, mientras que los estilos y las imágenes se sirven como recursos estáticos.

La capa de presentación no constituye una frontera de seguridad suficiente por sí misma. Su función es mostrar la información y recoger las acciones del usuario, pero la autenticación, la autorización, la validación definitiva de los datos y el aislamiento entre usuarios se vuelven a comprobar en el servidor. Esta decisión evita que una modificación de las peticiones desde el navegador permita acceder a consultas, sesiones de entrenamiento o funciones administrativas que no correspondan al usuario autenticado.

1.1.2 Capa HTTP y coordinación de los subsistemas funcionales

FastAPI constituye la entrada principal al servidor. La instancia se crea en `main.py`, donde se registra el ciclo de vida de la aplicación, se montan los recursos estáticos y se incorporan los routers funcionales. La organización por routers mantiene separadas las responsabilidades de acceso y evita concentrar todos los endpoints en un único módulo. La aplicación incorpora los routers de autenticación, diagnóstico, historial, cola de trabajos, laboratorio MLOps y administración.

Cada router recibe la petición HTTP, valida sus parámetros, obtiene la identidad del usuario cuando es necesario y delega la operación en el componente especializado. Así, `auth.py` coordina el registro, el inicio y el cierre de sesión; `inference.py` inicia las consultas de diagnóstico; `history.py` gestiona el historial; `trainer.py` coordina el laboratorio MLOps; `queue.py` expone el estado y la cancelación de trabajos; y `admin.py` restringe las operaciones de supervisión al rol de administrador. Esta distribución mantiene la relación entre las rutas de la API y los casos de uso sin convertir cada router en el responsable de la persistencia o del cálculo científico.

La instancia de FastAPI también concentra los elementos transversales que deben aplicarse a todas o a gran parte de las peticiones. `CSRFMiddleware` comprueba el token de protección en las operaciones que modifican el estado, `SecurityHeadersMiddleware` añade las cabeceras de seguridad configuradas y el limitador basado en `slowapi` controla la frecuencia de las operaciones sensibles, especialmente el acceso. Estos mecanismos se registran en el punto de creación de la aplicación para que no dependan de que cada router los aplique manualmente.

1.1.3 Servicios de aplicación y motores de inteligencia artificial

La lógica que no pertenece directamente al protocolo HTTP se organiza en el directorio `services`. Esta capa contiene servicios de aplicación y motores especializados que son invocados por los routers o por el worker de la cola. La separación es especialmente importante en `vitalXAI` porque las operaciones de una consulta clínica no se limitan a producir una etiqueta: también pueden generar un mapa de explicabilidad, crear un informe PDF y persistir el resultado asociado al usuario.

El servicio de autenticación encapsula la creación y comprobación de credenciales, el hash de contraseñas mediante `bcrypt`, la generación de tokens y la gestión de su renovación y revocación. `ml_engine.py` prepara la imagen, carga las arquitecturas disponibles y ejecuta la inferencia mediante TensorFlow/Keras o mediante modelos de Transformers. Las arquitecturas basadas en atención se obtienen a través de la configuración de Hugging Face utilizada por el proyecto. `xai_generator.py` produce los mapas de explicabilidad correspondientes a la arquitectura seleccionada, y `pdf_generator.py` transforma el resultado de una consulta en un informe descargable.

El laboratorio de experimentación se divide en servicios que reflejan sus distintas responsabilidades. `chatbot_service.py` encapsula la comunicación con la API de Groq y permite que el asistente interprete los parámetros del experimento; `trainer_engine.py` prepara los datos y ejecuta el entrenamiento; `mlops_engine.py` gestiona las sesiones, los modelos y los resultados; y `pdf_generator_mlops.py` genera los informes de las sesiones de experimentación. Esta división permite que el router del laboratorio coordine el flujo sin contener la implementación completa del entrenamiento o de la comunicación con el servicio externo.

Los servicios externos se tratan como fronteras de integración. La API de Groq se utiliza únicamente desde el servicio conversacional y requiere la variable de entorno `GROQ_API_KEY`. Los modelos de Transformers se resuelven desde Hugging Face cuando corresponde y se cargan en el motor de aprendizaje automático. Las respuestas o fallos de estos servicios no se exponen directamente al usuario como excepciones internas: la capa de aplicación los convierte en estados o mensajes que la interfaz puede mostrar y que el worker puede registrar como trabajos fallidos.

1.1.4 Persistencia relacional y ejecución asíncrona

La persistencia se implementa mediante MySQL y el conector oficial `mysql-connector-python`. El módulo `database.py` proporciona un `MySQLConnectionPool` reutilizable, obtiene los parámetros de conexión desde variables de entorno y expone la función que entrega conexiones a los routers y servicios que necesitan consultar o modificar datos. La inicialización de la aplicación verifica la existencia de las tablas principales: `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`.

Las relaciones de propiedad se mantienen en la base de datos mediante las claves ajenas que vinculan los trabajos y las consultas con los usuarios. De este modo, la identidad obtenida durante la autenticación puede utilizarse para filtrar el historial, las sesiones de entrenamiento y los trabajos de cada cuenta. El diseño también permite persistir el resultado de una operación larga después de que la petición HTTP inicial haya finalizado: el cliente conserva el identificador del trabajo y consulta posteriormente su estado y su resultado.

La ejecución asíncrona no depende de un broker externo. `queue.py` ofrece las operaciones de consulta y cancelación, mientras que `queue_worker.py` implementa el consumidor interno. Durante el ciclo de vida de FastAPI, `start_worker` restablece los trabajos que quedaron en ejecución tras una interrupción y crea una tarea `asyncio` que inspecciona periódicamente la tabla `job_queue`. El worker selecciona el siguiente trabajo pendiente, intenta reclamarlo de forma condicional y ejecuta el diagnóstico, el entrenamiento o la validación externa según el tipo registrado.

El procesamiento pesado se ejecuta mediante `run_in_executor`, evitando que el cálculo bloquee el bucle de eventos de la aplicación. Una vez terminada la operación, el worker almacena el resultado y marca el trabajo como completado; si se produce una excepción, registra el error y lo marca como fallido. Esta máquina de estados —`queued`, `running`, `completed` y `failed`— permite que la interfaz informe del progreso sin mantener abierta la petición que creó el trabajo. La cancelación se limita a los trabajos que aún están pendientes, lo que evita interrumpir de forma inconsistente una operación que ya ha comenzado.

1.1.5 Vista global de la arquitectura

La siguiente ilustración resume las relaciones entre los elementos principales de la solución. El navegador consume las páginas y recursos de presentación y se comunica con FastAPI mediante peticiones HTTP. FastAPI dirige cada operación hacia el router y el servicio correspondiente, utiliza MySQL para la información persistente y registra en `job_queue` las operaciones que deben ejecutarse fuera del ciclo inmediato de la petición. El worker recupera esos trabajos y utiliza los motores de inferencia, explicabilidad, entrenamiento y generación de informes antes de persistir sus resultados.

La arquitectura resultante conserva una separación clara entre la interacción del usuario, la coordinación HTTP, la lógica especializada, la persistencia y la ejecución de trabajos. La aplicación sigue siendo desplegable como una única solución, pero sus responsabilidades internas están suficientemente delimitadas para que una modificación en la interfaz no obligue a reescribir el motor de inferencia, que una nueva arquitectura de modelo se incorpore dentro de los servicios de aprendizaje automático y que los cambios en la presentación no alteren el mecanismo de cola. Esta organización materializa las dependencias identificadas en el análisis: los subsistemas de diagnóstico y laboratorio dependen de la identidad y la persistencia, y ambos utilizan el mecanismo transversal de trabajos asíncronos para ejecutar las operaciones de larga duración.

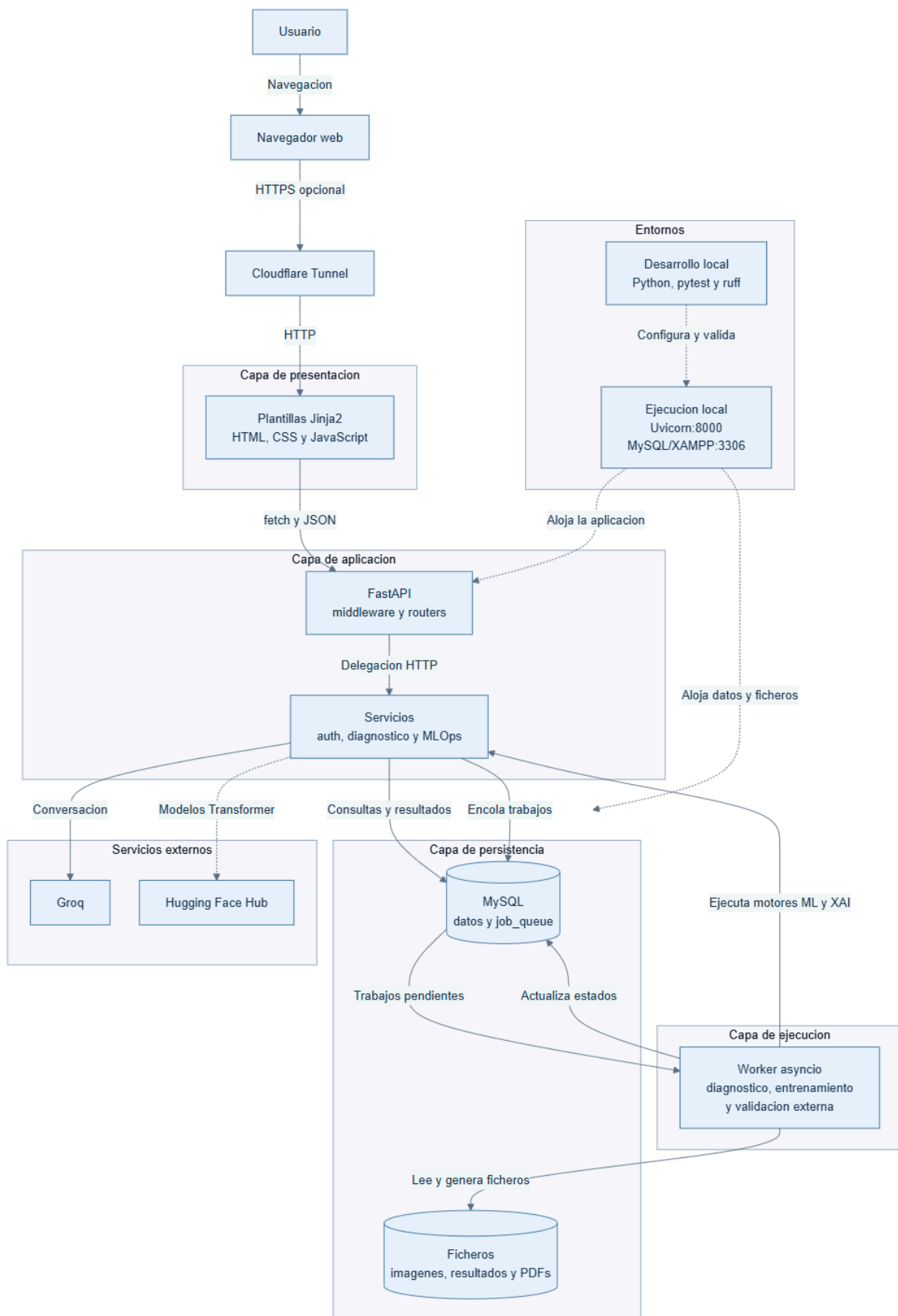


Ilustración x - Organización arquitectónica de vitalXAI

1.2 Condiciones de calidad, normativa y restricciones del diseño

La arquitectura definida en el apartado anterior no constituye una elección aislada de tecnologías. Cada una de sus decisiones responde a los requisitos no funcionales, a las normas aplicables al dominio sanitario y a las limitaciones del entorno en el que se construye y ejecuta vitalXAI. En consecuencia, este apartado relaciona las condiciones de calidad del capítulo 12 con las decisiones técnicas que las materializan, distingue los estándares que deben observarse durante el desarrollo y delimita las restricciones que condicionan la solución.

1.2.1 Requisitos no funcionales y respuesta arquitectónica

Los requisitos no funcionales se han agrupado en el análisis en siete dimensiones: seguridad y acceso, confidencialidad de los datos, rendimiento, sencillez de uso, robustez, reproducibilidad y documentación. En la fase de diseño no todos ellos se traducen en una clase o un módulo independiente. Algunos se materializan mediante middleware, configuración o separación de responsabilidades; otros corresponden a procedimientos del entorno de ejecución y a los documentos que acompañan al software. La siguiente tabla resume la relación entre cada grupo y la respuesta adoptada en la implementación actual.

Grupo de requisitos	Requisitos relacionados	Decisión de diseño y mecanismo aplicado
Seguridad del acceso	RNF-001 a RNF-011	Hash de contraseñas con bcrypt, tokens de acceso y refresco, cookies HttpOnly con SameSite=Lax, middleware CSRF, limitación de acceso mediante slowapi y cabeceras de seguridad. Las operaciones administrativas se protegen mediante el rol del usuario.
Confidencialidad de los datos	RNF-012 a RNF-018	Separación de los datos por user_id, comprobación de propiedad en las rutas privadas, uso de datasets anonimizados y exposición del servicio mediante HTTPS cuando se accede desde el exterior.
Rendimiento y respuesta	RNF-019 a RNF-022	Reutilización de modelos cargados, pool de conexiones MySQL, ejecución de trabajos mediante job_queue y worker asyncio, y delegación del cálculo pesado en run_in_executor.
Usabilidad y accesibilidad	RNF-023 a RNF-026	Interfaz HTML/Jinja2 con JavaScript vanilla, asistencia conversacional para configurar experimentos, servicio de traducciones, cambio de tema visual y diseño adaptable de las vistas.
Robustez y disponibilidad	RNF-027 a RNF-032	Estados persistentes de los trabajos, registro de errores, recuperación de trabajos que quedaron en ejecución al reiniciar el servidor y claves ajenas en MySQL. Las copias de seguridad dependen del procedimiento operativo del entorno MySQL.

Grupo de requisitos	Requisitos relacionados	Decisión de diseño y mecanismo aplicado
Reproducibilidad y sostenibilidad	RNF-033 a RNF-034	Versiones fijadas en requirements.txt, entorno virtual de Python, separación entre routers, servicios y persistencia, y ejecución de comprobaciones automatizadas mediante pytest y ruff.
Documentación y entregables	RNF-037 a RNF-038	Memoria estructurada por fases, guía de despliegue y documentación de configuración, instalación y uso de la plataforma.

Tabla x - Relación entre los requisitos no funcionales y las decisiones de diseño

1.2.1.1 Seguridad, identidad y control del acceso

La identidad del usuario constituye la primera condición de seguridad de la plataforma. El servicio de autenticación no almacena contraseñas originales, sino valores generados mediante bcrypt, y la tabla users conserva únicamente el campo password_hash. La sesión se articula mediante un token de acceso y un token de refresco. Ambos se entregan mediante cookies protegidas frente al acceso de JavaScript mediante el atributo HttpOnly, mientras que el token de refresco limita además su ruta de envío al endpoint de renovación. La rotación y revocación de los tokens se apoya en la tabla refresh_tokens, que conserva su hash, su fecha de expiración y su estado de revocación.

El diseño incorpora una protección adicional frente a peticiones CSRF. El middleware genera una cookie csrf_token accesible desde el código de la interfaz y exige que las peticiones que modifican el estado incluyan el mismo valor en la cabecera X-CSRF-Token. Las operaciones de lectura permanecen fuera de esta comprobación y la ruta de inicio de sesión se trata como excepción específica para permitir el acceso inicial. La comparación se realiza en el servidor antes de entregar la petición al router correspondiente.

La limitación de frecuencia se aplica especialmente al inicio de sesión, que se encuentra limitado a cinco peticiones por minuto mediante slowapi. Esta medida reduce la viabilidad de ataques automatizados de fuerza bruta sin tener que incorporar la lógica de limitación dentro del router de autenticación. A este mecanismo se suman las cabeceras emitidas por SecurityHeadersMiddleware, entre ellas Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security y Referrer-Policy.

El control de roles se aplica en el router de administración. El usuario administrador puede consultar el listado de usuarios y supervisar las consultas de una cuenta, mientras que un usuario ordinario no puede invocar esas operaciones. El aislamiento de datos se comprueba de forma independiente en cada operación que recupera o modifica una consulta, una sesión de entrenamiento o un trabajo. Esta comprobación no se delega en la interfaz, ya que el navegador puede ser manipulado por el usuario que realiza la petición.

Debe distinguirse entre el control de acceso administrativo y la auditoría formal de la actividad administrativa. La implementación actual restringe las rutas de administración mediante el rol, pero no introduce una tabla independiente de trazas con el detalle histórico de cada operación. Por ello, RNF-006 se mantiene como condición de calidad y de gobierno del sistema, mientras que el mecanismo de autorización sí forma parte de la arquitectura implementada. Esta precisión evita presentar como existente un componente de auditoría que no aparece en el código actual.

1.2.1.2 Confidencialidad y tratamiento de la información

La aplicación trabaja con cuentas de usuario, radiografías y resultados de diagnóstico. El diseño relacional vincula las consultas, los trabajos y los entrenamientos con el identificador del usuario propietario, y las rutas utilizan esa relación para limitar los resultados. Esta regla se aplica tanto al historial clínico como a las sesiones del laboratorio MLOps y a los trabajos almacenados en `job_queue`.

El cumplimiento de los requisitos de privacidad no se resuelve únicamente con una configuración de software. Los datasets de entrenamiento y validación deben estar anonimizados antes de incorporarse al proyecto, y el personal que sube una imagen debe garantizar que no contiene información identificable del paciente. La plataforma almacena rutas de ficheros y resultados asociados al usuario profesional, pero no incorpora un módulo de identificación del paciente ni debe utilizarse para introducir datos personales que no sean necesarios para la consulta.

La confidencialidad durante el tránsito depende del entorno de exposición. En el uso local, Uvicorn atiende la aplicación en el puerto 8000; cuando se publica el sistema mediante el túnel configurado para la demostración, el acceso exterior se realiza mediante HTTPS. Las cookies de sesión incorporan actualmente `HttpOnly` y `SameSite=Lax`; la protección del canal debe mantenerse como condición obligatoria del entorno de despliegue para evitar que las credenciales y los datos viajen sin cifrar.

1.2.1.3 Rendimiento, concurrencia y ejecución sin bloqueo

El requisito de rendimiento más relevante no consiste únicamente en reducir el tiempo de una función concreta, sino en impedir que una operación de aprendizaje automático monopolice el ciclo de petición. Por este motivo, el diagnóstico, el entrenamiento y la validación externa se registran primero como trabajos en la tabla `job_queue`. La petición HTTP puede devolver el identificador y el estado inicial del trabajo, mientras que el worker continúa el procesamiento de forma independiente.

El worker se inicia durante el ciclo de vida de FastAPI y consulta periódicamente los trabajos pendientes. Para evitar que el cálculo de TensorFlow, la generación de mapas XAI o la ejecución del entrenamiento bloqueen el bucle de eventos, el trabajo se envía a `run_in_executor`. El resultado o el mensaje de error se almacena después en MySQL. Así, el usuario puede consultar la cola mientras el procesamiento continúa y la plataforma puede atender otras peticiones.

La concurrencia de acceso a la persistencia se aborda mediante `MySQLConnectionPool`. El tamaño del pool se configura con `DB_POOL_SIZE`, mientras que los restantes datos de conexión se obtienen mediante variables de entorno. Esta decisión evita crear una conexión nueva de forma indiscriminada para cada operación y permite controlar el número de conexiones simultáneas en función del entorno disponible. La concurrencia no elimina la necesidad de validar la propiedad de cada registro: el rendimiento y el aislamiento de datos son condiciones simultáneas.

1.2.1.4 Usabilidad, internacionalización y presentación

La elección de plantillas Jinja2 y JavaScript vanilla reduce la complejidad del cliente y evita introducir una cadena de compilación independiente. Las páginas se entregan directamente desde el servidor y los scripts añaden únicamente las interacciones necesarias para cargar imágenes, consultar estados, mostrar resultados y actualizar el laboratorio. La arquitectura facilita que la interfaz pueda funcionar desde un navegador sin instalar software adicional.

El servicio lang.py centraliza la recuperación de textos traducidos y permite seleccionar el idioma mediante la cookie correspondiente. Esta capacidad se utiliza desde las vistas de autenticación, el panel de diagnóstico y el laboratorio. El cambio de tema visual se realiza en el cliente y se conserva como preferencia de la interfaz. La presentación de los resultados se organiza alrededor de las necesidades del usuario: diagnóstico y confianza, mapas de explicabilidad, histórico de consultas, progreso de trabajos y resultados del laboratorio.

1.2.1.5 Robustez, persistencia y recuperación

La tabla job_queue actúa como soporte persistente de la ejecución asíncrona. A diferencia de una cola exclusivamente residente en memoria, permite conservar el estado de un trabajo y recuperarlo después de reiniciar la aplicación. Al iniciar el worker, _reset_running_jobs() devuelve al estado queued los trabajos que quedaron en running, de modo que no permanezcan bloqueados indefinidamente. Cada ejecución puede terminar en completed o failed, y en el segundo caso se conserva un mensaje de error limitado en longitud.

Las claves ajenas de las tablas consultations, training_jobs, job_queue y refresh_tokens apuntan a users. Esta relación refuerza la integridad del modelo y permite eliminar los datos dependientes de un usuario cuando la operación está configurada con ON DELETE CASCADE. Las escrituras se confirman explícitamente mediante commit y las conexiones se cierran al finalizar cada operación.

La copia de seguridad periódica y la restauración de MySQL pertenecen al entorno operativo, no a un servicio interno de la aplicación. El diseño identifica esta dependencia para que el despliegue no confunda la persistencia local con una estrategia de recuperación completa. La guía de despliegue debe conservar, por tanto, las instrucciones necesarias para iniciar MySQL y verificar la disponibilidad de la base de datos antes de ejecutar la plataforma.

1.2.2 Estándares y normas aplicables

El sistema se desarrolla bajo normas legales y técnicas que afectan a distintos niveles de la solución. En materia de privacidad se aplica el Reglamento General de Protección de Datos, Reglamento (UE) 2016/679, junto con la Ley Orgánica 3/2018 de protección de datos personales y garantía de los derechos digitales. Estas normas condicionan la minimización de la información recogida, la anonimización de las imágenes, la asociación de los datos a una cuenta y la forma en que se documenta el tratamiento de la información sanitaria (Parlamento Europeo y Consejo de la Unión Europea, 2016; España, 2018).

Para la protección de las comunicaciones se adopta HTTPS apoyado en TLS. El túnel utilizado para exponer la aplicación debe proporcionar el canal cifrado entre el navegador y el entorno local, y las cabeceras de seguridad del servidor complementan esa protección. La existencia de HTTPS no sustituye a la autenticación, a la autorización ni a la protección CSRF; son mecanismos que operan en capas distintas y deben mantenerse conjuntamente (IETF, 2018).

El código Python sigue las convenciones de PEP 8. La aplicación utiliza ruff como comprobación automatizada de estilo y mantiene un fichero requirements.txt con versiones fijadas de sus dependencias principales. La documentación de la memoria y los manuales se elaboran conforme a la normativa del Trabajo Fin de Grado de la Escuela Politécnica Superior de la Universidad Pablo de Olavide, que constituye el marco formal de los entregables del proyecto (van Rossum, Warsaw, & Coghlan, 2001; Universidad Pablo de Olavide, 2014).

La siguiente tabla resume el ámbito de aplicación de las normas anteriores.

Estándar / Norma	Ámbito de aplicación
RGPD (Reglamento UE 2016/679)	Protección de datos y tratamiento de información biomédica.
LOPDGDD (Ley Orgánica 3/2018)	Adaptación del RGPD al ordenamiento jurídico español.
HTTPS / TLS	Protocolo seguro de comunicación entre cliente y servidor.
OWASP Top 10	Seguridad de las aplicaciones web.
PEP 8	Estándar de estilo y nomenclatura del código Python.

Tabla x - Normas y estándares aplicables al diseño

1.2.3 Restricciones técnicas y límites del entorno

La primera restricción técnica es el ecosistema de aprendizaje automático. La aplicación se ejecuta sobre Python 3.11 y depende de TensorFlow, Keras, OpenCV, scikit-learn y Transformers. Esta combinación condiciona la versión del intérprete, el sistema operativo y la disponibilidad de recursos de memoria. Los modelos convolucionales y Transformer no se consideran componentes ligeros del servidor: sus pesos deben cargarse en memoria y los entrenamientos pueden requerir una GPU compatible con el entorno de TensorFlow.

La segunda restricción es la capacidad de cómputo local. El entrenamiento y la validación de las arquitecturas demandan más recursos que una consulta web ordinaria, por lo que el entorno de desarrollo debe disponer de hardware suficiente, especialmente durante la ejecución del laboratorio MLOps. La interfaz y los routers no deben asumir que el resultado está disponible de forma inmediata; esta condición es precisamente la que justifica el diseño basado en trabajos persistidos y worker interno.

La tercera restricción es la disponibilidad de MySQL. La aplicación depende de una instancia accesible mediante los parámetros DB_HOST, DB_USER, DB_PASSWORD, DB_NAME y DB_POOL_SIZE. En el entorno de demostración, MySQL se ejecuta mediante XAMPP. Si el servicio no está iniciado o las credenciales no son válidas, la inicialización de la base de datos no puede completarse, aunque el ciclo de vida de FastAPI informa del problema y continúa hasta iniciar el worker.

La cuarta restricción procede de los servicios externos. El asistente conversacional requiere GROQ_API_KEY y conectividad con la API de Groq. La carga de los modelos Transformer puede requerir acceso a Hugging Face, salvo que los modelos ya estén disponibles localmente. Estas dependencias introducen límites de disponibilidad y latencia que el sistema debe tratar como fallos de integración, no como errores de lógica interna. Por este motivo, las pruebas unitarias sustituyen los servicios externos por mocks y la ejecución de producción debe conservar las variables de entorno fuera del código fuente.

Finalmente, la exposición remota mediante el túnel no convierte el entorno local en una infraestructura distribuida. La aplicación, el worker, los ficheros y la base de datos continúan alojados en la máquina de ejecución. El túnel únicamente proporciona el canal de acceso desde el navegador externo. Esta decisión simplifica la demostración y mantiene bajo control las versiones del entorno, pero limita la escalabilidad, la tolerancia a fallos hardware y la disponibilidad de la plataforma si la máquina anfitriona se apaga o pierde conectividad.

En conjunto, estas restricciones explican por qué el diseño prioriza una aplicación servidor sencilla, una persistencia relacional accesible, un mecanismo de trabajos basado en MySQL y una separación clara entre la petición web y la computación científica. No se introducen colas, bases de datos o servicios de despliegue adicionales que no sean necesarios para la implementación actual. La arquitectura conserva así una relación directa entre los requisitos del análisis, las condiciones de calidad, el entorno disponible y los componentes que realmente ejecuta vitalXAI.

1.3 Subsistemas de Diseño

Los subsistemas de diseño son la traducción técnica de los seis subsistemas de análisis definidos en el capítulo 13. La correspondencia se mantiene de forma directa: cada subsistema SS conserva sus responsabilidades funcionales y recibe un identificador SD que agrupa los componentes de implementación que las realizan. Esta decisión permite recorrer la trazabilidad desde los requisitos y casos de uso hasta los routers, servicios, mecanismos de persistencia y procesos que intervienen en la ejecución.

La correspondencia no implica que cada subsistema de diseño sea una aplicación independiente. Todos forman parte de la misma aplicación FastAPI y comparten la base de datos, la autenticación, la capa de presentación y el worker de trabajos. La separación se realiza por responsabilidades y límites funcionales, no mediante despliegues separados. De esta manera, el código mantiene una organización modular sin introducir una complejidad distribuida que no exige el entorno actual.

Subsistema de diseño	Subsistema de análisis	Responsabilidad principal	Componentes de implementación
SD-001	SS-001	Gestionar la identidad, las sesiones y el acceso a la plataforma.	routers/auth.py, services/auth_service.py, users, refresh_tokens y middleware de seguridad.
SD-002	SS-002	Recibir una radiografía, solicitar el diagnóstico y producir sus artefactos.	routers/inference.py, services/ml_engine.py, services/xai_generator.py, services/pdf_generator.py y job_queue.
SD-003	SS-003	Consultar y administrar el historial de diagnósticos del usuario.	routers/history.py, tabla consultations, ficheros de resultados y dashboard.js.
SD-004	SS-004	Configurar, ejecutar y consultar experimentos MLOps.	routers/trainer.py, chatbot_service.py, trainer_engine.py, mlops_engine.py y pdf_generator_mlops.py.
SD-005	SS-005	Supervisar usuarios, consultas y sesiones desde el ámbito administrativo.	routers/admin.py, consultas MySQL y funciones de consulta de mlops_engine.py.
SD-006	SS-006	Coordinar trabajos asíncronos y capacidades transversales de la interfaz.	routers/queue.py, services/queue_worker.py, services/lang.py, middleware global y recursos estáticos.

Tabla x - Correspondencia entre subsistemas de análisis y subsistemas de diseño

1.3.1 SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa el subsistema de análisis SS-001 y cubre el registro, el inicio y el cierre de sesión, la renovación de credenciales y la identificación del usuario en las áreas privadas. Su responsabilidad no se limita a validar el formulario de acceso: establece la identidad que utilizarán los demás subsistemas para aplicar el aislamiento de datos y las comprobaciones de rol.

El punto de entrada del subsistema es `routers/auth.py`. El router sirve las páginas de inicio de sesión y registro y procesa los formularios recibidos desde la interfaz. Antes de crear una cuenta valida el formato del nombre de usuario, la longitud mínima de la contraseña y la presencia de los datos personales básicos. A continuación, consulta MySQL para comprobar que el usuario no existe, genera el hash de la contraseña y almacena la cuenta en la tabla `users`. La contraseña original no se transmite a ningún componente de persistencia después de completar el proceso de hash.

La lógica criptográfica y la gestión del ciclo de vida de los tokens se concentra en `services/auth_service.py`. El token de acceso se firma con JWT y contiene el identificador del usuario y su fecha de expiración. El token de refresco se genera como un valor aleatorio, pero en la base de datos solo se conserva su hash SHA-256 junto con el usuario, la fecha de expiración y el indicador de revocación. Esta diferencia permite renovar la sesión sin guardar una credencial reutilizable en texto plano.

El cierre de sesión revoca el token de refresco y elimina las cookies de la respuesta. La rotación genera un token nuevo y revoca el anterior; si se detecta el uso de un token revocado fuera del periodo de gracia configurado, el servicio invalida todos los tokens del usuario. El mecanismo está diseñado para tolerar peticiones concurrentes de renovación durante un intervalo corto sin interpretar automáticamente cada repetición como un robo de sesión.

El subsistema se apoya también en los mecanismos transversales registrados en `main.py`. `CSRFMiddleware` protege las peticiones que modifican el estado, `SecurityHeadersMiddleware` refuerza las respuestas dirigidas al navegador y `slowapi` limita el endpoint de inicio de sesión. La sesión se almacena en cookies `HttpOnly` y con alcance `SameSite=Lax`, mientras que el resto de routers utiliza `get_user_id_from_token()` para obtener la identidad antes de realizar operaciones privadas. SD-001 es, por tanto, un subsistema transversal: SD-002, SD-003, SD-004, SD-005 y SD-006 dependen de la identidad que establece.

Desde el punto de vista de la trazabilidad, SD-001 realiza principalmente los casos de uso CU-001 a CU-004. CU-001 se materializa en la combinación de validación de los campos, consulta de unicidad, generación del hash e inserción en `users`; CU-002 añade la verificación de la contraseña y la creación de las dos cookies de sesión; CU-003 revoca el token de refresco y elimina las cookies; y CU-004 utiliza el servicio de idioma para adaptar la presentación. Los casos de uso de otros subsistemas no necesitan conocer cómo se firma un token: solo reciben la identidad resuelta o una respuesta de acceso no autorizado.

La configuración sensible permanece fuera del código. `JWT_SECRET_KEY`, `JWT_ACCESS_EXPIRE_MINUTES`, `JWT_REFRESH_EXPIRE_DAYS` y `REFRESH_ROTATION_GRACE_SECONDS` se obtienen del entorno, al igual que los parámetros de MySQL. Esta decisión evita que la clave criptográfica o la duración de las credenciales queden ligadas al repositorio. El servicio emite además una advertencia cuando se utiliza la clave de desarrollo por defecto, lo que convierte una configuración insegura en una condición visible durante el arranque.

El subsistema establece contratos sencillos con sus consumidores. Un componente que necesita identificar al usuario entrega el token de acceso a `get_user_id_from_token()` y recibe un identificador entero o `None`; no interpreta directamente el payload JWT ni duplica la lógica de expiración. Los routers, a su vez, traducen la ausencia de identidad en una respuesta HTTP 401. La comprobación del rol se realiza únicamente en las operaciones administrativas, por lo que la existencia de una sesión válida no equivale a disponer de privilegios elevados.

La principal decisión de SD-001 es conservar el token de acceso en la cookie y el token de refresco en una cookie con ruta restringida, en lugar de enviar las credenciales en el cuerpo de cada petición. Esto simplifica la interacción de las plantillas y los scripts del navegador y reduce la exposición accidental de los tokens en el almacenamiento de la interfaz. La protección efectiva depende además de que el entorno de publicación utilice HTTPS y de que la clave secreta de producción esté configurada correctamente.

1.3.2 SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 corresponde a SS-002 y concentra el flujo clínico de la plataforma. Su función es recibir una imagen válida, asociarla al usuario autenticado, registrar una solicitud de diagnóstico, ejecutar la inferencia con la arquitectura seleccionada y conservar los artefactos que permiten consultar y justificar el resultado.

`routers/inference.py` actúa como fachada HTTP del subsistema. La ruta `/predict` comprueba la existencia de una sesión válida y valida el tipo MIME de la imagen. Solo se aceptan imágenes JPEG y PNG, y se aplica un límite de 10 MB antes de escribir el fichero en `static/uploads`. Esta validación se realiza antes de encolar el trabajo, de modo que una petición inválida no ocupa una posición en la cola ni genera un registro de procesamiento incompleto.

Una vez almacenada temporalmente la imagen, el router crea un registro en `job_queue` con el tipo `diagnosis`, el identificador del usuario y un payload JSON que incluye la arquitectura, la ruta de la imagen y el idioma. La respuesta HTTP informa de que el trabajo ha quedado encolado, devuelve su identificador y calcula su posición. El router no carga el modelo ni ejecuta la predicción, ya que esas operaciones pertenecen al worker y a la capa de servicios.

El worker dirige el trabajo de diagnóstico hacia `services/ml_engine.py`, que prepara la imagen, carga o reutiliza el modelo y produce la etiqueta y la confianza de la predicción. La implementación combina arquitecturas convolucionales gestionadas mediante TensorFlow/Keras y arquitecturas basadas en Transformers. Después de la inferencia, `services/xai_generator.py` genera el mapa de explicabilidad adecuado para el modelo utilizado. Finalmente, `services/pdf_generator.py` genera el informe descargable con la imagen original, el resultado, la confianza y el artefacto XAI.

El resultado se registra en la tabla `consultations`, donde quedan asociados el usuario, el modelo utilizado, las rutas de los ficheros, la etiqueta de predicción, la confianza y el informe PDF. Esta persistencia permite que el historial y la interfaz administrativa consulten el resultado sin repetir la inferencia. Si cualquier fase falla, el worker marca el trabajo como `failed` y conserva el mensaje de error en `job_queue`, evitando que la petición original quede abierta indefinidamente.

El flujo interno de SD-002 se divide en cinco pasos. Primero, el router autentica al usuario y valida el fichero recibido. Segundo, conserva la imagen original en el área de cargas y crea el payload serializable del trabajo. Tercero, el worker reclama el registro y ejecuta la inferencia. Cuarto, genera los artefactos derivados, que son el mapa XAI y el informe PDF. Quinto, persiste la consulta y actualiza el estado de la cola. La división evita que una ruta HTTP tenga que mantener referencias abiertas a un fichero o a un modelo durante toda la operación.

El payload de un trabajo de diagnóstico contiene únicamente la información necesaria para repetir el procesamiento: `model_name`, `image_path` y `lang`, además del identificador del usuario almacenado en la propia fila de la cola. No se serializa el objeto del modelo ni la imagen completa dentro de MySQL. Los datos voluminosos se mantienen en el sistema de ficheros y la base de datos conserva sus rutas, de manera que la cola sigue siendo ligera y puede consultarse con rapidez.

La separación entre `ml_engine`, `xai_generator` y `pdf_generator` también delimita los fallos. Una arquitectura no reconocida debe fallar antes de producir un resultado ambiguo; una imagen no válida no debe alcanzar el motor; y un error al construir el informe no debe transformarse en una consulta marcada como completada. El worker agrupa el flujo completo dentro del procesamiento del trabajo y utiliza la transición a `failed` como contrato común para que la interfaz pueda mostrar que la operación terminó con error.

La reutilización de modelos cargados responde al RNF-019. El coste de la primera consulta puede incluir la lectura de los pesos y la construcción del modelo, mientras que las consultas posteriores pueden aprovechar el modelo disponible en memoria. Esta optimización pertenece al motor de aprendizaje automático y no al router, por lo que la capa HTTP permanece independiente de si la arquitectura seleccionada es convolucional o Transformer.

SD-002 también actúa como frontera de validación de ficheros. El tipo MIME y el tamaño se comprueban antes de escribir en disco, pero la validez científica de la imagen y la interpretación clínica del resultado siguen siendo responsabilidades distintas. El sistema ofrece una clasificación asistida, una confianza y explicaciones visuales; no sustituye la valoración del profesional sanitario. Esta separación de responsabilidades es importante para no convertir el diseño técnico del subsistema en una afirmación clínica que la implementación no puede garantizar.

1.3.3 SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa SS-003 y proporciona la recuperación y gestión de las consultas ya realizadas. Su entidad persistente principal es `consultations`, que almacena las rutas de los artefactos, la arquitectura utilizada, la predicción, la confianza, el nombre mostrado al usuario y la fecha de la operación.

`routers/history.py` expone la consulta del listado y las operaciones de actualización y eliminación. La consulta del historial filtra siempre por el `user_id` obtenido de la cookie de acceso y ordena los resultados por fecha descendente. Antes de modificar o eliminar una consulta, `_check_consultation_ownership()` comprueba que el registro existe y que pertenece al usuario solicitante. La misma función contempla la excepción controlada del administrador, que puede supervisar consultas desde SD-005.

El subsistema no vuelve a ejecutar el modelo para mostrar una consulta anterior. Recupera desde MySQL los metadatos y devuelve las rutas de la imagen original, el mapa XAI y el informe PDF para que la interfaz pueda presentarlos o descargarlos. Esta decisión reduce el coste de acceso al historial y conserva la separación entre una consulta nueva y la visualización de un resultado ya calculado.

La interfaz del historial se integra en el panel de diagnóstico y utiliza JavaScript para mostrar los resultados, cambiar el nombre visible de una consulta y solicitar su eliminación. El router devuelve códigos diferenciados para la falta de autenticación, la consulta inexistente y la ausencia de permisos. Esta distinción permite que la capa de presentación informe al usuario de la situación sin exponer detalles internos de la base de datos.

La relación entre SD-003 y SD-002 es de productor-consumidor de resultados. SD-002 crea el registro de `consultations` cuando finaliza un diagnóstico; SD-003 no modifica la predicción ni sus artefactos, sino que proporciona operaciones de consulta y organización. El nombre mostrado al usuario se actualiza sobre `patient_name`, que funciona como etiqueta de organización de la consulta y no como identificación clínica del paciente. Esta decisión mantiene separados los metadatos de presentación de los datos técnicos de la inferencia.

El listado devuelve los campos necesarios para construir las tarjetas o filas del historial: identificador, fecha, modelo, rutas de imágenes, etiqueta, confianza, nombre y PDF. Las fechas se convierten a una representación textual antes de formar la respuesta JSON, de modo que el navegador no necesita conocer el tipo específico de fecha de MySQL. Los artefactos se sirven mediante las rutas estáticas configuradas en la aplicación y no mediante una nueva ruta de cálculo.

La eliminación se implementa actualmente como una eliminación física de la fila de consulta. Por esa razón, el diseño no debe describir SD-003 como un subsistema de borrado lógico ni afirmar que conserva automáticamente una auditoría de cada eliminación. La integridad de la operación depende de que se compruebe la propiedad antes de ejecutar el `DELETE`, y de que la transacción se confirme solo después de que la actualización haya sido aceptada por MySQL. La decisión se documenta aquí porque afecta a la interpretación del historial y a futuras ampliaciones del modelo de persistencia.

El administrador constituye una excepción controlada al aislamiento ordinario. `_check_consultation_ownership()` permite continuar cuando el usuario tiene rol admin, pero el acceso administrativo se comprueba en el servidor y no se infiere de un parámetro enviado por el navegador. En todos los demás casos, una consulta de otro usuario produce una respuesta 403. Así, SD-003 mantiene una política única de propiedad y permite que SD-005 la utilice sin duplicar una segunda consulta de autorización.

1.3.4 SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 corresponde a SS-004 y constituye el bloque de mayor complejidad funcional. Gestiona la configuración conversacional del experimento, la selección del dataset, el entrenamiento de modelos, la ejecución de los análisis de explicabilidad, la comparación estadística, la validación externa, la consulta de resultados y la generación de informes.

`routers/trainer.py` se ha diseñado como una fachada ligera. Sus rutas comprueban la autenticación, validan que la sesión de experimentación pertenece al usuario y delegan las operaciones especializadas en `mlops_engine.py` o en los servicios correspondientes. Entre sus operaciones se encuentran la conversación con el asistente, la exploración de carpetas, el inicio de entrenamientos, la consulta de logs, la recuperación de sesiones, la consulta de resultados, el ranking, el recálculo de la comparativa, la validación externa y la generación de informes.

La configuración conversacional se encapsula en `services/chatbot_service.py`, que se comunica con Groq y devuelve una configuración estructurada cuando se han proporcionado los parámetros necesarios. El router no incorpora la lógica del modelo de lenguaje: únicamente valida la sesión y entrega el mensaje al servicio. Esta separación permite que la comunicación con Groq se trate como una frontera externa y que los errores del proveedor no se confundan con errores de persistencia o de entrenamiento.

`services/mlops_engine.py` organiza las sesiones del laboratorio y resuelve la lectura y escritura de los resultados en `training_results`. `trainer_engine.py` contiene la lógica de preparación del dataset, construcción de modelos, entrenamiento, cálculo del progreso y actualización del estado. Los resultados generados por el pipeline incluyen las métricas de rendimiento, los artefactos de explicabilidad, los datos de comparación y la información necesaria para la validación externa. Los informes finales se generan mediante `pdf_generator_mlops.py`.

El inicio de un entrenamiento crea la sesión, comprueba que la ruta del dataset existe y registra en `job_queue` un trabajo de tipo training con los modelos e hiperparámetros solicitados. La validación externa sigue el mismo mecanismo con un trabajo de tipo `external_validation`. El worker ejecuta ambos tipos fuera del ciclo de petición y actualiza el estado de la cola al finalizar. Esta decisión permite que el laboratorio conserve una interfaz interactiva aunque el entrenamiento tarde mucho más que una petición HTTP normal.

La propiedad de las sesiones es una regla central del subsistema. Antes de mostrar resultados, eliminar o renombrar una sesión, el router invoca la comprobación de propiedad de `mlops_engine`; el administrador puede acceder a estas operaciones de supervisión cuando la ruta lo permite. Así, la configuración, los resultados y los artefactos de una experimentación permanecen asociados a la cuenta que la creó.

La configuración de una sesión contiene los parámetros que necesita el pipeline: ruta del dataset, modelos seleccionados, número de épocas, tamaño de lote y tasa de aprendizaje. El asistente conversacional permite completar esa información mediante mensajes, pero la ejecución no comienza hasta que el router dispone de una configuración completa y válida. La conversación y el entrenamiento son responsabilidades distintas: Groq ayuda a interpretar la intención del usuario, mientras que mlops_engine y trainer_engine validan y ejecutan la configuración.

El lanzamiento de un entrenamiento sigue una secuencia concreta. La ruta comprueba la autenticación, verifica que existe la ruta del dataset, crea la sesión de entrenamiento y registra en job_queue un payload con la lista de modelos y los hiperparámetros. El identificador de la sesión se conserva tanto en la fila del trabajo como en el directorio de resultados. Esta doble referencia permite consultar el estado desde MySQL y recuperar los artefactos experimentales desde el sistema de ficheros.

La información de una sesión no se reduce a un único valor de rendimiento. El pipeline genera resultados por arquitectura y por pliegue, métricas agregadas, artefactos de explicabilidad, configuraciones y datos de comparación estadística. get_model_results_data() y get_session_ranking_data() transforman esos ficheros en estructuras que el frontend puede mostrar. Esta decisión evita trasladar al navegador la responsabilidad de recorrer todos los artefactos o de repetir los cálculos del ranking.

La comparación estadística tiene un flujo diferenciado del entrenamiento. La ruta de comparación comprueba la propiedad y programa run_statistical_comparison mediante BackgroundTasks, mientras que el entrenamiento y la validación externa se insertan explícitamente en job_queue para ser procesados por queue_worker.py. Esta diferencia es una característica de la implementación actual: ambos mecanismos permiten devolver pronto la respuesta, pero no deben documentarse como si fueran el mismo canal de ejecución. El estado del recálculo se consulta mediante una ruta específica de la sesión.

La validación externa también mantiene la separación entre entrenamiento y evaluación. La ruta verifica que el dataset externo existe y que el usuario tiene acceso a la sesión; después encola un trabajo con el identificador de la sesión y la ruta del dataset. El worker ejecuta la validación sobre los modelos disponibles sin modificar la configuración original del entrenamiento. Los resultados se almacenan como artefactos de la sesión y quedan disponibles para su consulta posterior.

El informe de MLOps se genera en una fase posterior, cuando la sesión dispone de los datos necesarios. pdf_generator_mlops.py recibe la información preparada por el motor y produce un documento descargable, evitando que el router conozca el formato interno del informe. De este modo, el laboratorio puede cambiar la estructura visual del PDF sin cambiar el contrato de las rutas que consultan los resultados.

Desde la perspectiva de los errores, SD-004 debe diferenciar una configuración inválida, un dataset inexistente, una sesión ajena y un fallo del entrenamiento. Las dos primeras situaciones se rechazan antes de crear o encolar el trabajo; la tercera devuelve 403; y la cuarta se registra durante el procesamiento y cambia el estado del trabajo. Esta clasificación permite que la interfaz muestre una corrección de datos, una falta de permisos o un fallo computacional sin confundirlos.

1.3.5 SD-005: Supervisión y administración

El subsistema SD-005 deriva de SS-005 y reúne las operaciones que solo puede realizar un usuario con rol de administrador. Su objetivo es proporcionar una visión supervisada del uso de la plataforma sin mezclar las operaciones administrativas con las rutas que utiliza un usuario ordinario.

`routers/admin.py` centraliza la comprobación de permisos mediante `_require_admin()`. La función obtiene primero la identidad desde el token de acceso y consulta el rol en la tabla `users`. El router diferencia entre una petición no autenticada, una petición autenticada sin permisos y una petición administrativa válida, devolviendo códigos HTTP diferentes en cada caso.

La consulta del listado administrativo combina la información de `users` con el número de consultas de `consultations` y obtiene también el número de sesiones de laboratorio a partir de la información disponible en `training_results`. Las rutas específicas permiten consultar las consultas de un usuario y visualizar el detalle de una consulta concreta. En la segunda operación se incluyen también las sesiones de entrenamiento recuperadas por `mlops_engine`.

SD-005 no duplica la lógica de propiedad del historial ni la del laboratorio. Su responsabilidad es establecer la autorización administrativa y coordinar las consultas globales; la lectura de los datos sigue utilizando las mismas tablas y servicios que el resto de la aplicación. Esta decisión reduce la duplicación y mantiene una frontera clara entre el control de permisos y la representación de la información.

El flujo administrativo comienza siempre por `_require_admin()`. La función identifica al solicitante, consulta su rol y devuelve un resultado ternario en la práctica: no hay identidad, existe identidad sin privilegios o existe identidad con rol de administrador. Cada endpoint interpreta ese resultado antes de abrir la consulta global correspondiente. Esta secuencia evita que una ruta administrativa ejecute una consulta de usuarios o de consultas antes de haber comprobado el permiso.

La vista global combina dos fuentes de información. Los usuarios y el número de diagnósticos se obtienen mediante una consulta agregada sobre `users` y `consultations`; el número de sesiones del laboratorio se calcula inspeccionando las configuraciones disponibles en `training_results`. Esta decisión refleja la persistencia híbrida actual de la plataforma: la información relacional se consulta en MySQL, mientras que parte de los artefactos de las sesiones se conserva en el sistema de ficheros.

Las rutas de supervisión mantienen la granularidad necesaria para la interfaz administrativa. Una ruta devuelve el listado de usuarios, otra recupera las consultas y sesiones asociadas a un usuario concreto y una tercera muestra el detalle de una consulta. El administrador puede acceder a la información de otros usuarios por su rol, pero el sistema no convierte esas rutas en operaciones de modificación general: su responsabilidad actual es de consulta y supervisión.

La protección del subsistema se completa con las medidas de SD-001 y SD-006. El router administrativo no implementa una segunda autenticación ni una segunda gestión de tokens; reutiliza la identidad de la sesión y consulta el rol en la base de datos. Tampoco escribe una traza de auditoría independiente en cada operación, por lo que el requisito de auditoría administrativa debe considerarse una condición pendiente del modelo operativo y no una tabla o servicio que ya exista.

1.3.6 SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa SS-006 y reúne las capacidades que sirven de apoyo a varios flujos funcionales. Su núcleo es la cola persistente de trabajos, utilizada por los diagnósticos, los entrenamientos y las validaciones externas. También incluye la consulta de idioma, la personalización de la interfaz y los mecanismos transversales que se aplican desde la configuración principal de FastAPI.

`routers/queue.py` expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente. La ruta de estado filtra por el usuario autenticado, devuelve los trabajos recientes y calcula la posición de los que permanecen en `queued`. También interpreta el payload según el tipo de trabajo para mostrar el nombre del modelo o el identificador de la sesión sin enviar al navegador el contenido interno completo del payload.

La cancelación se realiza mediante una actualización condicional: solo puede cambiar a `cancelled` un trabajo que pertenezca al usuario y que continúe en estado `queued`. Si el worker ya lo ha reclamado, la operación no lo interrumpe de forma abrupta. Esta restricción evita inconsistencias entre el estado persistido y el cálculo que ya se está ejecutando.

`services/queue_worker.py` es el consumidor de la cola. Al iniciar la aplicación restablece los trabajos que quedaron en estado `running`, selecciona el siguiente trabajo pendiente y lo reclama mediante una actualización que exige que siga en estado `queued`. Después ejecuta el flujo correspondiente en el executor y marca el resultado como completado o fallido. El worker comparte la persistencia con los routers, pero no comparte con ellos el ciclo de petición: esta es la frontera que permite mantener disponible la interfaz durante las tareas de larga duración.

La parte de internacionalización se concentra en `services/lang.py`, que obtiene el idioma seleccionado y devuelve los textos de la interfaz y de los mensajes de respuesta. Los recursos JavaScript aplican el idioma y el tema visual en el navegador. Estas capacidades no necesitan un servicio externo ni una nueva base de datos, por lo que permanecen como mecanismos transversales de presentación y aplicación.

La tabla de responsabilidades concluye la descomposición del diseño. SD-001 establece la identidad; SD-002 produce el diagnóstico; SD-003 conserva y recupera sus consultas; SD-004 gestiona la experimentación; SD-005 supervisa la plataforma; y SD-006 proporciona la ejecución asíncrona y las capacidades comunes. La dependencia entre ellos es deliberada y coincide con la verificación de consistencia del capítulo 15: los subsistemas funcionales necesitan la identidad de SD-001, los trabajos de SD-002 y SD-004 pasan por SD-006, y SD-003 y SD-005 acceden a la persistencia respetando reglas diferentes de propiedad y administración.

La cola aplica además una política de ordenación que prioriza los trabajos de diagnóstico frente a los de entrenamiento. En `_next_job()`, los trabajos de tipo `training` reciben una prioridad posterior, mientras que los diagnósticos y las validaciones externas pueden ser seleccionados antes cuando comparten el estado `queued`. Esta decisión intenta proteger la respuesta del flujo clínico sin eliminar la posibilidad de que los entrenamientos se ejecuten. La posición que observa el usuario se calcula desde el router de cola y se adapta al tipo de trabajo.

El mecanismo de reclamación evita que dos iteraciones del worker procesen el mismo registro. Primero se selecciona una fila pendiente y después `_claim_job()` ejecuta una actualización condicionada a que el estado siga siendo `queued`. Solo si la actualización afecta a una fila se considera que el worker ha adquirido el trabajo. Aunque la configuración actual inicia un worker interno, esta comprobación introduce una garantía de consistencia útil si el proceso evoluciona hacia una ejecución con más de un consumidor.

La máquina de estados de SD-006 es compartida por los subsistemas que generan trabajos. `queued` indica que la petición ha sido aceptada y espera procesamiento; `running` indica que el worker la ha reclamado; `completed` conserva un resultado; `failed` conserva el error; y `cancelled` identifica una cancelación solicitada por el usuario antes del inicio. No todos los estados se pueden alcanzar desde todas las rutas: la cancelación solo opera sobre `queued`, y un trabajo `running` no se interrumpe mediante una actualización administrativa.

El reinicio de la aplicación se trata como un evento de recuperación. Durante el lifespan de FastAPI se inicializa la base de datos y se llama a `start_worker()`. Este método restablece los trabajos `running` y crea la tarea de `worker_loop()`. La recuperación no pretende reconstruir el estado intermedio de un entrenamiento parcialmente ejecutado; vuelve a poner el trabajo en condición de ser procesado, por lo que los motores deben tolerar que una tarea se reinicie desde la información disponible en su payload y en sus artefactos.

El subsistema también resuelve aspectos que no deben mezclarse con la cola. `lang.py` proporciona textos traducidos para los routers y la interfaz, mientras que los scripts JavaScript gestionan el tema visual y la presentación de los estados. El middleware CSRF y el middleware de cabeceras se registran globalmente en `main.py`, aunque funcionalmente apoyan a todos los subsistemas. Se incluyen en la visión transversal de SD-006 por su alcance común, pero sus responsabilidades de seguridad no sustituyen las comprobaciones de propiedad específicas de SD-003 y SD-004.

El límite principal de SD-006 es que la cola está implementada sobre MySQL y el worker se ejecuta dentro del mismo proceso de la aplicación. Esta solución es suficiente para el entorno actual y evita introducir Redis, Celery o un sistema distribuido que no forma parte del código. A cambio, el diseño depende de la disponibilidad del proceso y de la base de datos local, y no proporciona por sí mismo escalado horizontal de workers ni una garantía independiente de disponibilidad. Esta limitación queda recogida para que las futuras decisiones de soporte no confundan la ejecución asíncrona actual con una arquitectura distribuida.

La descomposición de SD-006 permite finalmente que los demás subsistemas compartan un contrato común para las operaciones largas. SD-002 entrega un trabajo de diagnóstico; SD-004 entrega trabajos de entrenamiento y validación; SD-006 los procesa, actualiza sus estados y devuelve la información a los routers de consulta. El usuario observa siempre una misma idea de funcionamiento —petición aceptada, trabajo pendiente, procesamiento y resultado— aunque los motores internos sean diferentes. Esta uniformidad es la principal aportación arquitectónica del subsistema transversal.

1.3.7 Relaciones de integración entre los subsistemas de diseño

La separación anterior permite identificar qué subsistemas inician una operación, cuáles la ejecutan y cuáles presentan sus resultados. SD-001 constituye el punto de entrada común porque proporciona la identidad y el rol. SD-002 y SD-004 son los productores de los trabajos de mayor coste; SD-006 se encarga de procesarlos; SD-003 y SD-005 consumen la información persistida desde perspectivas diferentes. Esta organización evita que el flujo observable de un caso de uso dependa de una llamada directa desde la interfaz a cada motor interno.

En el flujo de diagnóstico, el navegador envía la imagen y la arquitectura a SD-002. El router obtiene el identificador del usuario desde SD-001, valida la carga y registra un trabajo en SD-006. El worker ejecuta el motor de SD-002, produce sus ficheros y escribe la consulta final en MySQL. Cuando el usuario abre el historial, SD-003 recupera la fila de consultations y entrega las rutas de los artefactos ya generados. Si el usuario dispone del rol adecuado, SD-005 puede recuperar una consulta desde la perspectiva administrativa, pero no altera el modo en que SD-002 calculó el resultado.

El flujo del laboratorio es más amplio. El usuario comienza una conversación que SD-004 dirige hacia Groq; después, la configuración devuelta se valida antes de crear una sesión. El lanzamiento registra los parámetros en los artefactos del laboratorio y en el payload de SD-006. El worker invoca a los motores de entrenamiento y evaluación, y los resultados se almacenan en el directorio de la sesión. Las rutas posteriores de SD-004 consultan esos artefactos, calculan o recuperan el ranking, solicitan comparativas y generan el informe. SD-005 puede contabilizar y consultar las sesiones asociadas a un usuario, pero la gestión detallada de los resultados sigue perteneciendo a SD-004.

Las dependencias de persistencia son deliberadamente asimétricas. SD-001 escribe usuarios y tokens; SD-002 escribe trabajos y consultas; SD-003 modifica únicamente los datos organizativos de las consultas; SD-004 combina MySQL para la cola y el sistema de ficheros para los resultados de entrenamiento; SD-005 realiza consultas globales; y SD-006 actualiza los estados operativos de los trabajos. Ningún subsistema debe acceder a los datos de otro sin aplicar la regla de propiedad o la autorización administrativa que corresponda.

La aplicación mantiene también una separación entre datos estructurados y artefactos. Las cuentas, las consultas, los tokens y los estados de la cola se guardan en MySQL porque necesitan filtros, relaciones y actualizaciones transaccionales. Las imágenes, los mapas XAI, los informes PDF, los logs y las configuraciones de entrenamiento se guardan en el sistema de ficheros porque son archivos o estructuras generadas por los motores. Las tablas conservan referencias a los artefactos, pero no incorporan el contenido binario completo en cada registro.

La gestión de errores sigue el límite del subsistema que los origina. Un error de autenticación se resuelve en SD-001 con una respuesta de acceso no autorizado; un fichero que no cumple las restricciones se rechaza en SD-002 antes de generar un trabajo; una consulta ajena se bloquea en SD-003; una configuración de entrenamiento inválida se rechaza en SD-004; un usuario sin rol recibe 403 en SD-005; y un fallo durante una tarea se registra mediante SD-006 como trabajo fallido. Esta distribución evita que los routers tengan que conocer detalles internos de todos los componentes para construir mensajes de error.

La extensibilidad del diseño se apoya en los puntos donde la variación es real. Añadir una nueva arquitectura de inferencia afecta principalmente a `ml_engine` y a las funciones XAI compatibles; añadir un nuevo tipo de tarea requiere ampliar la selección de `_execute_job()` y los datos que presenta `queue.py`; incorporar una nueva operación administrativa se restringe a SD-005; y ampliar los idiomas se concentra en los recursos gestionados por `lang.py` y los scripts de interfaz. Estas extensiones no exigen modificar el mecanismo general de autenticación ni reconstruir la persistencia completa.

El criterio de diseño adoptado es, por tanto, la cohesión funcional con dependencias explícitas. Los componentes de un subsistema colaboran porque comparten una responsabilidad y no únicamente porque estén escritos en Python. Los routers conocen los contratos HTTP; los servicios conocen la lógica de aplicación; el worker conoce la ejecución prolongada; MySQL conoce el estado estructurado; y el navegador conoce la representación y la interacción. Esta distribución hace posible comparar cada subsistema con los casos de uso del capítulo 12 y comprobar, durante las pruebas, si el fallo pertenece a la entrada, a la coordinación, al cálculo o a la persistencia.

Flujo de integración	Subsistema iniciador	Subsistema coordinador	Resultado persistido o presentado
Registro e inicio de sesión	SD-001	SD-001 y middleware global	Cookies de sesión y registro en users y refresh_tokens.
Diagnóstico asistido	SD-002	SD-006 mediante job_queue	Predicción, confianza, mapas XAI, PDF y fila en consultations.
Consulta del historial	SD-003	SD-001 para identidad	Listado de consultas y acceso a sus artefactos.
Entrenamiento MLOps	SD-004	SD-006 mediante job_queue	Sesión, modelos, métricas, artefactos y logs.
Validación externa	SD-004	SD-006 mediante job_queue	Resultados de validación asociados a la sesión.
Supervisión administrativa	SD-005	SD-001 para autenticación y rol	Listado de usuarios, consultas y sesiones supervisadas.
Consulta y cancelación de trabajos	SD-006	SD-001 para identidad	Estado, posición, error o cancelación del trabajo.

Tabla x - Relaciones de integración entre subsistemas de diseño

1.4 Operación segura y controles de protección

La operación de vitalXAI debe conservar las garantías de confidencialidad, integridad y disponibilidad definidas durante el análisis. Estas garantías no dependen de una única función, sino de la combinación de controles distribuidos entre la sesión, los routers, los servicios, la base de datos, el sistema de ficheros y el entorno de publicación. El propósito de este apartado es describir cómo debe funcionar la plataforma de forma segura cuando se inicia, recibe peticiones, procesa trabajos y se detiene, manteniendo la correspondencia con los requisitos RNF-001 a RNF-034 y con los casos de uso que implican autenticación, datos sanitarios o ejecución asíncrona.

La operación segura se organiza en seis ámbitos: control de identidad y permisos, protección de datos y ficheros, comunicaciones y secretos, seguridad de las tareas asíncronas, persistencia y recuperación y condiciones del despliegue. La siguiente tabla resume los controles principales y su punto de aplicación.

Ámbito operativo	Controles principales	Componentes implicados
Identidad y sesión	Hash de contraseñas, JWT con expiración, refresh tokens, rotación, revocación y cookies protegidas.	auth.py, auth_service.py y tabla refresh_tokens en MySQL.
Autorización	Comprobación de autenticación, propiedad de los recursos y rol de administrador.	Routers de history, trainer, queue y administración.
Protección web	CSRF, cabeceras de seguridad y limitación de peticiones.	CSRFMiddleware, SecurityHeadersMiddleware y slowapi.
Datos y ficheros	Datasets anonimizados, validación de imágenes, asociación por usuario y control de rutas.	MySQL, inference.py, consultations y directorios de resultados.
Trabajos asíncronos	Estados persistentes, reclamación condicional, aislamiento de errores y recuperación al reiniciar.	Tabla job_queue y ejecución en queue_worker.py.
Operación y recuperación	Variables de entorno, MySQL/XAMPP, HTTPS mediante el túnel y copias operativas de la base de datos.	main.py, .env, Uvicorn, XAMPP y Cloudflare Tunnel.

Tabla x - Controles de operación segura de vitalXAI

1.4.1 Control de identidad, sesión y autorización

El acceso a las funciones privadas comienza con la identificación del usuario a partir de la cookie `access_token`. El token contiene el identificador de la cuenta y una fecha de expiración, y se firma mediante HS256 con la clave configurada en `JWT_SECRET_KEY`. El servidor no confía en un identificador recibido como un campo ordinario de la petición: cada router obtiene el usuario desde la credencial de sesión y utiliza ese valor para realizar sus consultas.

Durante el registro, `auth.py` valida los campos recibidos y comprueba la unicidad del nombre de usuario antes de insertar la cuenta. La contraseña se transforma mediante `bcrypt` y solo el resultado se almacena en `password_hash`. Durante el inicio de sesión se recupera el hash de la cuenta y se compara con la contraseña recibida; si la comparación falla, no se crean las cookies de sesión. Este comportamiento evita que una petición con credenciales inválidas genere una identidad parcial o un token utilizable.

La sesión utiliza dos credenciales con funciones distintas. El token de acceso se emplea para identificar las peticiones ordinarias y tiene una duración configurable. El token de refresco se utiliza exclusivamente en `/api/token/refresh`, se registra mediante un hash en `refresh_tokens` y permite crear un nuevo acceso sin repetir el formulario de inicio de sesión. La rotación revoca el token anterior y genera otro valor aleatorio. Si se detecta el uso de una credencial revocada fuera del intervalo de tolerancia, el servicio revoca el conjunto de tokens asociado al usuario.

El cierre de sesión debe revocar el refresh token y eliminar las cookies desde la respuesta. La revocación se realiza en la base de datos, por lo que no depende de que el navegador elimine correctamente sus datos locales. El token de acceso puede seguir presente hasta su expiración natural, pero las operaciones de renovación quedan bloqueadas por el estado persistido del token de refresco. Esta diferencia debe considerarse al operar el sistema: cerrar la sesión evita la renovación, pero la protección del canal y la duración razonable del token de acceso siguen siendo necesarias.

La autorización se aplica después de la autenticación. En el historial, las consultas deben pertenecer al usuario salvo cuando la operación se realiza desde el ámbito administrativo. En el laboratorio, las sesiones y sus resultados se comprueban mediante `_verify_session_ownership()` antes de ser consultados, renombrados o eliminados. En la cola, tanto la consulta del estado como la cancelación filtran por `user_id`. En administración, `_require_admin()` consulta el rol en `users` y bloquea al usuario autenticado que no tenga el valor `admin`.

Este diseño evita dos errores habituales. El primero es confundir que un usuario esté autenticado con que pueda acceder a cualquier registro; la sesión solo proporciona identidad, mientras que la propiedad decide el alcance. El segundo es dejar la autorización en el cliente; aunque la interfaz oculte el panel administrativo a un usuario ordinario, el servidor vuelve a comprobar el rol cuando recibe la petición.

La protección CSRF se aplica a las operaciones que cambian el estado. El middleware genera una cookie de token y exige que el navegador copie el mismo valor en la cabecera `X-CSRF-Token`. Las peticiones de lectura no necesitan esta comprobación y el inicio de sesión se excluye para permitir que un visitante obtenga su primera sesión. Esta excepción no convierte el resto de las rutas públicas en operaciones sin protección: cada petición que modifica datos debe incluir el token correspondiente.

1.4.2 Protección de los datos y de los artefactos

La plataforma debe operar exclusivamente con imágenes y datasets anonimizados. La arquitectura no incorpora un componente que identifique o elimine automáticamente los datos personales que puedan aparecer dentro de una radiografía; por tanto, la anonimización previa de los ficheros es una condición del procedimiento de uso. La interfaz no debe utilizarse para introducir nombres, identificadores sanitarios u otros metadatos que no sean necesarios para el diagnóstico asistido.

La ruta de diagnóstico aplica controles básicos antes de escribir una imagen: restringe el tipo MIME a JPEG y PNG y limita el tamaño a 10 MB. Después genera un nombre basado en la fecha y conserva el fichero en static/uploads. El trabajo encolado contiene la ruta de la imagen y la arquitectura solicitada, mientras que la fila final de consultations conserva las rutas de los artefactos asociados. Las rutas no deben construirse a partir de concatenaciones posteriores de datos no validados, y el entorno debe restringir los permisos de escritura de los directorios de cargas y resultados al proceso que ejecuta la aplicación.

La asociación por usuario es el control principal de confidencialidad lógica. consultations, training_jobs, job_queue y refresh_tokens contienen un user_id que permite aplicar filtros explícitos. Las consultas de historial y de cola usan ese filtro directamente; las operaciones del laboratorio validan la propiedad de la sesión; y las operaciones administrativas se separan mediante el rol. Esta estrategia no sustituye a los permisos del sistema operativo sobre los directorios, pero evita que un usuario autenticado acceda a los metadatos de otra cuenta a través de las rutas funcionales.

Los ficheros generados por el diagnóstico incluyen la imagen original, el mapa de explicabilidad y el informe PDF. El informe puede contener el resultado, la confianza, el modelo utilizado y la imagen, por lo que debe considerarse parte de la información protegida aunque se sirva como un documento descargable. El sistema conserva las rutas y la interfaz muestra los artefactos dentro del flujo autorizado de la consulta. La configuración del servidor debe impedir que el directorio de resultados se convierta en un repositorio de ficheros accesible sin control cuando la aplicación se publique fuera del entorno local.

El acceso a MySQL se realiza mediante consultas parametrizadas y conexiones procedentes del pool. Las operaciones de lectura y escritura cierran la conexión al terminar, y las actualizaciones importantes ejecutan commit() de forma explícita. Las claves ajenas vinculan los registros dependientes con users, lo que evita mantener consultas huérfanas cuando se elimina una cuenta con las reglas configuradas en el esquema.

La protección lógica de los datos debe complementarse con una política de retención. La implementación actual permite eliminar una consulta de forma física desde el historial y no incorpora un subsistema de archivado o borrado lógico para las consultas. Por tanto, el operador debe conocer que la eliminación no equivale a una copia de seguridad ni a una auditoría histórica. Las copias de la base de datos y de los directorios de resultados deben gestionarse por separado si se necesita recuperar una operación eliminada.

1.4.3 Comunicaciones, configuración y secretos

En el entorno local, Uvicorn atiende la aplicación en el puerto 8000 y MySQL se ejecuta normalmente mediante XAMPP en el puerto 3306. Para permitir el acceso desde un navegador externo se utiliza el túnel configurado para la demostración. El túnel actúa como punto de publicación y debe proporcionar HTTPS hacia el exterior; el backend no debe exponerse directamente abriendo puertos de la base de datos a Internet.

La comunicación entre el navegador y las rutas de FastAPI transporta credenciales, imágenes y resultados. Por ello, el acceso remoto debe realizarse siempre mediante HTTPS basado en TLS. Las cabeceras añadidas por SecurityHeadersMiddleware complementan el cifrado del canal: Content-Security-Policy restringe los orígenes de recursos, X-Frame-Options evita la inclusión en marcos, X-Content-Type-Options bloquea la interpretación MIME indebida, Strict-Transport-Security comunica la preferencia por HTTPS y Referrer-Policy limita la información que se comparte entre páginas.

La aplicación utiliza variables de entorno para los valores que no deben formar parte del código fuente. Entre ellas se encuentran la clave JWT, las credenciales de MySQL, el tamaño del pool, las duraciones de los tokens y GROQ_API_KEY. El fichero .env, cuando se utiliza en desarrollo, debe quedar fuera del control de versiones y sus permisos deben limitarse al usuario que ejecuta la aplicación. La clave por defecto de desarrollo que auth_service.py detecta mediante una advertencia no es una configuración válida para producción.

La integración con Groq exige controlar tanto la disponibilidad como el secreto de acceso. chatbot_service.py es el único componente que necesita la clave de la API y traduce la respuesta del servicio externo al flujo conversacional del laboratorio. Los routers no deben incluir la clave en las respuestas JSON ni en los logs. Si Groq no está disponible, la conversación debe informar del error sin revelar la configuración del entorno ni interrumpir la disponibilidad del resto de la plataforma.

La integración con Hugging Face presenta una restricción diferente: puede requerir conectividad durante la descarga inicial de los modelos Transformer y memoria suficiente para mantenerlos cargados. Una vez disponibles, el motor puede reutilizar los modelos según su configuración. La operación debe distinguir entre un fallo de red durante la carga y una predicción inválida, registrando el trabajo como fallido cuando no pueda completarse y evitando devolver al usuario una salida parcial.

La configuración de desarrollo se debe validar antes de publicar el servicio. El operador debe comprobar que Python y las dependencias coinciden con requirements.txt, que MySQL está iniciado, que las variables de entorno están definidas, que existe la estructura de directorios necesaria y que el acceso al túnel funciona mediante HTTPS. Esta verificación se complementa con la ejecución de las pruebas automatizadas y evita que un fallo de configuración se interprete como un error del modelo o de la lógica de negocio.

1.4.4 Seguridad de la ejecución asíncrona

Los trabajos largos se consideran recursos operativos que requieren control de ciclo de vida. La petición que los crea no ejecuta el cálculo directamente: primero valida la identidad, registra el tipo de trabajo, asocia el payload al usuario y devuelve su identificador. El worker recupera la información desde MySQL y no debe aceptar órdenes procedentes directamente del navegador para ejecutar una función arbitraria. El tipo `job_type` determina si se llama al flujo de diagnóstico, entrenamiento o validación externa.

El payload contiene rutas y parámetros que deben haber sido validados por el router que creó el trabajo. El worker vuelve a interpretar el JSON antes de utilizarlo y falla de forma controlada si faltan claves obligatorias. Esta doble frontera es necesaria porque la fila podría conservar datos antiguos o haber sido alterada por un fallo de operación. El worker nunca debe tratar el contenido de un payload como código ejecutable ni utilizarlo para seleccionar un módulo fuera de los tipos explícitamente soportados.

La reclamación condicional del trabajo protege frente a duplicados. `_claim_job()` solo cambia el estado si sigue siendo `queued`; por tanto, un trabajo cancelado o reclamado por otro consumidor no se ejecuta de nuevo desde esa iteración. Después, `_finish_job()` guarda el resultado serializado y `_fail_job()` limita el mensaje de error a 500 caracteres. Este límite reduce el riesgo de llenar la base de datos con trazas descontroladas, aunque los diagnósticos detallados deben consultarse en los logs del entorno cuando sean necesarios.

El aislamiento de errores se basa en el tratamiento individual de cada trabajo. Una excepción durante la inferencia, la generación XAI o el entrenamiento se captura en el bloque de procesamiento y cambia la fila a `failed`. El bucle principal continúa con el siguiente trabajo y, si se produce un error externo al procesamiento, espera antes de volver a consultar la cola. Así, un dataset defectuoso o una arquitectura incompatible no detiene por completo el servicio.

La recuperación tras un reinicio es deliberadamente conservadora. `start_worker()` restablece los trabajos que quedaron en `running` a `queued`, porque no existe un registro externo que garantice en qué punto exacto del cálculo se encontraba el proceso. Esta estrategia puede provocar que una tarea vuelva a comenzar, pero evita dejarla invisible para el usuario. El diseño debe asumir que las operaciones de entrenamiento y validación pueden repetirse después de una interrupción y que los artefactos parciales no deben considerarse resultados válidos hasta que el trabajo se marque como completado.

La cancelación está restringida al estado `queued`. No se intenta finalizar abruptamente un hilo o un cálculo que ya está en `run_in_executor`, porque esa interrupción podría dejar ficheros incompletos, sesiones en un estado ambiguo o conexiones abiertas. El usuario recibe una respuesta de recurso no encontrado cuando el trabajo no existe, no le pertenece o ya ha comenzado. Este comportamiento prioriza la integridad del procesamiento frente a una cancelación inmediata sin garantías.

1.4.5 Persistencia, arranque y recuperación operativa

El arranque de la aplicación se concentra en la función `lifespan` de `main.py`. Primero se carga la configuración del entorno, después se ejecuta `init_db()` para comprobar o crear las tablas principales y, finalmente, se inicia el worker. Si MySQL no está disponible, la aplicación informa de la incidencia con un mensaje destinado al operador. Esta información no debe considerarse una sustitución de la recuperación: la plataforma necesita que el servicio de base de datos esté operativo para autenticar usuarios, consultar trabajos y persistir resultados.

El pool de conexiones se crea de forma diferida cuando se solicita la primera conexión. Esta decisión evita abrir conexiones innecesarias al importar el módulo y permite que los parámetros se obtengan del entorno ya cargado. El tamaño por defecto es cinco, pero puede modificarse con `DB_POOL_SIZE` según la capacidad real del servidor. El operador debe evitar configurar un pool superior al número de conexiones que MySQL puede atender con estabilidad.

Las tablas principales cumplen funciones diferentes. `users` y `refresh_tokens` sostienen la identidad; `consultations` conserva el historial clínico; `training_jobs` conserva información de entrenamientos; y `job_queue` permite coordinar los trabajos del diagnóstico, el entrenamiento y la validación externa. La persistencia de los artefactos se completa con `static/uploads`, `static/results` y `training_results`. Un procedimiento de copia completo debe incluir tanto el contenido relacional como los directorios de ficheros, ya que restaurar solo MySQL dejaría rutas sin sus documentos asociados.

La recuperación operativa debe contemplar tres fallos distintos. Ante un fallo de MySQL, no se pueden resolver las operaciones que dependen de la cuenta o de la cola; ante un fallo del sistema de ficheros, los registros pueden conservar rutas cuyos documentos no existen; y ante un fallo del worker, los trabajos running pueden volver a quedar encolados durante el siguiente arranque. La guía de despliegue debe permitir identificar cuál de estos componentes ha fallado antes de reiniciar indiscriminadamente toda la aplicación.

La integridad de los resultados requiere una comprobación adicional durante la operación. Una consulta marcada como `completed` debe disponer de sus artefactos esperados y de la fila de consulta correspondiente. Un trabajo `failed` debe conservar un mensaje suficiente para distinguir un error del modelo, del dataset, de la API externa o de la persistencia. El sistema actual registra estos estados, pero no incorpora un servicio separado de observabilidad; por ello, el operador debe revisar la salida del proceso y los logs disponibles cuando se investigue una incidencia.

1.4.6 Límites actuales y responsabilidades del operador

La solución actual está diseñada para un entorno local o de demostración controlada. La aplicación, MySQL, los ficheros y el worker pueden residir en la misma máquina, y el túnel proporciona el acceso remoto sin convertir el sistema en una plataforma distribuida. Esta decisión reduce la complejidad, pero implica que la disponibilidad depende del equipo anfitrión, de XAMPP, de la conectividad y de la cuota de Groq cuando se utiliza el asistente.

El operador debe iniciar MySQL antes de ejecutar la aplicación, verificar las variables de entorno, evitar el uso de la clave JWT de desarrollo, conservar las dependencias del entorno virtual y comprobar que el directorio de resultados tiene espacio suficiente. También debe controlar los datasets que se cargan, confirmar que están anonimizados y no exponer directamente los puertos 3306 ni otros servicios internos a Internet. La URL pública del túnel debe compartirse solo con los usuarios autorizados durante la demostración.

La implementación actual no proporciona alta disponibilidad, balanceo de workers, almacenamiento de secretos dedicado, cifrado de ficheros en reposo ni un sistema formal de auditoría administrativa. Estas ausencias no se ocultan detrás de nombres arquitectónicos: son límites operativos que deben tenerse en cuenta al valorar la plataforma. Los controles implementados —hashes, tokens, CSRF, cabeceras, rate limiting, filtros de propiedad, estados de cola y recuperación de trabajos— aportan una base de seguridad, pero no equivalen por sí solos a una certificación clínica ni a un entorno de producción de alta disponibilidad.

En consecuencia, el uso de vitalXAI debe mantenerse dentro del alcance declarado en el proyecto. La plataforma ofrece apoyo al diagnóstico y experimentación de modelos, pero la decisión clínica corresponde al profesional. Los datos de pacientes deben anonimizarse antes de la carga, los resultados deben interpretarse junto con los mapas de explicabilidad y los informes no deben considerarse una autorización para automatizar decisiones médicas sin supervisión. La operación segura incluye tanto las medidas técnicas como estas reglas de uso.

La combinación de controles de software y responsabilidades operativas completa el diseño de seguridad. SD-001 protege la identidad; SD-002 limita y procesa las entradas de diagnóstico; SD-003 y SD-004 restringen el acceso a los datos propios; SD-005 separa la supervisión administrativa; SD-006 controla los trabajos y sus estados; y el entorno mantiene MySQL, los ficheros, las claves y el canal HTTPS bajo condiciones conocidas. Esta distribución permite verificar cada requisito de operación en el punto donde realmente se aplica y evita atribuir a una única capa garantías que dependen del sistema completo.

2 Arquitectura de soporte y mecanismos transversales

La arquitectura de soporte reúne los elementos que permiten que los subsistemas funcionales operen de forma coordinada, segura y mantenible. A diferencia de los subsistemas de diseño del capítulo 18, estos componentes no representan una capacidad clínica o investigadora concreta para el usuario. Su responsabilidad es proporcionar persistencia, almacenamiento, configuración, protección, validación y ejecución común a las funcionalidades de autenticación, diagnóstico, historial, laboratorio y administración.

La separación entre funcionalidad y soporte no significa que ambos ámbitos estén desconectados. SD-002 depende del almacenamiento y de la cola; SD-004 necesita el sistema de ficheros, MySQL y la configuración de los servicios externos; SD-001 utiliza el middleware de seguridad; y todos los routers dependen del ciclo de vida de FastAPI. La arquitectura de soporte permite que estas dependencias se apliquen de forma uniforme, evitando que cada módulo implemente su propia conexión, su propio control de errores o su propia interpretación del entorno.

En la implementación actual se distinguen dos dimensiones. La primera está formada por los subsistemas de soporte, que agrupan servicios o recursos con una identidad técnica propia: persistencia relacional, almacenamiento de ficheros, entorno de ejecución e integración externa y calidad operativa. La segunda está formada por los mecanismos de diseño transversales, que son reglas aplicadas a varios componentes: gestión de configuración, validación de entradas, autenticación y autorización, manejo de errores, ejecución asíncrona, modularidad y pruebas automatizadas.

2.1 Subsistemas de Soporte

Los subsistemas de soporte descritos en esta sección no sustituyen a SD-006 ni duplican sus responsabilidades. SD-006 pertenece a la lógica de capacidades transversales que el usuario puede utilizar, como consultar o cancelar un trabajo. En cambio, los subsistemas de soporte proporcionan la infraestructura que hace posible esa capacidad: la conexión a MySQL, el sistema de ficheros, el entorno de ejecución y los servicios que se integran con la plataforma.

Subsistema de soporte	Responsabilidad	Componentes actuales
SSOP-001: Persistencia relacional	Crear conexiones, conservar entidades y asegurar la integridad básica de las relaciones.	database.py, MySQLConnectionPool, MySQL/XAMPP y tablas del esquema.
SSOP-002 Almacenamiento de ficheros y recursos	Conservar imágenes, mapas XAI, informes, resultados MLOps, plantillas y recursos estáticos.	static/, templates/, training_results/ y montajes de main.py.
SSOP-003 Entorno de ejecución e integración	Proporcionar el proceso web, las dependencias Python y el acceso a APIs y repositorios externos.	static/, templates/, training_results/ y montajes de main.py.
SSOP-004 Calidad y verificación	Comprobar el comportamiento de los componentes y detectar regresiones durante el desarrollo.	pytest, fixtures de tests/conftest.py, mocks, pytest-cov, ruff y CI.

Tabla x - Subsistemas de soporte de vitalXAI

2.1.1 SSOP-001: Persistencia relacional y conexiones de base de datos

El subsistema SSOP-001 encapsula el acceso de la aplicación a MySQL. Su componente principal es `database.py`, que mantiene una referencia al pool de conexiones y expone `get_db_connection()` para que los routers y servicios puedan solicitar una conexión sin conocer cómo se crea. La configuración se obtiene de `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` y `DB_POOL_SIZE`, con valores de desarrollo definidos cuando el entorno no proporciona una variable concreta.

El pool se crea de forma diferida. La primera operación que necesita una conexión invoca `_get_pool()`, que construye un `MySQLConnectionPool` con el tamaño configurado. Las operaciones posteriores reutilizan el mismo objeto y solicitan conexiones disponibles mediante `get_connection()`. Este comportamiento reduce el coste de crear conexiones repetidas y proporciona un punto único desde el que controlar el acceso concurrente a la base de datos.

La inicialización del esquema se realiza mediante `init_db()`, llamada durante el lifespan de FastAPI. El procedimiento comprueba la existencia de las tablas `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`, y crea las que no existan. Esta estrategia es adecuada para el entorno local y de demostración actual, donde la base de datos se inicializa desde la aplicación. No equivale a un sistema de migraciones versionadas: una evolución posterior del esquema deberá gestionarse con un procedimiento controlado para no depender únicamente de sentencias `CREATE TABLE IF NOT EXISTS`.

La persistencia relacional cumple dos funciones diferentes. Por un lado, conserva información de negocio, como las cuentas y consultas. Por otro, sirve de soporte operacional para la cola de trabajos y los tokens de refresco. En ambos casos se utilizan consultas parametrizadas, se confirma la escritura con `commit()` y se cierra la conexión tras terminar. Las claves ajenas de las tablas de trabajos, consultas y tokens apuntan a `users`, lo que mantiene la relación de propiedad a nivel de esquema.

Los componentes funcionales no comparten una conexión global abierta. Cada router o servicio solicita una conexión para una operación concreta, obtiene un cursor, ejecuta la consulta y cierra la conexión. Esta decisión limita el alcance de un error de conexión y evita que una petición retenga indefinidamente un recurso del pool. El coste de esta estrategia se controla mediante `DB_POOL_SIZE`, que debe ajustarse a la capacidad real de MySQL y al número de peticiones que se espera atender.

El subsistema también define el comportamiento ante indisponibilidad. Si MySQL no puede conectarse durante el arranque, `main.py` muestra un aviso operativo indicando que debe iniciarse XAMPP o revisarse la configuración. La aplicación puede continuar hasta iniciar el worker, pero las operaciones que requieren identidad, persistencia o cola no podrán completarse correctamente. Esta respuesta informa del problema sin ocultar que MySQL es una dependencia obligatoria.

2.1.2 SSOP-002: Almacenamiento de ficheros y recursos de aplicación

El subsistema SSOP-002 gestiona los artefactos que no son adecuados para almacenarse directamente como columnas relacionales. La aplicación utiliza `static/uploads` para las imágenes originales de diagnóstico, `static/results` para los mapas XAI y `training_results` para las configuraciones, logs y resultados del laboratorio. Los informes PDF se generan en las rutas que devuelven `pdf_generator.py` y `pdf_generator_mlops.py`. MySQL conserva las referencias a los ficheros cuando forman parte de una consulta o de un resultado.

La separación entre base de datos y sistema de ficheros evita insertar imágenes y documentos binarios grandes dentro de las tablas principales. La base de datos mantiene los metadatos que necesita el historial —modelo, etiqueta, confianza, usuario y rutas— y el sistema de ficheros conserva el contenido. Cuando SD-003 recupera una consulta, no recalcula ni copia el documento: utiliza las referencias persistidas para que la interfaz solicite el artefacto correspondiente.

Las plantillas Jinja2 y los recursos JavaScript pertenecen también a este subsistema de soporte, aunque se consumen desde la capa de presentación. `main.py` monta `static` como directorio de recursos y crea el directorio de resultados de entrenamiento si no existe. Las plantillas se resuelven mediante Jinja2Templates desde `templates`. Esta configuración permite que la aplicación sirva la interfaz sin un servidor frontend independiente ni un proceso de compilación adicional.

El almacenamiento se organiza según el origen del artefacto. Las cargas de diagnóstico reciben un nombre generado con una marca temporal y el nombre original; los resultados XAI se generan en un subdirectorio específico; y las sesiones de entrenamiento utilizan su identificador para agrupar configuración, logs y resultados. Esta organización facilita que el worker encuentre los artefactos a partir del payload y que el administrador pueda contabilizar sesiones inspeccionando sus configuraciones.

El control de acceso a los ficheros debe entenderse junto al control de acceso de los registros. La aplicación comprueba la propiedad de la consulta o de la sesión antes de devolver los metadatos que apuntan a los artefactos. Además, el entorno operativo debe limitar los permisos de escritura de los directorios al proceso de la aplicación y evitar la exposición directa de los directorios de datos cuando no sea necesaria. La implementación actual ofrece validación de tipo MIME y tamaño para las imágenes, pero no incorpora un antivirus ni un servicio independiente de análisis de contenido.

La recuperación de este subsistema requiere copiar tanto los directorios de ficheros como las tablas que contienen sus rutas. Una copia de MySQL sin `static/uploads`, `static/results` o `training_results` restauraría referencias incompletas; una copia de los ficheros sin la base de datos perdería la asociación con usuarios, consultas y sesiones. Esta dependencia debe reflejarse en el procedimiento operativo de copia y restauración.

2.1.3 SSOP-003: Entorno de ejecución e integraciones externas

SSOP-003 agrupa los elementos necesarios para iniciar la aplicación y comunicarse con los servicios que no forman parte del proceso local. El servidor se ejecuta con Uvicorn sobre Python 3.11 y utiliza FastAPI como framework web. El fichero requirements.txt fija las versiones de las dependencias principales, entre ellas TensorFlow, Keras, Transformers, mysql-connector-python, Groq, fpdf2, bcrypt, python-jose, slowapi, pytest y ruff.

El entorno virtual aísla estas dependencias de otras instalaciones de Python de la máquina. Antes de iniciar el sistema se debe comprobar que el intérprete activo corresponde al entorno del proyecto y que las versiones instaladas coinciden con el fichero de requisitos. Esta condición es especialmente importante para TensorFlow, Keras y Transformers, cuyas incompatibilidades pueden impedir la importación del motor de inferencia aunque el código de los routers sea correcto.

La aplicación se integra con Groq mediante chatbot_service.py. El servicio externo recibe los mensajes de configuración y devuelve una respuesta que el laboratorio procesa. El subsistema de soporte no expone una ruta genérica para llamar a Groq: la integración se encapsula en un único componente y utiliza GROQ_API_KEY desde el entorno. Esta frontera reduce la propagación de credenciales y permite sustituir o simular el proveedor durante las pruebas.

La carga de modelos Transformer constituye la otra integración externa relevante. ml_engine.py utiliza la configuración de Hugging Face para resolver las arquitecturas y pesos correspondientes. El primer uso puede requerir conectividad y espacio local para descargar o almacenar los modelos; los usos posteriores pueden beneficiarse de la caché disponible. La operación debe registrar como fallo de integración la imposibilidad de cargar un modelo, sin atribuir ese problema a la validación de la imagen o a la base de datos.

El entorno de publicación utiliza opcionalmente un túnel de Cloudflare para que un navegador externo acceda al servidor local. El túnel no aloja los datos ni ejecuta los modelos; solo proporciona el canal de entrada hasta Uvicorn. MySQL permanece en la máquina de ejecución y sus puertos no deben publicarse. Esta configuración conserva bajo control las versiones y los ficheros del proyecto, aunque no proporciona por sí misma alta disponibilidad ni escalado horizontal.

2.1.4 SSOP-003: Entorno de ejecución e integraciones externas

El subsistema SSOP-004 agrupa las herramientas que permiten comprobar que los cambios no rompen el comportamiento existente. Las pruebas unitarias se encuentran en tests/unit, las de integración en tests/integration y las fixtures compartidas en tests/conftest.py. Los servicios externos se simulan mediante mocks y la base de datos se sustituye en las pruebas de integración por el entorno definido para ese nivel.

La estrategia de pruebas refleja la arquitectura. Los routers se prueban comprobando respuestas, autenticación, propiedad y errores; los servicios de aprendizaje automático se prueban con modelos simulados; el worker se prueba con trabajos controlados; y los middlewares se prueban de forma aislada para confirmar CSRF, cabeceras y limitación. La batería existente incluye pruebas de autenticación, validación de entradas, base de datos, inferencia, XAI, MLOps, cola, seguridad e internacionalización, además del flujo de autenticación integrado.

pytest-cov permite medir la cobertura de los módulos principales de services, routers y database.py. ruff verifica el estilo del código y el workflow de integración continua ejecuta las comprobaciones automatizadas. La calidad no se concentra en una única prueba de aceptación: se distribuye entre verificaciones rápidas durante el desarrollo y comprobaciones más amplias antes de integrar cambios.

Las fixtures deben mantenerse deterministas y sin datos reales de pacientes. Los modelos, clientes externos y conexiones de producción se sustituyen por dobles de prueba en los niveles que requieren aislamiento. La prueba de integración debe comprobar la colaboración entre capas, pero no convertir el entorno de producción ni la API de Groq en dependencias necesarias para cada ejecución local. Esta política mejora la repetibilidad y evita que una interrupción externa produzca falsos fallos del código.

2.2 Mecanismos de diseño transversales

Los mecanismos de diseño son reglas que se aplican a varios subsistemas independientemente de la funcionalidad concreta que implementen. Se distinguen de los subsistemas de soporte porque no son necesariamente un servicio con una identidad propia: son decisiones repetidas sobre cómo validar, autenticar, persistir, ejecutar y comunicar errores. En vitalXAI se identifican seis mecanismos principales.

2.2.1 Configuración externa y separación de secretos

La configuración que cambia entre máquinas o que contiene información sensible se obtiene desde variables de entorno. Este mecanismo se aplica a autenticación, MySQL, Groq, duración de tokens y tamaño del pool. El código conserva valores por defecto únicamente para permitir el desarrollo local y emite advertencias cuando una configuración insegura puede afectar a la operación.

La separación de secretos evita que una clave JWT, una contraseña de MySQL o una clave de Groq quede incrustada en un router o en un servicio. Los componentes reciben la configuración en el momento de importación o de ejecución y utilizan nombres de variables comunes. El fichero de ejemplo o la guía de despliegue puede documentar los nombres requeridos, pero no debe contener valores válidos.

Este mecanismo también mejora la portabilidad. El mismo código puede ejecutarse con una base de datos local, una URL pública del túnel diferente o una duración de tokens ajustada al entorno sin modificar los routers. La configuración de desarrollo, pruebas y demostración debe mantenerse separada para evitar que una prueba utilice accidentalmente datos o credenciales del entorno operativo.

2.2.2 Validación en las fronteras de entrada

Toda información procedente del navegador se considera no confiable hasta ser validada. FastAPI valida tipos y parámetros declarados por las rutas; los routers añaden comprobaciones de negocio; y los servicios vuelven a comprobar las condiciones necesarias antes de utilizar los datos en operaciones sensibles.

En el registro se comprueba el correo, la longitud de la contraseña y los campos obligatorios. En el diagnóstico se validan MIME y tamaño de la imagen. En el laboratorio se comprueba la existencia del dataset y la propiedad de la sesión. En la cola se valida la identidad y la pertenencia del trabajo. La validación del cliente mediante JavaScript puede mejorar la experiencia, pero no es la fuente de verdad: la petición debe poder rechazarse correctamente aunque se construya con un cliente distinto del frontend oficial.

Las consultas SQL utilizan parámetros separados de las instrucciones, y los nombres de archivo se generan dentro del servidor antes de copiar el contenido recibido. Los datos estructurados del payload de la cola se serializan como JSON y se interpretan según un tipo de trabajo controlado. La combinación de estas decisiones reduce el riesgo de que una entrada del usuario se convierta directamente en una consulta, una ruta o una orden ejecutable.

2.2.3 Autenticación, autorización y propiedad de recursos

El mecanismo de identidad se reutiliza en todos los routers privados mediante `get_user_id_from_token()`. Esto evita que cada subsistema implemente una forma distinta de interpretar la sesión. Después de obtener el usuario, cada operación aplica el alcance correspondiente: propiedad del registro, rol administrativo o acceso general a una capacidad transversal.

La autorización se realiza cerca del recurso que se protege. El historial comprueba la propiedad de la consulta; el laboratorio comprueba la sesión; la cola comprueba el trabajo; y administración comprueba el rol. Esta decisión impide que un identificador enviado por el navegador sea suficiente para acceder a un objeto ajeno. Las funciones compartidas, como `_check_consultation_ownership()` y `_require_admin()`, concentran reglas repetidas sin convertirlas en un bypass general de seguridad.

CSRF, cabeceras y rate limiting se aplican mediante middleware o decoradores para que no dependan de que cada router recuerde incluirlos. No obstante, estos mecanismos son complementarios: un token CSRF válido no concede permisos sobre otro usuario y una cabecera de seguridad no sustituye a la comprobación de rol.

2.2.4 Persistencia de estado y transacciones simples

La aplicación utiliza MySQL como fuente de verdad para el estado de cuentas, consultas, tokens y trabajos. Las operaciones de escritura siguen una secuencia corta: obtener conexión, preparar una consulta parametrizada, ejecutar, confirmar y cerrar. Esta política reduce el tiempo durante el que se mantiene un recurso del pool y facilita localizar la operación que produjo un error.

Los estados de los trabajos se persisten en lugar de mantenerse solo en una variable del proceso. Esto permite que el router de cola y el worker observen la misma información y que un reinicio pueda recuperar trabajos que quedaron en ejecución. La actualización condicional de `_claim_job()` actúa como mecanismo sencillo de exclusión para evitar que dos consumidores reclamen la misma tarea.

La persistencia de MySQL y la de ficheros deben coordinarse de manera explícita. Una fila de consulta puede existir antes de que todos sus artefactos hayan terminado de generarse, pero solo debe presentarse como resultado completado cuando el worker haya concluido el flujo. Los fallos de escritura de un informe o de un mapa XAI deben dejar constancia en el estado del trabajo y no ocultarse con una fila incompleta.

2.2.5 Ejecución asíncrona y aislamiento de trabajos

La ejecución asíncrona se implementa mediante una cola persistida y un worker interno. Los routers registran el trabajo y devuelven la respuesta; `queue_worker.py` lo reclama y envía el cálculo pesado a `run_in_executor`. Este patrón libera el ciclo de petición y permite que la interfaz consulte el progreso sin esperar a que termine el entrenamiento o la inferencia.

El worker limita sus operaciones a los tipos explícitos `diagnosis`, `training` y `external_validation`. Cada tipo tiene una función de procesamiento que interpreta su `payload`, invoca al servicio adecuado y devuelve un resultado serializable. El bucle exterior captura los errores de un trabajo, los registra como `failed` y continúa con el siguiente. La recuperación de trabajos `running` al arrancar evita que una interrupción deje tareas invisibles.

La ejecución asíncrona no significa que todas las tareas tengan el mismo nivel de prioridad o el mismo mecanismo. Los entrenamientos y validaciones utilizan `job_queue`; el recálculo estadístico del laboratorio usa `BackgroundTasks`; y las peticiones de consulta permanecen síncronas porque solo recuperan información. Documentar esta diferencia evita que una futura modificación asuma garantías de persistencia para una tarea que actualmente se ejecuta como trabajo en segundo plano del proceso web.

2.2.6 Errores, estados y comunicación con el usuario

Los errores se traducen en respuestas que la interfaz puede interpretar. Las rutas devuelven 401 cuando falta autenticación, 403 cuando el usuario no tiene autorización, 404 cuando el recurso no existe y 400 cuando los datos recibidos no cumplen una condición de negocio. Las excepciones inesperadas se convierten en respuestas 500 y, cuando pertenecen a un trabajo asíncrono, se conservan en el estado `failed` de la cola.

La capa de presentación recibe mensajes localizados mediante `lang.py`, por lo que los routers no deberían mezclar en cada respuesta textos técnicos sin una razón concreta. Al mismo tiempo, el cliente no debe recibir claves secretas, trazas completas ni credenciales. El detalle necesario para diagnosticar se conserva en la salida del servidor o en el estado de error del trabajo, mientras que la interfaz obtiene un mensaje comprensible y seguro.

Los estados `queued`, `running`, `completed`, `failed` y `cancelled` forman un contrato de comunicación entre el worker, los routers y el navegador. El usuario puede saber si la petición fue aceptada, si se está ejecutando, si terminó correctamente o si debe revisar un error. La uniformidad de este contrato reduce la necesidad de que cada vista invente una representación diferente para los trabajos.

2.2.7 Modularidad, pruebas y evolución controlada

La modularidad se aplica separando routers, servicios, worker, persistencia y recursos de presentación. Los routers coordinan HTTP; los servicios encapsulan lógica especializada; database.py proporciona conexiones; queue_worker.py ejecuta trabajos; y las plantillas y scripts representan la interacción. La separación no es absoluta —los routers actuales acceden directamente a `get_db_connection()` en varias operaciones—, pero las responsabilidades principales siguen identificables y permiten probar cada grupo de forma aislada.

Las pruebas sirven como contrato evolutivo de esos límites. Los cambios en autenticación deben conservar las pruebas de tokens y cookies; los cambios en diagnóstico deben conservar la validación de imágenes y la integración con la cola; los cambios en MLOps deben conservar la propiedad de sesiones y los estados; y los cambios en middleware deben conservar las pruebas CSRF, cabeceras y rate limiting. Esta relación entre mecanismo de diseño y prueba reduce el riesgo de que una refactorización altere una garantía transversal sin que el cambio sea detectado.

2.2.8 Ciclo de vida y sustitución controlada de componentes

Los componentes de soporte deben poder iniciarse, utilizarse y detenerse sin dejar recursos abiertos ni estados imposibles de interpretar. Durante el arranque, FastAPI carga la configuración, comprueba la base de datos e inicia el worker. Durante la operación, los routers solicitan conexiones y ficheros únicamente cuando los necesitan. Durante una parada o reinicio, los trabajos que estaban ejecutándose deben quedar identificados para que el siguiente arranque pueda devolverlos a la cola. Esta secuencia vincula el ciclo de vida del proceso web con la persistencia de `job_queue`.

La separación de soporte también facilita sustituir un componente si el entorno lo exige. MySQL podría cambiarse por otro motor compatible si se mantiene el contrato de `get_db_connection()` y se adaptan las consultas y el esquema; Groq podría reemplazarse si el servicio conversacional conserva la entrada de mensajes y la salida estructurada; y el directorio local de artefactos podría trasladarse a un almacenamiento especializado si se mantiene la relación entre rutas, usuarios y resultados. Estas posibilidades no forman parte de la implementación actual, pero muestran que la dependencia se concentra en puntos identificables y no se dispersa por toda la interfaz.

La sustitución no debe realizarse sin revisar los mecanismos transversales. Cambiar MySQL afecta al pool, a las transacciones, a las claves ajenas y a la recuperación de la cola; cambiar el almacenamiento afecta a las rutas de imágenes, informes y resultados; y cambiar el proveedor externo afecta a las credenciales, los tiempos de espera y el tratamiento de errores. Por ello, cualquier cambio de infraestructura debe acompañarse de pruebas de integración y de una comprobación de que los estados y permisos siguen siendo equivalentes.

El entorno de construcción conserva este principio mediante dependencias fijadas y pruebas reproducibles. Una nueva versión de TensorFlow, Keras, Transformers o mysql-connector-python no debe incorporarse únicamente porque resuelva un problema local: debe instalarse en un entorno aislado, ejecutarse la batería de pruebas y comprobarse que los modelos, los ficheros y las consultas mantienen el comportamiento esperado. La configuración de producción y la de pruebas deben permanecer separadas para que una migración o una prueba de recuperación no altere los datos de uso.

La arquitectura de soporte concluye así la estructura técnica de vitalXAI. SSOP-001 sostiene las relaciones y estados persistentes; SSOP-002 conserva los artefactos y recursos; SSOP-003 proporciona el entorno y las integraciones; SSOP-004 verifica el comportamiento. Sobre ellos se aplican la configuración externa, la validación, la seguridad, la persistencia de estados, la asincronía, la comunicación de errores y la modularidad. Ninguno de estos elementos sustituye a los subsistemas funcionales, pero todos son necesarios para que puedan ejecutarse de manera coherente con los requisitos y restricciones del proyecto.

La responsabilidad queda distribuida de forma verificable: el desarrollador mantiene el código y las pruebas; el operador prepara el entorno, las credenciales y MySQL; y el sistema conserva los estados y errores necesarios para diagnosticar una ejecución. Esta distribución evita que una incidencia de infraestructura se oculte como un fallo funcional y facilita localizar el punto de actuación adecuado.

Esta arquitectura deja definidos los puntos que deben comprobarse cuando se implanta o modifica la plataforma: disponibilidad de MySQL y de los directorios, validez de las variables de entorno, compatibilidad de las dependencias, funcionamiento del worker, conservación de los estados de la cola y ejecución de las pruebas automatizadas. La operación de vitalXAI depende de que estos elementos funcionen conjuntamente; por ello, el soporte no es un añadido posterior, sino la base que permite que la lógica clínica y experimental se ejecute de forma repetible y controlada.

El resultado es una infraestructura proporcionada al alcance académico y técnico del proyecto.

3 Persistencia relacional y esquema de datos de vitalXAI

Este capítulo describe cómo vitalXAI conserva la información que produce y sobre la que opera a lo largo de su funcionamiento. Hasta aquí, el análisis del capítulo 14 identificó las entidades del dominio y el diseño de los capítulos 18 y 19 fijó la arquitectura y los componentes de soporte; queda por concretar la representación física de los datos. Esta concreción incluye las tablas que existen, las columnas que las componen, los tipos de datos que admiten, las claves que las relacionan y las restricciones que las gobiernan. El objetivo es que el esquema resultante quede definido con la precisión necesaria para poder reproducirlo, verificarlo y hacerlo evolucionar sin ambigüedades durante el resto del proyecto.

El motor sobre el que se sustenta esta persistencia es MySQL, un sistema de gestión de bases de datos relacional de código abierto, accedido desde la aplicación mediante el conector oficial `mysql-connector-python` y un pool de conexiones `MySQLConnectionPool`. La definición del esquema se realiza con sentencias `CREATE TABLE IF NOT EXISTS` que la función `init_db()` ejecuta durante el arranque de la aplicación; no se emplea un ORM ni un sistema de migraciones versionadas. Esta forma de trabajar simplifica la puesta en marcha en el entorno local y de demostración, pero tiene una consecuencia que debe tenerse presente: cualquier cambio posterior sobre las tablas, como añadir una columna o un índice, no se aplicará automáticamente y deberá planificarse como una operación controlada sobre la base de datos.

Conviene destacar, además, que el almacenamiento de vitalXAI no se limita a las tablas de MySQL. El sistema conserva en el sistema de ficheros los artefactos de mayor tamaño o de estructura variable —las imágenes radiológicas, los mapas de explicabilidad, los informes en PDF y los resultados del laboratorio de entrenamiento— y las tablas guardan únicamente las rutas o referencias que permiten localizarlos. Esta separación entre datos relacionales y artefactos por fichero es una decisión de diseño deliberada, coherente con el subsistema de soporte SSOP-002 del capítulo 19, y condiciona de forma directa el procedimiento de copia y recuperación de la información.

El presente capítulo se limita al esquema de persistencia: el apartado 20.1 presenta los criterios de diseño que lo rigen, los diagramas que lo representan y la descripción detallada de cada una de las cinco tablas que lo componen. El acceso a los datos —la configuración del pool, el ciclo de vida de las operaciones y la inicialización del esquema— se documenta en el capítulo 19 dentro del subsistema de soporte SSOP-001 y, por tanto, no se repite aquí.

3.1 Diseño del esquema de persistencia

Las entidades que el análisis identificó en el capítulo 14 no se traducen todas a una tabla relacional. La naturaleza híbrida del almacenamiento hace que unas entidades se materialicen directamente como tablas, que otras queden representadas mediante columnas de una tabla y que otras, en particular las del laboratorio de experimentación, se conserven íntegramente en el sistema de ficheros. El resultado es un esquema compuesto por cinco tablas, todas ellas organizadas en torno a la cuenta de usuario: `users`, `consultations`, `job_queue`, `refresh_tokens` y `training_jobs`.

Antes de describir cada tabla en detalle, conviene fijar los criterios que se aplican de forma uniforme a todo el esquema. Estos criterios responden a las necesidades del proyecto y explican por qué el esquema tiene la forma que tiene.

- **Identificadores enteros autoincrementales.** Cada tabla define su clave primaria como una columna `id` de tipo `INT` con la propiedad `AUTO_INCREMENT`, de modo que MySQL asigna un valor secuencial al insertar una nueva fila. Se evita así la generación externa de identificadores, que no aporta ventajas en un sistema de alcance local y que añadiría complejidad innecesaria a la inserción.
- **La cuenta de usuario como eje de propiedad.** Las cuatro tablas dependientes de `users` incorporan una clave ajena `user_id` que apunta a `users.id`. Esta clave traslada al esquema el aislamiento de datos entre cuentas declarado en el requisito RF-005: ninguna consulta, trabajo, token o entrenamiento puede existir sin una cuenta propietaria. La base de datos garantiza la existencia de la cuenta referenciada, y la aplicación complementa esa garantía con las comprobaciones de propiedad y de rol que aplica en sus rutas.
- **Tipos de datos ajustados al contenido.** Las cadenas acotadas se declaran con `VARCHAR` y una longitud máxima, los valores numéricos con `INT` o `FLOAT` según su naturaleza, las marcas temporales con `DATETIME` y los contenidos variables con `JSON` o `TEXT`. Esta elección mantiene las tablas compactas y permite a MySQL indexar y comparar los datos con eficiencia. La precisión de los valores de confianza y de progreso se conserva con `FLOAT`, suficiente para las probabilidades y porcentajes que maneja la aplicación.
- **Borrado en cascada frente a borrado por flujo de negocio.** La relación de `consultations` con `users` no especifica una acción de borrado, mientras que `job_queue`, `refresh_tokens` y `training_jobs` declaran `ON DELETE CASCADE`. La diferencia responde a la forma en que cada información desaparece del sistema: la eliminación de una consulta la gestiona el propio flujo de negocio del historial, que borra el registro y sus artefactos de forma definitiva conforme al derecho de supresión del RGPD (RF-016), mientras que los trabajos, los tokens y los entrenamientos dependientes deben desaparecer automáticamente cuando se elimina su cuenta, para no dejar datos huérfanos.
- **Persistencia relacional y artefactos por fichero.** El esquema no pretende guardar en columnas los datos binarios ni la información variable del laboratorio. Las imágenes, los mapas, los informes y los resultados de entrenamiento se escriben en directorios del sistema de ficheros, y las tablas conservan las rutas que los referencian. Esta decisión mantiene las tablas ligeras y evita que el crecimiento de los artefactos degrade el rendimiento de las consultas.

- **Creación y evolución del esquema sin migraciones versionadas.** Las tablas se crean mediante `CREATE TABLE IF NOT EXISTS` durante el arranque. El código Python actúa, por tanto, como fuente efectiva de definición del esquema. Esta estrategia permite levantar una instalación nueva, pero no modifica tablas existentes; la evolución del esquema deberá gestionarse como un cambio de base de datos planificado y probado, sin atribuir a las sentencias de creación una capacidad de migración que no tienen.

En cuanto a los índices, el esquema no declara índices adicionales explícitos: MySQL/InnoDB crea automáticamente los índices necesarios para las claves primarias, las restricciones de unicidad y las claves ajenas. Estos índices son suficientes para el volumen de datos del proyecto. Posibles optimizaciones futuras, como índices sobre las columnas de fecha del historial o sobre el estado de los trabajos, quedan fuera de la implementación actual.

La correspondencia entre el modelo conceptual y el esquema físico merece una precisión adicional. De las ocho entidades del capítulo 14, Usuario se materializa en `users`, ConsultaDiagnostico en `consultations` y TrabajoCola en `job_queue`; los mapas de explicabilidad (MapaXAI) quedan representados por la columna `xai_image_path` de `consultations`, que apunta al fichero generado. Las entidades del laboratorio —SesionExperimentacion, ModeloEntrenado, ResultadoModelo y ValidacionExterna— no tienen tabla propia: se conservan en el directorio `training_results`, organizadas por un identificador de sesión con formato `RUN_AAAAMMDD_HHMMSS`. Esta decisión se justifica por la estructura variable de los artefactos experimentales, cuya normalización en tablas exigiría un esquema mucho más complejo, desproporcionado para el alcance del proyecto. Ambos capítulos deben leerse de forma complementaria: el 14 describe las entidades conceptuales y este describe su materialización física.

El diagrama entidad-relación de la ilustración representa el esquema en notación Chen. En esta notación, las entidades se dibujan como rectángulos, las relaciones como rombos y las cardinalidades se indican sobre las aristas (Chen, 1976; Elmasri & Navathe, 2016). La entidad `USERS` actúa como raíz y participa en cuatro relaciones de tipo uno a muchos con el resto de las entidades. El diagrama se completa a continuación con un boceto en Mermaid que puede adaptarse y refinarse.

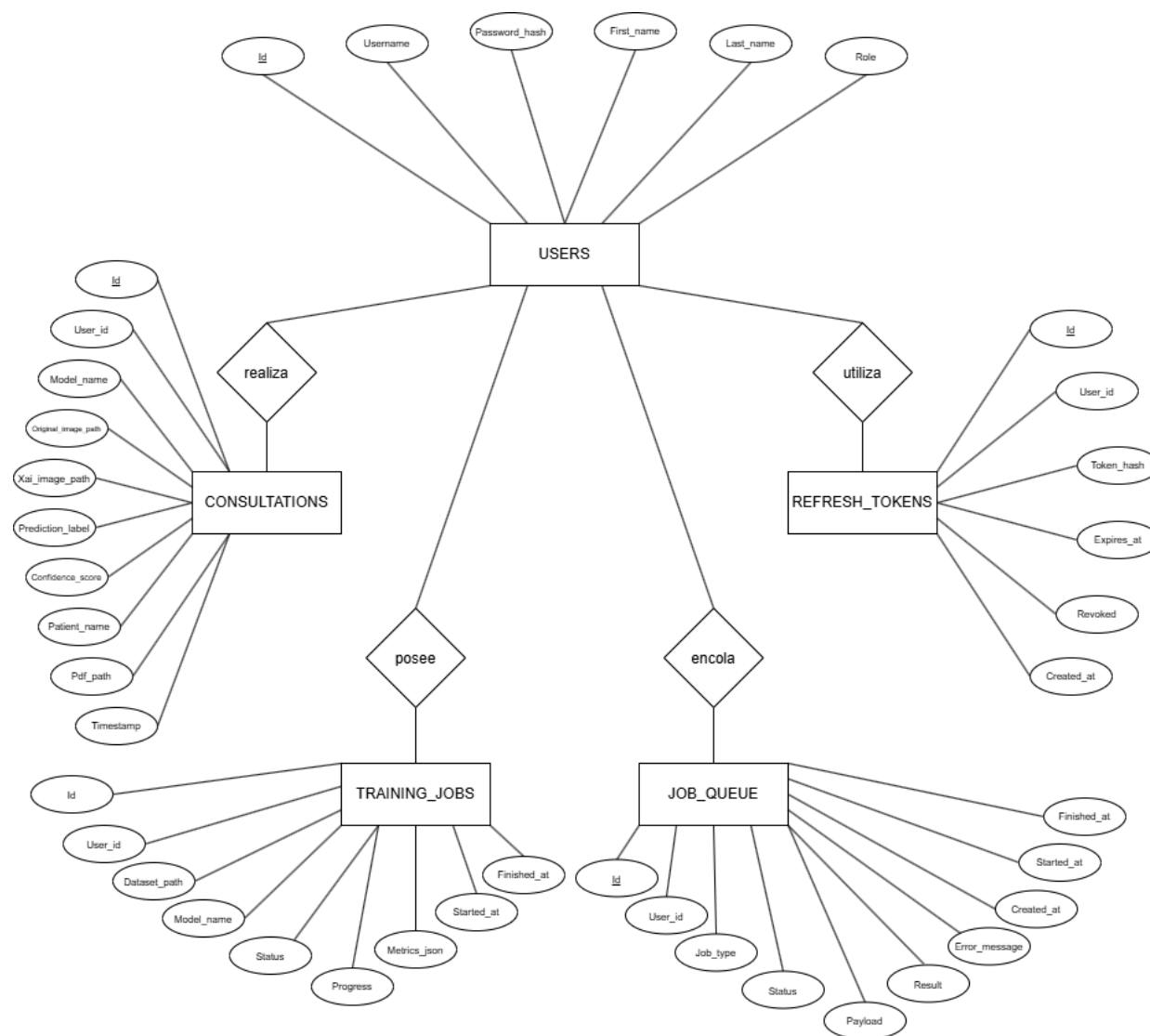


Ilustración x - Diagrama entidad-relación del esquema de persistencia en notación Che

El diagrama relacional de la siguiente ilustración representa el esquema desde la perspectiva de las tablas y sus claves. A diferencia del anterior, que se centra en las entidades y sus cardinalidades, este diagrama destaca las columnas que actúan como clave primaria y como clave ajena, y la forma en que las tablas se conectan entre sí y con el sistema de ficheros. Se incluye también como boceto en Mermaid, listo para completar.

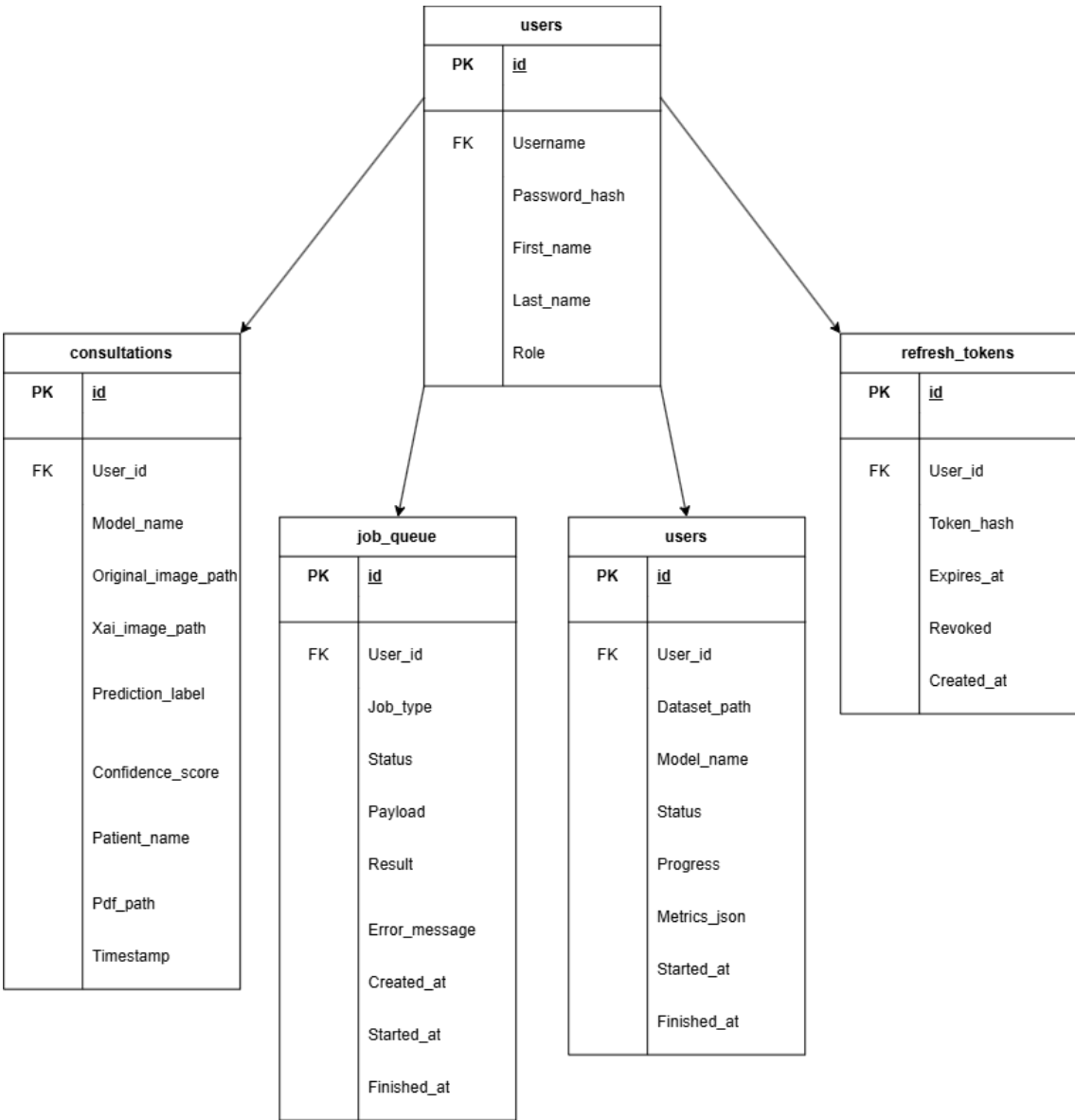


Ilustración x - Diagrama relacional de tablas y claves de vitalXAI

A continuación se describen, mediante una ficha por tabla, la estructura y las restricciones de cada una de las cinco tablas del esquema. Cada ficha indica el nombre, la descripción, los atributos con su tipo y su obligatoriedad, la clave primaria, las claves ajenas, las claves únicas y las restricciones que le aplican.

Nombre	users		
Descripción	Almacena los datos de identidad de las cuentas registradas. Es la entidad central del modelo y el origen de las relaciones de propiedad del resto de tablas.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno de la cuenta. Clave primaria autoincremental.
Username	VARCHAR(255)	S	Identificador de acceso, validado como correo electrónico.
Password_hash	VARCHAR(255)	S	Hash de la contraseña generado con bcrypt. Nunca se almacena la contraseña en claro.
First_name	VARCHAR(255)	S	Nombre mostrado del usuario.
Last_name	VARCHAR(255)	S	Apellidos mostrados del usuario.
Role	VARCHAR(255)	S	Perfil de autorización de la cuenta (ordinario o admin).
Clave primaria			
Nombre	Columnas		Secuencia
users_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino		Columnas
N/A	N/A		N/A
Claves únicas			
Nombre	Columnas		
users_username_key	Username		
Restricciones			
Nombre	Columnas		Restricción
Username	Username		Username
Username	Role		La aplicación restringe el rol a los valores ordinario y admin; el esquema no define un CHECK.

Tabla x - TB-01: users

La unicidad de username se aplica a nivel de base de datos y constituye la última barrera frente a cuentas duplicadas, incluso cuando dos peticiones de registro llegan de forma concurrente; antes de insertar, el router de autenticación valida el formato y comprueba la disponibilidad del identificador. La columna password_hash conserva el resultado de aplicar la función de hash bcrypt, de modo que la contraseña original nunca queda almacenada, en coherencia con el requisito RNF-001. La columna role es interpretada por la aplicación al proteger las rutas administrativas mediante la comprobación _require_admin() descrita en el capítulo 18; el esquema no introduce una tabla separada de roles, decisión suficiente para el alcance actual.

Nombre	consultations		
Descripción	Almacena el historial de consultas de diagnóstico de los usuarios. Cada fila representa una consulta completada y las rutas de sus artefactos asociados.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno de la consulta. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria de la consulta. Clave ajena a users.
Model_name	VARCHAR(100)	N	Arquitectura utilizada en el diagnóstico.
Original_image_path	VARCHAR(500)	N	Ruta de la imagen radiológica original.
Xai_image_path	VARCHAR(500)	N	Ruta del mapa de explicabilidad generado.
Prediction_label	VARCHAR(50)	N	Clasificación devuelta (PNEUMONIA o NORMAL).
Confidence_score	FLOAT	N	Confianza asociada a la predicción.
Patient_name	VARCHAR(255)	N	Nombre visible de organización de la consulta.
Pdf_path	VARCHAR(500)	N	Ruta del informe PDF generado.
Timestamp	DATETIME	S	Fecha y hora del registro, con valor por defecto CURRENT_TIMESTAMP.
Clave primaria			
Nombre	Columnas		Secuencia
consultations_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino		Columnas
consultations_user_id_fkey	users		User_id
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas		Restricción
consultations_user_id_not_null	User_id		Valor no nulo: toda consulta debe asociarse a una cuenta válida.
consultations_prediction_label_check	Prediction_label		La aplicación restringe la etiqueta a PNEUMONIA o NORMAL; el esquema no define un CHECK.

Tabla x - TB-02: consultations

Esta tabla se alimenta desde el worker de la cola cuando un diagnóstico termina: una vez obtenida la predicción y generados los artefactos de explicabilidad y el informe, el worker registra la consulta con sus metadatos y las rutas de los ficheros. Las columnas de rutas apuntan a los ficheros conservados en el sistema de ficheros, no a su contenido, de modo que reconstruir una consulta completa exige disponer tanto de la fila como de los ficheros referenciados; una copia de seguridad de MySQL sin los directorios correspondientes restauraría referencias incompletas. patient_name actúa como una etiqueta visible que el usuario puede renombrar para organizar su historial, y debe tratarse como un dato de presentación, no como una autorización para guardar identificadores personales del paciente. El estado operativo del diagnóstico no reside en esta tabla: se conserva en job_queue, y la presencia de una fila aquí indica que el diagnóstico alcanzó la fase de persistencia del resultado.

Nombre	job_queue		
Descripción	Almacena los trabajos asíncronos de la plataforma —diagnósticos, entrenamientos y validaciones externas— con su tipo, su estado, sus parámetros y su resultado.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del trabajo. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria del trabajo. Clave ajena a users.
Job_type	VARCHAR(20)	S	Tipo de trabajo: diagnosis, training o external_validation.
Status	VARCHAR(20)	S	Estado del ciclo de vida, con valor por defecto queued.
Payload	JSON	N	Parámetros necesarios para ejecutar la tarea.
Result	JSON	N	Resultado serializado del trabajo.
Error_message	TEXT	N	Motivo de un fallo, si lo hubo.
Created_at	DATETIME	S	Fecha de encolado, con valor por defecto CURRENT_TIMESTAMP.
Started_at	DATETIME	N	Fecha de inicio de la ejecución.
Finished_at	DATETIME	N	Fecha de finalización o cancelación.
Clave primaria			
Nombre	Columnas		Secuencia
job_queue_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino		Columnas
job_queue_user_id_fkey	users		User_id
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
job_queue_user_id_not_null	User_id	Valor no nulo: todo trabajo debe asociarse a una cuenta válida.	
job_queue_job_type_check	Job_type	La aplicación restringe el tipo a diagnosis, training o external_validation; el esquema no define un CHECK.	
job_queue_status_check	Status	La aplicación restringe el estado a queued, running, completed, failed o cancelled; el esquema no define un CHECK.	

Tabla x - TB-03: job_queue

Esta tabla es el soporte persistente de la ejecución asíncrona. Permite que el worker reclame y ejecute los trabajos, que el usuario consulte su progreso o los cancele, y que un reinicio del servidor recupere el estado de las tareas que quedaron en marcha. Las columnas de fecha reconstruyen la evolución temporal de cada trabajo. Las columnas payload y result, de tipo JSON, permiten guardar parámetros heterogéneos sin una columna por valor posible; a cambio, su contenido debe validarse en la aplicación, ya que MySQL no conoce su estructura interna. La tabla se asocia al usuario mediante user_id, pero no mantiene claves ajenas hacia consultations ni hacia las sesiones de entrenamiento: la relación entre un trabajo y su resultado es lógica y se expresa mediante el job_type y el contenido del payload.

Nombre	refresh_tokens		
Descripción	Almacena las credenciales de refresco de las sesiones, con su hash, su expiración y su estado de revocación.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del registro. Clave primaria autoincremental.
User_id	INT	S	Cuenta asociada al token. Clave ajena a users.
Token_hash	VARCHAR(255)	S	Hash del valor aleatorio del token.
Expires_at	DATETIME	S	Fecha y hora de caducidad del token.
Revoked	BOOLEAN	S	Indica si la credencial ha sido invalidada, con valor por defecto false.
Created_at	DATETIME	S	Fecha de creación, con valor por defecto CURRENT_TIMESTAMP.
Clave primaria			
Nombre	Columnas		Secuencia
refresh_tokens_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino	Columnas	
refresh_tokens_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
refresh_tokens_user_id_not_null	User_id	Valor no nulo: todo token debe asociarse a una cuenta válida.	
refresh_tokens_token_hash_not_null	Token_hash	Valor no nulo: solo se persiste el hash del token, nunca su valor original.	
refresh_tokens_expires_at_not_n ull	Expires_at	Valor no nulo: todo token debe declarar su fecha de caducidad.	

Tabla x - TB-04: refresh_tokens

El valor original del token de refresco nunca llega a la base de datos; solo se conserva su hash, de modo que una exposición de los datos no comprometa directamente las sesiones. Durante una renovación, el servicio de autenticación calcula el hash del valor recibido y busca una fila que coincida, no esté revocada y no haya caducado; la rotación invalida la fila anterior y crea una nueva. Si se detecta el uso de un token revocado fuera del periodo de tolerancia, el servicio revoca todos los tokens del usuario. La relación con users declara ON DELETE CASCADE, de modo que al eliminar la cuenta desaparecen sus credenciales de refresco, sin sustituir la revocación explícita que realiza el flujo de cierre de sesión.

Nombre	training_jobs		
Descripción	Representación estructurada prevista de los trabajos de entrenamiento de modelos. Definida en el esquema como parte de la inicialización de la base de datos.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del entrenamiento. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria del entrenamiento. Clave ajena a users.
Dataset_path	VARCHAR(500)	N	Ruta del dataset utilizado.
Model_name	VARCHAR(100)	N	Arquitectura entrenada.
Status	VARCHAR(50)	S	Estado del entrenamiento, con valor por defecto In Progress.
Progress	FLOAT	S	Porcentaje de progreso, con valor por defecto 0.0.
Metrics_json	TEXT	N	Métricas serializadas del entrenamiento.
Started_at	DATETIME	S	Fecha de inicio, con valor por defecto CURRENT_TIMESTAMP.
Finished_at	DATETIME	N	Fecha de finalización.
Clave primaria			
Nombre	Columnas	Secuencia	
training_jobs_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
training_jobs_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
training_jobs_user_id_not_null	User_id	Valor no nulo: todo entrenamiento debe asociarse a una cuenta válida.	

Tabla x - TB-05: training_jobs

Esta ficha debe leerse con una salvedad importante. La tabla training_jobs está definida en el esquema y se crea durante la inicialización, pero no se utiliza operativamente en la implementación actual: el flujo del laboratorio no inserta filas ni las consulta. La persistencia real del entrenamiento se realiza mediante el sistema de ficheros, en el directorio training_results, y el motor que lanza los entrenamientos (ml_ops_engine.py) no accede a la base de datos. Existen restos de lógica de actualización sobre la tabla en trainer_engine.py, un módulo que no participa en el flujo productivo, y un volcado histórico contiene filas antiguas, lo que sugiere un uso en versiones previas del proyecto. En consecuencia, esta tabla se documenta como una previsión del esquema cuya materialización se ha trasladado a los ficheros, sin atribuirle un uso que el código actual no realiza.

Los valores de los campos de dominio se validan en la capa de servicios y no mediante restricciones CHECK del esquema: el estado de la cola admite queued, running, completed, failed y cancelled; el tipo de trabajo admite diagnosis, training y external_validation; y la etiqueta de predicción admite PNEUMONIA y NORMAL. Esta decisión centraliza la lógica de validación en la aplicación y evita acoplar el esquema a la evolución de esos valores. La consistencia del esquema se apoya en tres niveles: MySQL aplica claves, obligatoriedad, unicidad y referencias; los routers y servicios aplican propiedad, roles y comprobaciones de entrada; y el worker coordina la relación temporal entre un trabajo encolado, sus artefactos y su resultado.

El esquema actual prioriza la trazabilidad directa con el código y la facilidad de puesta en marcha. Utiliza identificadores enteros, columnas JSON para parámetros variables y rutas de ficheros para los artefactos. Esta solución es suficiente para el alcance del TFG, pero tiene límites conocidos: no incorpora índices específicos para todas las consultas, no versiona automáticamente el esquema, no normaliza las métricas MLOps y no ofrece una entidad física que vincule cada fila de job_queue con una consulta o sesión concreta. Si el volumen de datos creciera, la evolución debería comenzar por los índices de las consultas frecuentes, una estrategia de migraciones versionadas y una política de retención, conservando la compatibilidad con el resto del sistema. Con ello, el esquema queda alineado con el estado real del código: MySQL sostiene las entidades y los estados relacionales, el sistema de ficheros conserva los artefactos, y la aplicación mantiene las reglas que no pueden expresarse solo con claves y tipos SQL.

3.2 Acceso a la capa de persistencia

El esquema descrito en el apartado 20.1 define dónde y cómo se almacenan los datos, pero no basta con ello: es necesario fijar también la forma en que los componentes del sistema leen y escriben esa información, porque de esa forma dependen tanto la corrección del comportamiento como la capacidad de evolucionar el sistema sin romper lo existente. En vitalXAI, el acceso a la capa de persistencia no pasa por una capa intermedia de mapeo objeto-relacional ni por un patrón de repositorios que abstraiga las consultas en clases independientes. La aplicación trabaja directamente con SQL parametrizado sobre conexiones obtenidas del pool de MySQLConnectionPool, que el módulo database.py construye de forma diferida y entrega a través de la función `get_db_connection()`. Esta elección responde a la simplicidad y a la trazabilidad: el código que consulta la base de datos es explícito, y la relación entre una consulta y la tabla a la que accede es inmediata.

La configuración del pool, su ciclo de vida y el proceso de inicialización del esquema se describen con detalle en el capítulo 19 dentro del subsistema SSOP-001. Este apartado se centra, por tanto, en dos aspectos complementarios: cómo se distribuye el acceso a las tablas entre los componentes del sistema, y qué consecuencias tiene esa distribución para la concurrencia, el rendimiento y la integridad de los datos. Ambos aspectos son necesarios para comprender el esquema no solo como una estructura estática, sino como un recurso que la aplicación utiliza de forma continuada.

Esta forma de trabajo tiene dos consecuencias directas. La primera es que el contrato entre la lógica de aplicación y la base de datos es pequeño y explícito: cualquier componente que necesite persistir obtiene una conexión, ejecuta sus consultas y la cierra, sin depender de una capa intermedia que abstraiga el SQL ni de clases de repositorio que oculten dónde se realiza cada operación. La segunda es que la correspondencia entre el esquema y el código es directa y trazable: cada tabla del modelo físico tiene asociados unos componentes concretos que la leen y la escriben, y esa asociación permite localizar con rapidez dónde se introduce o se modifica cada dato.

Tabla de persistencia	Operaciones principales	Componentes de acceso
<i>users</i>	Insertar cuenta, consultar credenciales y perfil, consultar rol.	routers/auth.py, routers/admin.py, routers/history.py, routers/trainer.py.
<i>consultations</i>	Insertar resultado de diagnóstico, consultar historial, renombrar y eliminar.	services/queue_worker.py, routers/history.py, routers/admin.py.
<i>job_queue</i>	Encolar trabajo, reclamarlo, actualizar su estado, cancelarlo.	routers/inference.py, routers/trainer.py, services/queue_worker.py, routers/queue.py.
<i>refresh_tokens</i>	Crear token, verificarlo, revocarlo.	services/auth_service.py, routers/auth.py.
<i>training_jobs</i>	Tabla prevista; sin acceso operativo en la implementación actual.	Ninguno (persistencia real por ficheros en training_results).

Tabla x - Distribución del acceso a las tablas del esquema

La tabla anterior permite verificar que cada tabla tiene un responsable claro en la capa de aplicación, y que la lógica de negocio que rodea a cada entidad reside en los componentes que la utilizan. Por ejemplo, la escritura de consultations es competencia del worker de la cola, que la realiza al finalizar un diagnóstico, mientras que la lectura y la gestión del historial corresponden al router de historial y, en el ámbito administrativo, al router de administración. La cola de trabajos concentra la mayor diversidad de operaciones, porque es el punto de encuentro entre las peticiones que encolan tareas y el worker que las ejecuta. Esta separación evita que la responsabilidad sobre una misma tabla se disperse sin criterio y facilita localizar dónde se introduce o se modifica cada dato durante la depuración y el mantenimiento.

Todas las operaciones de acceso siguen un ciclo de vida homogéneo, que conviene describir con precisión porque constituye el patrón que se repite en todos los componentes. En una lectura, el componente obtiene una conexión del pool, crea un cursor —normal o de diccionario, según necesite los resultados como tuplas o como objetos con nombres de columna—, ejecuta una consulta parametrizada en la que los valores viajan separados de la instrucción, procesa el resultado para transformarlo en la estructura que necesita la capa HTTP o de servicio, y cierra la conexión. En una escritura, la secuencia añade la confirmación explícita mediante `commit()` antes de cerrar, de modo que los cambios quedan persistidos de forma atómica para la operación. El uso de parámetros evita construir sentencias concatenando datos recibidos del usuario, una garantía que se aplica de forma uniforme en todas las consultas del sistema, desde la autenticación hasta la cola de trabajos, y que constituye la defensa principal frente a la inyección SQL.

Esta uniformidad tiene una implicación práctica sobre la concurrencia. Cada petición solicita una conexión del pool durante el tiempo estrictamente necesario para su operación y la devuelve al finalizar, lo que limita el número de conexiones simultáneas al tamaño configurado en `DB_POOL_SIZE`. Las operaciones que combinan acceso a la base de datos con procesamiento costoso —como la generación de artefactos, la ejecución de un modelo o la espera de servicios externos— deben ejecutar su consulta sin retener la conexión durante todo el cálculo, porque de lo contrario agotarían rápidamente el pool y provocarían esperas en el resto de las peticiones. El diseño del worker, que reclama un trabajo, ejecuta el procesamiento pesado fuera del bloque de la consulta y solo vuelve a la base de datos para actualizar el estado o persistir el resultado, respeta este principio.

El aislamiento de datos entre usuarios, por su parte, no depende solo de las claves ajenas del esquema. Las consultas del historial y de la cola filtran por el `user_id` obtenido de la sesión, de modo que un usuario no puede leer filas de otro aunque técnicamente la tabla las contenga. Las operaciones administrativas, que sí acceden a datos de terceros, comprueban previamente el rol del actor y se limitan a las funciones de supervisión definidas en el análisis. Esta doble comprobación —la clave ajena que garantiza la existencia del propietario y el filtro de aplicación que garantiza la autorización— es la que materializa en la práctica el requisito RF-005, y conviene tenerla presente porque la primera sin la segunda no bastaría para proteger los datos.

En cuanto al rendimiento, el esquema se apoya en los índices que MySQL/InnoDB crea de forma implícita para las claves primarias, las restricciones de unicidad y las claves ajenas. Estos índices cubren las consultas más habituales, que en su mayoría filtran por el identificador propio de cada tabla y por `user_id`: el historial se consulta por el usuario y se ordena por fecha; los trabajos se filtran por propietario y, en el worker, por estado; y los tokens se localizan por su hash. La cola de trabajos accede por estado, por tipo y por fecha de encolado al reclamar y actualizar trabajos; el volumen actual no exige un índice adicional, pero la evolución del sistema

debería considerar índices sobre `job_queue.status` y sobre las columnas de fecha del historial si el número de trabajos y de consultas creciera de forma significativa. La carga de los modelos de inferencia, que no es una operación de base de datos, se resuelve en memoria dentro del motor de aprendizaje automático y no interviene en este apartado.

Conviene precisar, además, la relación entre el acceso a la base de datos y el acceso a los artefactos del sistema de ficheros. Cuando una consulta se completa, el worker escribe los ficheros en el sistema de ficheros y, a continuación, inserta en `consultations` las rutas que los referencian. La operación inversa, la lectura, recupera primero la fila y después entrega los ficheros desde las rutas almacenadas. Esta secuencia introduce una coordinación que debe ser correcta: si el fichero se genera pero la inserción falla, la fila no existe y el resultado no aparece en el historial; si la inserción se completa pero el fichero no llegó a escribirse, la fila quedaría referenciando un artefacto inexistente. El worker gestiona esta coordinación registrando el estado del trabajo y, en caso de error, dejando constancia en `job_queue` y evitando presentar como completada una consulta cuyos artefactos no están disponibles.

El acceso a la capa de persistencia queda, así, alineado con la naturaleza híbrida del almacenamiento descrita en el apartado 20.1. Las tablas conservan los datos estructurados que necesitan consultas y relaciones, y se accede a ellas mediante conexiones del pool y consultas parametrizadas; los artefactos grandes se escriben y se leen directamente en el sistema de ficheros, y la base de datos únicamente guarda las rutas que los referencian. Esta combinación mantiene la correspondencia entre el esquema físico y el código que lo utiliza, y deja identificadas las operaciones y los componentes responsables de cada tabla, de modo que la evolución del sistema pueda partir de un conocimiento preciso de dónde reside cada dato y de quién lo gestiona.

4 Diseño de los casos de uso

El diseño de los casos de uso constituye la etapa de la parte de diseño en la que las interacciones descritas en el capítulo 12 dejan de ser especificaciones de comportamiento y se convierten en decisiones técnicas concretas de implementación. El análisis explica qué debe hacer el sistema desde la perspectiva del usuario, mientras que el diseño de los casos de uso determina cómo lo hace internamente: qué componentes participan, qué servicios invocan, qué validaciones aplican, qué operaciones de persistencia realizan y cómo se comunican con la interfaz. Este capítulo recoge esas determinaciones para cada uno de los casos de uso de vitalXAI, de modo que la transición del análisis al código quede documentada de forma inequívoca y que un futuro mantenedor pueda localizar con rapidez dónde y cómo se materializa cada interacción (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004).

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño definida en el capítulo 18. Cada subsistema de diseño (SD-001 a SD-006) agrupa los casos de uso que comparten una responsabilidad funcional común, de acuerdo con la correspondencia establecida entre los subsistemas de análisis y los casos de uso en los capítulos 13 y 15. Para cada subsistema se presentan tres elementos: el diagrama de casos de uso, que reproduce la estructura de interacciones del análisis; los casos de uso reales, que recogen las decisiones técnicas de diseño de cada interacción; y los diagramas de interacción entre objetos, que muestran cómo colaboran los componentes del sistema para llevar a cabo los flujos más significativos de cada subsistema. Esta estructura es coherente con la organización que el propio diseño adoptó para los subsistemas y facilita la lectura transversal de la parte de diseño de la memoria.

El contenido de este capítulo se apoya en dos fuentes complementarias. Por un lado, las especificaciones de los casos de uso del capítulo 12, que definen el comportamiento observable y las condiciones que deben cumplirse. Por otro lado, la arquitectura de los subsistemas de diseño del capítulo 18 y los mecanismos de soporte del capítulo 19, que fijan los componentes, los servicios y las infraestructuras disponibles para materializar ese comportamiento. El diseño de cada caso de uso no introduce funcionalidades nuevas ni contradice el análisis: concreta, con las decisiones técnicas del proyecto, aquello que el análisis ya declaró, y lo hace de forma coherente con la implementación descrita en los capítulos precedentes.

4.1 Subsistema de diseño SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa el subsistema de análisis SS-001 y se ocupa de controlar el acceso a la plataforma y de gestionar la identidad de los usuarios registrados. Cubre el registro de nuevas cuentas, el inicio y el cierre de sesión, la renovación de credenciales y la identificación del usuario en las áreas privadas. Su responsabilidad no se limita a validar el formulario de acceso: establece la identidad que utilizarán el resto de los subsistemas para aplicar el aislamiento de datos entre cuentas y las comprobaciones de rol, por lo que constituye una condición transversal para el funcionamiento de SD-002, SD-003, SD-004, SD-005 y SD-006. Esta característica, ya señalada en el capítulo 18, explica que los demás subsistemas no reproduzcan la lógica de autenticación, sino que consuman la identidad que SD-001 establece.

El subsistema se apoya principalmente en `routers/auth.py`, que sirve las páginas de acceso y procesa los formularios recibidos desde la interfaz, y en `services/auth_service.py`, que concentra la lógica criptográfica y el ciclo de vida de las credenciales. Junto a ellos intervienen los mecanismos transversales registrados en `main.py`, como la protección CSRF, las cabeceras de seguridad y la limitación del endpoint de inicio de sesión. Los detalles de su arquitectura y de las decisiones de implementación se describen con profundidad en el capítulo 18; aquí se presentan, en los apartados siguientes, el diagrama de casos de uso del subsistema, los casos de uso reales con sus determinaciones técnicas y los diagramas de interacción entre objetos. Los casos de uso que agrupa son CU-001 a CU-004, tal y como se estableció en la especificación del capítulo 12 y en la correspondencia del capítulo 13.

4.1.1 Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-001 recoge las cuatro interacciones que el subsistema pone a disposición de los actores y es el mismo que se definió en el análisis para el módulo de gestión del acceso y de la cuenta, adaptado aquí al ámbito del subsistema de diseño. Dos de las interacciones —la creación de la cuenta (CU-001) y el inicio de sesión (CU-002)— las realiza el visitante, que accede a la plataforma sin autenticarse; el cierre de sesión (CU-003) lo realiza el usuario autenticado; y el cambio del idioma de la interfaz (CU-004) está disponible tanto para el visitante como para el usuario autenticado, porque el idioma puede adaptarse desde el área pública y desde el área privada. El diagrama se representa a continuación.

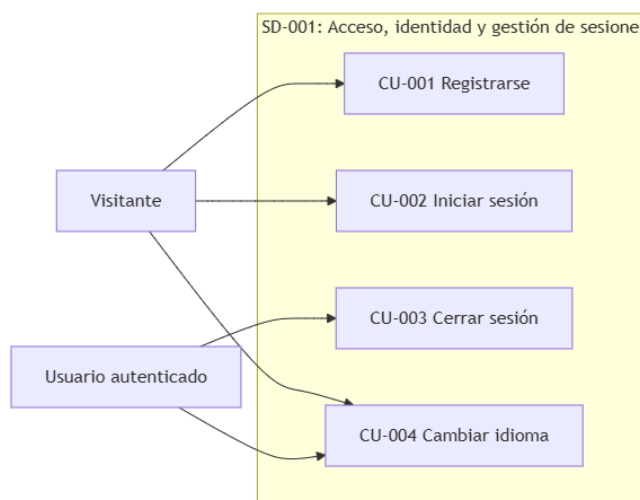


Ilustración x - Diagrama de casos de uso del subsistema SD-001

El diagrama distingue dos actores, en coherencia con la definición de actores del sistema del capítulo 12 (Jacobson, Booch, & Rumbaugh, 1999; Cockburn, 2001). El visitante representa a toda persona que accede sin haber iniciado sesión, y sus interacciones se limitan al registro, al inicio de sesión y al cambio de idioma. El usuario autenticado representa a todos los perfiles con cuenta registrada, incluido el administrador, porque las operaciones de gobierno de este último se recogen en el subsistema SD-005 y no en este diagrama. Esta decisión, ya adoptada en el análisis, evita que el administrador aparezca como un actor diferenciado en un subsistema donde no desempeña ninguna operación específica, y mantiene la coherencia entre el ámbito de SD-001 y el ámbito de SD-005.

Los cuatro casos de uso se representan como interacciones directas e independientes del actor con el sistema. No existe una relación de inclusión entre ellos, porque ninguno incorpora pasos obligatorios definidos por otro caso de uso, ni una relación de extensión, porque ninguno amplía de forma opcional el flujo de otro. La creación de la cuenta y el inicio de sesión comparten la necesidad de una identidad, pero se modelan por separado: el registro crea la identidad y el acceso la valida, sin que el segundo dependa del primero en la misma sesión. Esta estructura, idéntica a la del análisis, se mantiene en el diseño porque el flujo real de la plataforma no introduce relaciones nuevas entre estos casos de uso, y porque cada uno de ellos se implementa como un flujo propio e independiente en el subsistema.

Desde la perspectiva del diseño, el diagrama refleja una característica relevante del subsistema SD-001: actúa como puerta de entrada al resto de la plataforma, pero sus casos de uso no invocan servicios de otros subsistemas. Cuando el visitante se registra o inicia sesión, el subsistema opera exclusivamente sobre la tabla de usuarios y sobre las credenciales de sesión; cuando el usuario autenticado cierra sesión, revoca sus credenciales y elimina las cookies. El resto de los subsistemas consumen la identidad resultante mediante el mecanismo común de identificación descrito en el capítulo 18, pero no participan en estos flujos. Esta separación mantiene la cohesión del subsistema y facilita que las decisiones técnicas de SD-001 se describan de forma aislada en los apartados siguientes, sin dependencias circulares con el resto de la plataforma.

4.1.2 Casos de Uso Reales

Los casos de uso reales concretan cada interacción del diagrama en decisiones técnicas de implementación: qué componentes del subsistema intervienen, qué validaciones aplican, qué operaciones de persistencia realizan, qué respuesta devuelven al cliente y qué medidas de seguridad incorporan. Cada caso de uso real corresponde a un caso de uso del análisis y no añade funcionalidades nuevas, pero sí precisa determinaciones que el análisis dejaba abiertas, como el algoritmo de cifrado, el formato de las credenciales o los códigos de respuesta. Los cuatro casos de uso reales de SD-001 se describen a continuación.

CU-001 Registrarse

El registro se materializa en el endpoint POST /api/register del router routers/auth.py, que procesa los datos del formulario y traduce el flujo del análisis en una secuencia de validaciones y operaciones sobre la tabla users. El diseño mantiene la puerta de entrada autónoma del análisis: el alta no requiere la intervención de un administrador, y el proceso completo se resuelve en una única petición. Las decisiones técnicas del caso de uso real son las siguientes:

- **Validación de formato:** la función `_validate_register_inputs()` comprueba el nombre de usuario mediante una expresión regular de correo electrónico, exige una contraseña de al menos 8 caracteres y la presencia de nombre y apellidos. Ante cualquier incumplimiento, el sistema responde HTTP 400 con el código `validation_error` e indica el campo afectado, sin crear el registro.
- **Comprobación de unicidad:** antes de insertar, el sistema ejecuta una consulta de existencia sobre el nombre de usuario (`SELECT id FROM users WHERE username = %s`). Si el usuario ya está registrado, responde HTTP 400 con el código `user_exists` y abandona el proceso.
- **Cifrado de la contraseña:** la contraseña se convierte con `hash_password()` de `services/auth_service.py`, que aplica el algoritmo `bcrypt` con un salt generado en cada operación. La contraseña original no se transmite a ningún componente de persistencia tras completarse el hash, en coherencia con la postcondición del análisis de que nunca quede en texto plano.
- **Persistencia:** la cuenta se almacena mediante `INSERT INTO users (username, password_hash, first_name, last_name, role)`, que materializa la creación del registro con el rol indicado en el formulario.
- **Sesión tras el registro:** una vez creada la cuenta, el sistema genera el token de acceso y el token de refresco y los establece en las mismas cookies de sesión que el inicio de sesión, de modo que el visitante que acaba de registrarse accede directamente al panel. Esta determinación, que el análisis no fijaba —preveía redirigir al inicio de sesión—, se adopta en el diseño para eliminar un paso redundante al usuario que acaba de crear su cuenta.
- **Gestión de errores:** si la operación de persistencia falla, el sistema responde HTTP 500 con el código `server_error`, de modo que un fallo interno no se confunde con una condición de validación del formulario.

CU-002 Iniciar sesión

El inicio de sesión se materializa en el endpoint `POST /login`, protegido con la limitación `@limiter.limit("5/minute")` del mecanismo `slowapi`. El diseño conserva la protección descrita en el análisis: la verificación no distingue entre nombre de usuario y contraseña incorrectos, y el límite de peticiones bloquea temporalmente los intentos reiterados desde una misma dirección. Las decisiones técnicas del caso de uso real son las siguientes:

- **Verificación de credenciales:** el sistema recupera el registro mediante `SELECT id, password_hash FROM users WHERE username = %s` y comprueba la contraseña con `verify_password()`, que compara el valor introducido contra el hash almacenado con `bcrypt.checkpw`. Si la verificación falla, responde con una redirección a la página de inicio con un mensaje genérico que no revela qué parte de las credenciales era incorrecta.
- **Token de acceso:** `create_access_token()` de `services/auth_service.py` firma un JWT con el algoritmo `HS256` que contiene el identificador del usuario (`sub`) y su fecha de expiración (`exp`), utilizando la clave `JWT_SECRET_KEY` del entorno y la duración `JWT_ACCESS_EXPIRE_MINUTES`.
- **Token de refresco:** `create_refresh_token()` genera un valor aleatorio mediante `secrets.token_urlsafe`, y en la tabla `refresh_tokens` solo se conserva su hash `SHA-256` junto con el identificador del usuario, la fecha de expiración y el indicador de revocación. Esta decisión permite renovar la sesión sin guardar una credencial reutilizable en texto plano.
- **Cookies de sesión:** el token de acceso se establece en una cookie `HttpOnly` con `SameSite=Lax`, y el token de refresco en una cookie `HttpOnly` con `SameSite=Lax` y ruta restringida a `/api/token/refresh`. La restricción de ruta limita el envío del token de refresco al único endpoint que debe consumirlo.
- **Renovación de la sesión:** el endpoint `POST /api/token/refresh` ejecuta `rotate_refresh_token()`, que verifica el token actual, revoca el anterior y emite uno nuevo junto con un token de acceso renovado. El parámetro `REFRESH_ROTATION_GRACE_SECONDS` (60 segundos por defecto) tolera las peticiones concurrentes de renovación, y el uso de un token revocado fuera de ese periodo de gracia se interpreta como un posible robo: el servicio invalida todos los tokens del usuario.
- **Respuesta al cliente:** si las credenciales son válidas, el sistema redirige con `HTTP 303` al panel de diagnóstico; el propio registro de la sesión se completa en el navegador mediante las cookies.

CU-003 Cerrar sesión

El cierre de sesión se materializa en el endpoint GET /logout, y su diseño refleja la doble naturaleza de las credenciales de sesión: el token de refresco se revoca de forma definitiva en la base de datos, mientras que el token de acceso se descarta del navegador y caduca por sí mismo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Revocación del token de refresco:** si existe una cookie de refresco, el sistema ejecuta `revoke_refresh_token()`, que actualiza la fila correspondiente de `refresh_tokens` marcando el indicador de revocación. A partir de ese momento, el endpoint de renovación rechaza el token, por lo que la sesión no puede prolongarse.
- **Eliminación de cookies:** el sistema elimina del navegador tanto la cookie del token de acceso como la del token de refresco, con la ruta restringida correspondiente, mediante las operaciones de borrado de cookies de la respuesta.
- **Respuesta al cliente:** el sistema redirige con HTTP 303 a la página de inicio de sesión. El acceso a las áreas privadas queda bloqueado de inmediato porque el token de refresco ya no es válido y el token de acceso, aunque aún no haya expirado, no se conserva en el navegador.

CU-004 Cambiar el idioma de la interfaz

El cambio de idioma se materializa en el selector de idioma que está presente tanto en las páginas públicas (inicio de sesión y registro) como en las privadas (panel de diagnóstico y laboratorio de entrenamiento), y combina un mecanismo de cliente con un servicio de traducciones del lado del servidor. El diseño conserva las postcondiciones del análisis: la preferencia persiste durante la sesión y el cambio no interrumpe el estado de navegación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Selección en el navegador:** el script `static/js/i18n.js`, mediante la función `changeLanguage()`, lee el idioma elegido en el selector, lo guarda en `localStorage['appLang']` y actualiza dinámicamente los textos de la interfaz marcados con `data-i18n`, sin recargar la página. Esta decisión hace que el cambio sea inmediato y preserve el estado de la vista actual.
- **Servicio de traducciones del servidor:** `services/lang.py` centraliza el diccionario de mensajes que genera el backend. La función `get_lang_from_cookie()` lee la cookie `appLang`, valida que el idioma esté dentro del conjunto permitido —español, inglés, chino e hindú— y utiliza el español como valor por defecto si la preferencia es inválida o está ausente.
- **Aplicación en los resultados generados:** los servicios que producen contenido traducible leen la misma cookie para alinear sus salidas con el idioma de la interfaz: las etiquetas de predicción del motor de diagnóstico, los títulos de los mapas de explicabilidad, los textos del informe PDF y los mensajes del asistente conversacional. De este modo, el idioma seleccionado por el actor se aplica de forma coherente en toda la plataforma.
- **Coherencia entre cliente y servidor:** el mecanismo separa la responsabilidad de la presentación, que pertenece al navegador, de la de los mensajes generados por el backend, que se resuelven con el diccionario de `services/lang.py`; ambos consultan la misma preferencia de idioma, lo que evita que la interfaz muestre un idioma distinto del que utiliza el sistema en sus respuestas.

4.1.3 Diagramas de interacción entre objetos

Los diagramas de interacción muestran cómo colaboran los componentes del subsistema para realizar cada caso de uso real. Siguiendo la notación de los diagramas de secuencia del UML (Larman, 2004), se representan los participantes reales del diseño —la interfaz del navegador, el router `routers/auth.py`, el servicio `services/auth_service.py` y la base de datos MySQL—, los mensajes que intercambian en el tiempo y las alternativas que se resuelven durante cada flujo. Las cajas de activación indican el periodo en que cada participante ejecuta una operación, y los bloques alternativos reflejan las decisiones condicionales descritas en los casos de uso reales. Los diagramas son fieles a la implementación: cada mensaje corresponde a una invocación real de los componentes, de modo que la secuencia puede contrastarse directamente con el código. La numeración automática de los mensajes facilita la correspondencia entre el diagrama y los pasos de cada flujo.

CU-001 Registrarse

El diagrama de la figura muestra el flujo completo del registro. La interfaz envía los datos del formulario al router, que ejecuta la validación de formato y, según su resultado, responde con el error correspondiente o continúa con la comprobación de unicidad. Si el usuario no existe, el router encarga al servicio el cálculo del hash con `bcrypt`, inserta el registro en la tabla `users` y, a continuación, solicita la creación de los dos tokens de sesión; el servicio conserva en la base de datos solo el hash del token de refresco. El flujo culmina con la respuesta de éxito y el establecimiento de las dos cookies, lo que materializa la decisión de diseño de que el visitante quede autenticado al completar el registro. Las respuestas de error se mantienen diferenciadas: la validación de formato devuelve el campo afectado, mientras que la existencia previa devuelve la condición de duplicidad, lo que permite a la interfaz mostrar mensajes precisos sin revelar información sensible.

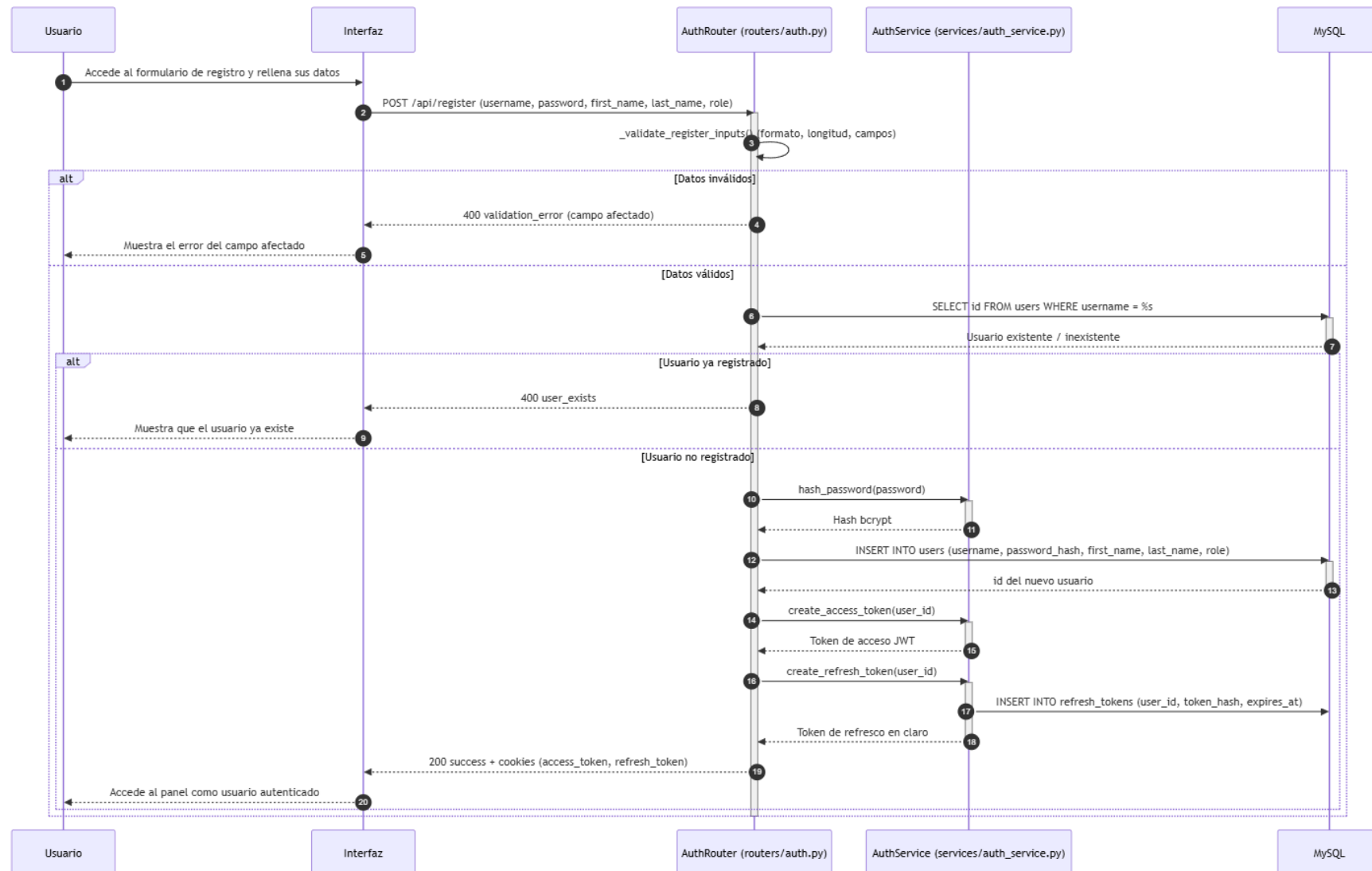


Ilustración x - Diagrama de secuencia del CU-001 Registrarse

CU-002 Iniciar sesión

El diagrama de la figura muestra el inicio de sesión. El router recupera el registro del usuario, y las dos alternativas que se suceden —usuario inexistente y contraseña incorrecta— responden con el mismo mensaje genérico, sin revelar qué parte de las credenciales falló, en coherencia con el análisis. Cuando la verificación con `bcrypt.checkpw` es correcta, el servicio genera el token de acceso y el token de refresco, y el router establece las cookies y redirige al panel. La secuencia refleja la separación de responsabilidades: el router se ocupa de la orquestación y de la respuesta HTTP, mientras que el servicio concentra la verificación criptográfica y la generación de las credenciales. El límite de peticiones del endpoint, aplicado por `slowapi` antes de procesar la solicitud, no se representa en el diagrama porque es una condición transversal que actúa en el punto de entrada del flujo.

El mecanismo de renovación de la sesión, que complementa al CU-002 y permite prolongar la sesión sin que el usuario vuelva a introducir sus credenciales, se muestra en la figura. Se activa cuando el token de acceso se acerca a su expiración: la interfaz envía el token de refresco al endpoint de rotación, y el servicio verifica su estado en la base de datos. Si el token está activo o fue revocado dentro del periodo de gracia, el servicio revoca el anterior, emite uno nuevo y lo devuelve; si fue revocado fuera del periodo de gracia, el sistema interpreta un posible robo de sesión y revoca todos los tokens del usuario, respondiendo la rotación como inválida. El diseño tolera así las renovaciones concurrentes sin tratar cada repetición como un ataque, pero invalida la sesión completa ante el uso real de una credencial revocada. Si la rotación es válida, el router emite un nuevo token de acceso y renueva las dos cookies; en caso contrario, responde con HTTP 401.

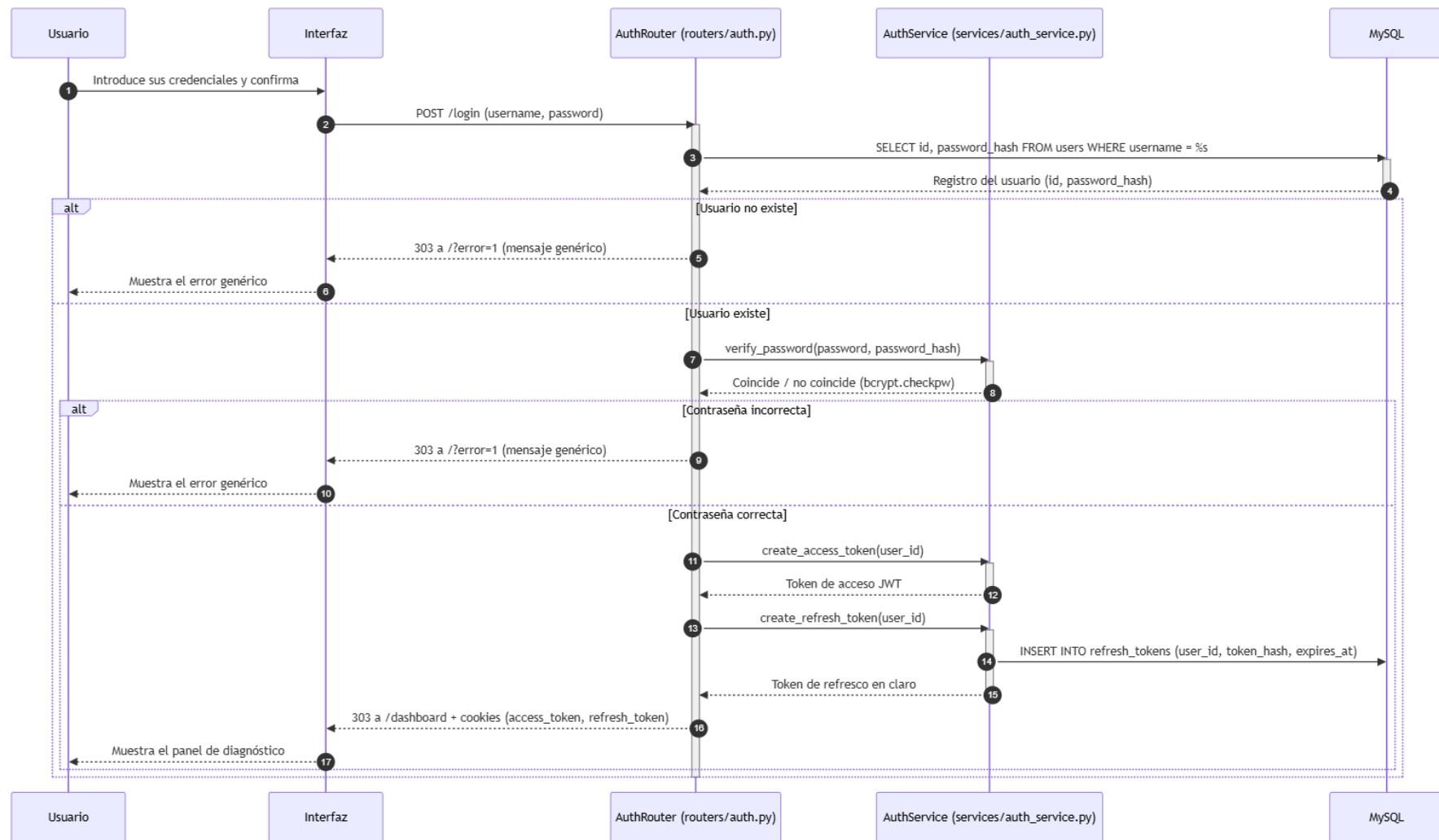


Ilustración x - Diagrama de secuencia del CU-001 Registrarse

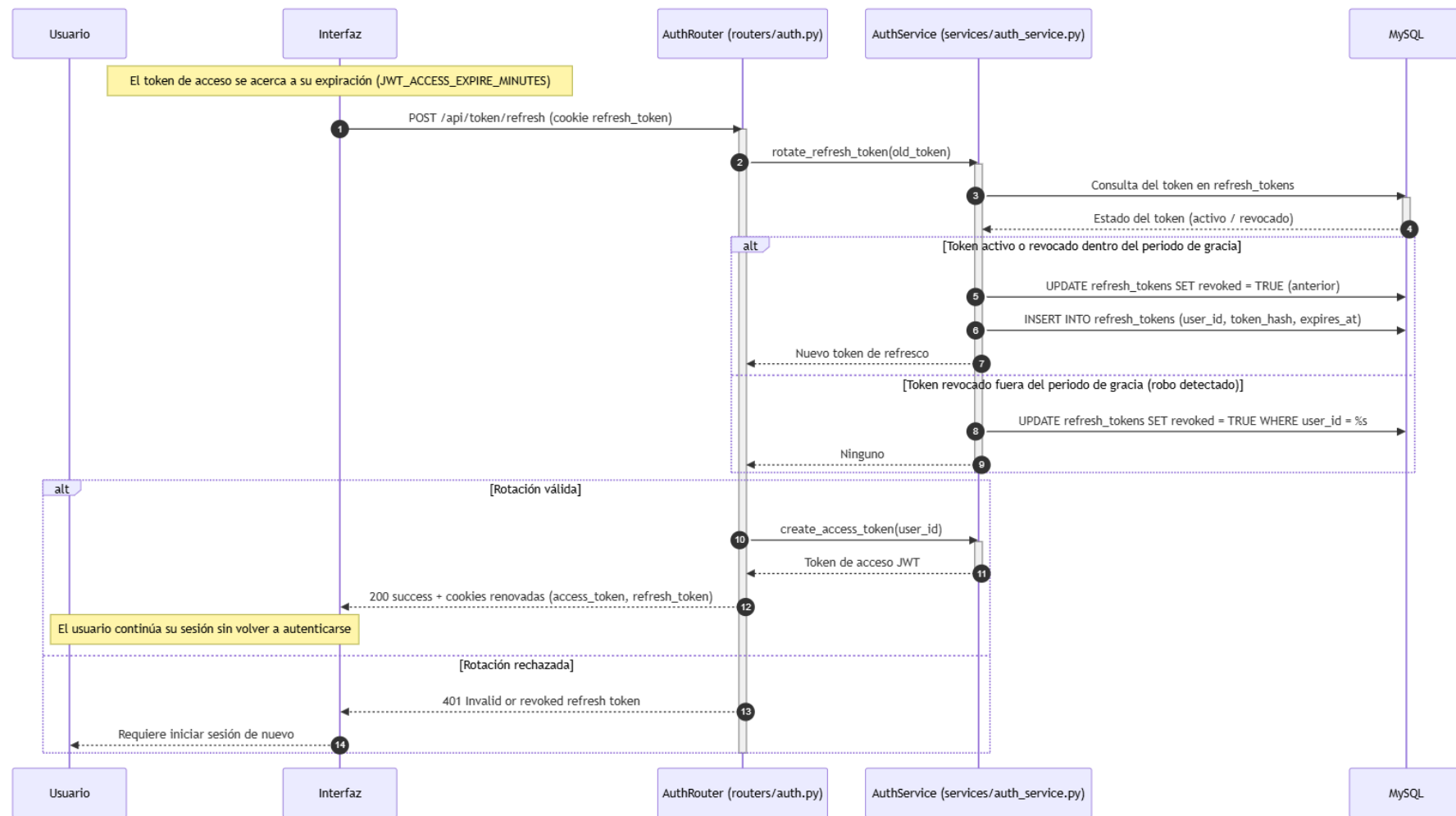


Ilustración x - Diagrama de secuencia de la renovación de la sesión

CU-003 Cerrar sesión

El diagrama de la figura muestra el cierre de sesión. El router solicita al servicio la revocación del token de refresco, que marca la fila correspondiente en la tabla `refresh_tokens`, y a continuación elimina las dos cookies de la respuesta y redirige a la página de inicio de sesión. La secuencia refleja la doble naturaleza de las credenciales: la revocación en la base de datos impide prolongar la sesión mediante la rotación, mientras que el borrado de las cookies descarta el token de acceso, que al ser un JWT autónomo no se puede revocar directamente y debe retirarse del navegador. La condición del diagrama cubre el caso en que no exista cookie de refresco —por ejemplo, una sesión sin rotación previa—, en cuyo caso la revocación se omite y la operación se limita al borrado de las cookies.

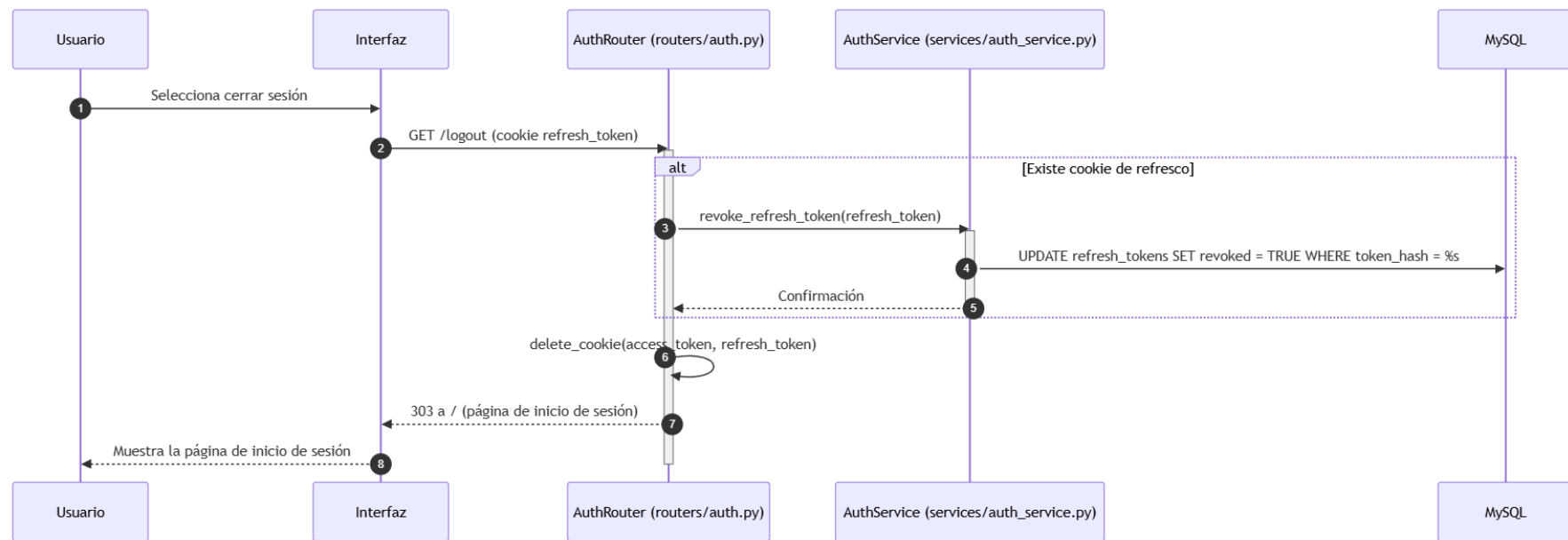


Ilustración x - Diagrama de secuencia del CU-003 Cerrar sesión

CU-004 Cambiar el idioma de la interfaz

El diagrama de la figura muestra el cambio de idioma. En el navegador, el script `i18n.js` lee la opción seleccionada, la guarda como preferencia y actualiza los textos de la interfaz sin recargar la página; este tramo se representa con auto-mensajes porque no interviene el servidor. En el tramo del servidor, los routers y servicios consultan al `LangService` el idioma aplicable mediante `get_lang_from_cookie()`, que valida la preferencia contra el conjunto permitido de idiomas y aplica el español por defecto cuando la preferencia es inválida o está ausente. La secuencia muestra la separación entre la presentación, que se resuelve en el navegador, y los mensajes generados por el backend, que se traducen con el diccionario centralizado de `services/lang.py`, de modo que las salidas del sistema —mensajes de error, etiquetas de predicción, títulos de los mapas de explicabilidad, informes PDF y mensajes del asistente— se mantienen alineadas con el idioma elegido por el actor.

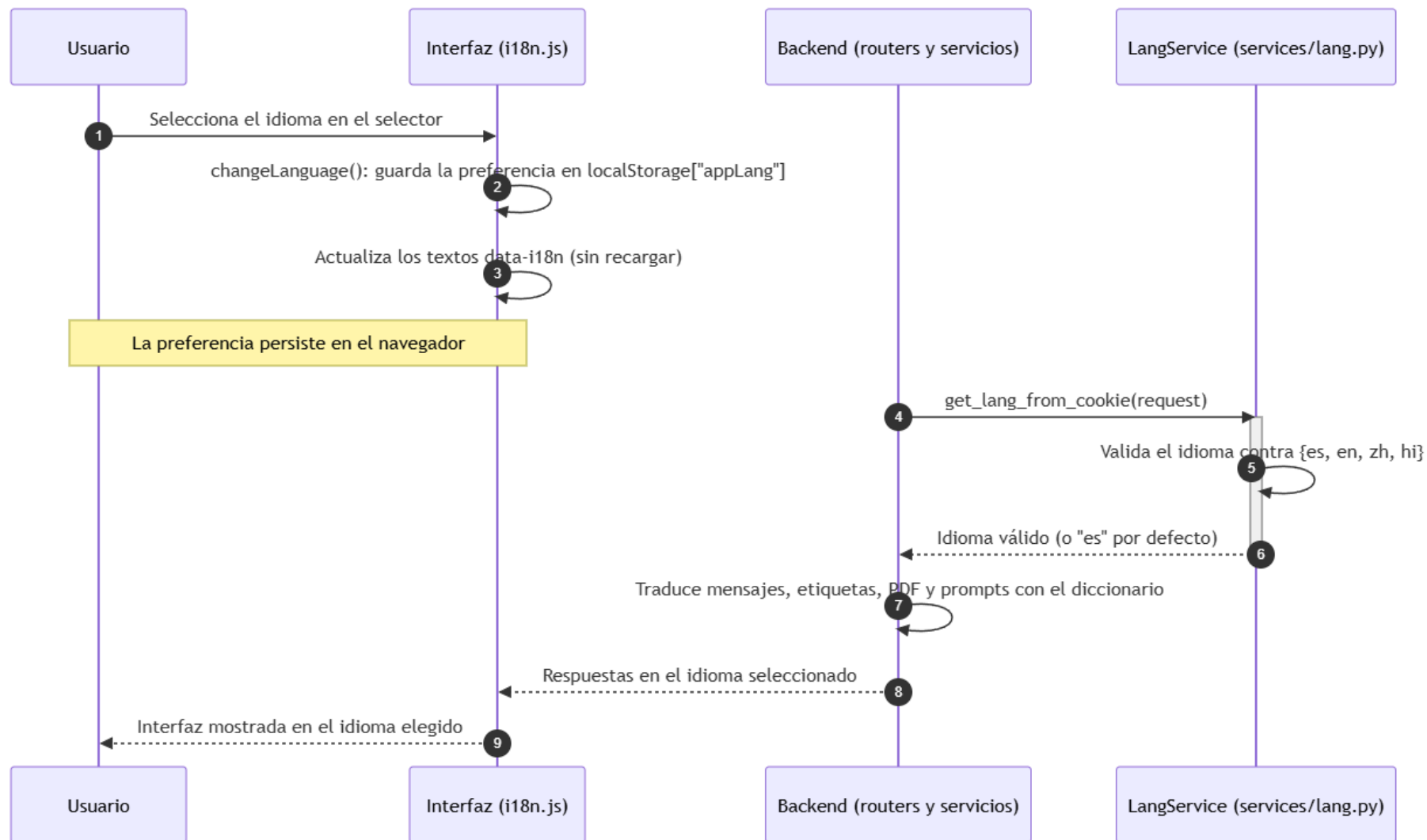


Ilustración x - Diagrama de secuencia del CU-004 Cambiar el idioma de la interfaz

4.2 Subsistema de diseño SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 corresponde al subsistema de análisis SS-002 y concentra el flujo clínico de la plataforma. Su función es recibir una imagen válida, asociarla al usuario autenticado, registrar una solicitud de diagnóstico, ejecutar la inferencia con la arquitectura seleccionada y conservar los artefactos que permiten consultar y justificar el resultado. Agrupa los casos de uso CU-005 a CU-010 y CU-037, que materializan el ciclo completo de una consulta clínica: el acceso al panel, la subida de la radiografía, la selección de la arquitectura, la solicitud del diagnóstico, la visualización del resultado y de los mapas de explicabilidad, y la generación del informe PDF. La gestión del historial de consultas (CU-011 a CU-014) no pertenece a este subsistema, sino a SD-003, que se describe en el apartado siguiente.

El subsistema se apoya en `routers/inference.py`, que actúa como fachada HTTP, y en un conjunto de servicios que se ejecutan fuera del ciclo de la petición: `services/ml_engine.py`, que prepara la imagen y produce la etiqueta y la confianza; `services/xai_generator.py`, que genera el mapa de explicabilidad adecuado al modelo; y `services/pdf_generator.py`, que construye el informe descargable. La ejecución de la inferencia se delega en la cola de trabajos y en su worker, que procesa el diagnóstico en segundo plano y persiste el resultado en la tabla `consultations`, de modo que la interfaz permanece operativa durante el análisis (RNF-020). El router, por tanto, no carga el modelo ni ejecuta la predicción: valida la petición, conserva la imagen y crea el registro de trabajo con un payload serializable que solo contiene el modelo, la ruta de la imagen y el idioma.

SD-002 participa en el sistema con tres dependencias claras. Se apoya en SD-001 para obtener la identidad del usuario a través del mecanismo común de identificación; utiliza la cola de trabajos del subsistema de ejecución asíncrona (SD-006) para procesar los diagnósticos sin bloquear la interfaz; y entrega a SD-003 los registros de `consultations` que este último consulta en el historial. Además, actúa como frontera de validación de ficheros: el tipo MIME y el tamaño se comprueban antes de escribir en disco, mientras que la validez científica de la imagen y la interpretación clínica del resultado quedan fuera del ámbito del subsistema, que no sustituye la valoración del profesional sanitario.

4.2.1 Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-002 recoge las siete interacciones que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de interfaz de diagnóstico asistido definido en el análisis. Se conservan únicamente los casos de uso del flujo clínico y del informe (CU-005 a CU-010 y CU-037), porque la gestión del historial (CU-011 a CU-014) pertenece a SD-003 y se representa en su propio diagrama. El diagrama no dibuja relaciones de extensión entre estos casos de uso: la cadena de navegación por la que el informe se genera desde el detalle de una consulta cruza la frontera del subsistema —CU-037 extiende CU-012, que pertenece a SD-003—, por lo que esa relación se describe en el texto y se refleja en los diagramas de interacción de ambos subsistemas. El diagrama se representa a continuación.

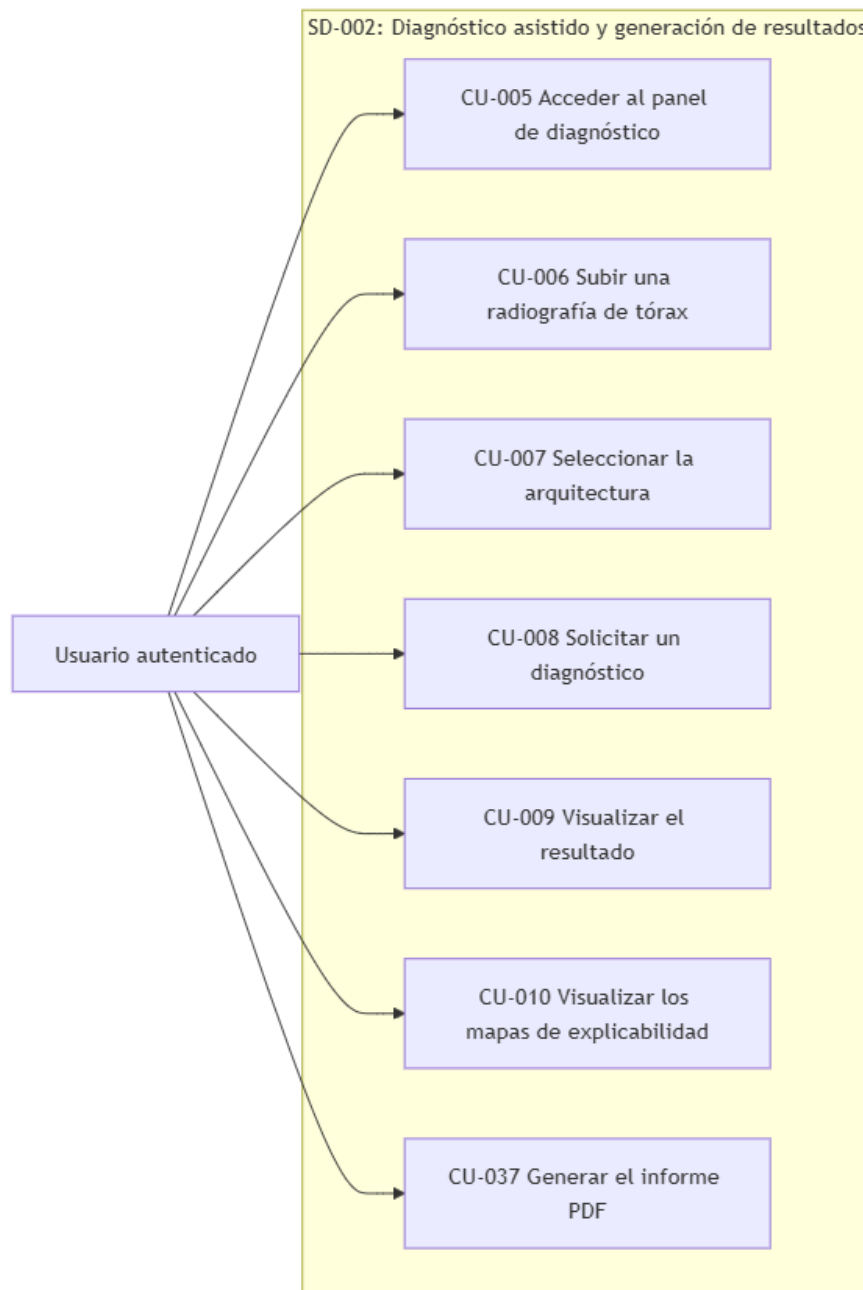


Ilustración x - Diagrama de casos de uso del subsistema SD-002

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los profesionales con cuenta registrada. El administrador no aparece como actor diferenciado porque sus operaciones de supervisión sobre las consultas se recogen en el subsistema SD-005. Los siete casos de uso se representan como interacciones directas e independientes del actor con el sistema. La subida de la imagen (CU-006), la selección de la arquitectura (CU-007) y la solicitud del diagnóstico (CU-008) forman la cadena de entrada del flujo clínico; la visualización del resultado (CU-009) y de los mapas de explicabilidad (CU-010) constituyen su salida; y el informe PDF (CU-037) su materialización documental. Los casos de uso CU-006, CU-007 y CU-008 se representan por separado, como en el análisis, aunque en la implementación se procesan en una única petición HTTP, tal y como se detalla en el apartado siguiente.

4.2.2 Casos de uso reales del subsistema

Los casos de uso reales de SD-002 concretan cada interacción del diagrama en decisiones técnicas de implementación. A diferencia de SD-001, cuyos flujos son síncronos, la mayor parte de este subsistema se organiza en torno al procesamiento asíncrono: la validación y el encolado se resuelven en el ciclo HTTP, mientras que la predicción, la generación de los mapas y del informe se ejecutan en el worker. Cada caso de uso real se describe a continuación, indicando los componentes que intervienen, las validaciones, las operaciones de persistencia y las respuestas al cliente.

CU-005 Acceder al panel de diagnóstico

El acceso al panel se materializa en el endpoint GET /dashboard del router routers/auth.py, que es la puerta de entrada al entorno clínico de la plataforma. El diseño aplica la condición transversal de SD-001: solo el usuario con una sesión válida alcanza el panel, y cualquier intento sin sesión se redirige al punto de entrada del sistema. Las decisiones técnicas del caso de uso real son las siguientes:

- **Verificación de la sesión:** el router obtiene el identificador del usuario mediante `get_user_id_from_token()` de SD-001, a partir de la cookie `access_token`. Si no hay sesión, responde con una redirección HTTP 303 a la página de inicio de sesión.
- **Carga del perfil:** con el identificador resuelto, se consultan el nombre y el rol del usuario en la tabla `users`, de modo que el panel se personaliza con el nombre completo y con el indicador de administrador cuando corresponde.
- **Control de caché:** la respuesta del panel incluye las cabeceras `Cache-Control: no-store`, que impiden que el navegador o los intermediarios sirvan la página privada desde caché tras un cierre de sesión.

CU-006 Subir una radiografía de tórax

La subida de la radiografía se materializa en la selección del archivo desde el panel de diagnóstico y en la validación que el router realiza al recibir el formulario. El diseño mantiene una doble frontera: la interfaz descarta de forma temprana los archivos claramente no válidos, y el router vuelve a comprobar el tipo y el tamaño antes de escribir nada en disco. Las decisiones técnicas del caso de uso real son las siguientes:

- **Selección del archivo:** la zona de carga del panel (`file-input`) permite adjuntar el fichero, y la interfaz habilita el envío únicamente cuando hay una imagen seleccionada.
- **Validación del tipo MIME:** el router comprueba que `file.content_type` pertenezca al conjunto permitido `{image/jpeg, image/png, image/jpg}`. Ante cualquier otro tipo, responde HTTP 400 con el mensaje de solo imágenes, sin escribir el fichero.
- **Validación del tamaño:** el router mide el archivo y rechaza con HTTP 400 los que superen los 10 MB, en coherencia con el límite declarado en el análisis.
- **Conservación de la imagen:** el fichero se guarda en el área `static/uploads` con un nombre basado en la fecha y hora de la recepción, lo que evita colisiones y no depende del nombre original del archivo.
- **Integración con la solicitud:** la imagen se envía en el mismo formulario que la solicitud del diagnóstico (CU-008), de modo que subida y solicitud se materializan en una única petición POST `/predict`.

CU-007 Seleccionar la arquitectura para el diagnóstico

La selección de la arquitectura se materializa en el selector de modelos del panel, que presenta las arquitecturas disponibles del proyecto —arquitecturas convolucionales como InceptionV3 o Xception y arquitecturas Transformer como DeiT, Swin y ViT—. El diseño reparte la responsabilidad de la selección entre la interfaz y el motor de aprendizaje, sin duplicar la lista de modelos en el router. Las decisiones técnicas del caso de uso real son las siguientes:

- **Elección en el panel:** el usuario elige la arquitectura en model-selector y el identificador del modelo viaja como campo del formulario de la solicitud (model_name).
- **Validación diferida:** el router no comprueba el modelo contra una lista, porque esa verificación pertenece al motor de inferencia: si los pesos de la arquitectura no existen, el worker marca el trabajo como fallido con un mensaje claro. Esta decisión evita mantener la lista de modelos en dos capas distintas.
- **Reutilización de modelos en memoria:** el motor conserva los modelos cargados en un diccionario global, de modo que la primera consulta con una arquitectura asume el coste de cargar los pesos, mientras que las posteriores reutilizan el modelo en memoria (RNF-019). Esta optimización pertenece al motor y no al router.

CU-008 Solicitar un diagnóstico

La solicitud del diagnóstico se materializa en el endpoint POST /predict de routers/inference.py, que traduce la petición del usuario en la creación de un trabajo de la cola. El diseño sigue el orden del análisis —validar, encolar y notificar—, pero traslada la ejecución de la inferencia al worker para que la interfaz no se bloquee. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación:** el router resuelve la identidad con SD-001 y responde HTTP 401 con el mensaje de no autenticado si no hay sesión válida.
- **Validación antes de encolar:** el tipo MIME y el tamaño se comprueban antes de crear el trabajo, de modo que una petición inválida no ocupa una posición en la cola y no consume recursos del worker.
- **Conservación de la imagen:** la imagen se guarda en static/uploads y la ruta resultante se incorpora al payload del trabajo.
- **Encolado:** se inserta un registro en job_queue con el tipo diagnosis, el identificador del usuario y un payload JSON serializable que solo contiene model_name, image_path y lang. Ni el modelo ni la imagen completa se serializan en la base de datos.
- **Posición en la cola:** el router calcula la posición del trabajo mediante la consulta de posición, que ordena los diagnósticos antes que el resto de tipos de trabajo.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el estado queued, el identificador del trabajo, su posición y un mensaje localizado, de modo que la interfaz informa al usuario de que el diagnóstico ha sido encolado.

CU-009 Visualizar el resultado del diagnóstico

La visualización del resultado se materializa en la consulta periódica del estado del trabajo y en el render del panel cuando este pasa a completado. El diseño separa la obtención del estado, que se resuelve con el router de la cola, de la presentación del resultado, que se apoya en el registro persistido por el worker. Las decisiones técnicas del caso de uso real son las siguientes:

- **Consulta del estado:** la interfaz ejecuta un sondeo de GET /api/queue/status cada dos segundos, con un límite de intentos, que refleja la ejecución en segundo plano del diagnóstico. El router de la cola devuelve los últimos trabajos del usuario, su estado y su posición.
- **Presentación del resultado:** cuando el trabajo pasa a completed, el panel muestra la predicción (Neumonía o Normal), el nivel de confianza y el modelo empleado, y refresca el historial de consultas.
- **Persistencia sin reinferencia:** el resultado queda registrado por el worker en la tabla consultations con su etiqueta, su confianza y las rutas de los artefactos, de modo que las visualizaciones posteriores no repiten la inferencia.
- **Localización:** la etiqueta de la predicción se adapta al idioma de la sesión mediante el servicio de idioma, en coherencia con el CU-004 de SD-001.

CU-010 Visualizar los mapas de explicabilidad

La visualización de los mapas se materializa en la imagen explicativa que el worker genera junto con la predicción y que la interfaz muestra en el detalle de la consulta. El diseño separa la generación, que pertenece al motor de explicabilidad, de la entrega, que se resuelve con los recursos estáticos de la aplicación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Generación del mapa:** services/xai_generator.py produce una figura con la radiografía original, el mapa de Saliency, el mapa de SmoothGrad y la superposición del Grad-CAM (para arquitecturas convolucionales) o del mapa de atención (para arquitecturas Transformer), en función de la arquitectura utilizada.
- **Conservación del artefacto:** la imagen se guarda en static/results con un nombre derivado del de la radiografía, y su ruta se persiste en la columna xai_image_path de la consulta.
- **Entrega al usuario:** la imagen se sirve como recurso estático y se muestra en el detalle de la consulta junto a la radiografía original, con la comprobación de propiedad que impone el historial: solo el usuario propietario accede a sus consultas y a sus artefactos (RF-005).
- **Inspección visual:** la interfaz permite ampliar la radiografía y el mapa en un visor, de modo que el profesional pueda inspeccionar con detalle las regiones que justifican la predicción.

CU-037 Generar el informe PDF del diagnóstico

El informe PDF se materializa durante el procesamiento del trabajo, mediante `services/pdf_generator.py`, que construye el documento descargable de la consulta. El diseño adelanta la generación del informe al momento del diagnóstico: cuando la consulta se completa, el documento ya existe, de modo que la descarga posterior no depende de una nueva operación del sistema. Las decisiones técnicas del caso de uso real son las siguientes:

- **Construcción del documento:** el generador produce un informe A4 con la fecha, el modelo utilizado, el diagnóstico, el nivel de confianza, la radiografía original y el mapa de explicabilidad, aplicando un color de diagnóstico acorde con el resultado.
- **Conservación del informe:** el PDF se guarda en `static/reports` con un nombre basado en la fecha, y su ruta se persiste en la columna `pdf_path` de la consulta.
- **Entrega al usuario:** el informe se sirve como recurso estático descargable desde el detalle de la consulta, de modo que el actor descarga el documento a su equipo tal y como declara el análisis.
- **Protección de la información:** el informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial (RGPD).

4.2.3 Diagramas de interacción entre objetos

Los diagramas de interacción de SD-002 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. A diferencia de SD-001, cuyas interacciones son síncronas, la colaboración del diagnóstico se divide en dos planos: el ciclo de petición HTTP, en el que participan la interfaz, el router y la base de datos, y el procesamiento asíncrono, en el que el worker reclama el trabajo de la cola y orquesta el motor de predicción, el generador de mapas y el generador de informes. Esta división es deliberada: el worker no acepta órdenes directamente del navegador, sino que reclama condicionalmente los trabajos que se encuentran en estado `queued`, lo que impide el doble procesamiento y mantiene la interfaz desbloqueada durante la inferencia. Los diagramas se presentan a continuación, con la misma notación de secuencia del UML empleada en el 21.1.3.

CU-005 Acceder al panel de diagnóstico

El diagrama de la figura muestra el acceso al panel. La interfaz solicita el panel y el router consulta a SD-001 la identidad contenida en la cookie de acceso. Si hay sesión válida, el router recupera el perfil del usuario y devuelve la vista del panel con el control de caché; si no la hay, redirige al punto de entrada del sistema. La secuencia refleja la condición transversal de la autenticación: el panel no se sirve en ningún caso sin identidad resuelta.

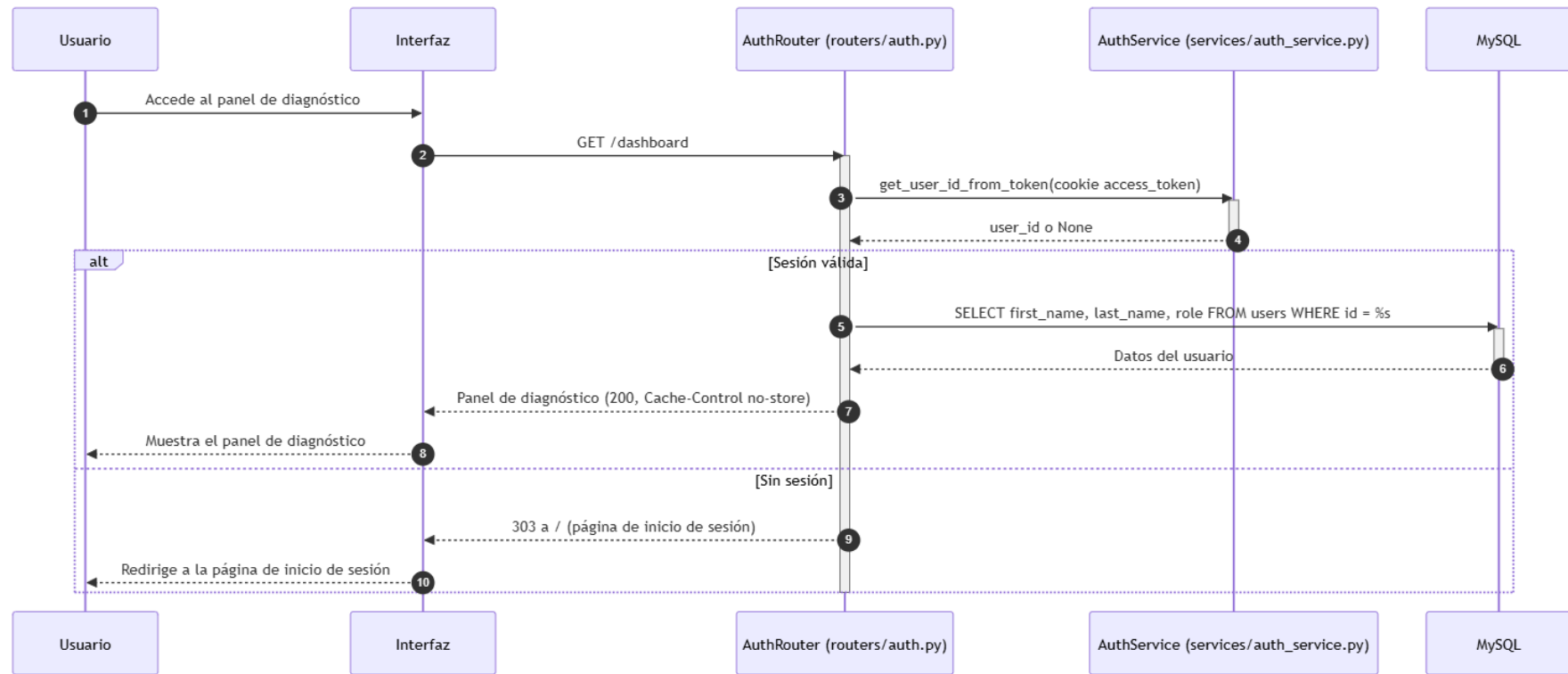


Ilustración x - Diagrama de secuencia del CU-005 Acceder al panel de diagnóstico

CU-008 Solicitar un diagnóstico

El diagrama de la figura muestra la solicitud del diagnóstico, que materializa también la subida de la radiografía (CU-006) y la selección de la arquitectura (CU-007), porque ambas se procesan en la misma petición. La interfaz adjunta la imagen y el modelo seleccionado, y el router resuelve la identidad. A continuación se suceden las validaciones de la frontera: sin sesión responde 401, y ante un tipo o un tamaño no permitidos responde 400 sin tocar la cola. Solo un fichero válido se conserva en disco y se encola: el router inserta el trabajo de tipo diagnosis, calcula su posición y devuelve el estado queued. El diagrama deja fuera la ejecución de la inferencia, que corresponde al procesamiento asíncrono de la figura 58.

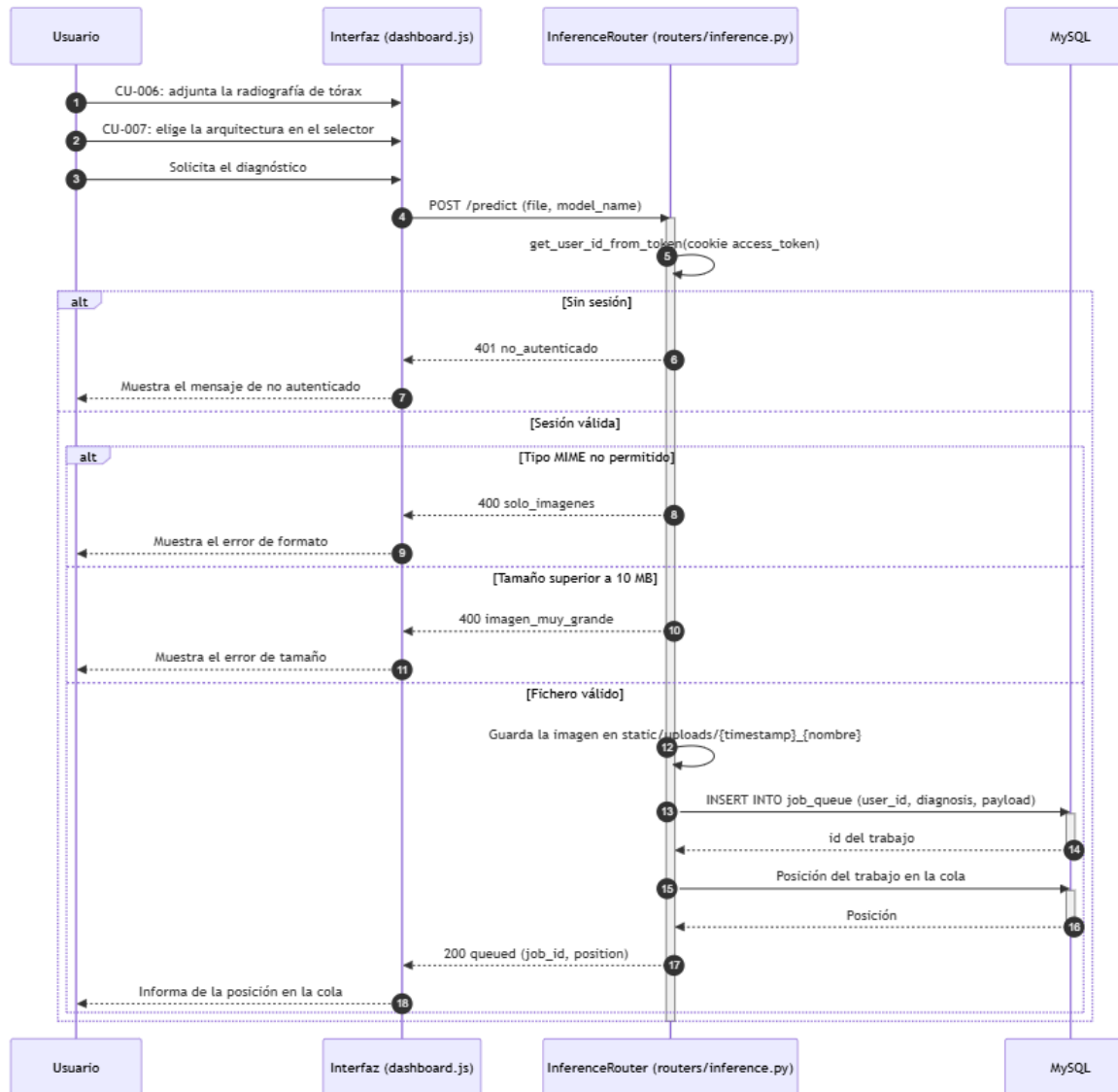


Ilustración x - Diagrama de secuencia del CU-008 Solicitar un diagnóstico

Procesamiento asíncrono del diagnóstico

El diagrama de la figura muestra la colaboración del worker con los servicios del subsistema. El bucle de la cola reclama el primer trabajo queued, con prioridad para los diagnósticos, y lo marca como running mediante una actualización condicional que solo afecta a una fila si el trabajo seguía en cola; de este modo, dos instancias no pueden procesar el mismo trabajo. A continuación el worker encarga al motor de predicción la inferencia, al generador de mapas la explicación visual y al generador de informes el documento PDF, y persiste el resultado en consultations con todas las rutas de los artefactos. Finalmente, marca el trabajo como completed con el resultado en formato JSON. Si cualquier paso falla, el trabajo pasa a failed con el mensaje de error limitado, de modo que un fallo del motor, de la imagen o del informe no se transforma en una consulta completada con datos ambiguos.

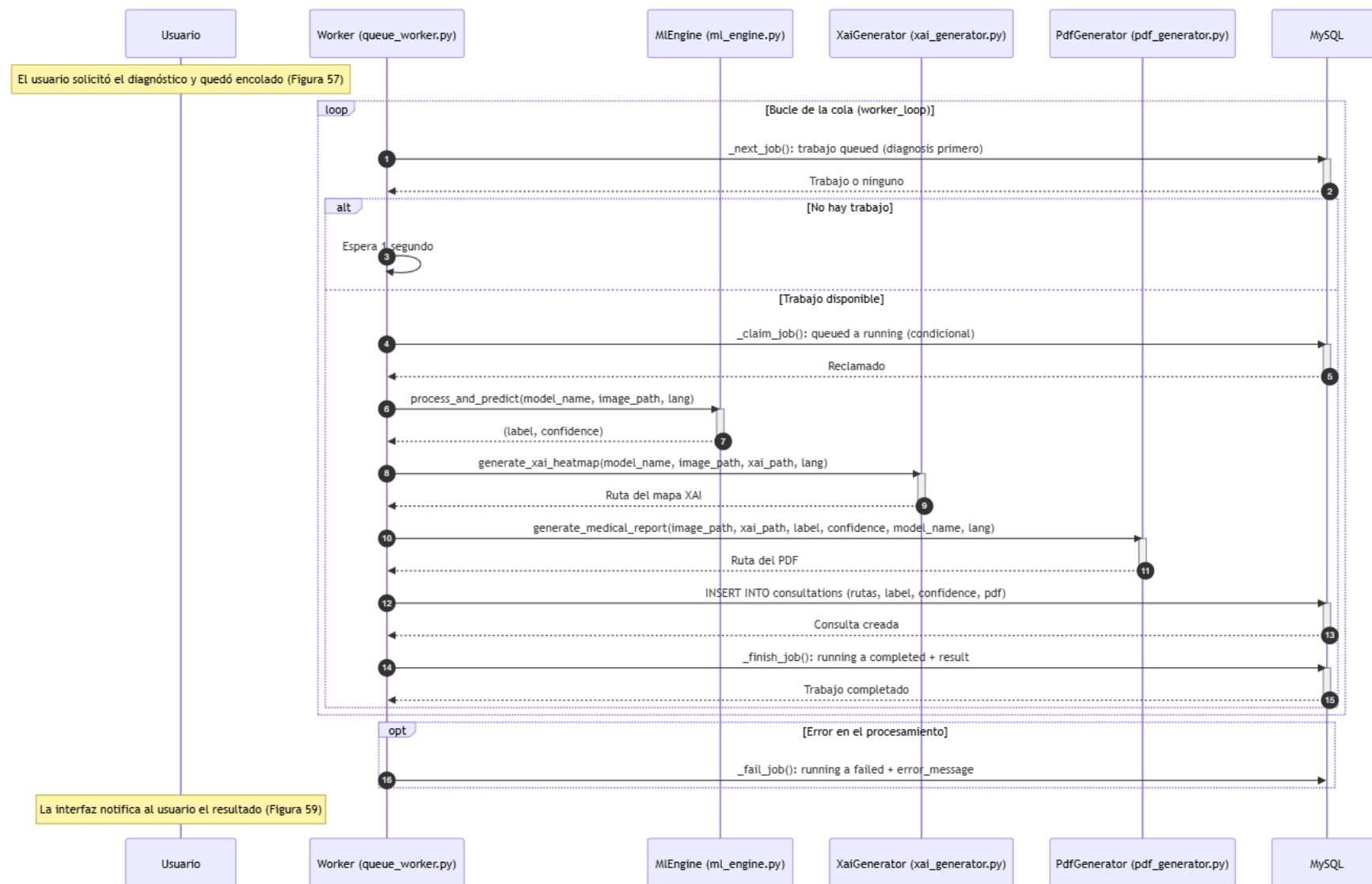


Ilustración x - Diagrama de secuencia del procesamiento asíncrono del diagnóstico

CU-009 y CU-010 Visualizar el resultado y los mapas de explicabilidad

El diagrama de la figura muestra la visualización del resultado y de los mapas. La interfaz sondea el estado del trabajo con el router de la cola; cuando el trabajo pasa a completed, muestra la predicción, la confianza y el modelo (CU-009) y refresca el historial. Desde el detalle de la consulta, la interfaz solicita la radiografía original y el mapa de explicabilidad al almacenamiento estático (CU-010). Si el trabajo terminó en failed, la interfaz muestra el error recibido. La secuencia refleja que el resultado no se calcula en la visualización, sino que se recupera del trabajo completado y de los artefactos persistidos.

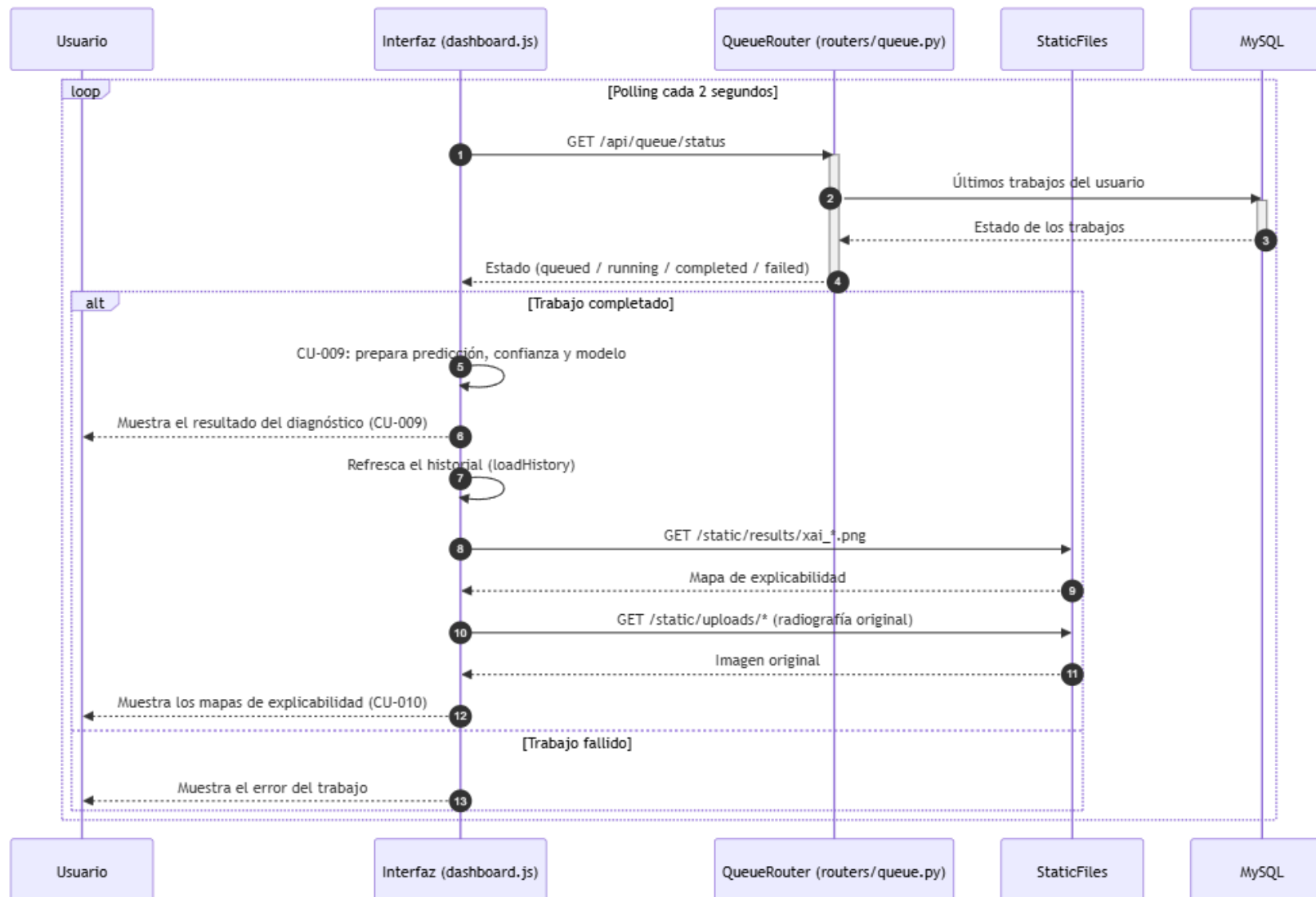


Ilustración x - Diagrama de secuencia del procesamiento asíncrono del diagnóstico

CU-037 Generar el informe PDF del diagnóstico

El diagrama de la figura muestra la entrega del informe PDF. La generación del documento ya ocurrió durante el procesamiento asíncrono, por lo que la interacción del actor se limita a recuperar la ruta del informe desde el historial y a descargar el recurso estático. La interfaz consulta la consulta en el router del historial, obtiene la ruta del PDF persistida y solicita el documento al almacenamiento estático, que lo entrega como archivo descargable. El diseño garantiza que la descarga siempre disponga del documento, porque el informe existe desde el momento en que la consulta se completa.

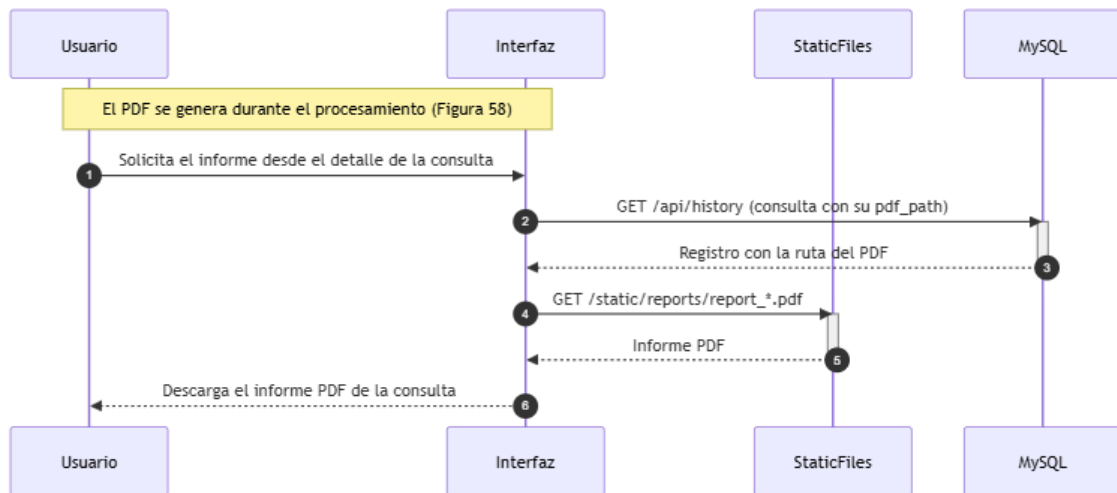


Ilustración x - Diagrama de secuencia del CU-037 Generar el informe PDF

4.3 Subsistema de diseño SD-003: Historial y gestión de consultas

El subsistema SD-003 corresponde al subsistema de análisis SS-003 y proporciona la recuperación y la gestión de las consultas ya realizadas. Su entidad persistente principal es la tabla consultations, que almacena las rutas de los artefactos, la arquitectura utilizada, la predicción, la confianza, el nombre mostrado al usuario y la fecha de la operación. Agrupa los casos de uso CU-011 a CU-014: la consulta del historial, la visualización del detalle, el renombrado y la eliminación. La relación con SD-002 es de productor-consumidor de resultados: SD-002 crea el registro de consultations cuando finaliza un diagnóstico, y SD-003 ofrece sobre él operaciones de consulta y organización, sin modificar en ningún caso la predicción ni sus artefactos.

El subsistema se apoya en routers/history.py, que expone la consulta del listado y las operaciones de actualización y eliminación, y en la interfaz del historial integrada en el panel de diagnóstico, que utiliza JavaScript para mostrar los resultados, cambiar el nombre visible de una consulta y solicitar su eliminación. La entidad de persistencia y la no ejecución del modelo condicionan el diseño: el subsistema recupera desde MySQL los metadatos y devuelve las rutas de la imagen original, del mapa XAI y del informe PDF, de modo que la visualización de una consulta anterior no repite la inferencia. El nombre mostrado al usuario se actualiza sobre patient_name, que funciona como etiqueta de organización de la consulta y no como identificación clínica del paciente, y la eliminación se implementa actualmente como una eliminación física de la fila, por lo que el subsistema no realiza un borrado lógico ni conserva automáticamente una auditoría de cada eliminación.

4.3.1 Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-003 recoge las cuatro interacciones que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de interfaz de diagnóstico asistido definido en el análisis (figura 4), limitada a los casos de uso de la gestión del historial. El diagrama conserva las relaciones de extensión de la cadena de navegación que el análisis declaró para este bloque: el detalle (CU-012) se alcanza de forma opcional desde el listado (CU-011), y desde el detalle pueden ejecutarse, de forma opcional, el renombrado (CU-013) o la eliminación (CU-014). El informe PDF (CU-037) también extiende CU-012, pero pertenece a SD-002, que se ocupa de su generación, por lo que esa relación se materializa en la frontera entre subsistemas y no se dibuja aquí. El diagrama se representa a continuación.

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los profesionales con cuenta registrada. El administrador no aparece como actor diferenciado, aunque las operaciones de este subsistema le afectan: la comprobación de propiedad contempla una excepción controlada para el rol administrativo, que permite supervisar consultas desde SD-005, pero esa excepción es una decisión de diseño del caso de uso real y no una interacción distinta del actor. Los cuatro casos de uso se representan como interacciones del actor con el sistema, con las relaciones de extensión que ordenan la navegación entre el listado, el detalle y las operaciones que se ejecutan desde este último.

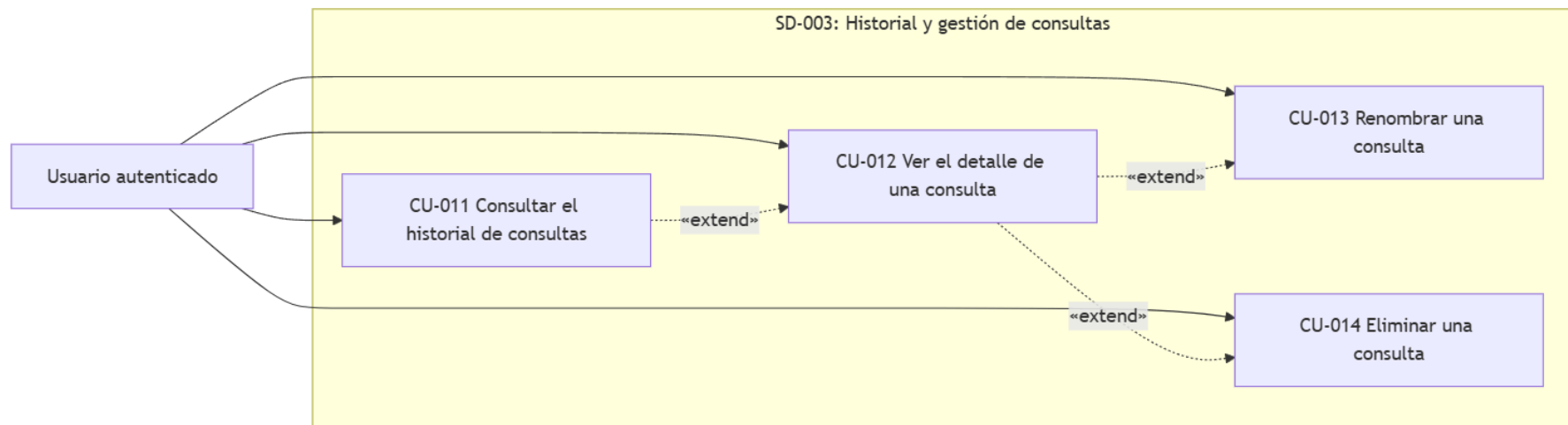


Ilustración x - Diagrama de casos de uso del subsistema SD-003

4.3.2 Casos de uso reales del subsistema

Los casos de uso reales de SD-003 concretan cada interacción del diagrama en decisiones técnicas de implementación. A diferencia de SD-002, las operaciones de este subsistema son síncronas y no requieren la cola de trabajos: recuperan o modifican registros ya persistidos, sin volver a ejecutar el modelo. Cada caso de uso real se describe a continuación, indicando los componentes que intervienen, las comprobaciones de propiedad, las operaciones de persistencia y las respuestas al cliente.

CU-011 Consultar el historial de consultas

La consulta del historial se materializa en el endpoint GET /api/history de routers/history.py, que devuelve exclusivamente las consultas del usuario autenticado. El diseño impone el aislamiento de datos en la propia consulta de persistencia: el filtro por user_id se aplica en el servidor y no se confía en la interfaz. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación:** el router resuelve la identidad con SD-001 y responde HTTP 401 con el mensaje de no autenticado si no hay sesión válida.
- **Filtrado por propietario:** la consulta recupera las columnas del historial desde consultations filtrando por WHERE user_id = %s, de modo que ningún usuario puede recibir las consultas de otro (RF-005).
- **Ordenación:** el listado se ordena por fecha descendente, de modo que las consultas más recientes aparecen en primer lugar.
- **Formato de los datos:** las fechas se convierten a una representación textual antes de formar la respuesta JSON, de modo que el navegador no necesita conocer el tipo de fecha de MySQL.
- **Presentación en la interfaz:** la vista del historial agrupa las consultas por modelo y construye las tarjetas con la imagen, la fecha, la etiqueta y la confianza, enlazando cada tarjeta con el detalle de la consulta.
- **Sin reinferencia:** la consulta no ejecuta el modelo; recupera los metadatos y las rutas de los artefactos ya persistidos, lo que reduce el coste de acceso al historial.

CU-012 Ver el detalle de una consulta del historial

La visualización del detalle se materializa en la interfaz del panel de diagnóstico, que construye la vista de detalle a partir de los datos que ya recibió en la consulta del historial. El diseño evita un endpoint específico de detalle para el usuario normal: como el listado ya está filtrado por propietario, los datos del detalle provienen de una respuesta que el servidor restringió al usuario, y los artefactos se sirven mediante las rutas estáticas configuradas en la aplicación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Origen de los datos:** la interfaz conserva en la tarjeta del historial el identificador, las rutas de la imagen y del mapa, la etiqueta, la confianza, el modelo, el nombre y la fecha; al abrir el detalle, presenta estos campos sin necesidad de una nueva petición al servidor.
- **Artefactos visuales:** la vista muestra la radiografía original y el mapa de explicabilidad servidos como recursos estáticos, junto con el resultado del diagnóstico, su nivel de confianza, el modelo empleado y los metadatos de la consulta.
- **Comprobación de propiedad implícita:** la propiedad queda garantizada por la cadena de datos: el detalle solo puede construirse a partir de una consulta que el servidor ya devolvió al usuario propietario.
- **Acciones desde el detalle:** desde la vista de detalle se ejecutan, de forma opcional, las operaciones de renombrado (CU-013), eliminación (CU-014) y descarga del informe PDF (CU-037, de SD-002), en coherencia con las relaciones de extensión del diagrama.

CU-013 Renombrar una consulta del historial

El renombrado se materializa en el endpoint POST `/api/history/update_name` de `routers/history.py`, precedido de la solicitud del nuevo nombre en la interfaz. El diseño antepone la comprobación de propiedad a la modificación y valida el nuevo nombre antes de enviarlo al servidor. Las decisiones técnicas del caso de uso real son las siguientes:

- **Solicitud del nombre:** la interfaz muestra un diálogo con el nombre actual de la consulta y recoge el nuevo valor.
- **Validación del nombre:** la interfaz rechaza el envío si el nuevo nombre está vacío o no ha cambiado, de modo que no se envíen peticiones innecesarias.
- **Comprobación de propiedad:** el router invoca `_check_consultation_ownership()`, que verifica que la consulta existe y pertenece al usuario solicitante, con la excepción controlada del rol admin que permite la supervisión desde SD-005. La comprobación distingue tres resultados: consulta inexistente, consulta de otro usuario y propiedad confirmada.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 cuando la consulta no existe y HTTP 403 cuando pertenece a otro usuario, de modo que la interfaz informa de la situación sin exponer detalles internos de la base de datos.
- **Actualización:** si la propiedad se confirma, el sistema ejecuta `UPDATE consultations SET patient_name = %s WHERE id = %s` y confirma la transacción, actualizando únicamente la etiqueta de organización de la consulta, sin tocar los datos técnicos de la inferencia.
- **Refresco de la vista:** la interfaz actualiza el nombre en la tarjeta del historial y en el detalle abierto, y recarga el listado para mantener la coherencia.

CU-014 Eliminar una consulta del historial

La eliminación se materializa en el endpoint POST `/api/history/delete` de `routers/history.py`, precedido de la confirmación del usuario en la interfaz. El diseño sigue el requisito del análisis de solicitar confirmación previa, en línea con el derecho de supresión del RGPD, y ejecuta la eliminación como un borrado físico de la fila. Las decisiones técnicas del caso de uso real son las siguientes:

- **Confirmación previa:** la interfaz solicita al usuario la confirmación de la eliminación antes de enviar la petición; si no se confirma, la operación se abandona y la consulta permanece intacta.
- **Comprobación de propiedad:** antes del borrado, el router invoca `_check_consultation_ownership()` con la misma lógica de los tres resultados —inexistente, ajena y propia— y con la excepción controlada del rol admin.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 y HTTP 403 en los casos de inexistencia y falta de permisos, y HTTP 200 cuando la eliminación se confirma.
- **Borrado físico:** la eliminación se ejecuta con `DELETE FROM consultations WHERE id = %s` y se confirma la transacción solo después de que MySQL la acepte. El diseño no conserva automáticamente una auditoría de la eliminación, tal y como se declaró en el capítulo 18.
- **Refresco de la vista:** la interfaz cierra el detalle abierto y recarga el historial, de modo que la consulta eliminada desaparece del listado junto con el acceso a sus artefactos.

4.3.3 Diagramas de interacción entre objetos

Los diagramas de interacción de SD-003 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. A diferencia de SD-002, todas las interacciones de este subsistema son síncronas: no interviene la cola de trabajos, porque las operaciones recuperan o modifican registros ya persistidos. El control de propiedad constituye el elemento central de los diagramas de renombrado y eliminación, donde la comprobación se resuelve antes de cualquier modificación y condiciona la respuesta al cliente. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-011 Consultar el historial de consultas

El diagrama de la figura muestra la consulta del historial. El usuario accede a su historial y la interfaz solicita el listado al router, que resuelve la identidad. Con sesión válida, el router recupera únicamente las consultas del usuario ordenadas por fecha descendente, formatea las fechas y devuelve el listado; la interfaz lo presenta agrupado por modelo. Si no hay sesión, el router responde 401 y la interfaz conduce al inicio de sesión. La secuencia refleja que el aislamiento de datos se aplica en la consulta de persistencia y no en la presentación.

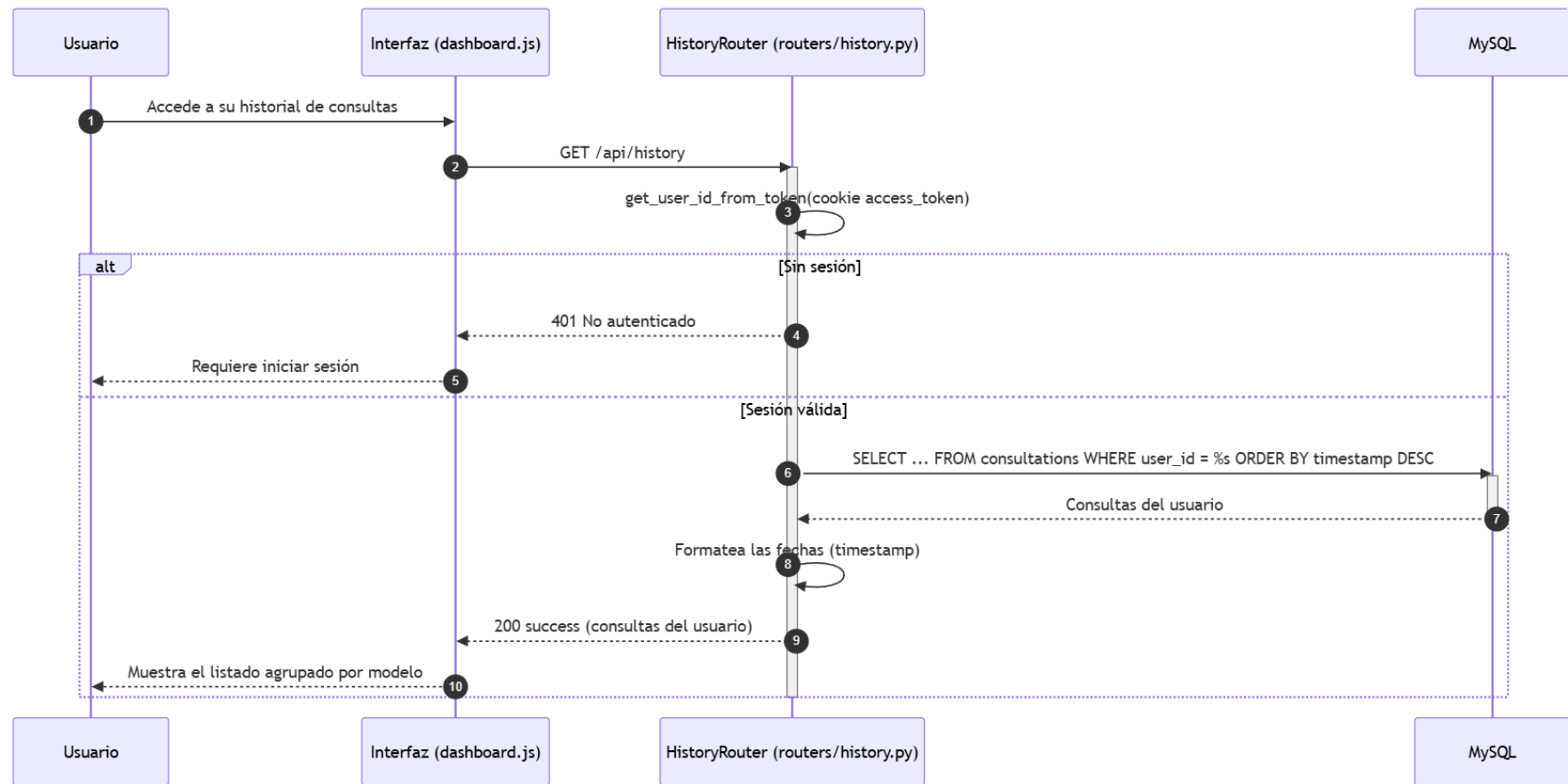


Ilustración x - Diagrama de secuencia del CU-011 Consultar el historial de consultas

CU-012 Ver el detalle de una consulta del historial

El diagrama de la figura muestra la visualización del detalle. El usuario selecciona una consulta del listado y la interfaz construye el detalle con los datos que ya recibió del historial, sin realizar una petición adicional al servidor. A continuación solicita al almacenamiento estático la radiografía original y el mapa de explicabilidad, y presenta la consulta completa al usuario. La secuencia refleja la decisión de diseño de no disponer de un endpoint de detalle para el usuario normal: la propiedad queda garantizada por la cadena de datos del listado, ya restringido al propietario.

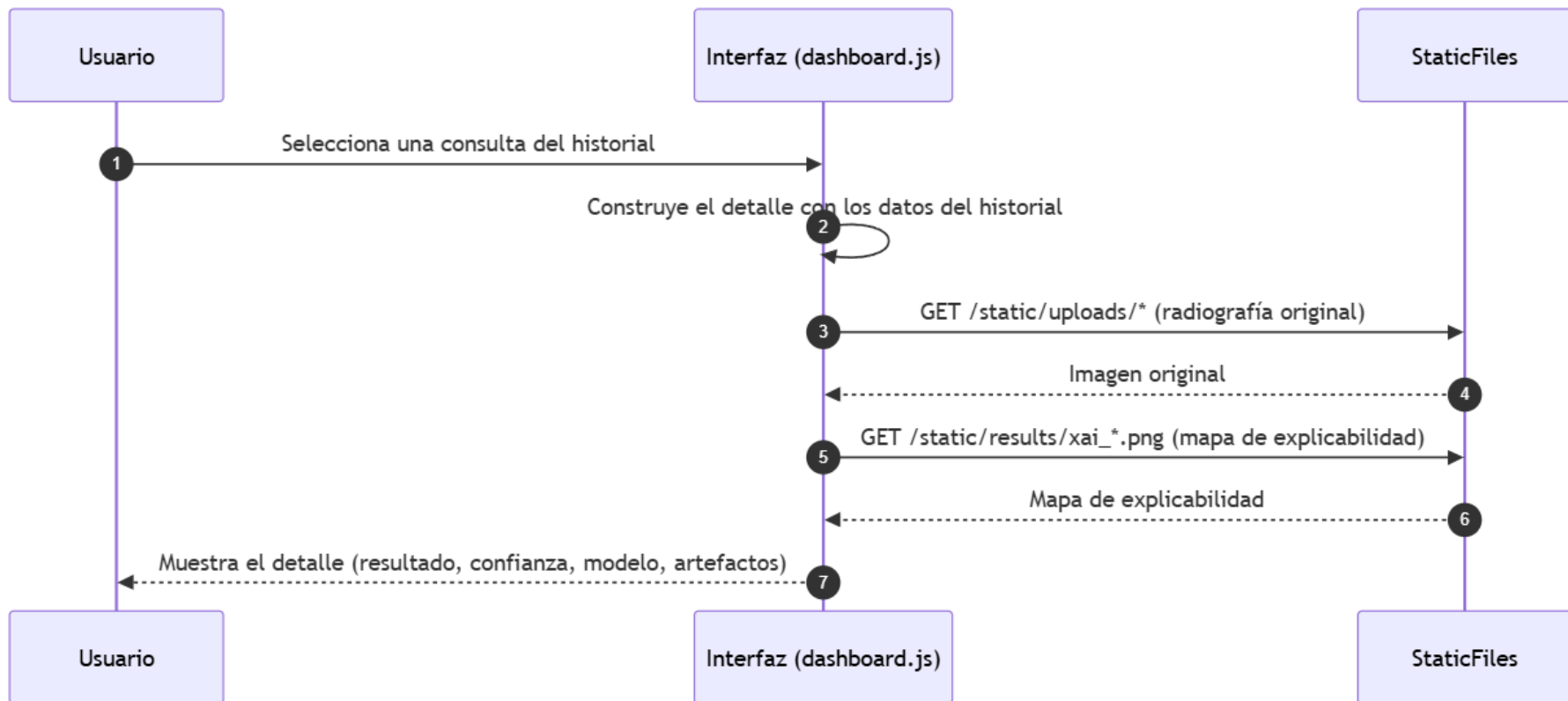


Ilustración x -Diagrama de secuencia del CU-012 Ver el detalle de una consulta del historial

CU-013 Renombrar una consulta del historial

El diagrama de la figura muestra el renombrado. El usuario indica el nuevo nombre en el diálogo de la interfaz, que valida que no esté vacío y envía la petición al router. El router resuelve la identidad y comprueba la propiedad de la consulta mediante `_check_consultation_ownership()`. Según el resultado, responde 404 si la consulta no existe, 403 si pertenece a otro usuario, o ejecuta la actualización del nombre cuando la propiedad se confirma y devuelve éxito; la interfaz muestra el nuevo nombre. La secuencia muestra que la modificación queda condicionada a la comprobación previa de propiedad.

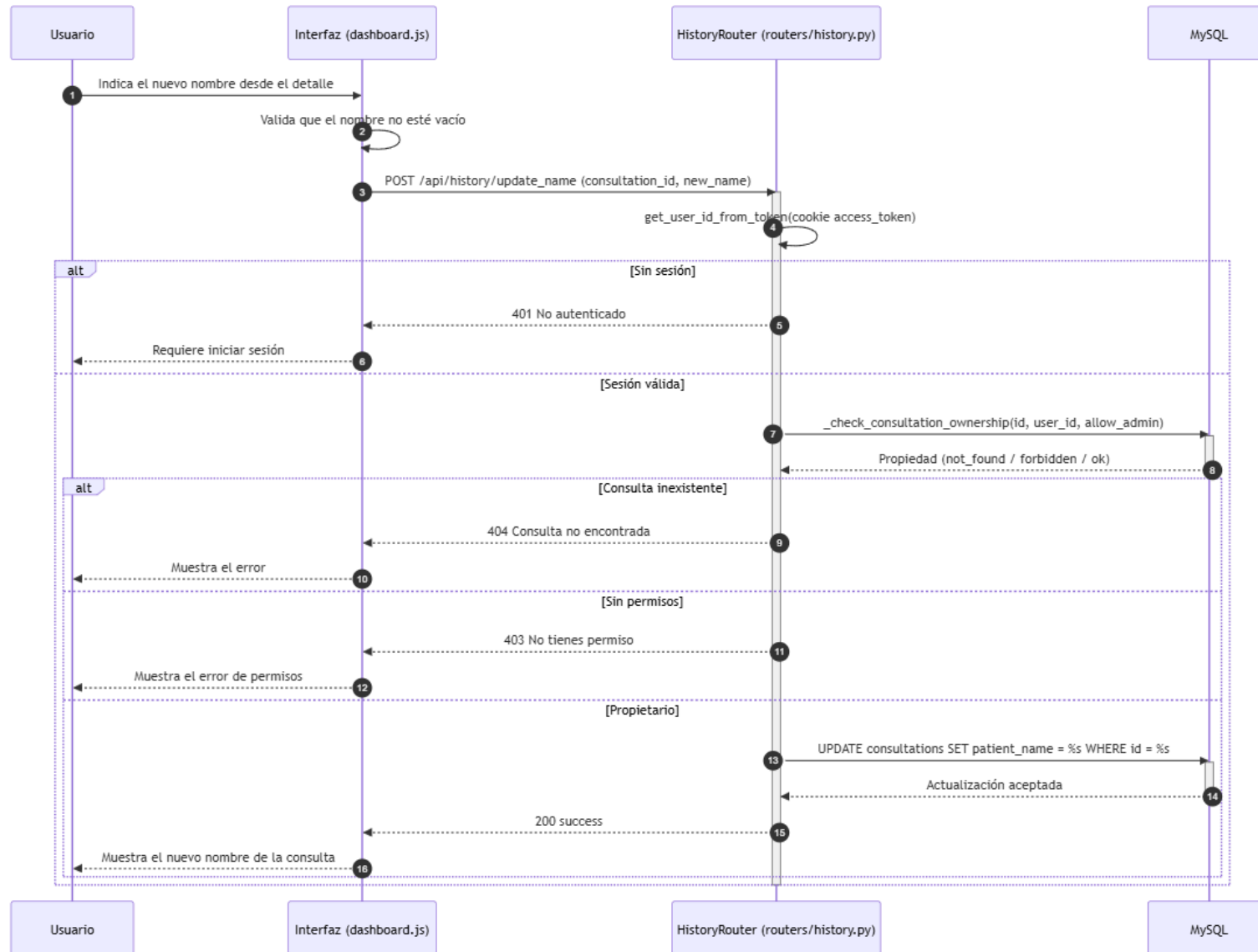


Ilustración x - Diagrama de secuencia del CU-013 Renombrar una consulta del historial

CU-014 Eliminar una consulta del historial

El diagrama de la figura muestra la eliminación. El usuario solicita eliminar la consulta desde el detalle y la interfaz pide confirmación; únicamente tras la confirmación se envía la petición al router. El router resuelve la identidad y comprueba la propiedad; con propiedad confirmada, ejecuta el borrado físico y confirma la transacción, y la interfaz cierra el detalle y refresca el historial. La secuencia refleja tanto la confirmación previa del análisis como el control de propiedad previo a la destrucción del registro.

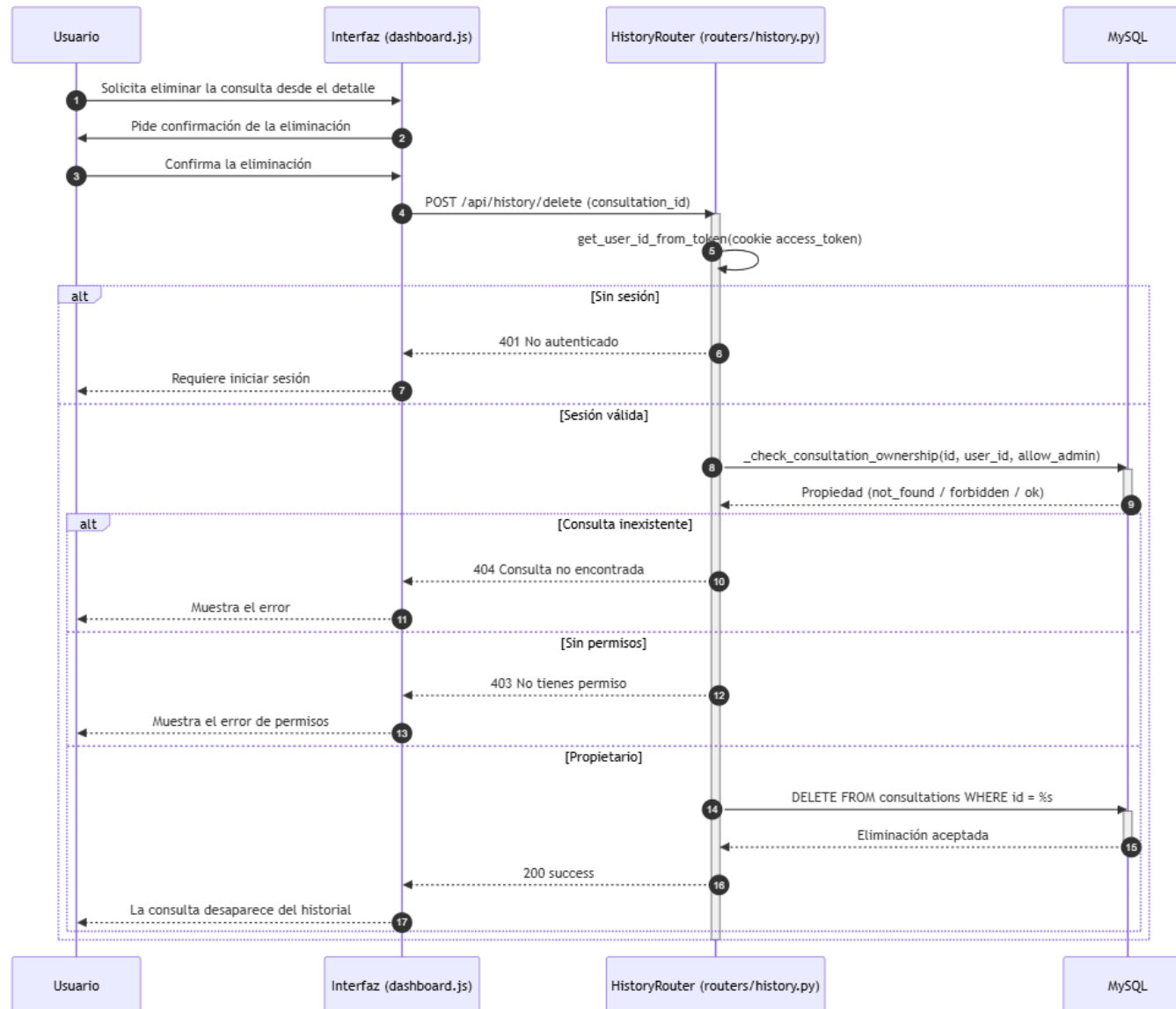


Ilustración x - Diagrama de secuencia del CU-014 Eliminar una consulta del historial

4.4 Subsistema de diseño SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 corresponde al subsistema de análisis SS-004 y constituye el bloque de mayor complejidad funcional de la plataforma. Gestiona la configuración conversacional del experimento, la selección del dataset, el entrenamiento de los modelos, la ejecución de los análisis de explicabilidad, la comparación estadística, la validación externa, la consulta de resultados y la generación de informes. Agrupa los casos de uso CU-015 a CU-030 y CU-039, que materializan los diecisiete requisitos del laboratorio: la configuración y el lanzamiento de los experimentos, la consulta de las sesiones y de sus resultados, el análisis XAI, la comparación estadística, la validación externa, la generación de informes y la gestión de las sesiones.

El subsistema se apoya en `router/trainer.py`, que actúa como fachada ligera, y en un conjunto de servicios especializados: `services/chatbot_service.py`, que resuelve la configuración conversacional del experimento mediante el asistente externo; `services/mlops_engine.py`, que organiza las sesiones del laboratorio y resuelve la lectura y escritura de los resultados en el directorio `training_results`; los scripts de entrenamiento de `pneumoniaccnn-main/code`, que ejecutan el pipeline de preparación, entrenamiento, análisis XAI y comparación estadística; y `services/pdf_generator_mlops.py`, que genera los informes consolidados de la sesión. La ejecución de las tareas de larga duración se delega en la cola de trabajos y en su worker, de modo que el laboratorio conserva una interfaz interactiva aunque el entrenamiento tarde mucho más que una petición HTTP normal.

El diseño de SD-004 introduce dos decisiones que condicionan su arquitectura. En primer lugar, la persistencia es híbrida: MySQL conserva la cola de trabajos y el estado, mientras que el sistema de ficheros alberga los artefactos de cada sesión, de modo que el identificador de la sesión se conserva tanto en la fila del trabajo como en el directorio de resultados. En segundo lugar, existen dos canales de ejecución diferenciados: el entrenamiento y la validación externa se procesan a través de la cola de trabajos, mientras que la comparación estadística se programa con las tareas en segundo plano de FastAPI (`BackgroundTasks`). La regla central del subsistema es la propiedad de las sesiones: antes de mostrar resultados, eliminar o renombrar una sesión, el router invoca la comprobación de propiedad de `mlops_engine`, con la excepción controlada del administrador cuando la ruta lo permite.

4.4.1 Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-004 recoge las diecisiete interacciones que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de laboratorio de experimentación MLOps definido en el análisis (figura 5), limitada al ámbito del subsistema de diseño. El diagrama conserva las relaciones del análisis: la inclusión de la configuración conversacional en el lanzamiento del experimento (CU-018 incluye CU-016, porque la ejecución no comienza hasta que el router dispone de una configuración completa y válida), y las extensiones que ordenan la navegación entre la consulta de sesiones y las vistas de resultados, ranking, comparativa e informe. El diagrama se representa a continuación.

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los profesionales con cuenta registrada. La estructura de los casos de uso refleja el ciclo de vida del laboratorio. La entrada se compone de CU-015 a CU-018: el acceso al laboratorio, la configuración conversacional con el asistente, la selección de la carpeta del dataset y el lanzamiento del experimento, que incorpora la configuración conversacional como paso obligatorio mediante la inclusión. La consulta se organiza desde las sesiones (CU-019), desde las que se accede de forma opcional a los resultados de un modelo (CU-020), al ranking (CU-022), a la comparativa estadística (CU-023) y al informe PDF (CU-028); desde los resultados de un modelo se alcanzan sus mapas de explicabilidad (CU-021), y desde la comparativa se solicita su recálculo (CU-024). El resto de operaciones —la ejecución del análisis XAI (CU-025), la validación externa (CU-026 y CU-027), el renombrado y la eliminación de sesiones (CU-029 y CU-030) y la comprobación de la limitación de entrenamientos (CU-039)— son interacciones directas e independientes del actor con el sistema.

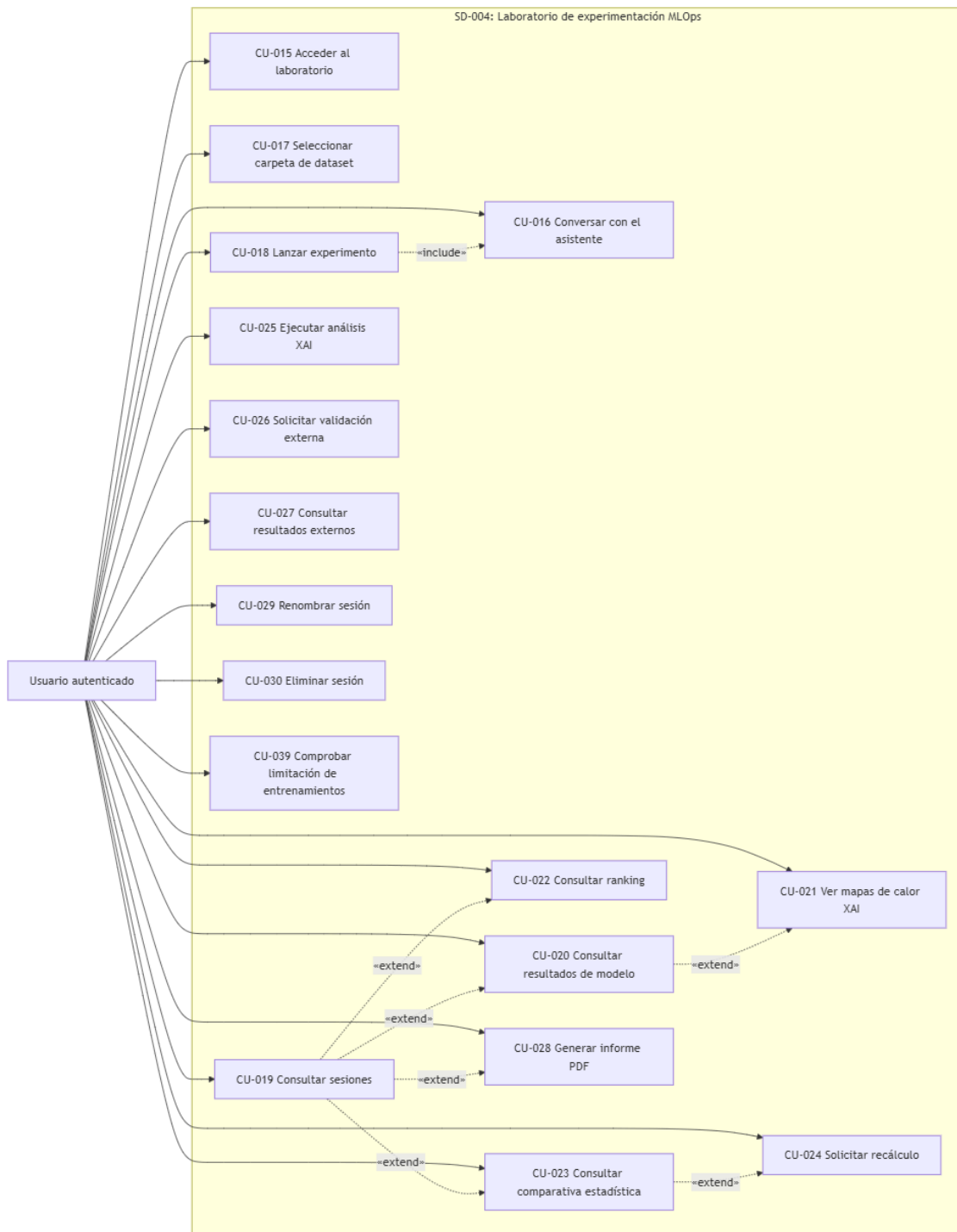


Ilustración x - Diagrama de casos de uso del subsistema SD-004

4.4.2 Casos de uso reales del subsistema

Los casos de uso reales de SD-004 concretan cada interacción del diagrama en decisiones técnicas de implementación. El subsistema combina operaciones síncronas de configuración y consulta con operaciones asíncronas de ejecución, repartidas entre la cola de trabajos y las tareas en segundo plano. Para facilitar la lectura, los casos de uso reales se presentan agrupados en cuatro bloques: la entrada del laboratorio, la consulta de resultados, el análisis y la validación, y la gestión e informes.

CU-015 Acceder al laboratorio de entrenamiento

El acceso al laboratorio se materializa en el endpoint GET /training del router routers/auth.py, que aplica la misma política de sesión que el panel de diagnóstico. Las decisiones técnicas del caso de uso real son las siguientes:

- **Verificación de la sesión:** el router obtiene el identificador del usuario mediante `get_user_id_from_token()` de SD-001; sin sesión válida, responde con una redirección HTTP 303 a la página de inicio de sesión.
- **Carga del perfil:** con el identificador resuelto, se consultan el nombre y el rol del usuario en la tabla users para personalizar la vista del laboratorio.
- **Control de caché:** la respuesta incluye las cabeceras Cache-Control: no-store, de modo que la página del laboratorio no se sirve desde caché tras un cierre de sesión.

CU-016 Conversar con el asistente para configurar un experimento

La configuración conversacional se materializa en el endpoint POST /api/chat, que delega en services/chatbot_service.py. El diseño trata al proveedor del asistente como una frontera externa: los errores del proveedor no se confunden con los de persistencia o entrenamiento. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comunicación con el asistente:** el servicio envía el mensaje del usuario al asistente externo (Groq) y devuelve una configuración estructurada del experimento, solicitando los parámetros que falten (arquitecturas, épocas, lote, tasa de aprendizaje).
- **Frontera externa:** el único componente que necesita la clave de la API del asistente es chatbot_service.py; los routers no la incluyen en las respuestas ni en los registros.
- **Localización de los prompts:** el servicio selecciona el prompt del asistente según el idioma de la sesión, en coherencia con el mecanismo de internacionalización de SD-001.
- **Condición del lanzamiento:** la configuración conversacional constituye el paso obligatorio que el lanzamiento del experimento (CU-018) incluye, de modo que la ejecución no comienza hasta que la configuración es completa y válida.

CU-017 Seleccionar la carpeta del dataset

La selección de la carpeta del dataset se materializa en el endpoint GET /api/train/browse, que delega en la exploración de mlops_engine. El diseño confina la selección dentro del directorio raíz permitido, evitando que la ruta escape del ámbito de datos del sistema. Las decisiones técnicas del caso de uso real son las siguientes:

- **Exploración del sistema de ficheros:** la función browse_folder() devuelve la ruta del dataset configurada mediante la variable de entorno TFG_DEMO_DATASET (o TFG_DEMO_EXTERNAL_DATASET para la validación externa) o, en su defecto, abre el selector de carpetas del sistema.
- **Selección confinada:** la ruta seleccionada se restringe al directorio permitido, de modo que el usuario no puede indicar una ruta arbitraria fuera del ámbito de datos.
- **Respuesta al cliente:** el router devuelve la ruta seleccionada para que la interfaz la inserte en la configuración del experimento, o responde HTTP 500 si la exploración no pudo completarse.

CU-018 Lanzar un experimento de entrenamiento

El lanzamiento del experimento se materializa en el endpoint POST /api/train/start, que crea la sesión de entrenamiento y encola su ejecución asíncrona. El diseño exige una configuración completa y válida antes de encolar, en coherencia con la inclusión de la configuración conversacional (CU-016). Las decisiones técnicas del caso de uso real son las siguientes:

- **Validación de la ruta del dataset:** el router comprueba que la ruta del dataset exista en el sistema de ficheros; en caso contrario, responde HTTP 400 con el mensaje de ruta inexistente y no crea la sesión.
- **Creación de la sesión:** create_training_session() crea el directorio de la sesión en training_results y escribe el config.json con la configuración del experimento, los modelos, la ruta del dataset y el identificador del usuario propietario.
- **Encolado del entrenamiento:** el router inserta un registro en job_queue con el tipo training y un payload JSON serializable que contiene el identificador de la sesión, los modelos, la ruta del dataset y los hiperparámetros; ni el modelo ni los datos se serializan en la base de datos.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el estado queued, el identificador del trabajo y el de la sesión, de modo que la interfaz informa del encolado y comienza a consultar el estado.

CU-039 Comprobar la limitación de entrenamientos simultáneos y encolados

La limitación de los entrenamientos se materializa en el mecanismo de ejecución de la cola de trabajos, que procesa un único trabajo de entrenamiento cada vez y prioriza de forma diferenciada los tipos de trabajo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Procesamiento secuencial:** el worker reclama el primer trabajo queued mediante una actualización condicional que solo tiene efecto si el trabajo seguía en cola, de modo que dos instancias no pueden ejecutar simultáneamente el mismo entrenamiento.
- **Prioridad de la cola:** la selección del siguiente trabajo ordena por tipo, de modo que el entrenamiento no compite por la misma prioridad que el diagnóstico; la interfaz de la cola refleja la posición de cada trabajo y los recuentos de pendientes.
- **Consulta del estado:** el endpoint de estado de la cola devuelve los trabajos del usuario con su estado y posición, de modo que el usuario puede comprobar el encolado y la progresión de sus experimentos.

CU-019 Consultar las sesiones de entrenamiento

La consulta de sesiones se materializa en el endpoint GET /api/train/models, que devuelve únicamente las sesiones del usuario autenticado. El diseño aplica el aislamiento de datos en el propio motor: la enumeración de las sesiones filtra por propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Enumeración de sesiones:** get_trained_sessions() lista los directorios de training_results que contienen al menos un modelo con resultados, en orden descendente, y filtra por el identificador del usuario mediante la configuración de cada sesión.
- **Aislamiento de datos:** cada usuario recibe únicamente sus propias sesiones, en coherencia con el requisito de aislamiento de datos entre cuentas (RF-005).
- **Respuesta al cliente:** el router devuelve la lista de sesiones con sus modelos, y la interfaz las muestra en el panel lateral del laboratorio con su fecha de creación.

CU-020 Consultar los resultados de un modelo de la sesión

La consulta de los resultados de un modelo se materializa en el endpoint GET /api/train/results/{session_id}/{model_name}, precedida de la comprobación de propiedad de la sesión. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad:** el router invoca _require_ownership(), que delega en la verificación de mlops_engine; si la sesión no pertenece al usuario, responde HTTP 403 con el mensaje de permiso denegado.
- **Lectura de resultados:** get_model_results_data() lee las métricas de validación cruzada del kfold_results.csv del modelo, las métricas de calibración, las métricas XAI cuantitativas y las rutas de los artefactos visuales.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 si el modelo no dispone de resultados y HTTP 200 con las métricas y las rutas de los artefactos cuando la consulta es correcta.
- **Servicio de artefactos:** las imágenes del modelo se sirven mediante las rutas estáticas del directorio training_results, sin un endpoint de cálculo adicional.

CU-021 Visualizar los mapas de calor de explicabilidad de un modelo

La visualización de los mapas de calor se materializa en la galería de imágenes que la consulta de resultados devuelve, correspondientes al análisis XAI cualitativo del modelo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Artefactos de la sesión:** los mapas de calor se generan durante el entrenamiento o mediante la ejecución manual del análisis XAI, y se conservan en el directorio del modelo con el prefijo xai_example_.
- **Presentación:** la interfaz muestra la galería de mapas de calor sobre imágenes de ejemplo, servidos desde el almacenamiento estático de training_results.
- **Origen de los datos:** la visualización no repite el análisis; recupera los artefactos ya persistidos por el pipeline de explicabilidad.

CU-022 Consultar el ranking de modelos de la sesión

La consulta del ranking se materializa en el endpoint GET /api/train/session/{session_id}/ranking, precedida de la comprobación de propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Lectura del ranking:** get_session_ranking_data() lee el session_ranking.csv, que ordena los modelos por la media del AUC de la validación cruzada con su desviación típica, junto con la configuración de la sesión y el mapa de calor del test de Wilcoxon.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 con el mensaje de sesión no comparada si la sesión aún no dispone de ranking, y HTTP 200 con el ranking, el heatmap y la configuración cuando existe.
- **Presentación:** la interfaz muestra la tabla de ranking con los mejores modelos y la matriz de significancia estadística.

CU-023 Consultar la comparativa estadística de la sesión

La consulta de la comparativa estadística se materializa en el mismo endpoint del ranking, que devuelve la matriz de significación junto con el ranking. Las decisiones técnicas del caso de uso real son las siguientes:

- **Matriz de significación:** la comparativa incluye los p-valores del test de Wilcoxon entre los modelos y, si la sesión dispone de validación externa, los p-valores del test de DeLong sobre las curvas ROC.
- **Artefacto visual:** el heatmap de significancia se sirve como imagen estática junto con los datos del ranking.
- **Condición de disponibilidad:** la comparativa solo existe cuando el pipeline de estadística se ha ejecutado sobre la sesión; en caso contrario, la consulta responde con la condición de sesión no comparada.

CU-024 Solicitar el recálculo de la comparativa estadística

El recálculo se materializa en el endpoint POST /api/train/session/compare, que programa la comparación estadística en segundo plano mediante las tareas de fondo de FastAPI, en lugar de la cola de trabajos. Esta decisión diferencia el canal de ejecución de la comparativa del canal del entrenamiento. Las decisiones técnicas del caso de uso real son las siguientes:

- **Programación asíncrona:** el router registra run_statistical_comparison() como tarea de fondo, de modo que el recálculo se ejecuta sin bloquear la petición y sin ocupar la cola de trabajos.
- **Regeneración de resultados:** la tarea ejecuta el script de estadística que regenera el ranking y la matriz de Wilcoxon, y registra la finalización mediante un marcador de estado en la sesión.
- **Consulta del estado:** la interfaz sondea el endpoint de estado del recálculo, que devuelve running o completed, y recarga la vista cuando la comparativa queda regenerada.

CU-025 Ejecutar el análisis de explicabilidad de un modelo

La ejecución del análisis XAI se materializa en el endpoint POST /api/train/run_eval, que permite regenerar los artefactos de explicabilidad de un modelo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad y de ruta:** el router verifica la propiedad de la sesión y resuelve la ruta del dataset; si no existe, responde HTTP 400 con el mensaje de dataset no encontrado.
- **Ejecución del análisis:** el motor lanza los scripts de análisis XAI cualitativo y cuantitativo sobre el modelo, regenerando los mapas de calor y las métricas de fidelidad.
- **Respuesta al cliente:** el sistema responde HTTP 200 con un mensaje de generación completada, y la interfaz recarga la vista de resultados para mostrar los nuevos artefactos.

CU-026 Solicitar la validación externa de la sesión

La solicitud de validación externa se materializa en el endpoint POST /api/train/session/external_validation, que encola la evaluación sobre la cohorte externa. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobaciones previas:** el router verifica la propiedad de la sesión y que la ruta del dataset externo exista; en caso contrario, responde HTTP 403 o HTTP 400 con el mensaje de ruta externa inválida.
- **Encolado:** se inserta un registro en job_queue con el tipo external_validation y un payload con el identificador de la sesión y la ruta del dataset externo.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el estado queued y el identificador del trabajo, de modo que la interfaz informa del encolado de la validación.
- **Separación de la configuración:** la validación externa se ejecuta sobre los modelos disponibles de la sesión sin modificar la configuración original del experimento.

CU-027 Consultar los resultados de la validación externa

La consulta de los resultados externos se materializa en el endpoint `GET /api/train/session/{session_id}/external_results`, precedida de la comprobación de propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Lectura de resultados:** `get_external_results_data()` lee las métricas sobre la cohorte externa, la curva ROC y la matriz de significación de DeLong.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 con el mensaje de resultados externos no disponibles si la validación aún no ha producido resultados, y HTTP 200 con las métricas, la ROC y la matriz de DeLong cuando existen.
- **Presentación:** la interfaz muestra las métricas de la validación externa junto con los artefactos visuales de la sesión.

CU-028 Generar el informe PDF de la sesión

La generación del informe PDF se materializa en el endpoint `GET /api/train/session/{session_id}/report`, que delega en `services/pdf_generator_mlops.py`. El diseño separa el conocimiento del formato del informe del router: el generador recibe la información preparada por el motor y construye el documento. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad:** el router verifica la propiedad de la sesión antes de generar el informe, respondiendo HTTP 403 ante una sesión ajena.
- **Construcción del documento:** `generate_pdf_report()` compone un informe consolidado con la configuración del experimento, el ranking de modelos con el heatmap de Wilcoxon, los resultados de la validación externa con la ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor.
- **Conservación del informe:** el documento se guarda en el directorio de la sesión y se sirve como descarga mediante `FileResponse`.
- **Fase posterior:** el informe se genera cuando la sesión dispone de los datos necesarios; si la sesión no existe, el generador responde HTTP 404.

CU-029 Renombrar una sesión de entrenamiento

El renombrado se materializa en el endpoint `POST /api/train/session/rename`, precedido de la comprobación de propiedad y de la validación del nuevo nombre en el motor. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad:** el router responde HTTP 403 con el mensaje de permiso de renombrado si la sesión no pertenece al usuario.
- **Sanitización del nombre:** `safe_rename()` valida el nuevo nombre —caracteres alfanuméricos, espacio, guion y guion bajo— y lo rechaza con HTTP 400 si es inválido o ya existe.
- **Renombrado en disco:** el motor ejecuta el renombrado del directorio de la sesión y devuelve el resultado al router, que lo refleja en la respuesta.
- **Refresco de la vista:** la interfaz actualiza el título de la sesión y recarga el panel lateral del laboratorio.

CU-030 Eliminar una sesión de entrenamiento

La eliminación se materializa en el endpoint DELETE /api/train/session/{session_id}, precedido de la confirmación del usuario y de la comprobación de propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Confirmación previa:** la interfaz solicita la confirmación de la eliminación antes de enviar la petición; si no se confirma, la operación se abandona.
- **Comprobación de propiedad:** el router responde HTTP 403 con el mensaje de permiso de eliminación ante una sesión ajena.
- **Eliminación de los resultados:** delete_session() elimina el directorio de la sesión y sus artefactos mediante el borrado recursivo del sistema de ficheros.
- **Refresco de la vista:** la interfaz oculta los paneles de la sesión y recarga el listado, de modo que la sesión desaparece del laboratorio junto con sus resultados.

4.4.3 Diagramas de interacción entre objetos

Los diagramas de interacción de SD-004 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. El subsistema combina dos canales de ejecución: la cola de trabajos, por la que transitan el entrenamiento y la validación externa, y las tareas en segundo plano, por las que se ejecuta la comparación estadística. Todos los diagramas reflejan la persistencia híbrida —MySQL para la cola y el sistema de ficheros para los resultados— y la regla de propiedad de las sesiones, que se aplica antes de consultar, modificar o eliminar cualquier sesión. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-015 Acceder al laboratorio de entrenamiento

El diagrama de la figura muestra el acceso al laboratorio, cuya secuencia es equivalente a la del panel de diagnóstico: la interfaz solicita la página y el router consulta a SD-001 la identidad contenida en la cookie de acceso. Con sesión válida, recupera el perfil del usuario y devuelve la vista del laboratorio con el control de caché; sin sesión, redirige al punto de entrada del sistema.

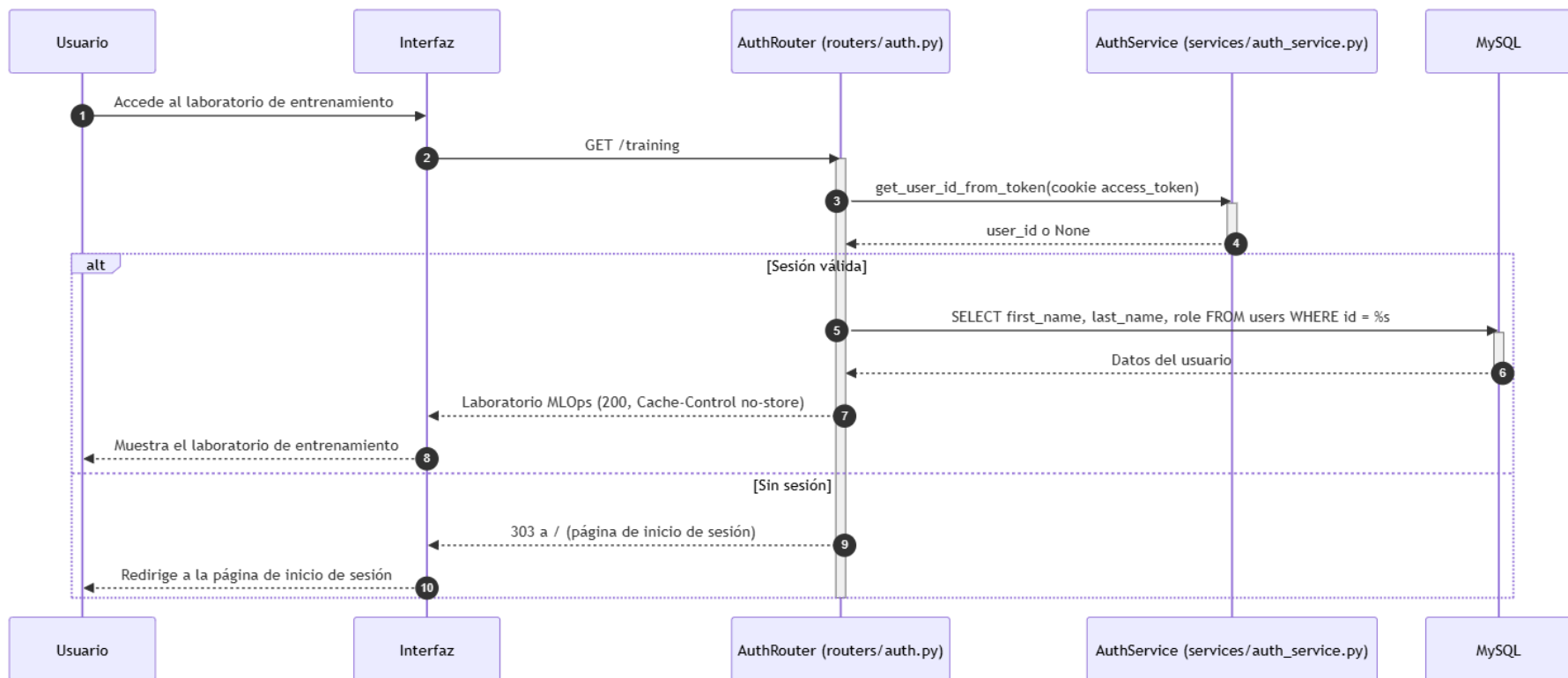


Ilustración x - Diagrama de secuencia del CU-015 Acceder al laboratorio de entrenamiento

CU-018 Lanzar un experimento de entrenamiento

El diagrama de la figura muestra el lanzamiento de un experimento, que materializa también la configuración conversacional (CU-016) y la selección de la carpeta del dataset (CU-017). El usuario describe el experimento en lenguaje natural y el servicio de chat solicita la configuración estructurada al asistente externo; a continuación selecciona la carpeta del dataset mediante la exploración del motor. Cuando la configuración está completa, el router valida la ruta del dataset, crea la sesión con su configuración y encola el entrenamiento, respondiendo el estado `queued`. La secuencia refleja la inclusión de CU-016 en CU-018: el lanzamiento solo se produce con una configuración completa y válida.

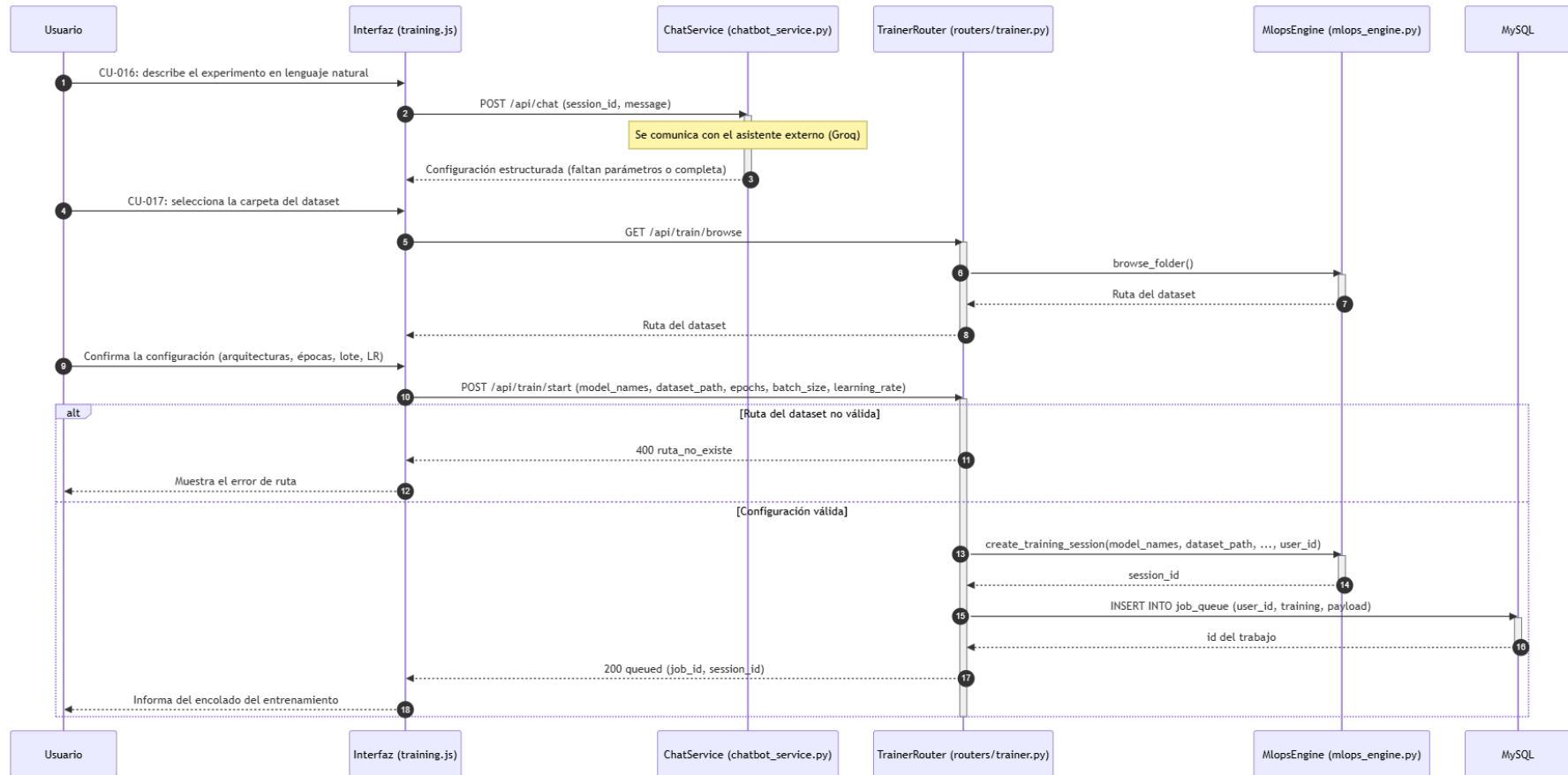


Ilustración x - Diagrama de secuencia del CU-018 Lanzar un experimento de entrenamiento

Procesamiento del entrenamiento

El diagrama de la figura muestra el procesamiento asíncrono del entrenamiento. El worker reclama el trabajo de tipo training de la cola, lo marca como running y encarga al motor el entrenamiento completo. El motor lanza secuencialmente, para cada modelo de la sesión, los scripts de entrenamiento K-fold y de análisis XAI, y finalmente el script de estadística que genera el ranking y la matriz de Wilcoxon; los scripts escriben los artefactos en el directorio de la sesión. Al terminar, el worker marca el trabajo como completed con el resultado. Si cualquier paso falla, el trabajo pasa a failed con el mensaje de error, de modo que un fallo del entrenamiento no se confunde con un estado completado.

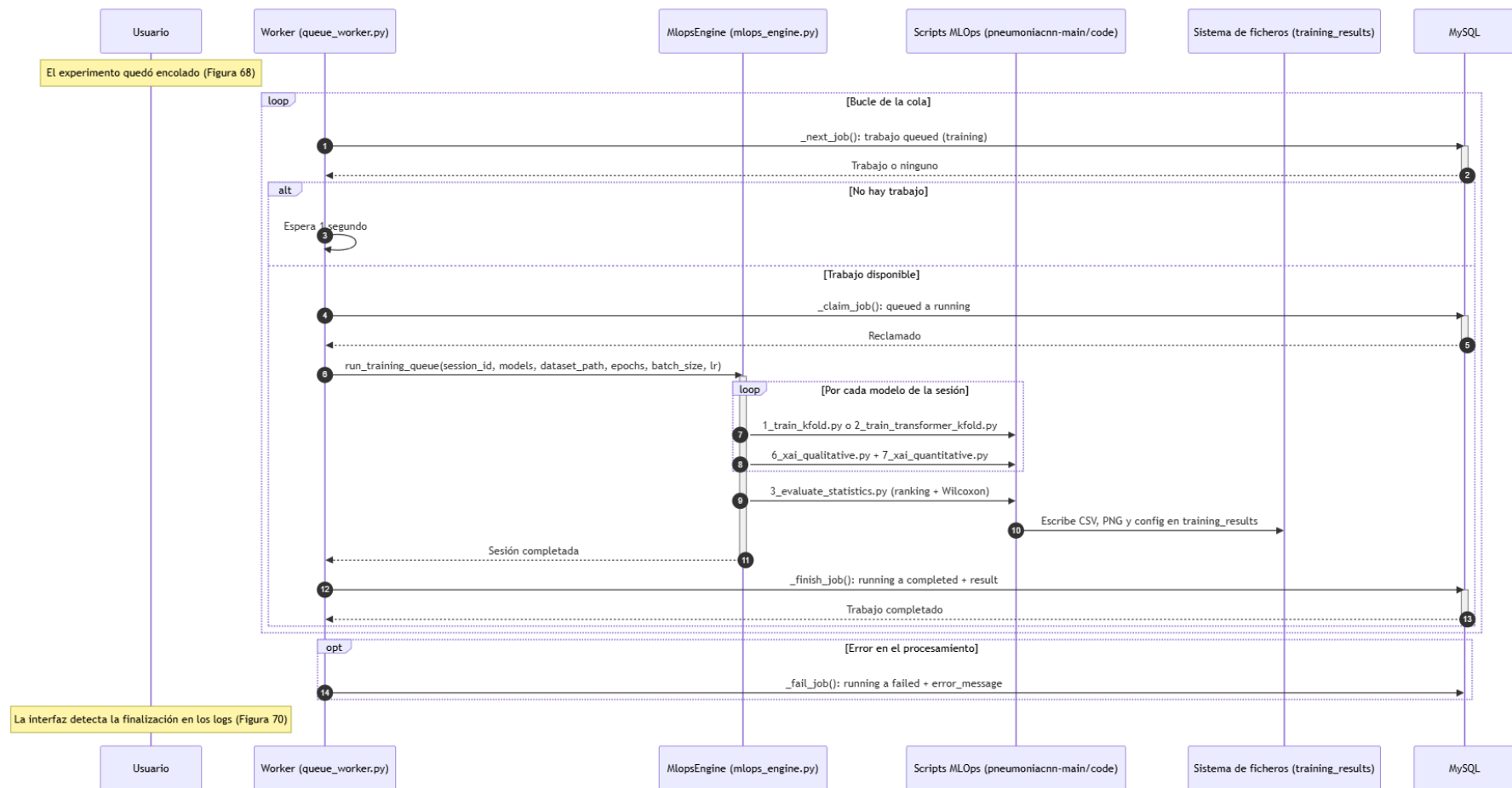


Ilustración x - Diagrama de secuencia del procesamiento del entrenamiento

CU-019, CU-020 y CU-022 Consultar sesiones y resultados

El diagrama de la figura muestra la consulta de las sesiones y de los resultados de un modelo. El usuario consulta sus sesiones y el motor enumera los directorios de training_results filtrando por propiedad. Desde el listado, el usuario consulta los resultados de un modelo: el router comprueba la propiedad y el motor lee las métricas K-fold y los artefactos del modelo, devolviendo los datos para su presentación. La secuencia refleja que la consulta no ejecuta el modelo: lee los resultados ya persistidos durante el entrenamiento.

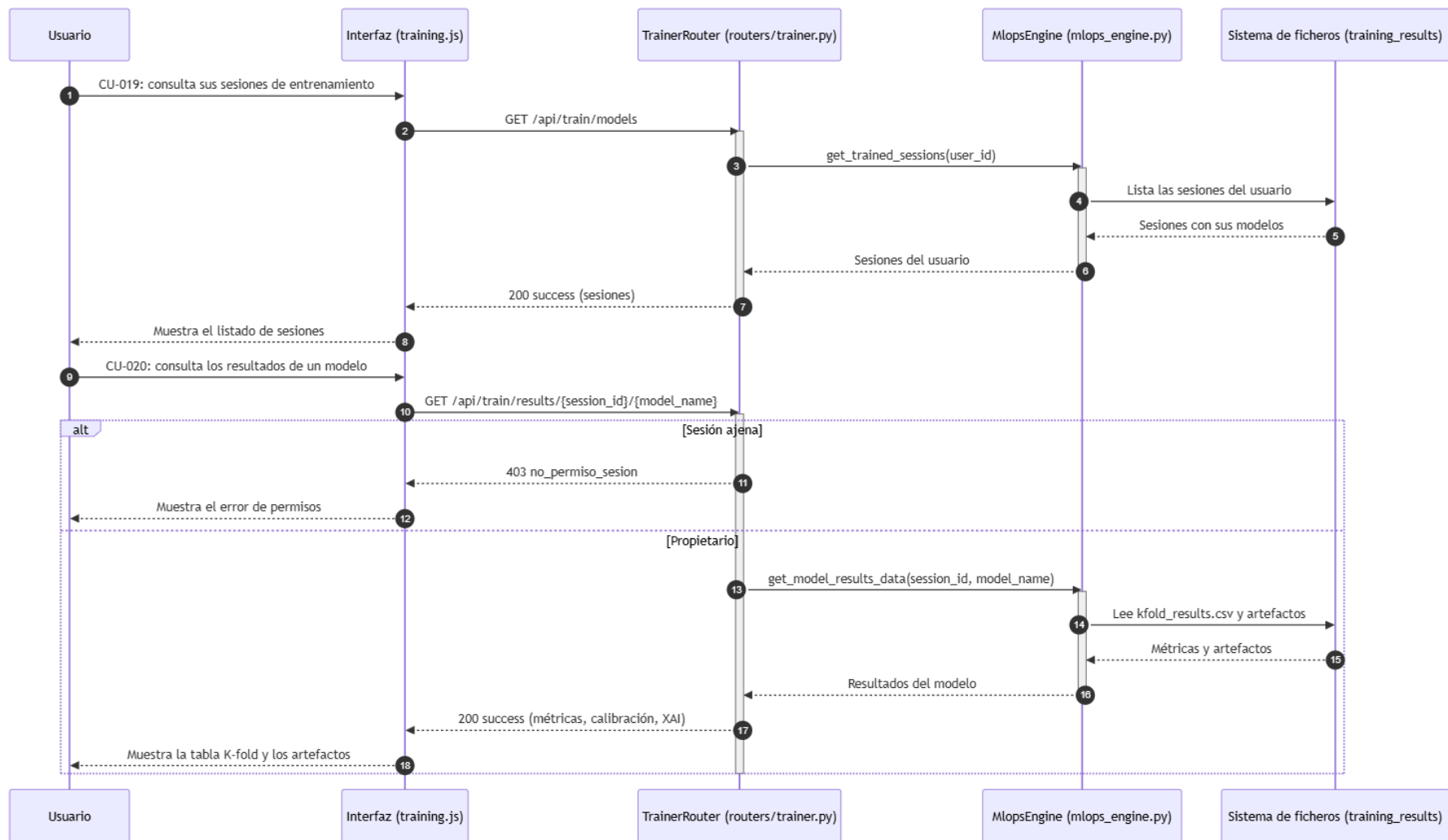


Ilustración x - Diagrama de secuencia de los CU-019, CU-020 y CU-022 Consultar sesiones y resultados

CU-023 y CU-024 Comparativa estadística y recálculo

El diagrama de la figura muestra la consulta de la comparativa estadística y su recálculo. El usuario consulta el ranking de la sesión y el motor lee el ranking y el heatmap de Wilcoxon. Cuando solicita el recálculo, el router programa la comparación estadística como tarea de fondo en lugar de encolarla, y responde el inicio del recálculo; la interfaz sondea el estado hasta que la comparativa queda regenerada. La secuencia refleja el canal de ejecución diferenciado de la comparativa, distinto de la cola de trabajos del entrenamiento.

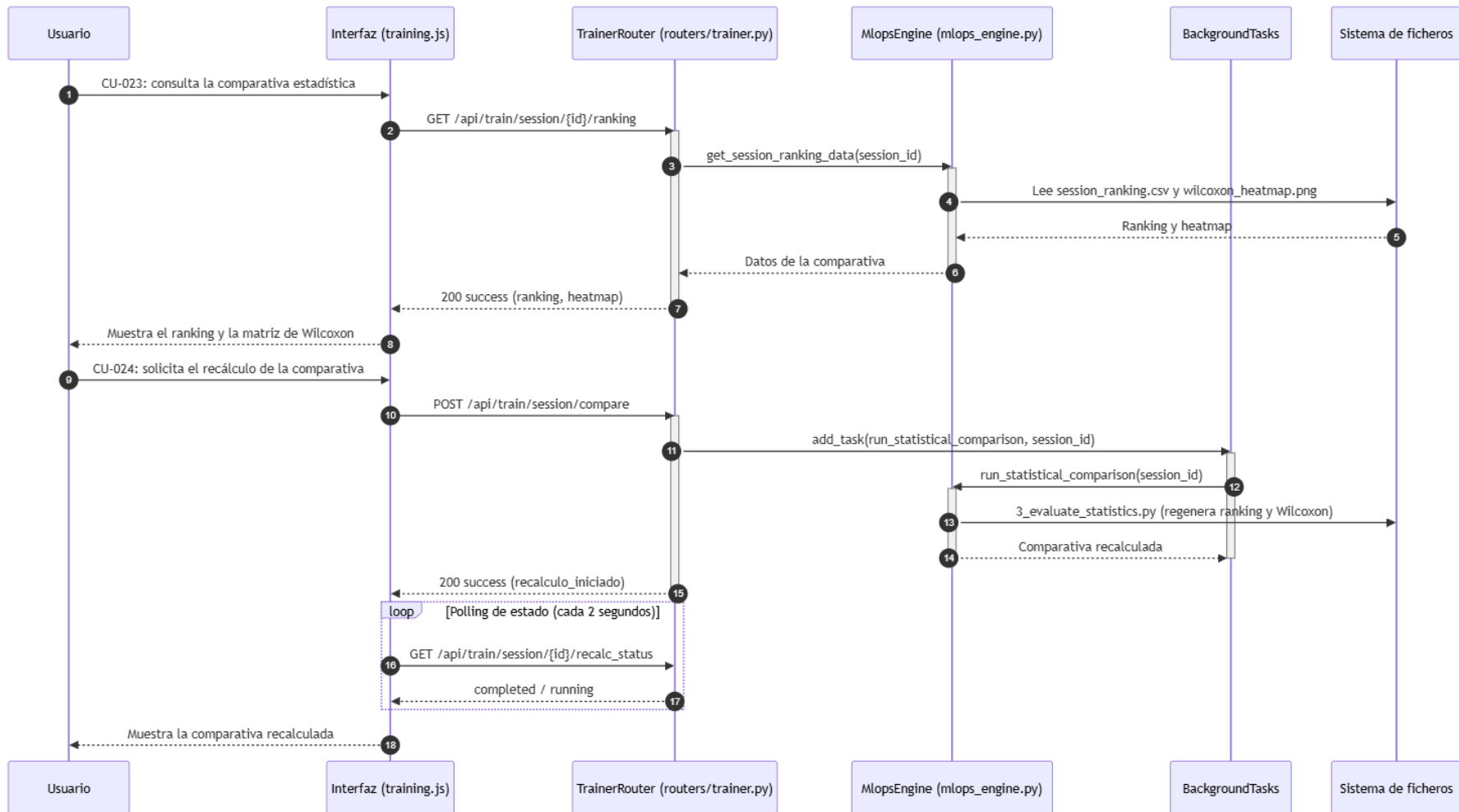


Ilustración x - Diagrama de secuencia de los CU-023 y CU-024 Comparativa estadística y recálculo

CU-026 y CU-027 Validación externa

El diagrama de la figura muestra la solicitud de la validación externa y la consulta de sus resultados. El usuario solicita la validación con la ruta del dataset externo; el router valida la propiedad y la ruta y encola el trabajo de tipo `external_validation`. El worker reclama el trabajo y el motor ejecuta la validación sobre los modelos disponibles, escribiendo las métricas, la curva ROC y la matriz de DeLong. Cuando la validación termina, el usuario consulta los resultados: el router comprueba la propiedad y el motor lee los artefactos de la validación externa. La secuencia refleja la separación de la validación externa respecto de la configuración original del experimento.

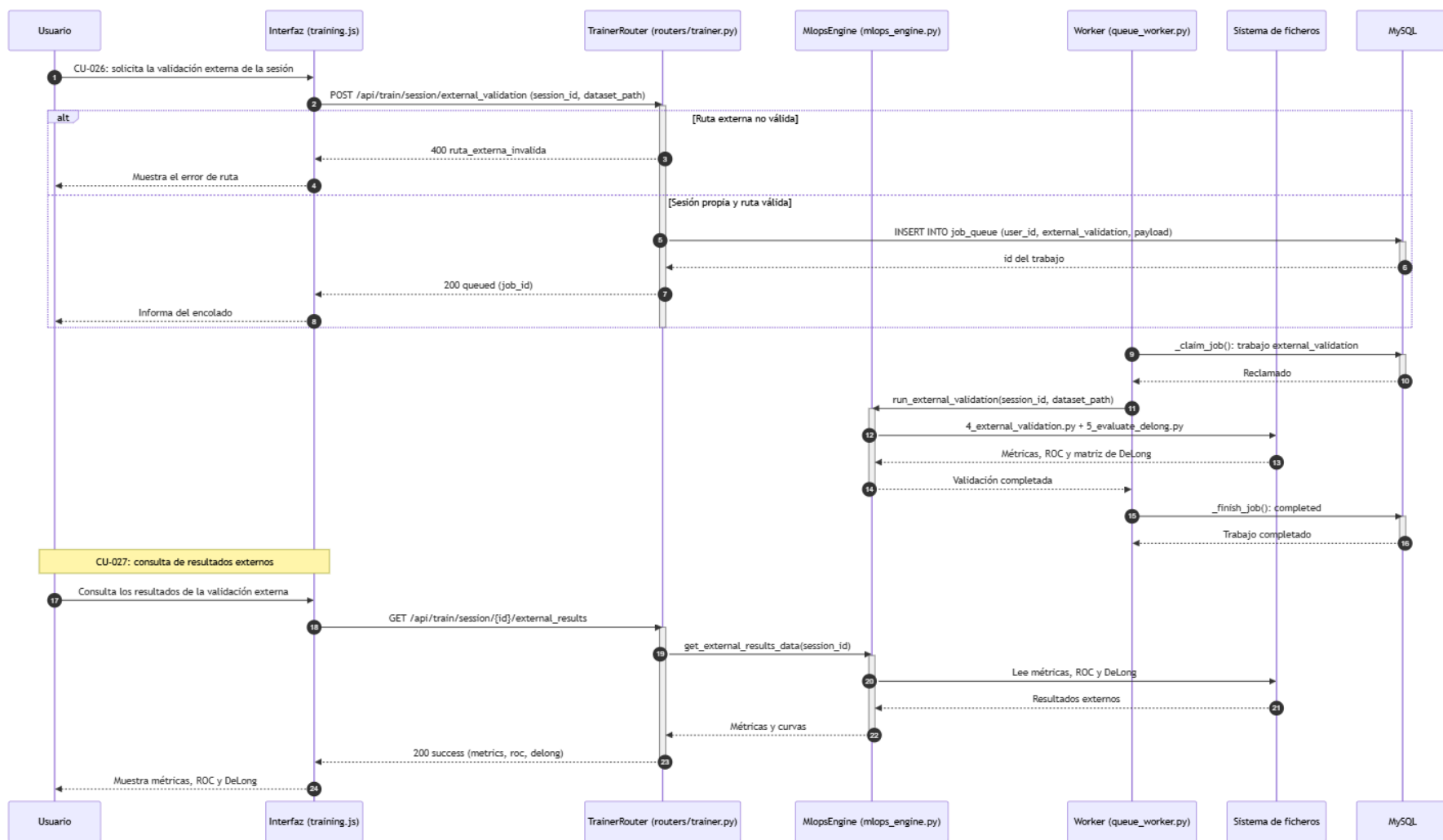


Ilustración x - Diagrama de secuencia de los CU-026 y CU-027 Validación externa

CU-028 Generar el informe PDF de la sesión

El diagrama de la figura muestra la generación y descarga del informe PDF. El usuario solicita el informe de la sesión y el router verifica la propiedad; el generador de informes lee la configuración, el ranking, el heatmap de Wilcoxon, la validación externa y las métricas XAI de cada modelo desde el directorio de la sesión, compone el documento y lo escribe como artefacto. El router sirve el PDF como descarga. La secuencia refleja que el informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios.

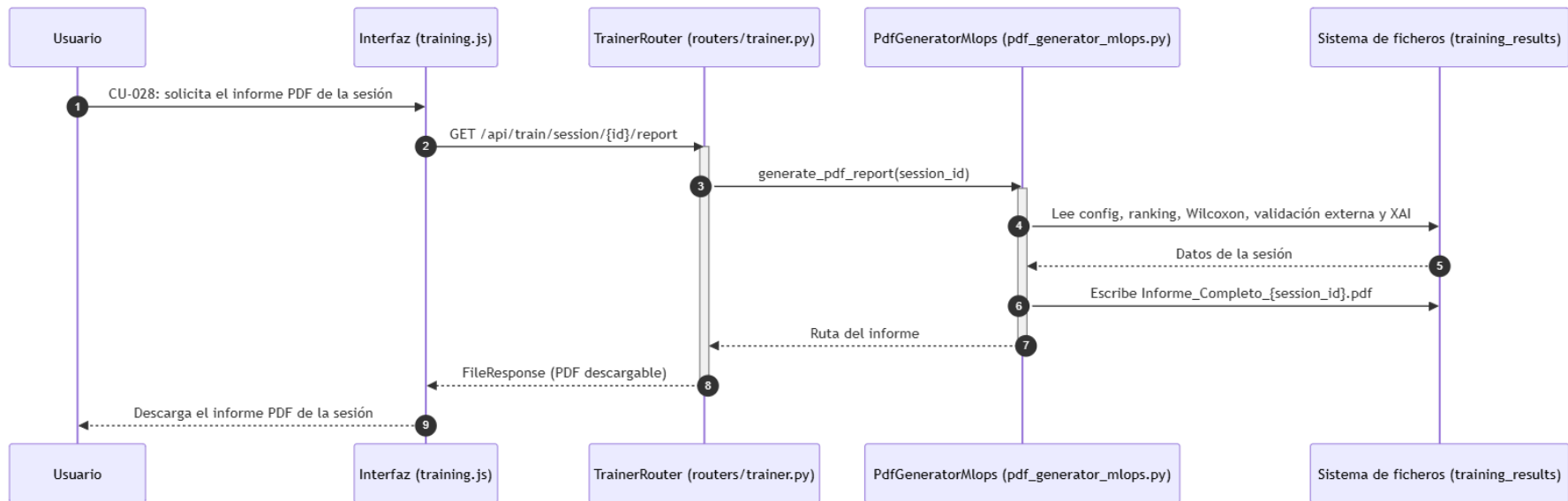


Ilustración x - Diagrama de secuencia del CU-028 Generar el informe PDF de la sesión

CU-029 y CU-030 Renombrar y eliminar una sesión

El diagrama de la figura muestra el renombrado y la eliminación de una sesión. Para renombrar, el usuario indica el nuevo nombre, el router comprueba la propiedad y el motor valida y ejecuta el renombrado del directorio. Para eliminar, la interfaz pide confirmación y, tras ella, el router comprueba la propiedad y el motor borra el directorio de la sesión y sus artefactos. La secuencia refleja que ambas operaciones quedan condicionadas a la propiedad de la sesión, que se resuelve antes de cualquier modificación del sistema de ficheros.

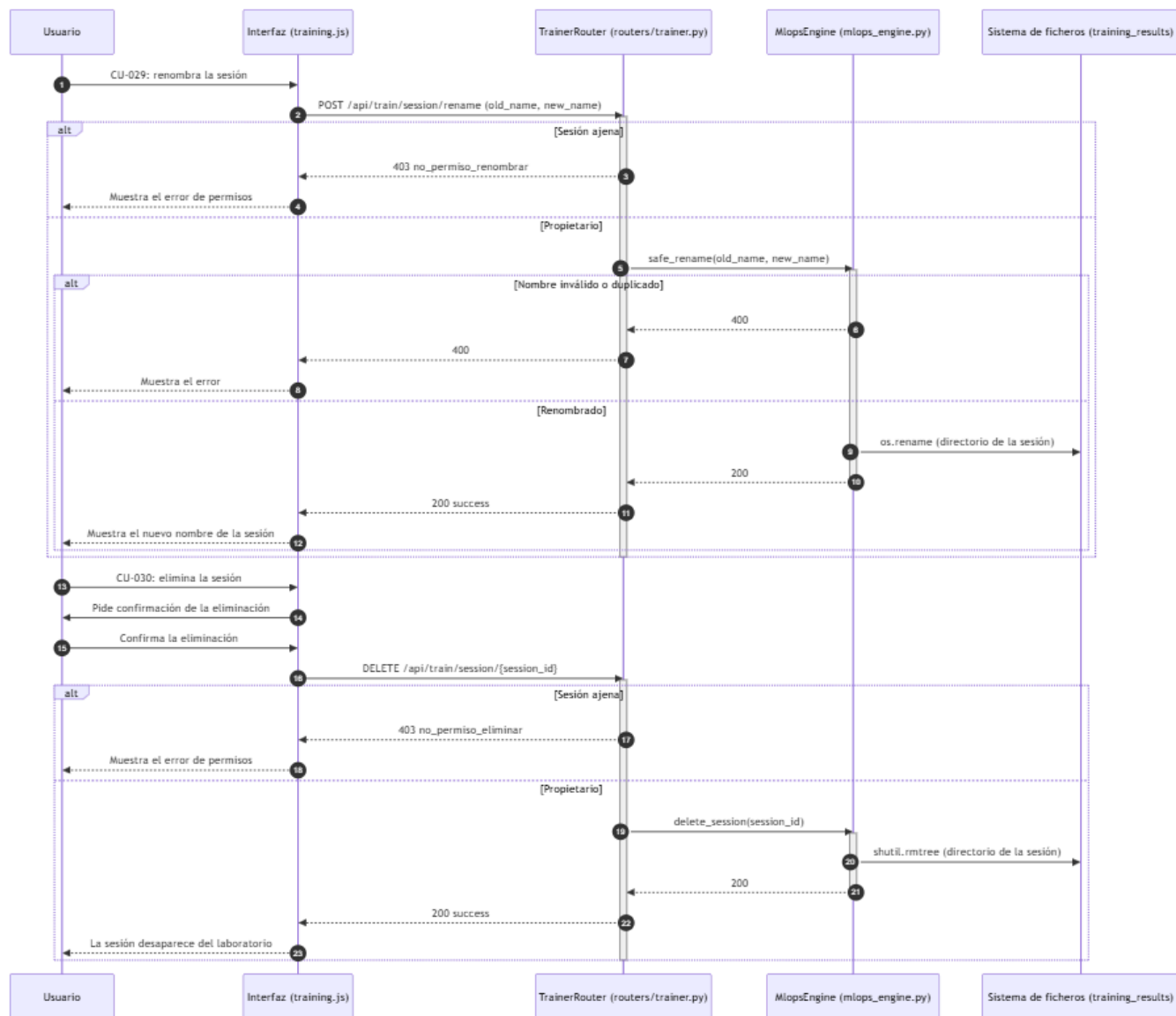


Ilustración x - Diagrama de secuencia de los CU-029 y CU-030 Renombrar y eliminar una sesión

4.5 Subsistema de diseño SD-005: Supervisión y administración

El subsistema SD-005 deriva del subsistema de análisis SS-005 y reúne las operaciones que solo puede realizar un usuario con rol de administrador. Su objetivo es proporcionar una visión supervisada del uso de la plataforma sin mezclar las operaciones administrativas con las rutas que utiliza un usuario ordinario. Agrupa los casos de uso CU-031 a CU-033, que materializan la consulta del listado de usuarios, la consulta de las consultas de un usuario concreto y la visualización del detalle de una de esas consultas. El análisis asignó a SS-005 también la gestión de cuentas de usuario (CU-038), pero esa operación no se materializa en el diseño actual: la responsabilidad presente de SD-005 es de consulta y supervisión, tal y como se declaró en el capítulo 18, de modo que no se describen aquí casos de uso reales para una funcionalidad no implementada.

El subsistema se apoya en `routers/admin.py`, que centraliza la comprobación de permisos mediante `_require_admin()`. La función obtiene primero la identidad desde el token de acceso de SD-001 y consulta el rol en la tabla `users`, devolviendo en la práctica un resultado ternario: no hay identidad, existe identidad sin privilegios o existe identidad con rol de administrador. Cada endpoint interpreta ese resultado antes de abrir la consulta global correspondiente, de modo que una ruta administrativa nunca ejecuta una consulta de usuarios o de consultas antes de haber comprobado el permiso. El router diferencia así una petición no autenticada (HTTP 401), una petición autenticada sin permisos (HTTP 403) y una petición administrativa válida.

SD-005 no duplica la lógica de propiedad del historial ni del laboratorio. Su responsabilidad es establecer la autorización administrativa y coordinar las consultas globales; la lectura de los datos sigue utilizando las mismas tablas y servicios que el resto de la aplicación. La vista global combina además las dos fuentes de persistencia de la plataforma: los usuarios y el número de diagnósticos se obtienen mediante una consulta agregada sobre `users` y `consultations`, mientras que el número de sesiones del laboratorio se calcula inspeccionando las configuraciones disponibles en el directorio `training_results`. La protección del subsistema reutiliza la identidad de SD-001 y no implementa una segunda autenticación ni una gestión de tokens propia, y el registro de auditoría administrativa que el análisis contempla (RNF-006) debe considerarse una condición pendiente del modelo operativo, porque el subsistema no escribe actualmente una traza independiente en cada operación.

4.5.1 Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-005 recoge las tres interacciones de supervisión que el subsistema pone a disposición del administrador, y es una adaptación del diagrama del módulo de supervisión y administración definido en el análisis (figura 6), limitada a los casos de uso que se materializan en el diseño. El diagrama conserva las relaciones de extensión de la cadena de navegación del análisis: desde el listado de usuarios (CU-031) se accede a las consultas de un usuario (CU-032), y desde ellas al detalle completo de una consulta (CU-033), de modo que el administrador puede profundizar progresivamente en la información que necesita. La gestión de cuentas (CU-038), que el análisis representaba como operación independiente, no se dibuja aquí por ser una condición pendiente del diseño. El diagrama se representa a continuación.

El diagrama distingue un único actor, el administrador, que es un usuario autenticado con el rol de administración. Los tres casos de uso se encadenan mediante relaciones de extensión que reflejan la navegación del administrador desde el listado general hasta el caso concreto: partiendo de la visión global de las cuentas, puede descender a la actividad de una cuenta y, si lo necesita, al detalle de una consulta individual con su imagen, su resultado y sus metadatos. Esta progresión, idéntica a la del análisis, se conserva en el diseño porque las tres operaciones comparten la misma política de autorización administrativa y la misma frontera de consulta, sin operaciones de modificación general.

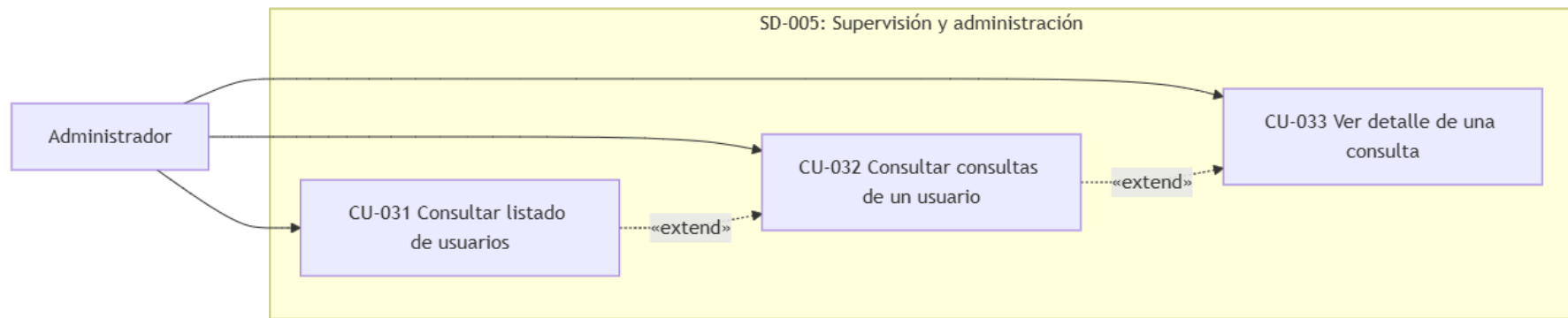


Ilustración x - Diagrama de casos de uso del subsistema SD-005

4.5.2 Casos de uso reales del subsistema

Los casos de uso reales de SD-005 concretan cada interacción del diagrama en decisiones técnicas de implementación. El elemento común de los tres es la comprobación de permisos que abre cada endpoint: ninguna consulta administrativa se ejecuta sin haber resuelto antes la identidad y el rol del solicitante. Cada caso de uso real se describe a continuación.

CU-031 Consultar el listado de usuarios

La consulta del listado se materializa en el endpoint GET /api/admin/users, que combina la información relacional de los usuarios con la información del sistema de ficheros sobre las sesiones del laboratorio. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de permisos:** el router invoca `_require_admin()`, que resuelve la identidad con SD-001 y consulta el rol en la tabla users. Sin identidad responde HTTP 401, y con identidad sin rol de administrador responde HTTP 403; el listado solo se abre con rol administrativo.
- **Consulta agregada:** el router ejecuta una consulta que combina users con consultations mediante una unión izquierda, obteniendo el número de diagnósticos de cada usuario mediante un recuento distinto sobre las consultas, y ordena el resultado por nombre de usuario.
- **Conteo de sesiones del laboratorio:** el router inspecciona el directorio training_results y, a partir de las configuraciones de cada sesión, calcula el número de sesiones por usuario, incorporándolo a cada fila del listado. Esta decisión refleja la persistencia híbrida de la plataforma: la información relacional se consulta en MySQL y parte de los artefactos de las sesiones se conserva en el sistema de ficheros.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el listado de usuarios, sus recuentos de diagnósticos y de sesiones del laboratorio, y la interfaz administrativa lo presenta como el punto de partida de la supervisión.

CU-032 Consultar las consultas de un usuario

La consulta de las consultas de un usuario se materializa en el endpoint GET `/api/admin/users/{user_id}/consultations`, que recupera la actividad clínica de una cuenta concreta junto con sus sesiones del laboratorio. El diseño conserva la granularidad de la interfaz administrativa: una ruta por cada nivel de supervisión. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de permisos:** el router aplica la misma `_require_admin()` de CU-031, con las mismas respuestas HTTP 401 y 403.
- **Existencia del usuario objetivo:** antes de recuperar la actividad, el router comprueba que el usuario consultado exista; en caso contrario, responde HTTP 404 con el mensaje de usuario no encontrado.
- **Recuperación de las consultas:** el router selecciona las consultas del usuario desde `consultations`, filtradas por el identificador del usuario y ordenadas por fecha descendente, y formatea las fechas antes de formar la respuesta JSON.
- **Recuperación de las sesiones del laboratorio:** la misma operación recupera las sesiones de entrenamiento del usuario mediante `get_trained_sessions()` de `mlops_engine`, de modo que la vista administrativa ofrece el panorama completo de la actividad de la cuenta.
- **Respuesta al cliente:** el sistema responde HTTP 200 con las consultas y las sesiones del usuario, y la interfaz las muestra como el segundo nivel de la supervisión, desde el que se alcanza el detalle de una consulta.

CU-033 Ver el detalle de una consulta de un usuario

La visualización del detalle se materializa en el endpoint GET `/api/admin/consultations/{consultation_id}`, que recupera la consulta completa para la auditoría de un caso concreto. El diseño aprovecha la misma política de autorización administrativa, sin duplicar la comprobación de propiedad del historial: el administrador accede por su rol, no por ser propietario de la consulta. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de permisos:** el router aplica `_require_admin()`, con las mismas respuestas HTTP 401 y 403 que el resto de las operaciones administrativas.
- **Recuperación de la consulta:** el router selecciona la consulta completa desde `consultations` por su identificador, incluidas las rutas de la imagen, del mapa XAI y del informe PDF, la predicción, la confianza, el nombre y la fecha.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 si la consulta no existe y HTTP 200 con el detalle cuando existe, de modo que la interfaz distingue la ausencia del registro de un fallo de permisos.
- **Presentación del detalle:** la interfaz muestra el detalle con la radiografía original, el mapa de explicabilidad, el resultado, la confianza, el modelo y los metadatos, servidos mediante las rutas estáticas de la aplicación, completando así la cadena de supervisión desde el listado general hasta la consulta individual.

4.5.3 Diagramas de interacción entre objetos

Los diagramas de interacción de SD-005 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. Los tres comparten el mismo punto de partida: el router resuelve la identidad y el rol del solicitante mediante `_require_admin()` antes de abrir cualquier consulta global, de modo que las rutas administrativas no ejecutan operaciones de lectura sobre usuarios o consultas sin la autorización correspondiente. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-031 Consultar el listado de usuarios

El diagrama de la figura muestra la consulta del listado de usuarios. La interfaz solicita el listado y el router comprueba la autorización, con las alternativas de ausencia de identidad (401) y de identidad sin privilegios (403). Con rol administrativo, el router consulta los usuarios con el recuento de diagnósticos en la base de datos, inspecciona el sistema de ficheros para contar las sesiones del laboratorio por usuario y devuelve el listado completo. La secuencia refleja la doble fuente de información de la vista global: MySQL para los datos relacionales y el sistema de ficheros para las sesiones.

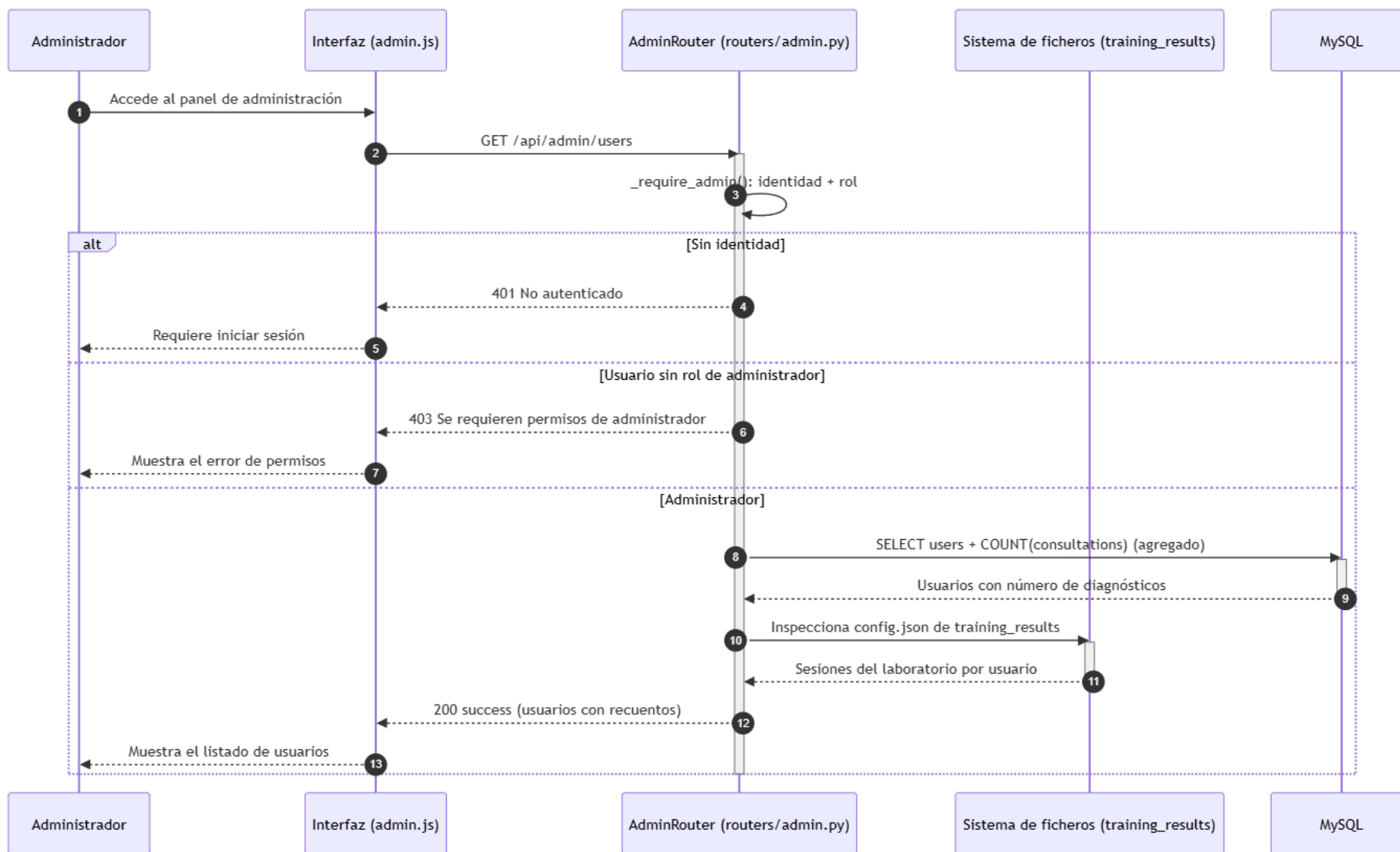


Ilustración x - Diagrama de secuencia del CU-031 Consultar el listado de usuarios

CU-032 Consultar las consultas de un usuario

El diagrama de la figura muestra la consulta de las consultas de un usuario. El administrador selecciona un usuario del listado y el router comprueba la autorización. Con rol administrativo, comprueba que el usuario exista (404 en caso contrario) y recupera sus consultas desde la base de datos, además de sus sesiones del laboratorio mediante el motor MLOps. La secuencia refleja que SD-005 coordina la consulta global reutilizando los mismos servicios del resto de la aplicación, sin duplicar la lógica de acceso a los datos.

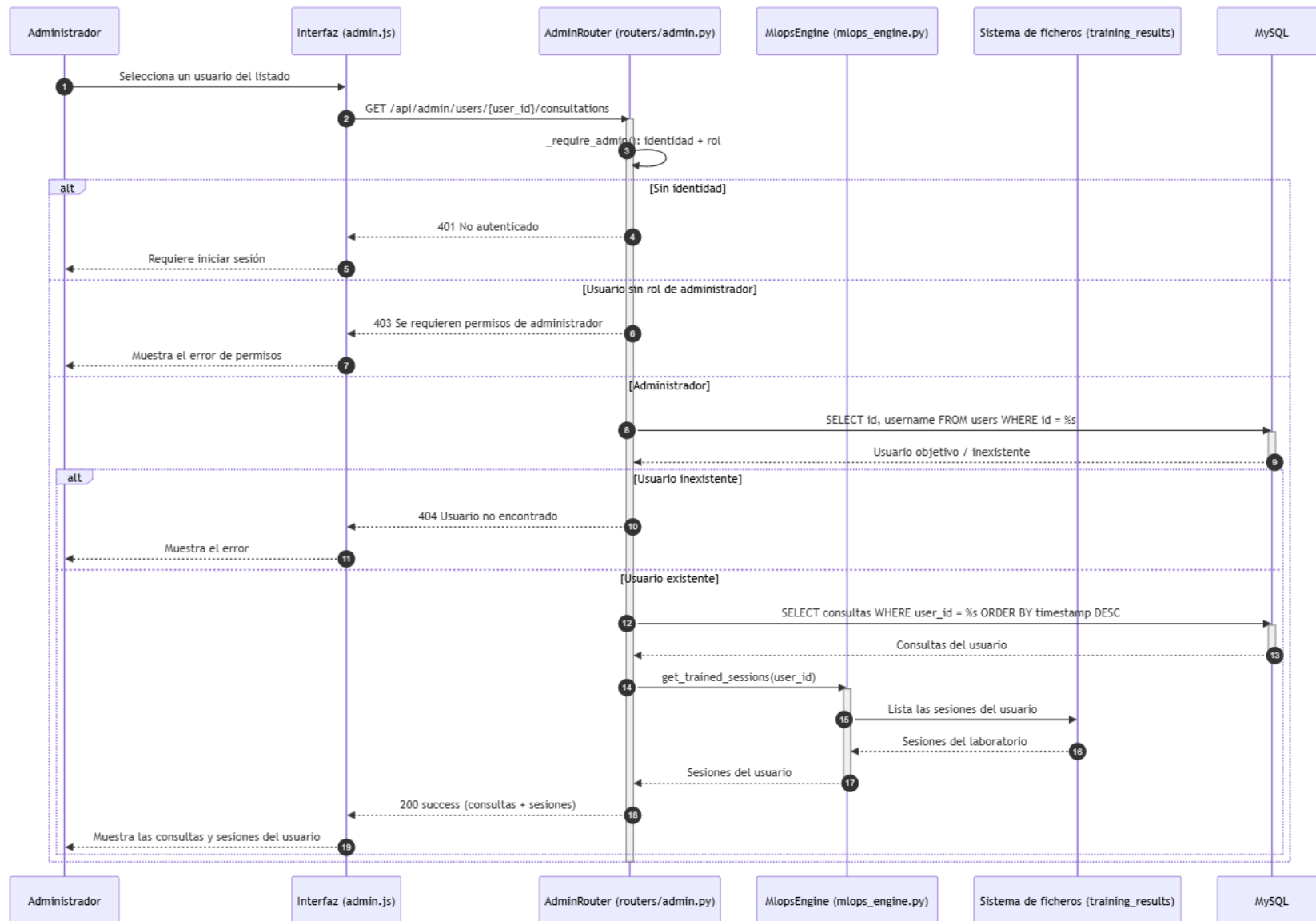


Ilustración x - Diagrama de secuencia del CU-032 Consultar las consultas de un usuario

CU-033 Ver el detalle de una consulta de un usuario

El diagrama de la figura muestra la visualización del detalle de una consulta. El administrador selecciona una consulta de las del usuario y el router comprueba la autorización; con rol administrativo, recupera la consulta por su identificador (404 si no existe) y la devuelve a la interfaz, que carga la imagen y el mapa desde el almacenamiento estático. La secuencia refleja que el acceso administrativo se concede por el rol y no por la propiedad, sin duplicar la comprobación de propiedad del historial.

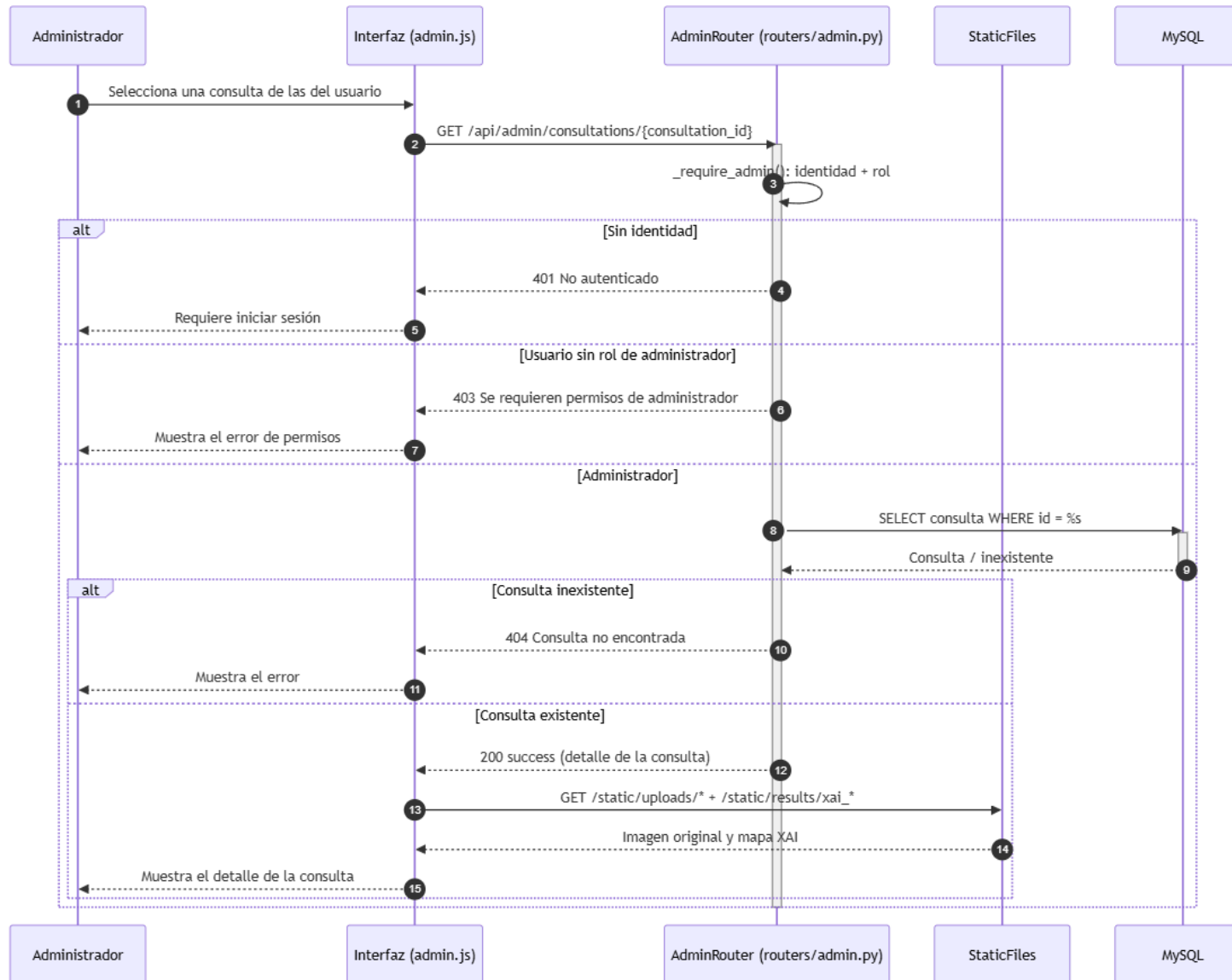


Ilustración x - Diagrama de secuencia del CU-033 Ver el detalle de una consulta

4.6 Subsistema de diseño SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa el subsistema de análisis SS-006 y reúne las capacidades que sirven de apoyo a varios flujos funcionales. Su núcleo es la cola persistente de trabajos, utilizada por los diagnósticos (SD-002) y por los entrenamientos y las validaciones externas (SD-004). Junto a la cola, el subsistema incluye las capacidades transversales de la interfaz: la consulta del estado de los trabajos, la cancelación de un trabajo pendiente y la alternancia del tema visual. Agrupa los casos de uso CU-034 a CU-036, disponibles para todo usuario autenticado con independencia del núcleo funcional en el que trabaje, y constituye el servicio común de ejecución asíncrona sobre el que se apoyan los subsistemas funcionales.

El subsistema se apoya en `routers/queue.py`, que expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente, y en `services/queue_worker.py`, que actúa como consumidor de la cola: al iniciar la aplicación restablece los trabajos que quedaron en estado `running`, selecciona el siguiente trabajo pendiente y lo reclama mediante una actualización condicionada a que siga en `queued`, para después ejecutar el flujo correspondiente en el executor y marcar el resultado como completado o fallido. El worker comparte la persistencia con los routers, pero no comparte con ellos el ciclo de petición: esta es la frontera que permite mantener disponible la interfaz durante las tareas de larga duración. La parte de internacionalización se concentra en `services/lang.py`, y los recursos JavaScript aplican el idioma y el tema visual en el navegador.

La cola define una máquina de estados compartida por los subsistemas que generan trabajos. El estado `queued` indica que la petición ha sido aceptada y espera procesamiento; `running` indica que el worker la ha reclamado; `completed` conserva un resultado; `failed` conserva el error; y `cancelled` identifica una cancelación solicitada por el usuario antes del inicio. No todos los estados se alcanzan desde todas las rutas: la cancelación solo opera sobre los trabajos en `queued`, y un trabajo en `running` no se interrumpe mediante una actualización administrativa. La cola aplica además una política de ordenación que prioriza los trabajos de diagnóstico frente a los de entrenamiento, de modo que la posición que observa el usuario se calcula desde el router y se adapta al tipo de trabajo.

4.6.1 Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-006 recoge las tres interacciones transversales que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de capacidades transversales definido en el análisis (figura 7), limitada al ámbito del subsistema de diseño. El diagrama no presenta relaciones de inclusión o extensión: la consulta de la cola, la cancelación de trabajos y el cambio de tema se ejecutan de forma autónoma, sin delegar pasos obligatorios en otros casos ni ampliar ningún flujo base. La cancelación de un trabajo (CU-035) se describe de forma coherente con la consulta de la cola (CU-034), porque es desde el panel de la cola donde el usuario identifica el trabajo que desea cancelar, aunque ambas operaciones se representan como casos de uso independientes. El diagrama se representa a continuación.

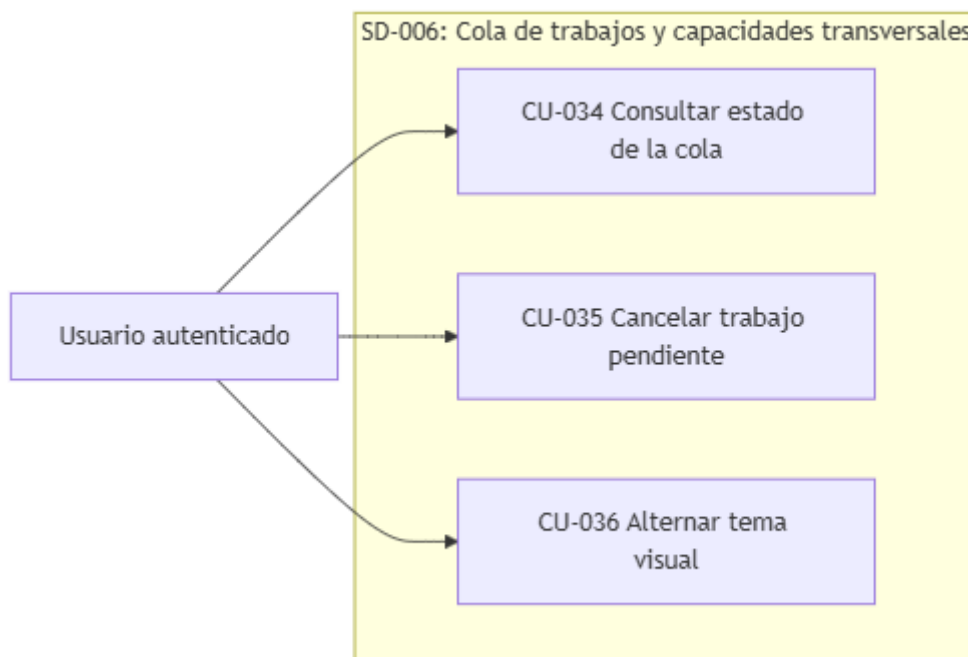


Ilustración x - Diagrama de casos de uso del subsistema SD-006

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los perfiles con cuenta registrada, incluido el administrador. Los tres casos de uso se representan como interacciones directas e independientes del actor con el sistema, en coherencia con la naturaleza transversal del subsistema: cualquiera de ellos puede ejecutarse desde cualquier núcleo funcional, sin relaciones de dependencia entre sí. El panel de la cola, que materializa la consulta del estado (CU-034), es el punto de partida natural de la cancelación (CU-035), pero el análisis mantiene ambas operaciones separadas porque la cancelación no forma parte obligatoria del flujo de consulta.

4.6.2 Casos de uso reales del subsistema

Los casos de uso reales de SD-006 concretan cada interacción del diagrama en decisiones técnicas de implementación. Los dos primeros se apoyan en el router de la cola y reflejan la máquina de estados del subsistema; el tercero se resuelve íntegramente en el navegador, sin intervención del servidor. Cada caso de uso real se describe a continuación.

CU-034 Consultar el estado de la cola de trabajos

La consulta del estado de la cola se materializa en el endpoint GET /api/queue/status, que devuelve los trabajos recientes del usuario autenticado y su estado. El diseño interpreta el payload de cada trabajo según su tipo, de modo que la interfaz muestra el nombre del modelo o el identificador de la sesión sin recibir el contenido interno completo del payload. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación:** el router resuelve la identidad con SD-001 y responde HTTP 401 si no hay sesión válida.
- **Filtrado por usuario:** la consulta recupera los últimos veinte trabajos del usuario, de modo que el panel solo refleja la actividad de la cuenta que lo consulta (RF-005).
- **Interpretación del payload:** el router deserializa el payload y expone los campos relevantes según el tipo: el nombre del modelo para los diagnósticos, el identificador de la sesión y los modelos para los entrenamientos, y el identificador de la sesión con los modelos leídos de su configuración para las validaciones externas. El payload completo no se envía al navegador.
- **Posición en la cola:** para los trabajos en queued, el router calcula su posición mediante la consulta de posición, que respeta la prioridad de la cola según el tipo de trabajo.
- **Respuesta al cliente:** el sistema responde HTTP 200 con los trabajos, su estado, el indicador de trabajos pendientes y el recuento de encolados; la interfaz actualiza el panel de forma periódica para reflejar la progresión de los diagnósticos, los entrenamientos y las validaciones externas.

CU-035 Cancelar un trabajo de la cola

La cancelación se materializa en el endpoint DELETE /api/queue/cancel/{job_id}, que aplica una actualización condicional sobre el trabajo. El diseño adopta una restricción deliberada: solo puede cancelarse un trabajo que pertenezca al usuario y que continúe en estado queued. Si el worker ya lo ha reclamado, la operación no lo interrumpe de forma abrupta, de modo que la cancelación no produce inconsistencias entre el estado persistido y el cálculo que ya se está ejecutando. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación y propiedad:** el router resuelve la identidad con SD-001 y condiciona la actualización al identificador del usuario, de modo que un trabajo ajeno no puede cancelarse.
- **Actualización condicional:** la cancelación ejecuta una actualización que exige que el trabajo continúe en queued; solo si la actualización afecta a una fila se considera cancelado el trabajo.
- **Respuestas diferenciadas:** si la actualización afectó a una fila, el sistema responde HTTP 200 con el trabajo cancelado; en caso contrario, responde HTTP 404 indicando que el trabajo no fue encontrado o ya no está en cola.
- **Alcance de la cancelación:** la operación cubre los trabajos pendientes de diagnóstico, entrenamiento y validación externa, pero no interrumpe los trabajos en ejecución. Esta decisión difiere del flujo del análisis, que preveía interrumpir los trabajos en ejecución cuando resultara técnicamente posible; el diseño actual documenta esa limitación y mantiene el estado running intacto.

CU-036 Alternar el tema visual de la interfaz

La alternancia del tema se materializa en el navegador, mediante los recursos JavaScript de la interfaz, sin intervención del servidor. El diseño trata el tema como una preferencia de presentación que se aplica de forma inmediata y persiste durante la navegación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Cambio en el navegador:** la función toggleTheme() de los recursos JavaScript aplica o elimina la clase del tema oscuro en el elemento raíz de la página, de modo que el cambio se propaga a toda la interfaz sin recargar.
- **Persistencia de la preferencia:** el tema seleccionado se guarda en el almacenamiento local del navegador, de modo que se restaura en las sucesivas vistas del usuario.
- **Restauración automática:** al cargar la interfaz, la preferencia guardada se aplica y, en ausencia de preferencia, se respeta la configuración del sistema operativo del usuario mediante la consulta de preferencia de color.
- **Independencia del servidor:** la operación no requiere una nueva ruta ni una modificación de estado persistida en el servidor; se resuelve como un mecanismo transversal de presentación de la interfaz.

4.6.3 Diagramas de interacción entre objetos

Los diagramas de interacción de SD-006 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. Los dos primeros reflejan la máquina de estados de la cola y su frontera con el ciclo de petición, mientras que el tercero se resuelve íntegramente en el navegador. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-034 Consultar el estado de la cola de trabajos

El diagrama de la figura muestra la consulta del estado de la cola. La interfaz consulta periódicamente el router de la cola, que recupera los últimos trabajos del usuario. Para los trabajos de validación externa, el router lee la configuración de la sesión en el sistema de ficheros para obtener los modelos; a continuación interpreta el payload según el tipo y calcula la posición de los trabajos encolados, devolviendo el estado al panel. La secuencia refleja que el panel se actualiza de forma periódica y que el payload completo de los trabajos no llega al navegador.

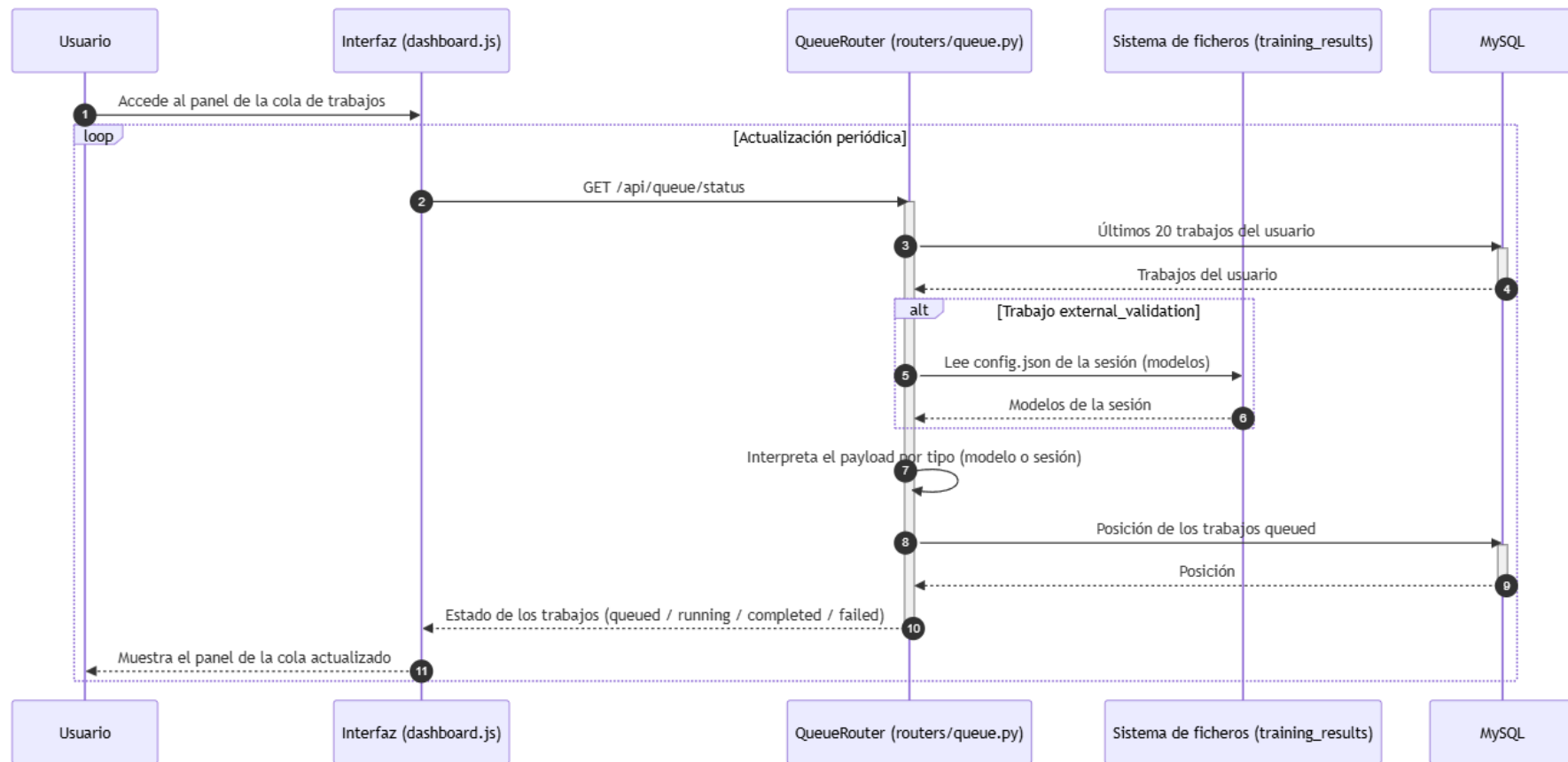


Ilustración x - Diagrama de secuencia del CU-034 Consultar el estado de la cola de trabajos

CU-035 Cancelar un trabajo de la cola

El diagrama de la figura muestra la cancelación de un trabajo. El usuario solicita la cancelación desde el panel y el router resuelve la identidad; con sesión válida, ejecuta la actualización condicionada a que el trabajo pertenezca al usuario y continúe en queued. Si la actualización afecta a una fila, el trabajo queda cancelado y desaparece del panel; si no, el sistema responde 404 informando de que el trabajo ya no está en cola, lo que cubre los trabajos en ejecución y los finalizados. La secuencia refleja la restricción deliberada del diseño: un trabajo reclamado por el worker no se interrumpe.

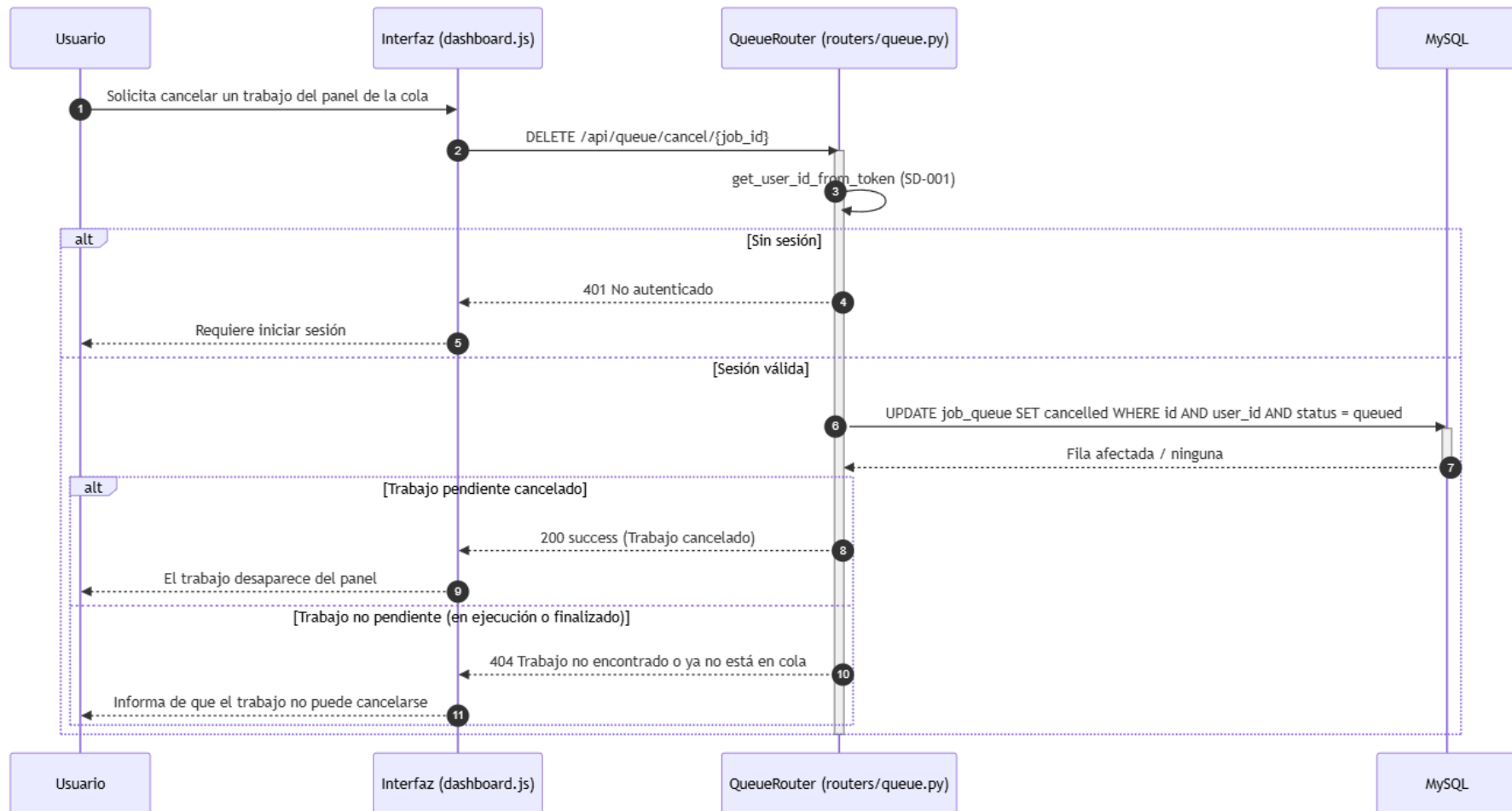


Ilustración x - Diagrama de secuencia del CU-035 Cancelar un trabajo de la cola

CU-036 Alternar el tema visual de la interfaz

El diagrama de la figura muestra la alternancia del tema visual. El usuario activa el cambio y la interfaz aplica o elimina la clase del tema oscuro en el elemento raíz, guarda la preferencia en el almacenamiento local y presenta la interfaz con el tema seleccionado. La secuencia se resuelve íntegramente en el navegador: no interviene el servidor, de modo que el cambio es inmediato y no interrumpe la navegación ni el estado del trabajo en curso.

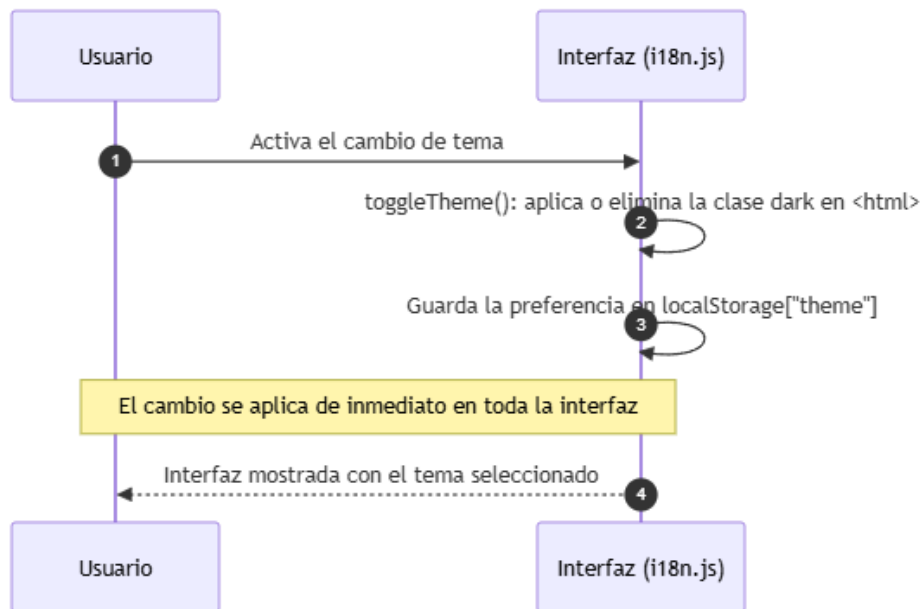


Ilustración x - Diagrama de secuencia del CU-036 Alternar el tema visual de la interfaz

5 Estructura de clases del diseño

El diseño de clases constituye la etapa del diseño que concreta la estructura estática de los subsistemas definidos en el capítulo 18: determina qué clases forman parte de cada subsistema de diseño, qué responsabilidades asume cada una y cómo colaboran entre sí para materializar los casos de uso reales descritos en el capítulo 21. Mientras que los diagramas de interacción muestran el comportamiento de los objetos en el tiempo, el modelo de clases captura la estructura persistente de ese comportamiento: los atributos que conserva cada componente, las operaciones que ofrece y las asociaciones que lo vinculan con el resto (Larman, 2004). El diseño de clases refina así los componentes identificados en la arquitectura y los presenta con el nivel de detalle necesario para comprender y mantener la implementación.

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño del capítulo 18: cada apartado corresponde a un subsistema (SD-001 a SD-006) y agrupa las clases que materializan su responsabilidad. Para cada subsistema se presentan dos elementos, conforme a la guía de diseño de la memoria (punto 5): el diagrama de clases, que representa gráficamente las clases del subsistema con sus atributos, sus operaciones y sus relaciones —incluidas las clases abstractas, las herencias y las asociaciones cuando existen—, y la especificación de las clases, que detalla cada clase mediante la ficha formal de clase CL-NNNN con sus atributos, sus operaciones y sus comentarios. Para las clases más complejas se añade, cuando procede, un diagrama de transición de estados que permite comprender la funcionalidad soportada por dicha clase.

Las clases de diseño de vitalXAI se corresponden con los componentes reales del sistema descritos en el capítulo 18. Aunque la implementación está orientada a módulos funcionales —routers, servicios y el worker—, cada módulo se modela aquí como una clase de diseño que agrupa sus responsabilidades, de modo que la notación UML permite expresar de forma homogénea la estructura de todos los subsistemas y su colaboración. La trazabilidad con el diseño de casos de uso se mantiene en cada ficha: las operaciones de cada clase materializan los mensajes que los diagramas de secuencia del capítulo 21 asignaron al componente correspondiente, y las asociaciones del modelo de clases reflejan las dependencias de colaboración entre routers y servicios.

5.1 *Subsistema SD-001: Acceso, identidad y gestión de sesiones*

El subsistema SD-001 materializa las responsabilidades de acceso, identidad y gestión de sesiones descritas en el capítulo 18 y agrupa dos clases de diseño propias. La clase AuthRouter se corresponde con el router `routers/auth.py` y actúa como fachada HTTP del subsistema: sirve las páginas de inicio de sesión y registro, procesa los formularios y gestiona las cookies de sesión. La clase AuthService se corresponde con el servicio `services/auth_service.py` y concentra la lógica criptográfica y el ciclo de vida de las credenciales: el hash de las contraseñas, la firma y verificación de los tokens de acceso, la gestión de los tokens de refresco y la rotación con detección de robo. El subsistema depende además del servicio de idioma LangService, que pertenece a SD-006 y proporciona los mensajes localizados que el router emplea en sus respuestas.

5.1.1 Diagrama de clases del subsistema

El modelo de clases de SD-001 se representa en la figura 83. El diagrama muestra las dos clases propias del subsistema con sus atributos y sus operaciones, y la dependencia hacia el servicio de idioma, que se representa como una clase de otro subsistema para reflejar la colaboración sin atribuirle la propiedad. La clase AuthRouter presenta atributos de configuración interna —la expresión regular de validación del correo electrónico y el motor de plantillas— y las operaciones que sirven las páginas y procesan los formularios; la clase AuthService presenta los parámetros de configuración de las credenciales y las operaciones criptográficas y de gestión de tokens.



Ilustración x - Diagrama de clases del subsistema SD-001

El diagrama identifica la asociación principal del subsistema: la dependencia de AuthRouter hacia AuthService, que materializa la delegación de toda la lógica criptográfica y de tokens del router al servicio. Esta separación de responsabilidades, ya declarada en el capítulo 18, mantiene al router como una fachada ligera que orquesta las peticiones HTTP y delega en el servicio las operaciones que requieren configuración sensible. La dependencia punteada hacia LangService refleja que el router consume los mensajes localizados del servicio de idioma sin que este forme parte del subsistema. El modelo no presenta herencias ni clases abstractas: las dos clases son clases concretas de diseño, y la colaboración se expresa exclusivamente mediante asociaciones de dependencia.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases se corresponden con los mensajes de los diagramas de secuencia del capítulo 21. Las operaciones login(), process_register(), logout() y token_refresh() de AuthRouter materializan los puntos de entrada de los CU-001, CU-002, CU-003 y de la renovación de la sesión, y las operaciones de AuthService —hash_password(), verify_password(), create_access_token(), create_refresh_token(), verify_refresh_token(), revoke_refresh_token() y rotate_refresh_token()— implementan las decisiones criptográficas que los diagramas de secuencia encargaron al servicio.

5.1.2 Especificación de las clases

A continuación se definen las dos clases propias de SD-001.

CL-0001	AuthRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/auth.py. Actúa como fachada HTTP del subsistema de acceso: sirve las páginas de inicio de sesión, registro, panel de diagnóstico y laboratorio de entrenamiento; procesa los formularios de registro e inicio de sesión; gestiona el cierre de sesión y la renovación de credenciales, y establece las cookies de sesión en las respuestas. Valida los datos de entrada del registro y delega toda la lógica criptográfica en AuthService.		
Atributos	Nombre	Tipo	Descripción
	_EMAIL_RE	re.Pattern	Expresión regular privada que valida el formato del nombre de usuario como dirección de correo electrónico en el registro.
	templates	Jinja2Templates	Motor de plantillas que compone las páginas HTML servidas por el router (inicio de sesión, registro, panel y laboratorio).
Operaciones	Nombre	Descripción	
	login_page(request, error)	Sirve la página de inicio de sesión, mostrando el error genérico cuando la autenticación previa falló.	
	login(request, username, password)	Procesa el formulario de inicio de sesión: verifica las credenciales mediante AuthService, establece las cookies de sesión y redirige al panel, o redirige al inicio con un mensaje genérico si la verificación falla. Aplicada la limitación de cinco peticiones por minuto.	
	register_page(request)	Sirve el formulario de registro de nuevas cuentas.	
	process_register(username, password, first_name, last_name, role)	Procesa el formulario de registro: valida el formato de los campos, comprueba la unicidad del usuario, solicita el hash de la contraseña a AuthService, crea la cuenta y establece las cookies de sesión.	

	logout(request)	Cierra la sesión: revoca el token de refresco mediante AuthService, elimina las cookies y redirige a la página de inicio de sesión.
	token_refresh(refresh_token)	Renueva la sesión: rota el token de refresco mediante AuthService y emite un nuevo token de acceso, actualizando las cookies.
	dashboard(request)	Sirve el panel de diagnóstico del usuario autenticado, redirigiendo a la página de inicio si no existe sesión válida.
	training_lab(request)	Sirve el laboratorio de entrenamiento del usuario autenticado, con la misma comprobación de sesión que el panel.
	_validate_register_inputs(username, password, first_name, last_name)	Operación privada que valida el formato del correo electrónico, la longitud mínima de la contraseña y la presencia del nombre y los apellidos, devolviendo el campo inválido o None.
Comentarios	La clase no mantiene estado de sesión propio: las credenciales se conservan en las cookies HttpOnly y SameSite=Lax, y la identidad se resuelve mediante AuthService. La operación token_refresh() cubre la renovación de credenciales descrita en el capítulo 18 y materializada en el 21.1.3. El rol de administrador no se valida en esta clase, sino en las operaciones administrativas de SD-005.	

Tabla x - Especificación de clase CL-0001

CL-0002	AuthService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/auth_service.py. Concentra la lógica criptográfica y el ciclo de vida de las credenciales del subsistema: genera y verifica el hash de las contraseñas con bcrypt, firma y valida los tokens de acceso JWT, gestiona los tokens de refresco en la base de datos mediante su hash SHA-256 y aplica la rotación con detección de uso de credenciales revocadas.		
Atributos	Nombre	Tipo	Descripción
	JWT_SECRET_KEY	str	Clave secreta de firma de los tokens de acceso JWT, obtenida del entorno; se emite una advertencia cuando se utiliza la clave de desarrollo por defecto.
	JWT_ACCESS_EXPIRE_MINUTES	int	Duración en minutos del token de acceso, configurada mediante variable de entorno.
	JWT_REFRESH_EXPIRE_DAYS	int	Duración en días del token de refresco, configurada mediante variable de entorno.
	REFRESH_ROTATION_GRACE_SECONDS	int	Periodo de gracia en segundos que tolera las renovaciones concurrentes del token de refresco sin interpretarlas como un robo.
Operaciones	Nombre	Descripción	
	hash_password(password)	Genera el hash bcrypt de la contraseña con un salt aleatorio en cada operación.	
	verify_password(password, password_hash)	Compara la contraseña introducida con el hash almacenado mediante bcrypt.checkpw.	
	create_access_token(user_id)	Firma un token de acceso JWT con el identificador del usuario y su fecha de expiración, usando la clave secreta del entorno.	
	verify_access_token(token)	Valida el token de acceso y devuelve el identificador del usuario, o None si el token es inválido o ha expirado.	
	create_refresh_token(user_id)	Genera un token de refresco aleatorio y persiste solo su hash SHA-256 en la tabla refresh_tokens, con la fecha de expiración y el indicador de revocación.	
	verify_refresh_token(raw_token)	Verifica el token de refresco: acepta el token activo, tolera el token revocado dentro del periodo de gracia y revoca todos los tokens del usuario si se detecta el uso de una credencial revocada fuera de él.	
	revoke_refresh_token(raw_token)	Marca como revocado el token de refresco en la base de datos.	
	rotate_refresh_token(old_raw_token)	Rota el token de refresco: verifica el token actual, revoca el anterior y emite uno nuevo; devuelve None si la rotación no es válida.	

	get_user_id_from_token(token)	Extrae el identificador del usuario de un token de acceso, devolviendo None si el token falta o no es válido; es el mecanismo común de identificación que consumen el resto de los subsistemas.
Comentarios	La clase no conserva la contraseña original en ningún atributo: una vez completado el hash, la credencial no se transmite a ningún componente de persistencia. La configuración sensible permanece fuera del código y se obtiene del entorno en tiempo de ejecución. Las operaciones de verificación y rotación de tokens de refresco implementan la política de seguridad de sesiones descrita en el capítulo 18 y materializada en los diagramas de secuencia del capítulo 21.	

Tabla x - Especificación de clase CL-0002

5.2 Subsistema SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 materializa el flujo clínico de la plataforma y agrupa cinco clases de diseño propias. La clase `InferenceRouter` se corresponde con el router `routers/inference.py` y actúa como fachada HTTP del diagnóstico: valida la imagen recibida, la conserva en el área de cargas y encola el trabajo de diagnóstico. Las clases `MIEngine`, `XaiGenerator` y `PdfGenerator` se corresponden con los servicios `services/ml_engine.py`, `services/xai_generator.py` y `services/pdf_generator.py`, y concentran respectivamente la predicción, la generación de los mapas de explicabilidad y la construcción del informe PDF. La clase `PDFReport` es la clase concreta del generador de informes que hereda del generador de la librería `fpdf`, lo que introduce la única herencia real del subsistema. El procesamiento de los trabajos encolados no pertenece a SD-002: lo ejecuta el worker de la cola de SD-006, que invoca las clases de predicción, explicabilidad e informe del presente subsistema.

5.2.1 Diagrama de clases del subsistema

El modelo de clases de SD-002 se representa en la figura 84. El diagrama muestra las cinco clases propias del subsistema con sus atributos y sus operaciones, la herencia de la clase `PDFReport` sobre la clase `FPDF` de la librería, y las dependencias hacia el servicio de idioma `LangService`, que pertenece a SD-006 y proporciona las etiquetas y los mensajes localizados. Las asociaciones entre las clases del subsistema reflejan la colaboración del flujo de diagnóstico: el generador de explicabilidad depende del motor de predicción para obtener el modelo cargado, y el generador de informes utiliza la clase concreta del documento PDF.

El diagrama identifica la única herencia del subsistema: la clase `PDFReport` especializa la clase `FPDF` de la librería de generación de documentos, redefiniendo la cabecera y el pie de página del informe médico. La clase `FPDF` se representa como una clase externa de la librería, marcada con el estereotipo de librería y con las operaciones básicas de generación que el informe hereda y utiliza —añadir página, configurar la fuente y el color, escribir celdas, insertar imágenes y producir el documento—, sin documentar su implementación interna por ser responsabilidad de la dependencia externa. El resto de las relaciones son asociaciones de dependencia. La dependencia de `XaiGenerator` hacia `MIEngine` refleja que la generación de los mapas de explicabilidad requiere el modelo cargado por el motor; las dependencias hacia `LangService` reflejan la localización de las etiquetas, los títulos y los mensajes; y la dependencia de `PdfGenerator` hacia `PDFReport` materializa la construcción del documento concreto. La clase `InferenceRouter` no presenta dependencias con el resto de las clases del subsistema porque su colaboración se limita al encolado del trabajo: el procesamiento de la predicción, de la explicabilidad y del informe lo orquesta el worker de SD-006, que invoca estas clases desde el ciclo asíncrono.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `predict()` de `InferenceRouter` implementa el CU-008 y las validaciones del CU-006, las operaciones de `MIEngine`, `XaiGenerator` y `PdfGenerator` materializan los pasos del procesamiento asíncrono de la figura y la operación `generate_medical_report()` de `PdfGenerator` materializa la generación del informe del CU-037.

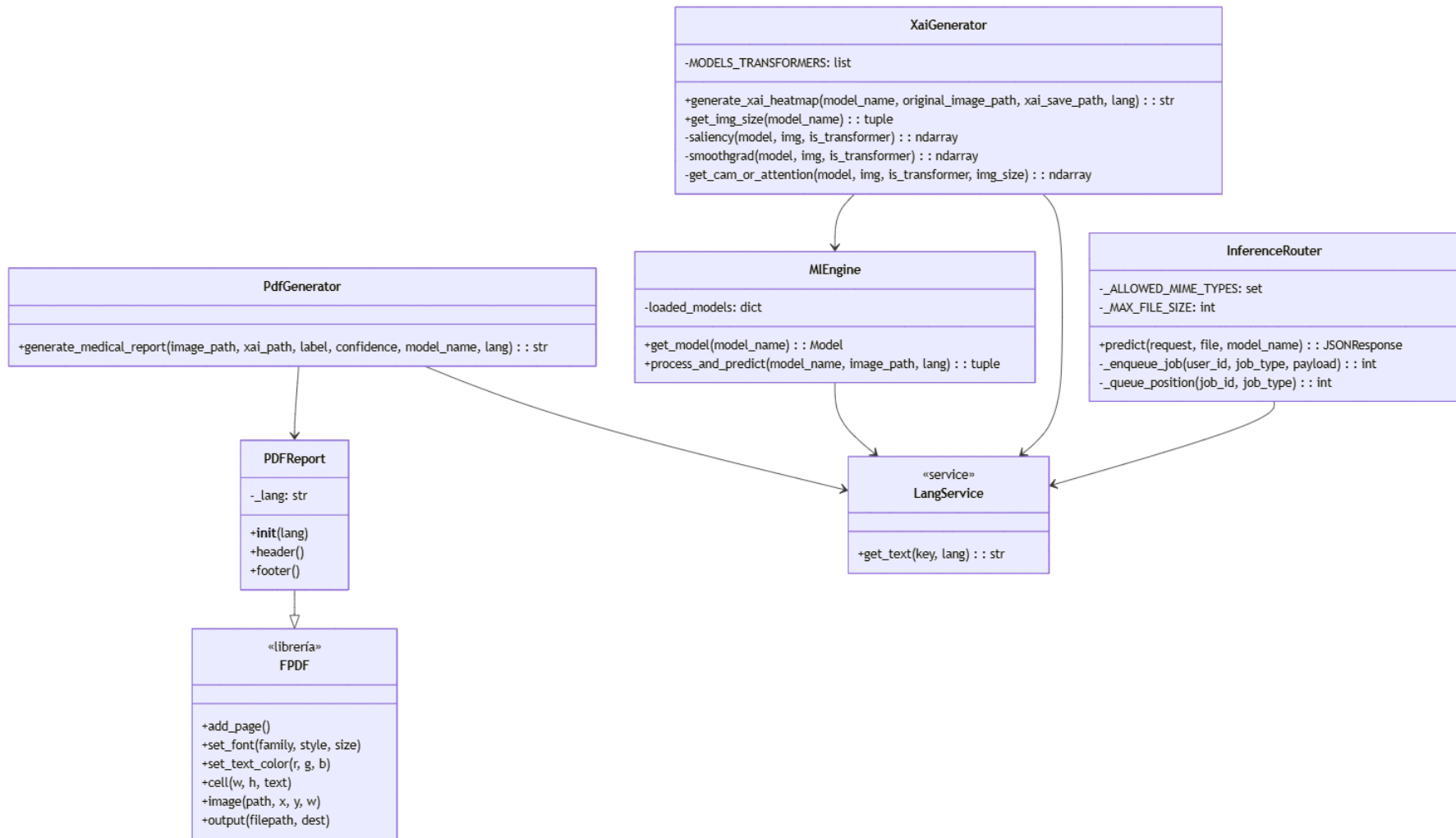


Ilustración x - Diagrama de clases del subsistema SD-002

5.2.2 Especificación de las clases

A continuación se definen las cinco clases propias de SD-002.

CL-0003	InferenceRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/inference.py. Actúa como fachada HTTP de la solicitud de diagnóstico: resuelve la identidad del usuario mediante AuthService, valida el tipo MIME y el tamaño de la imagen, la conserva en el área de cargas y crea el trabajo de diagnóstico en la cola con un payload serializable. No ejecuta la inferencia ni carga modelos.		
Atributos	Nombre	Tipo	Descripción
	_ALLOWED_MIME_TYPES	set	Conjunto de tipos MIME permitidos para la radiografía: JPEG y PNG.
	_MAX_FILE_SIZE	int	Tamaño máximo de la imagen en bytes: 10 MB.
Operaciones	Nombre	Descripción	
	predict(request, file, model_name)	Procesa la solicitud de diagnóstico: autentica al usuario (401 sin sesión), valida el tipo y el tamaño de la imagen (400 si no se permiten), guarda el fichero con un nombre basado en la fecha, encola el trabajo de tipo diagnosis y responde el estado queued con el identificador y la posición.	
	_enqueue_job(user_id, job_type, payload)	Operación privada que inserta un trabajo en la tabla job_queue con el tipo y el payload indicados, devolviendo el identificador del trabajo creado.	
	_queue_position(job_id, job_type)	Operación privada que calcula la posición del trabajo en la cola, respetando la prioridad de los diagnósticos frente al resto de tipos.	
Comentarios	La validación del fichero se realiza antes de encolar, de modo que una petición inválida no ocupa una posición en la cola ni consume recursos del worker. El payload del trabajo contiene únicamente el modelo, la ruta de la imagen y el idioma; ni el modelo ni la imagen completa se serializan en la base de datos.		

Tabla x - Especificación de clase CL-0003

CL-0004	MIEngine		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/ml_engine.py. Concentra la inferencia del diagnóstico: carga y reutiliza los modelos de aprendizaje profundo en memoria, prepara la imagen recibida y produce la etiqueta de la predicción y su nivel de confianza. Combina arquitecturas convolucionales y arquitecturas Transformer, y localiza las etiquetas mediante el servicio de idioma.		
Atributos	Nombre	Tipo	Descripción
	loaded_models	dict	Diccionario global que conserva los modelos cargados en memoria, indexado por nombre de arquitectura, de modo que la primera consulta paga la carga de los pesos y las posteriores reutilizan el modelo.
Operaciones	Nombre	Descripción	
	get_model(model_name)	Carga el modelo de la arquitectura indicada y lo conserva en la caché de modelos; si el modelo ya está cargado, lo devuelve sin volver a leer los pesos.	
	process_and_predict(model_name, image_path, lang)	Preprocesa la imagen según la arquitectura, ejecuta la inferencia y devuelve la etiqueta de la predicción (Neumonía o Normal) y el nivel de confianza, localizados mediante LangService.	
Comentarios	La reutilización de los modelos en memoria implementa el requisito de tiempo de respuesta de la inferencia (RNF-019): la optimización pertenece al motor y no al router. Un modelo no reconocido falla antes de producir un resultado ambiguo, de modo que la validación de la arquitectura queda diferida al motor.		

Tabla x - Especificación de clase CL-0004

CL-0005	XaiGenerator		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/xai_generator.py. Genera el mapa de explicabilidad adecuado a la arquitectura utilizada: los métodos basados en gradientes para las arquitecturas convolucionales y los mapas de atención para las arquitecturas Transformer, y compone la figura con la radiografía original y los tres mapas superpuestos.		
Atributos	Nombre	Tipo	Descripción
	MODELS_TRANSFORMERS	list	Lista de las arquitecturas Transformer del proyecto, utilizada para bifurcar el método de explicabilidad.
Operaciones	Nombre	Descripción	
	generate_xai_heatmap(model_name, original_image_path, xai_save_path, lang)	Genera la figura de explicabilidad (original, Saliency, SmoothGrad y superposición de Grad-CAM o atención), la guarda en la ruta indicada y devuelve dicha ruta.	
	get_img_size(model_name)	Devuelve el tamaño de entrada de la imagen según la arquitectura (299×299, 384×384 o 224×224).	
	saliency(model, img, is_transformer)	Operación privada que calcula el mapa de Saliency mediante los gradientes de la puntuación respecto a la imagen.	
	smoothgrad(model, img, is_transformer)	Operación privada que promedia los mapas de Saliency con ruido gaussiano y normaliza el resultado.	
	get_cam_or_attention(model, img, is_transformer, img_size)	Operación privada que calcula el Grad-CAM sobre la última capa convolucional para las arquitecturas convolucionales o el mapa de atención para las Transformer, con un fallback basado en gradientes.	
Comentarios	La generación de los mapas se realiza sobre el modelo cargado por MEngine, de modo que la clase depende del motor de predicción. La figura resultante se sirve como recurso estático y su ruta se persiste en la consulta, sin repetir el cálculo en las visualizaciones posteriores.		

Tabla x - Especificación de clase CL-0005

CL-0006	PdfGenerator		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/pdf_generator.py. Construye el informe PDF del diagnóstico a partir de la imagen original, el mapa de explicabilidad, la etiqueta, la confianza y el modelo utilizado, empleando la clase concreta PDFReport y los textos localizados del servicio de idioma.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: construye el informe a partir de los parámetros recibidos en la operación.
Operaciones	Nombre	Descripción	
	generate_medical_report(image_path, xai_path, label, confidence, model_name, lang)	Compone el informe A4 con la fecha, el modelo, el diagnóstico con su color según el resultado, la confianza y las dos imágenes (original y mapa), lo guarda en static/reports con un nombre basado en la fecha y devuelve la ruta del documento.	
Comentarios	La generación del informe se produce durante el procesamiento del diagnóstico, de modo que el documento está disponible cuando la consulta se completa y la descarga posterior no depende de una nueva operación del sistema. El informe se trata como parte de la información protegida de la consulta.		

Tabla x - Especificación de clase CL-0006

CL-0007	PDFReport		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase concreta del informe PDF de diagnóstico, correspondiente a la clase PDFReport del módulo services/pdf_generator.py. Especializa la clase FPDF de la librería de generación de documentos y redefine la cabecera y el pie de página del informe, además de conservar el idioma de los textos.		
Atributos	Nombre	Tipo	Descripción
	_lang	str	Idioma utilizado para localizar los textos del informe (cabecera, pie y etiquetas).
Operaciones	Nombre	Descripción	
	__init__(lang)	Inicializa el documento de la librería base y conserva el idioma para localizar los textos.	
	header()	Redefine la cabecera de cada página del informe con el título y el color corporativo.	
	footer()	Redefine el pie de página con el número de página.	
Comentarios	Esta clase constituye la herencia real del subsistema: su relación con FPDF se representa en el modelo de clases como una especialización. El generador de informes la instancia para componer el documento, de modo que el conocimiento del formato queda encapsulado en la clase y fuera del router.		

Tabla x - Especificación de clase CL-0007

5.3 Subsistema SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa la recuperación y la gestión de las consultas ya realizadas y agrupa una única clase de diseño propia. La clase HistoryRouter se corresponde con el router routers/history.py y concentra las operaciones del historial: la consulta del listado, la actualización del nombre y la eliminación de una consulta, todas ellas precedidas de la comprobación de propiedad. El subsistema no dispone de clases de servicio propias, porque su lógica se limita a la recuperación de los registros persistidos por SD-002 y a la actualización de la etiqueta de organización; la comprobación de propiedad se resuelve dentro de la propia clase. HistoryRouter depende de AuthService, perteneciente a SD-001, para resolver la identidad del usuario, y de LangService, perteneciente a SD-006, para los mensajes localizados de las respuestas.

5.3.1 Diagrama de clases del subsistema

El modelo de clases de SD-003 se representa en la figura 85. El diagrama muestra la clase propia del subsistema con sus operaciones, y las dependencias hacia las clases AuthService y LangService, que pertenecen a otros subsistemas y se representan con las operaciones que el historial consume. El modelo no presenta herencias ni clases abstractas: la clase HistoryRouter es una clase concreta que orquesta las operaciones del historial y delega la identidad en el subsistema de acceso.

El diagrama refleja la simplicidad estructural del subsistema: una sola clase concentra la consulta del listado, el renombrado y la eliminación, y la operación privada `_check_consultation_ownership()` que protege las dos operaciones de modificación. La dependencia hacia AuthService materializa el mecanismo común de identificación de SD-001, del que el historial obtiene el usuario para filtrar y comprobar la propiedad de las consultas. La dependencia hacia LangService localiza los mensajes de error y de respuesta. La relación de SD-003 con SD-002 es de productor-consumidor de datos y no se representa como una asociación entre clases: los registros de consultations que el historial lee son creados por el worker de SD-002, y la colaboración se produce a través de la persistencia compartida.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `get_history()` implementa el CU-011; la operación `update_patient_name()` implementa el CU-013; la operación `delete_history_record()` implementa el CU-014; y la operación privada `_check_consultation_ownership()` materializa la comprobación de propiedad que condiciona las respuestas HTTP 404 y 403 de los diagramas de secuencia.

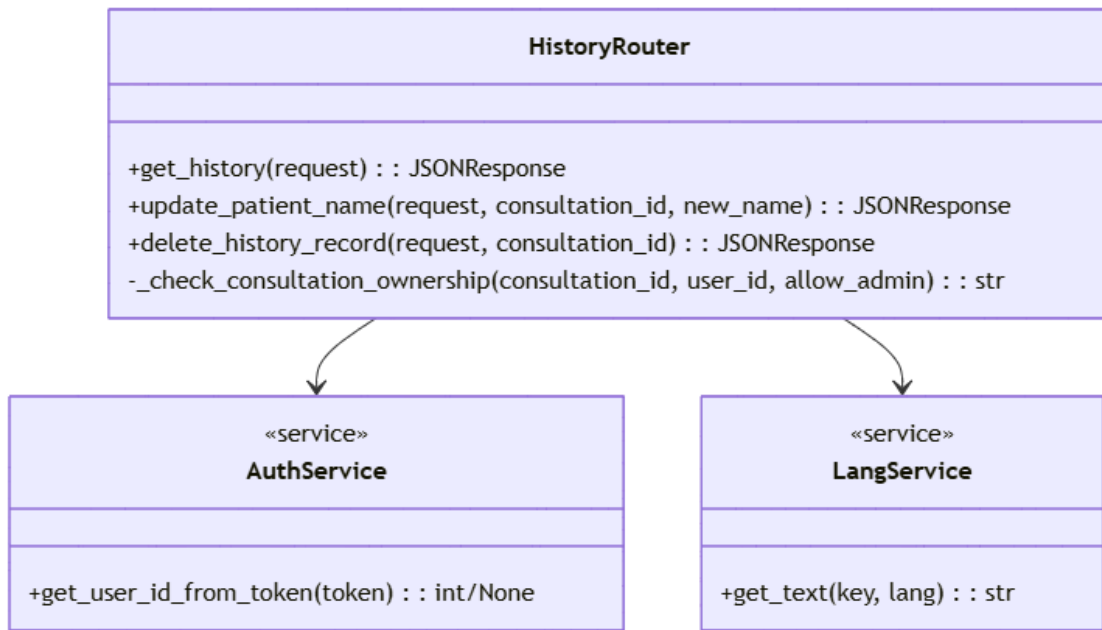


Ilustración x - Diagrama de clases del subsistema SD-003

5.3.2 Especificación de las clases

A continuación se define la clase propia de SD-003.

CL-0008	HistoryRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/history.py. Proporciona la recuperación y la gestión de las consultas de diagnóstico de un usuario: devuelve el listado del historial filtrado por el propietario, actualiza el nombre mostrado de una consulta y elimina una consulta de forma física, aplicando en las operaciones de modificación la comprobación de propiedad con la excepción controlada del administrador.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: cada operación resuelve la identidad y accede a la persistencia a partir de los parámetros recibidos.
Operaciones	Nombre	Descripción	
	get_history(request)	Devuelve el listado de consultas del usuario autenticado, filtrado por user_id y ordenado por fecha descendente; responde HTTP 401 sin sesión y formatea las fechas antes de formar la respuesta.	
	update_patient_name(request, consultation_id, new_name)	Actualiza el nombre mostrado de una consulta: comprueba la propiedad, responde HTTP 404 si la consulta no existe y HTTP 403 si pertenece a otro usuario, y ejecuta la actualización del nombre cuando la propiedad se confirma.	
	delete_history_record(request, consultation_id)	Elimina de forma física una consulta: comprueba la propiedad, responde HTTP 404 o HTTP 403 en los casos de inexistencia o falta de permisos, y ejecuta el borrado cuando la propiedad se confirma.	
	_check_consultation_ownership(consultation_id, user_id, allow_admin)	Operación privada que comprueba que la consulta existe y pertenece al usuario solicitante, permitiendo la excepción del rol admin; devuelve not_found, forbidden u ok.	
Comentarios	La clase no vuelve a ejecutar el modelo para mostrar una consulta anterior: recupera los metadatos y las rutas de los artefactos ya persistidos. El nombre actualizado se aplica sobre la columna patient_name, que funciona como etiqueta de organización y no como identificación clínica del paciente. La eliminación es un borrado físico, sin auditoría automática, tal y como se declaró en el capítulo 18.		

Tabla x - Especificación de clase CL-0008

5.4 Subsistema SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 materializa el laboratorio de experimentación MLOps y agrupa cinco clases de diseño propias. La clase `TrainerRouter` se corresponde con el router `routers/trainer.py` y actúa como fachada ligera del laboratorio: expone los endpoints de configuración, lanzamiento, consulta y gestión de las sesiones, y delega la lógica en los servicios. La clase `ChatbotService` se corresponde con `services/chatbot_service.py` y resuelve la configuración conversacional del experimento mediante el asistente externo. La clase `MlopsEngine` se corresponde con `services/mlops_engine.py` y concentra la organización de las sesiones, la ejecución de los pipelines de entrenamiento y la lectura de los resultados. Las clases `PdfGeneratorMlops` y `MedicalReport`, correspondientes a `services/pdf_generator_mlops.py`, generan el informe consolidado de la sesión; `MedicalReport` es la clase concreta que hereda de la librería `FPDF`, segunda herencia real del diseño de clases de `vitalXAI`.

5.4.1 Diagrama de clases del subsistema

El modelo de clases de SD-004 se representa en la figura 86. El diagrama muestra las cinco clases propias del subsistema, la herencia de `MedicalReport` sobre la clase `FPDF` de la librería, la dependencia del servicio conversacional hacia el proveedor externo de inteligencia artificial, y las dependencias del router hacia las clases propias y hacia los servicios transversales `AuthService` y `LangService`. El router concentra las asociaciones del subsistema: delega la exploración, el entrenamiento y la consulta de resultados en `MlopsEngine`, la conversación en `ChatbotService` y la generación del informe en `PdfGeneratorMlops`.

El diagrama identifica la herencia de `MedicalReport` sobre la clase `FPDF` de la librería de generación de documentos, que se representa con las operaciones básicas que el informe hereda y utiliza. El proveedor externo de inteligencia artificial se modela como una clase externa con el estereotipo de proveedor, con la operación de generación de respuestas que el servicio conversacional invoca; la clave de la API no forma parte de la interfaz de la clase, sino del atributo de configuración de `ChatbotService`. Las dependencias hacia `AuthService` y `LangService` reflejan los servicios transversales de SD-001 y SD-006 que el router consume. La clase `MlopsEngine` constituye el núcleo del subsistema: orquesta los scripts de entrenamiento, el análisis XAI y la comparación estadística, y organiza la lectura y escritura de los resultados en el sistema de ficheros, sin depender del servicio de idioma porque sus mensajes de registro son textos fijos del pipeline.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los casos de uso reales del capítulo 21. Las operaciones de `TrainerRouter` exponen los endpoints de los CU-015 a CU-030 y CU-039; las operaciones de `MlopsEngine` implementan los flujos del procesamiento del entrenamiento y de la validación externa; la operación `generate_pdf_report()` de `PdfGeneratorMlops` materializa el CU-028; y las operaciones de `ChatbotService` materializan la configuración conversacional del CU-016.

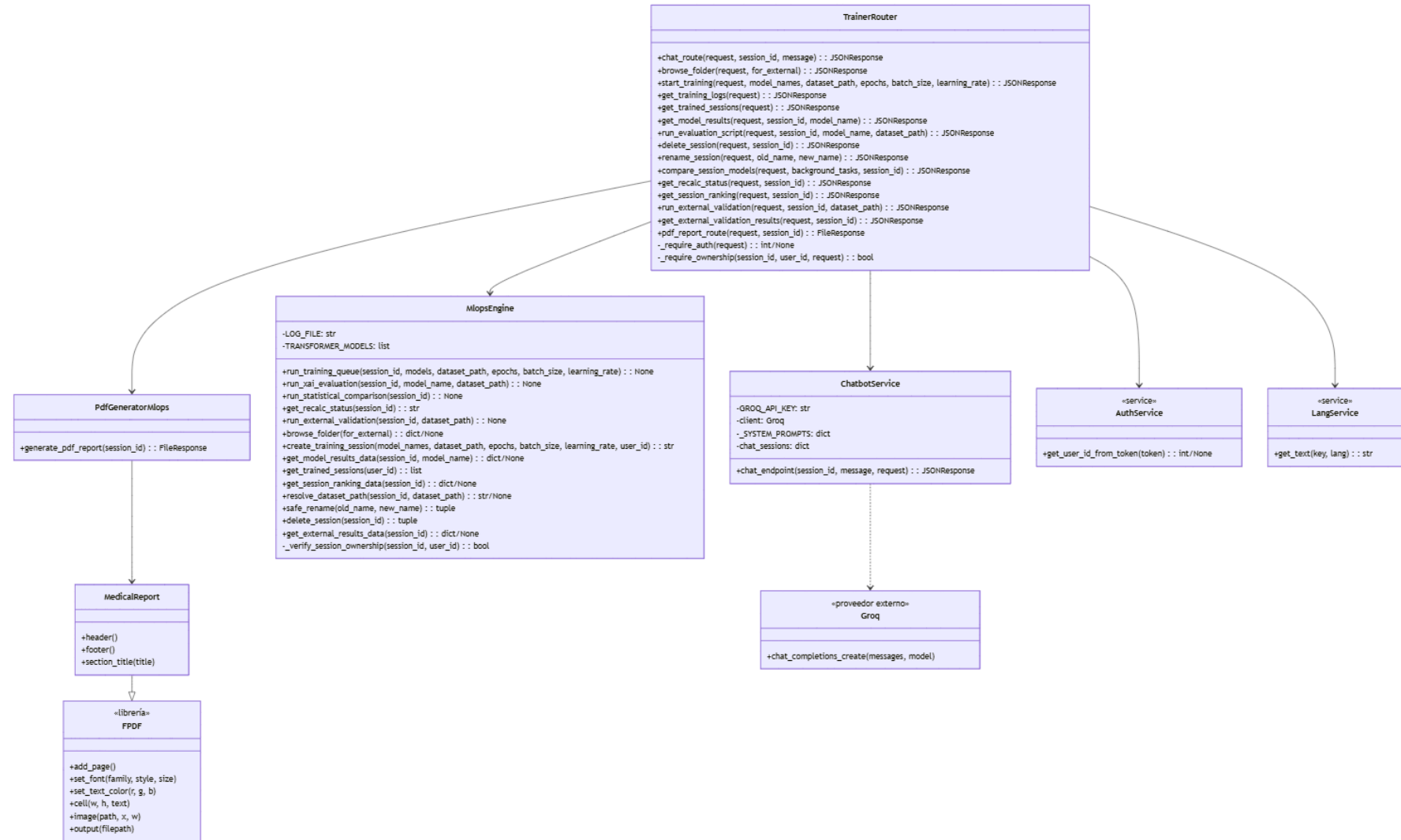


Ilustración x - Diagrama de clases del subsistema SD-004

5.4.2 Especificación de las clases

A continuación se definen las cinco clases propias de SD-004.

CL-0009	TrainerRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/trainer.py. Actúa como fachada ligera del laboratorio MLOps: expone los endpoints de conversación, exploración del dataset, lanzamiento de entrenamientos, consulta de logs, sesiones y resultados, ejecución del análisis XAI, comparativa estadística, validación externa, informes, renombrado y eliminación de sesiones. Comprueba la autenticación y la propiedad de las sesiones y delega la lógica en los servicios del subsistema.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: reexporta constantes del motor para compatibilidad y delega el resto en los servicios.
Operaciones	Nombre	Descripción	
	chat_route(request, session_id, message)	Gestiona la conversación con el asistente de configuración, delegando en ChatbotService.	
	browse_folder(request, for_external)	Explora la carpeta del dataset de entrenamiento o de validación externa mediante MlopsEngine.	
	start_training(request, model_names, dataset_path, epochs, batch_size, learning_rate)	Lanza un experimento: valida la ruta del dataset, crea la sesión mediante MlopsEngine y encola el trabajo de tipo training.	
	get_training_logs(request)	Devuelve las últimas líneas del registro de entrenamiento.	
	get_trained_sessions(request)	Devuelve las sesiones de entrenamiento del usuario mediante MlopsEngine.	
	get_model_results(request, session_id, model_name)	Devuelve los resultados de un modelo de la sesión, con las métricas K-fold, la calibración y las métricas XAI.	
	update_patient_name(request, session_id, model_name, dataset_path, consultation_id, new_name)	Ejecuta el análisis XAI manual de un modelo mediante MlopsEngine.	

	delete_session(request, session_id)	Elimina una sesión y sus artefactos mediante MlopsEngine, tras comprobar la propiedad.
	rename_session(request, old_name, new_name)	Renombra una sesión mediante MlopsEngine, tras comprobar la propiedad.
	compare_session_models(request, background_tasks, session_id)	Programa el recálculo de la comparativa estadística como tarea en segundo plano.
	get_recalc_status(request, session_id)	Devuelve el estado del recálculo de la comparativa (running o completed).
	get_session_ranking(request, session_id)	Devuelve el ranking de modelos y el heatmap de Wilcoxon mediante MlopsEngine.
	run_external_validation(request, session_id, dataset_path)	Encola la validación externa de la sesión con el tipo external_validation.
	get_external_validation_results(request, session_id)	Devuelve las métricas, la curva ROC y la matriz de DeLong de la validación externa.
	pdf_report_route(request, session_id)	Genera y sirve el informe PDF de la sesión mediante PdfGeneratorMlops.
	_require_auth(request)	Operación privada que resuelve la identidad del usuario mediante AuthService, devolviendo None sin sesión.
	_require_ownership(session_id, user_id, request)	Operación privada que comprueba la propiedad de la sesión mediante MlopsEngine, con la excepción del rol admin cuando la ruta lo permite.
Comentarios	La clase concentra un gran número de operaciones porque el laboratorio expone numerosos puntos de consulta y gestión; todas ellas mantienen la frontera de autorización: la comprobación de propiedad se resuelve antes de abrir cualquier operación sobre una sesión. Las operaciones de lanzamiento de entrenamiento y de validación externa encolan trabajos en la cola de SD-006.	

Tabla x - Especificación de clase CL-0009

CL-0010	ChatbotService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/chatbot_service.py. Gestiona la configuración conversacional del experimento: recibe los mensajes del usuario en lenguaje natural, los envía al asistente externo y devuelve una configuración estructurada del entrenamiento, solicitando los parámetros que falten. Trata al proveedor de inteligencia artificial como una frontera externa.		
Atributos	Nombre	Tipo	Descripción
	GROQ_API_KEY	str	Clave de la API del proveedor del asistente, obtenida del entorno; el único atributo del subsistema que conserva la credencial del proveedor.
	client	Groq	Cliente del proveedor externo, instanciado con la clave de la API; es None si la clave no está configurada.
	_SYSTEM_PROMPTS	dict	Diccionario de prompts del sistema por idioma, que guían la configuración conversacional.
	chat_sessions	dict	Diccionario que conserva el historial de mensajes de cada conversación, indexado por el identificador de la sesión de chat.
Operaciones	Nombre	Descripción	
	chat_endpoint(session_id, message, request)	Procesa un mensaje de la conversación: selecciona el prompt del sistema según el idioma, mantiene el historial de la sesión, solicita la configuración al proveedor externo y devuelve la respuesta con los parámetros de la configuración o la solicitud de los faltantes.	
Comentarios	La clase es el único componente del sistema que necesita la clave de la API del asistente; los routers no la incluyen en las respuestas ni en los registros. Los errores del proveedor externo se tratan como condiciones de la frontera y no se confunden con los errores de persistencia o de entrenamiento.		

Tabla x - Especificación de clase CL-0010

CL-0011	MlopsEngine		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/mlops_engine.py. Constituye el núcleo del laboratorio: organiza las sesiones de entrenamiento en el sistema de ficheros, orquesta la ejecución de los scripts del pipeline —entrenamiento K-fold, análisis XAI, comparación estadística y validación externa— y resuelve la lectura de los resultados y la comprobación de propiedad de las sesiones.		
Atributos	Nombre	Tipo	Descripción
	LOG_FILE	str	Ruta del archivo de registro del entrenamiento, donde el motor escribe la progresión del pipeline.
	TRANSFORMER_MODELS	list	Lista de las arquitecturas Transformer del proyecto, utilizada para bifurcar el script de entrenamiento.
Operaciones	Nombre	Descripción	
	run_training_queue(session_id, models, dataset_path, epochs, batch_size, learning_rate)	Orquesta el entrenamiento completo de la sesión: por cada modelo lanza el script de entrenamiento CNN o Transformer y los scripts de explicabilidad, y al finalizar ejecuta la comparación estadística que genera el ranking y la matriz de Wilcoxon.	
	run_xai_evaluation(session_id, model_name, dataset_path)	Ejecuta el análisis XAI cualitativo y cuantitativo de un modelo en modo manual.	
	run_statistical_comparison(session_id)	Regenera la comparativa estadística de la sesión, gestionando el marcador de estado del recálculo.	
	get_recalc_status(session_id)	Devuelve completed o running según el marcador de estado del recálculo.	
	run_external_validation(session_id, dataset_path)	Ejecuta la validación externa de la sesión y el test estadístico de DeLong sobre la cohorte independiente.	
	browse_folder(for_external)	Devuelve la ruta del dataset configurada por entorno o abre el selector de carpetas; distingue el dataset de entrenamiento del de validación externa.	
	create_training_session(model_names, dataset_path, epochs, batch_size, learning_rate, user_id)	Crea el directorio de la sesión en training_results y escribe su configuración, con el identificador del usuario propietario; devuelve el identificador de la sesión.	
	get_model_results_data(session_id, model_name)	Lee las métricas K-fold, la calibración y las métricas XAI de un modelo, junto con las rutas de sus artefactos; devuelve None si el modelo no dispone de resultados.	
	get_trained_sessions(user_id)	Enumera las sesiones con resultados que pertenecen al usuario, en orden descendente, con sus modelos.	

	get_session_ranking_data(session_id)	Lee el ranking de modelos, el heatmap de Wilcoxon y la configuración de la sesión; devuelve None si no hay ranking.
	resolve_dataset_path(session_id, dataset_path)	Devuelve la ruta del dataset indicada o la conservada en la sesión.
	safe_rename(old_name, new_name)	Valida y sanitiza el nuevo nombre de la sesión y ejecuta el renombrado del directorio, devolviendo el estado y el contenido de la respuesta.
	delete_session(session_id)	Elimina el directorio de la sesión y sus artefactos, devolviendo el estado y el contenido de la respuesta.
	get_external_results_data(session_id)	Lee las métricas, la curva ROC y la matriz de DeLong de la validación externa; devuelve None si no existen resultados.
	_verify_session_ownership(session_id, user_id)	Operación privada que comprueba que la configuración de la sesión contiene el identificador del usuario, como regla de propiedad del laboratorio.
Comentarios	La clase combina la persistencia híbrida de la plataforma: coordina la escritura de los artefactos en el sistema de ficheros y conserva en MySQL únicamente los trabajos de la cola. La comprobación de propiedad de las sesiones se resuelve en esta clase, que la comparte con el router sin duplicarla. Los scripts del pipeline se invocan como procesos externos que leen la configuración desde variables de entorno.	

Tabla x - Especificación de clase CL-0011

CL-0012	PdfGeneratorMlops		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/pdf_generator_mlops.py. Construye el informe consolidado de una sesión de entrenamiento: recopila la configuración del experimento, el ranking con el heatmap de Wilcoxon, la validación externa con la curva ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor, empleando la clase concreta MedicalReport.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	generate_pdf_report(session_id)	Compone el informe de la sesión, lo guarda en el directorio de la sesión con el nombre Informe_Completo_{session_id}.pdf y lo devuelve como descarga; responde HTTP 404 si la sesión no existe.	
Comentarios	La generación del informe separa el conocimiento del formato del router: el generador recibe la información preparada por el motor y construye el documento, de modo que el router no conoce el formato interno del informe. El informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios.		

Tabla x - Especificación de clase CL-0012

CL-0013	PdfGeneratorMlops		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase concreta del informe MLOps, correspondiente a la clase MedicalReport del módulo services/pdf_generator_mlops.py. Especializa la clase FPDF de la librería de generación de documentos y redefine la cabecera corporativa, el pie de página y el formato de los títulos de sección del informe.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	header()	Redefine la cabecera de cada página del informe con el título corporativo y el encabezado del protocolo MLOps.	
	footer()	Redefine el pie de página con el número de página y la indicación de generación automática.	
	section_title(title)	Escribe un título de sección con el formato propio del informe.	
Comentarios	Esta clase constituye la segunda herencia real del diseño de clases de vitalXAI: su relación con FPDF se representa en el modelo de clases como una especialización. El generador de informes la instancia para componer el documento consolidado de la sesión.		

Tabla x - Especificación de clase CL-0013

5.5 Subsistema SD-005: Supervisión y administración

El subsistema SD-005 materializa las operaciones de supervisión y administración reservadas al rol de administrador y agrupa una única clase de diseño propia. La clase AdminRouter se corresponde con el router routers/admin.py y concentra las tres operaciones de consulta administrativa: el listado de usuarios, las consultas de un usuario concreto y el detalle de una consulta, todas ellas precedidas de la comprobación de la autorización administrativa. El subsistema no dispone de clases de servicio propias, porque su responsabilidad es establecer la autorización y coordinar las consultas globales reutilizando los servicios existentes. AdminRouter depende de AuthService, perteneciente a SD-001, para resolver la identidad y consultar el rol, y de MlopsEngine, perteneciente a SD-004, para recuperar las sesiones del laboratorio de un usuario en la operación de consulta de su actividad.

5.5.1 Diagrama de clases del subsistema

El modelo de clases de SD-005 se representa en la figura 87. El diagrama muestra la clase propia del subsistema con sus operaciones, y las dependencias hacia las clases AuthService y MlopsEngine, pertenecientes a SD-001 y SD-004, representadas con las operaciones que la supervisión consume. El modelo no presenta herencias ni clases abstractas: la clase AdminRouter es una clase concreta que centraliza la comprobación de permisos y coordina las consultas globales.

El diagrama refleja la simplicidad estructural del subsistema y su decisión de diseño central: la operación privada `_require_admin()` se antepone a todas las consultas administrativas, de modo que ninguna ruta ejecuta una operación de lectura sobre usuarios o consultas sin haber resuelto antes la identidad y el rol del solicitante. La dependencia hacia AuthService materializa el mecanismo de identificación de SD-001, del que el router obtiene el usuario y consulta su rol; la dependencia hacia MlopsEngine materializa la recuperación de las sesiones del laboratorio en la consulta de la actividad de un usuario. El subsistema no depende del servicio de idioma porque sus mensajes de autorización son textos fijos, y no duplica la lógica de propiedad del historial ni del laboratorio: accede a los datos con las mismas tablas y servicios que el resto de la aplicación.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `admin_users()` implementa el CU-031; la operación `admin_user_consultations()` implementa el CU-032; la operación `admin_get_consultation()` implementa el CU-033; y la operación privada `_require_admin()` materializa la comprobación de permisos que condiciona las respuestas HTTP 401 y 403 de los diagramas de secuencia.

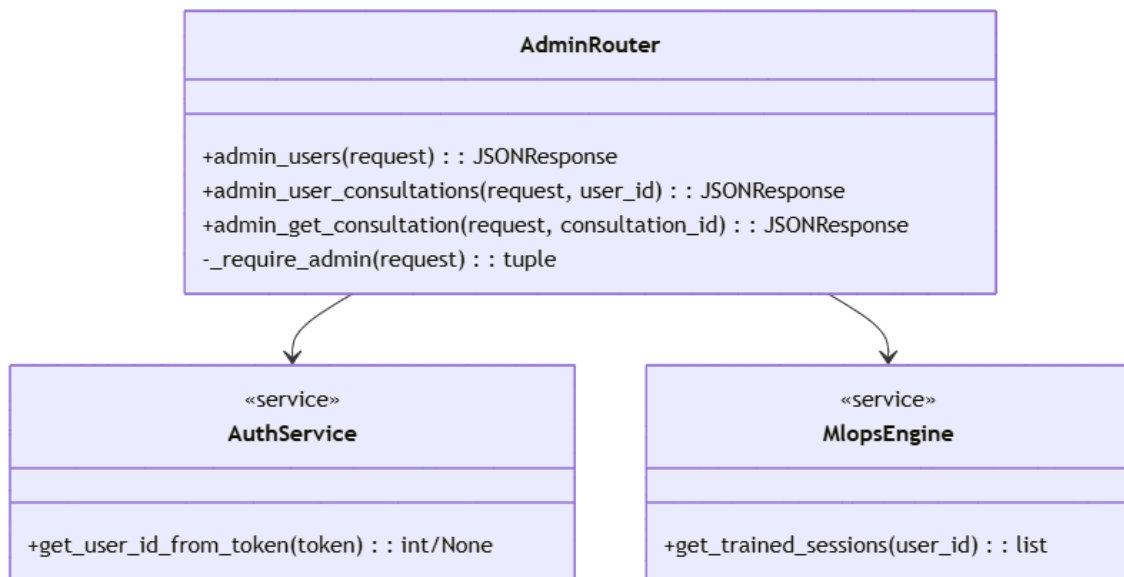


Ilustración x - Diagrama de clases del subsistema SD-005

5.5.2 Especificación de las clases

A continuación se define la clase propia de SD-005.

CL-0014	AdminRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/admin.py. Proporciona las operaciones de supervisión administrativa: el listado de usuarios con sus recuentos de diagnósticos y de sesiones del laboratorio, las consultas y las sesiones de un usuario concreto, y el detalle de una consulta. Centraliza la comprobación de la autorización administrativa en la operación privada _require_admin() y reutiliza los servicios existentes sin duplicar la lógica de propiedad.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	admin_users(request)	Devuelve el listado de usuarios con el número de diagnósticos, obtenido mediante una consulta agregada sobre users y consultations, y con el número de sesiones del laboratorio, calculado inspeccionando las configuraciones del sistema de ficheros.	
	admin_user_consultations(request, user_id)	Devuelve las consultas de diagnóstico de un usuario concreto, ordenadas por fecha descendente, junto con sus sesiones de entrenamiento recuperadas mediante MlopsEngine; responde HTTP 404 si el usuario no existe.	
	admin_get_consultation(request, consultation_id)	Devuelve el detalle completo de una consulta por su identificador, con las rutas de los artefactos, la predicción, la confianza y los metadatos; responde HTTP 404 si la consulta no existe.	
	_require_admin(request)	Operación privada que resuelve la identidad mediante AuthService y consulta el rol en la tabla users, devolviendo una tupla con el identificador y un indicador de rol administrativo; distingue la ausencia de identidad, la identidad sin privilegios y la identidad administrativa.	
Comentarios	La clase mantiene la frontera entre el control de permisos y la representación de la información: no duplica la lógica de propiedad del historial ni del laboratorio, y la autorización administrativa se comprueba en el servidor, no se infiere de parámetros del navegador. La protección reutiliza la identidad de SD-001 sin una segunda autenticación, y la traza de auditoría administrativa permanece como condición pendiente. tal y como se declaró en el capítulo 18.		

Tabla x - Especificación de clase CL-0014

5.6 Subsistema SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa la cola persistente de trabajos y las capacidades transversales de la plataforma, y agrupa cuatro clases de diseño propias. La clase `QueueRouter` se corresponde con el router `routers/queue.py` y expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente. La clase `QueueWorker` se corresponde con el servicio `services/queue_worker.py` y actúa como consumidor de la cola: reclama los trabajos pendientes, ejecuta el flujo correspondiente en el executor y actualiza el estado del trabajo. La clase `LangService` se corresponde con el servicio `services/lang.py` y concentra la internacionalización de la plataforma. La clase `QueueJob` modela la entidad persistente de la cola, la fila de la tabla `job_queue`, que constituye la clase más compleja del subsistema por su máquina de estados, compartida por los subsistemas que generan trabajos.

5.6.1 Diagrama de clases del subsistema

El modelo de clases de SD-006 se representa en la figura 88. El diagrama muestra las cuatro clases propias del subsistema, la dependencia del router hacia el servicio de identidad de SD-001, y las dependencias del worker hacia las clases `MIEngine` y `MlopsEngine`, pertenecientes a SD-002 y SD-004, que representan los flujos que el worker ejecuta según el tipo de trabajo. La clase `QueueJob` se representa con los atributos de su estado persistente y aparece asociada tanto al router, que consulta y cancela los trabajos, como al worker, que los reclama y actualiza.

El diagrama refleja la frontera entre el ciclo de petición y la ejecución asíncrona: el router y el worker comparten la persistencia de la cola, materializada en la clase `QueueJob`, pero no comparten el ciclo de petición. El worker despacha el trabajo según su tipo: para los diagnósticos invoca la clase `MIEngine` de SD-002, y para los entrenamientos y las validaciones externas invoca la clase `MlopsEngine` de SD-004; esas clases se representan con las operaciones que el worker utiliza. La clase `LangService` no presenta dependencias de colaboración en el diagrama porque sus consumidores pertenecen a otros subsistemas: el mecanismo de internacionalización se ofrece como un servicio transversal que cualquier router o servicio invoca. El modelo no presenta herencias ni clases abstractas.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `queue_status()` de `QueueRouter` implementa el CU-034; la operación `cancel_job()` implementa el CU-035; y las operaciones del worker materializan el procesamiento asíncrono que orquesta los flujos de SD-002 y SD-004. La clase `QueueJob` concentra la máquina de estados de la cola, que se describe en su definición.

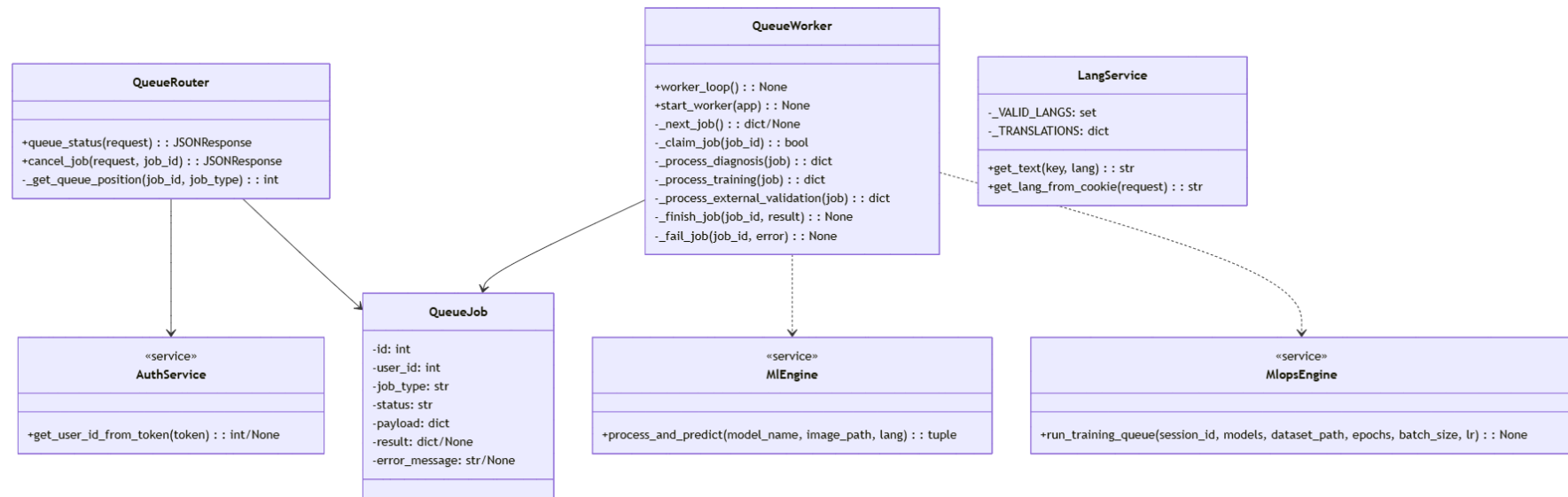


Ilustración x - Diagrama de clases del subsistema SD-006

5.6.2 Especificación de las clases

A continuación se definen las cuatro clases propias de SD-006.

CL-0015	QueueRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/queue.py. Expone la consulta del estado de los trabajos de la cola y la cancelación de un trabajo pendiente: devuelve los trabajos recientes del usuario con su estado y su posición, interpreta el payload según el tipo de trabajo y aplica la cancelación mediante una actualización condicional.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	queue_status(request)	Devuelve los últimos veinte trabajos del usuario con su estado, sus fechas y la posición de los trabajos encolados; interpreta el payload por tipo (modelo del diagnóstico, sesión y modelos del entrenamiento, o modelos de la validación externa leídos de su configuración) sin enviar el payload completo al navegador.	
	cancel_job(request, job_id)	Cancela un trabajo mediante una actualización condicionada a que pertenezca al usuario y continúe en queued; responde HTTP 200 si la actualización afectó a una fila y HTTP 404 si el trabajo no está en cola.	
	_get_queue_position(job_id, job_type)	Operación privada que calcula la posición de un trabajo encolado, respetando la prioridad de la cola según el tipo de trabajo.	
Comentarios	La cancelación solo opera sobre trabajos pendientes: un trabajo reclamado por el worker no se interrumpe, en coherencia con la máquina de estados de la cola. La consulta del estado filtra siempre por el usuario autenticado, aplicando el aislamiento de datos entre cuentas.		

Tabla x - Especificación de clase CL-0015

CL-0016	QueueWorker		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/queue_worker.py. Actúa como consumidor de la cola de trabajos: al iniciar la aplicación restablece los trabajos que quedaron en estado running, selecciona el siguiente trabajo pendiente, lo reclama mediante una actualización condicionada a que siga en queued y ejecuta el flujo correspondiente en el executor, marcando el resultado como completado o fallido. No comparte el ciclo de petición con los routers.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	worker_loop()	Bucle infinito del worker: selecciona el siguiente trabajo pendiente, lo reclama y ejecuta el flujo correspondiente en el executor, esperando cuando la cola está vacía.	
	start_worker(app)	Restablece los trabajos en running a queued y crea la tarea asíncrona del bucle del worker, conservándola en el estado de la aplicación.	
	_next_job()	Operación privada que selecciona el primer trabajo queued, con prioridad para los diagnósticos frente a los entrenamientos.	
	_claim_job(job_id)	Operación privada que reclama un trabajo mediante una actualización condicionada a que siga en queued, devolviendo True solo si la actualización afectó a una fila.	
	_process_diagnosis(job)	Operación privada que procesa un trabajo de diagnóstico: invoca MIEngine, XaiGenerator y PdfGenerator de SD-002 y persiste la consulta con sus artefactos.	
	_process_training(job)	Operación privada que procesa un trabajo de entrenamiento, delegando en MlopsEngine.	
	_process_external_validation(job)	Operación privada que procesa una validación externa, delegando en MlopsEngine.	
	_finish_job(job_id, result)	Operación privada que marca el trabajo como completed con su resultado en formato JSON.	
	_fail_job(job_id, error)	Operación privada que marca el trabajo como failed con el mensaje de error limitado.	
Comentarios	La reclamación condicional evita que dos iteraciones del worker procesen el mismo registro, lo que introduce una garantía de consistencia útil si la ejecución evoluciona hacia más de un consumidor. Los flujos de procesamiento se ejecutan en el executor para no bloquear el bucle de eventos de la aplicación.		

Tabla x - Especificación de clase CL-0016

CL-0017	LangService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/lang.py. Concentra la internacionalización de la plataforma: mantiene el diccionario de mensajes del backend en los cuatro idiomas soportados, obtiene el idioma seleccionado por el usuario y devuelve los textos de la interfaz y de las respuestas. No requiere un servicio externo ni una base de datos adicional.		
Atributos	Nombre	Tipo	Descripción
	_VALID_LANGS	set	Conjunto de los idiomas válidos de la plataforma: español, inglés, chino e hindú.
	_TRANSLATIONS	dict	Diccionario de traducciones de los mensajes del backend, indexado por idioma y por clave de mensaje.
Operaciones	Nombre	Descripción	
	get_text(key, lang)	Devuelve el mensaje correspondiente a la clave en el idioma indicado, aplicando el español como valor por defecto cuando el idioma o la clave no están disponibles.	
	get_lang_from_cookie(request)	Lee la cookie de idioma de la petición, valida que pertenezca al conjunto permitido y devuelve el idioma aplicable, con el español como valor por defecto.	
Comentarios	La clase constituye el mecanismo transversal de presentación del idioma: los routers y los servicios de los subsistemas la invocan para localizar sus mensajes, las etiquetas de predicción, los títulos de los mapas de explicabilidad, los textos de los informes y los mensajes del asistente, en coherencia con el caso de uso CU-004.		

Tabla x - Especificación de clase CL-0017

CL-0018	QueueJob		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño que modela la entidad persistente de la cola de trabajos, correspondiente a la fila de la tabla job_queue. Conserva el estado de cada tarea asíncrona —diagnóstico, entrenamiento o validación externa— junto con el usuario propietario, el tipo, el payload serializable y el resultado o el error. Constituye la clase más compleja del subsistema por su máquina de estados, que determina las transiciones permitidas entre los estados de un trabajo.		
Atributos	Nombre	Tipo	Descripción
	id	int	Identificador del trabajo, asignado automáticamente por la persistencia.
	user_id	int	Identificador del usuario propietario del trabajo, que condiciona la consulta y la cancelación.
	job_type	str	Tipo del trabajo: diagnosis, training o external_validation.
	status	str	Estado del trabajo en la máquina de estados: queued, running, completed, failed o cancelled.
	payload	dict	Configuración serializable del trabajo, que contiene el modelo y la ruta de la imagen, o la sesión y los hiperparámetros.
	result	dict	Resultado del trabajo cuando se completa, en formato JSON; None en el resto de los estados.
	error_message	str	Mensaje de error cuando el trabajo falla, limitado en longitud; None en el resto de los estados.
Operaciones	Nombre	Descripción	
	-	La clase no expone operaciones propias: las transiciones de su estado las ejecutan el router (cancelación) y el worker (reclamación, finalización y fallo), de modo que el comportamiento de la máquina de estados se describe mediante el diagrama de transición de estados siguiente.	
Comentarios	La máquina de estados de la clase es compartida por los subsistemas que generan trabajos: SD-002 crea los diagnósticos, SD-004 crea los entrenamientos y las validaciones externas, y el worker de SD-006 aplica las transiciones. La cancelación solo opera sobre queued, y un trabajo en running no se interrumpe mediante una actualización administrativa, de modo que la reclamación condicional del worker mantiene la coherencia entre el estado persistido y la ejecución en curso.		

Tabla x - Especificación de clase CL-0018

6 Diseño de interfaces de los subsistemas

El diseño de interfaces constituye la etapa del diseño que concreta las ventanas del sistema y la navegación entre ellas, refinando la estructura de interfaz definida en el análisis. Mientras que el capítulo 21 determinó el comportamiento de los casos de uso y el capítulo 22 la estructura de las clases, este capítulo establece cómo se presenta esa funcionalidad al usuario: qué ventanas componen la aplicación, qué campos y acciones contiene cada una, cómo se navega entre ellas y qué informes se generan (Sharp, Rogers, & Preece, 2019). El diseño de interfaces aplica los principios de usabilidad que orientan la interacción del profesional sanitario con la plataforma: la claridad de las acciones, la consistencia entre ventanas y la adecuación de los mensajes de error al contexto (Nielsen, 1994).

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño del capítulo 18: cada apartado corresponde a un subsistema (SD-001 a SD-006) y describe las interfaces que materializan su responsabilidad. Para cada subsistema se presentan tres elementos, conforme a la guía de diseño de la memoria (punto 6): el flujo de navegación, que define la navegación definitiva entre las ventanas del subsistema; la especificación de las interfaces, que documenta cada ventana mediante la ficha formal de interfaz IU-NNNN con sus campos, sus botones y sus enlaces; y los informes del subsistema, que describen cada documento generado mediante la ficha formal IF-NNNN con sus datos y sus agrupaciones. Las ventanas de la aplicación corresponden a las plantillas reales del sistema: la página de inicio de sesión, el registro, el panel de diagnóstico y el laboratorio de entrenamiento, junto con las vistas que se integran en ellas.

El diagrama de la figura representa la navegación global de la aplicación. El visitante entra por la página de inicio de sesión (IU-0001), desde la que accede al registro (IU-0002); una vez autenticado o registrado, el sistema conduce al panel de diagnóstico (IU-0003), que constituye el centro de la aplicación. Desde el panel se alcanzan las vistas integradas del historial y el detalle (IU-0004), la administración (IU-0006, solo para el rol administrador), la cola de trabajos (IU-0007) y el laboratorio de entrenamiento (IU-0005); desde el laboratorio también se alcanza la cola de trabajos.

La navegación global refleja la estructura funcional de la plataforma: las ventanas públicas de acceso y registro (SD-001) preceden al resto, y el panel de diagnóstico se sitúa como punto central desde el que se accede al laboratorio y a las vistas integradas. Las vistas del historial, la administración y la cola de trabajos no constituyen ventanas independientes de la aplicación, sino secciones y diálogos integrados en el panel de diagnóstico o en el laboratorio, que se describen en los apartados de sus subsistemas correspondientes.

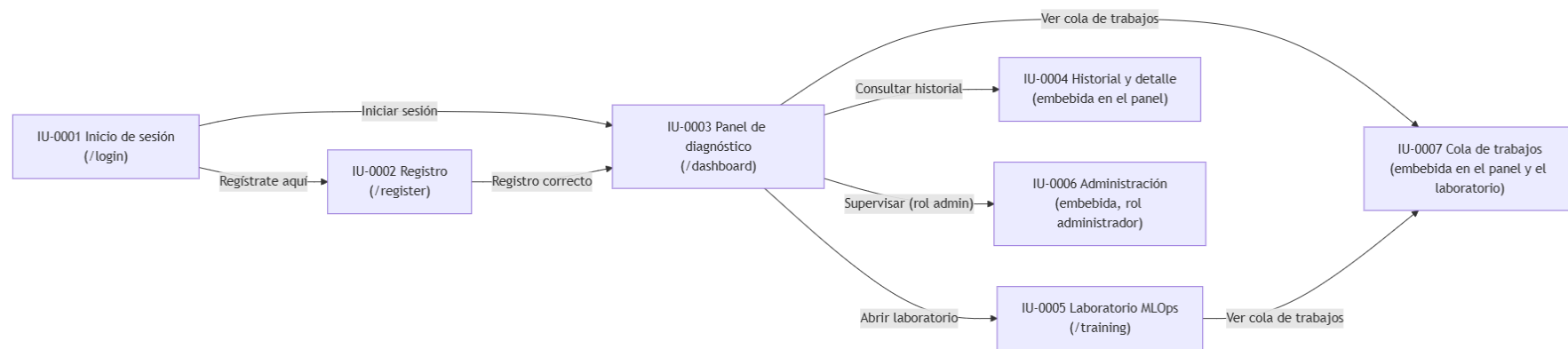


Ilustración x - Diagrama de navegación global de la aplicación

6.1 Subsistema SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa las interfaces de acceso de la plataforma: la página de inicio de sesión y la página de registro. Ambas constituyen las únicas ventanas públicas de la aplicación, accesibles sin autenticación, y comparten los mecanismos transversales de presentación: el selector de idioma y el cambio del tema visual. La página de inicio de sesión permite al visitante autenticarse con sus credenciales, y la página de registro permite crear una cuenta; en ambos casos, la autenticación correcta conduce al panel de diagnóstico del subsistema SD-002.

6.1.1 Flujo de navegación

El flujo de navegación del subsistema SD-001, representado en la figura, refleja la progresión del visitante hasta la entrada al área privada. El visitante llega a la página de inicio de sesión, desde la que puede desplazarse a la página de registro mediante el enlace correspondiente y volver a la página de inicio desde el registro. Tras completar el inicio de sesión o el registro, el sistema conduce al usuario al panel de diagnóstico, que pertenece a SD-002; la navegación entre SD-001 y SD-002 materializa la frontera entre el área pública y el área privada.

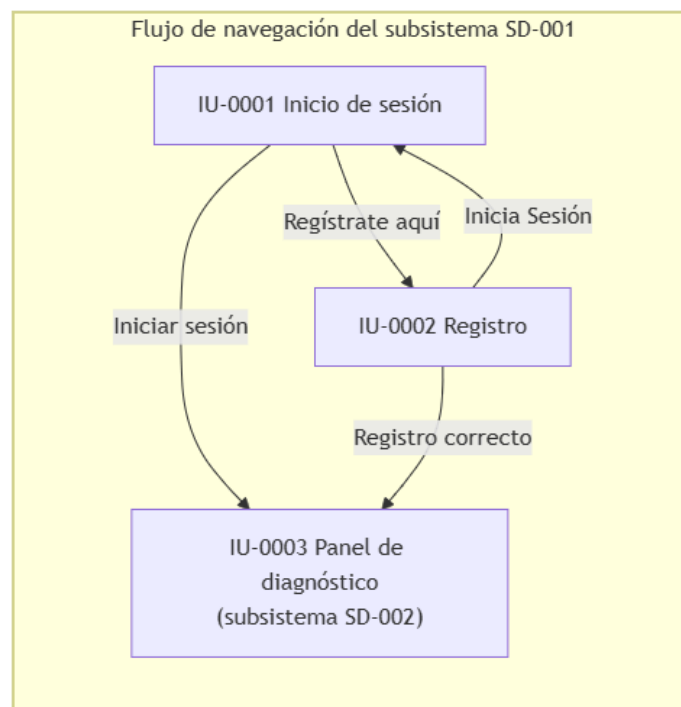


Ilustración x - Flujo de navegación del subsistema SD-001

El flujo no presenta rutas de navegación a otras ventanas del subsistema: las dos ventanas se enlazan mutuamente y ambas desembocan en el panel de diagnóstico. La página de inicio de sesión recibe el parámetro de error en la URL cuando una autenticación falla, de modo que muestra el mensaje genérico sin revelar qué parte de las credenciales era incorrecta, en coherencia con el CU-002 del análisis.

6.1.2 Especificación de las interfaces

A continuación se especifican las dos interfaces propias de SD-001.

IU-0001: Inicio de sesión					
Descripción	Ventana de acceso a la plataforma, correspondiente a la plantilla login.html (ruta /). Presenta el formulario de inicio de sesión con el nombre de usuario y la contraseña, el mensaje de error genérico, el selector de idioma y el cambio del tema visual. Ante una autenticación correcta, redirige al panel de diagnóstico; ante un fallo, muestra un mensaje genérico sin revelar la causa.				
Campos	Nombre	Tipo Datos	Editable/Consulta	Oblig.	Descripción
	username	Texto	Editable	Sí	Nombre de usuario o correo electrónico con el que se identifica la cuenta.
	password	Contraseña	Editable	Sí	Contraseña de la cuenta; se introduce enmascarada.
Botones/Enlaces	Nombre	Acción			
	Entrar al Sistema	Envía el formulario al endpoint POST /login; con credenciales válidas, el sistema establece las cookies de sesión y redirige al panel de diagnóstico; con credenciales inválidas, redirige a la página con el parámetro de error para mostrar el mensaje genérico.			
	Regístrate aquí	Navega a la página de registro (/register).			
	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.			
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.			

Tabla x - Especificación de interfaz IU-0001

La página constituye la puerta de entrada del sistema: todos los flujos privados comienzan en ella. El mensaje de error es genérico para no revelar si el fallo corresponde al nombre de usuario o a la contraseña, y el endpoint de acceso está limitado a cinco peticiones por minuto. La ventana se sirve sin almacenamiento de caché, de modo que no se reutiliza una sesión anterior.

IU-0002: Registro					
Descripción	Ventana de creación de cuenta, correspondiente a la plantilla register.html (ruta /register). Presenta el formulario de registro con el nombre, el apellido, la especialidad, el nombre de usuario y la contraseña, junto con el selector de idioma y el cambio del tema visual. Envía los datos al endpoint de registro de forma asíncrona; ante un registro correcto muestra la confirmación y conduce al panel de diagnóstico, y ante un error muestra el mensaje correspondiente.				
Campos	Nombre	Tipo Datos	Editable/ Consulta	Oblig.	Descripción
	first_name	Texto	Editable	Sí	Nombre del profesional.
	last_name	Texto	Editable	Sí	Apellido del profesional.
	role	Lista de selección	Editable	Sí	Especialidad del profesional: Médico Residente, Radiólogo Especialista, Médico de Familia o Neumólogo.
	username	Texto	Editable	Sí	Nombre de usuario o correo electrónico con el que se identificará la cuenta.
	password	Contraseña	Editable	Sí	Contraseña de la cuenta; el servidor exige una longitud mínima de ocho caracteres.
Botones/Enlaces	Nombre	Acción			
	Registrarme	Envía el formulario al endpoint POST /api/register de forma asíncrona; ante un registro correcto muestra la confirmación y redirige al panel de diagnóstico, y ante un error muestra el mensaje del campo afectado o de la duplicidad.			
	Inicia Sesión	Navega a la página de inicio de sesión (/).			
	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.			
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.			

Tabla x - Especificación de interfaz IU-0002

La ventana mantiene la puerta de entrada autónoma del análisis: el alta no requiere la intervención de un administrador. La validación del formato del correo, de la longitud de la contraseña y de la unicidad del usuario se resuelve en el servidor, y la interfaz muestra los errores devueltos sin interrumpir la sesión de navegación. Tras el registro correcto, el sistema establece las cookies de sesión y el visitante queda autenticado, en coherencia con el CU-001 y con la decisión de diseño del capítulo 21.

6.1.3 Informes del subsistema

El subsistema SD-001 no genera informes: sus ventanas se limitan a la autenticación y a la creación de cuentas, y no producen documentos descargables. Los informes de la plataforma corresponden a los subsistemas funcionales que generan resultados: el informe PDF del diagnóstico, que se describe en el apartado de SD-002, y el informe consolidado de las sesiones de entrenamiento, que se describe en el apartado de SD-004. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

6.2 Subsistema SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 materializa la interfaz clínica de la plataforma: el panel de diagnóstico, correspondiente a la plantilla `dashboard.html` (ruta `/dashboard`), que constituye el entorno central de la aplicación y sobre el que se realiza el ciclo completo de una consulta clínica. La ventana integra además las vistas del historial y el detalle de las consultas, la cola de trabajos y el panel de administración, que pertenecen a los subsistemas SD-003, SD-006 y SD-005, de modo que la ficha de interfaz se centra en las funciones de diagnóstico y se remite a los apartados correspondientes para el resto de las vistas. El subsistema genera además el informe PDF del diagnóstico (IF-0001), que se produce durante el procesamiento de la consulta.

6.2.1 Flujo de navegación

El flujo de navegación del subsistema SD-002, representado en la figura, sitúa el panel de diagnóstico como el punto central de la navegación del usuario autenticado. El usuario accede al panel tras el inicio de sesión o el registro, y desde él realiza el ciclo de diagnóstico sin abandonar la ventana: carga la radiografía, selecciona el modelo, solicita el análisis y visualiza el resultado y los mapas de explicabilidad en la misma vista. Desde el panel se alcanzan el historial y el detalle de las consultas (SD-003), el laboratorio de entrenamiento (SD-004) y la cola de trabajos (SD-006).

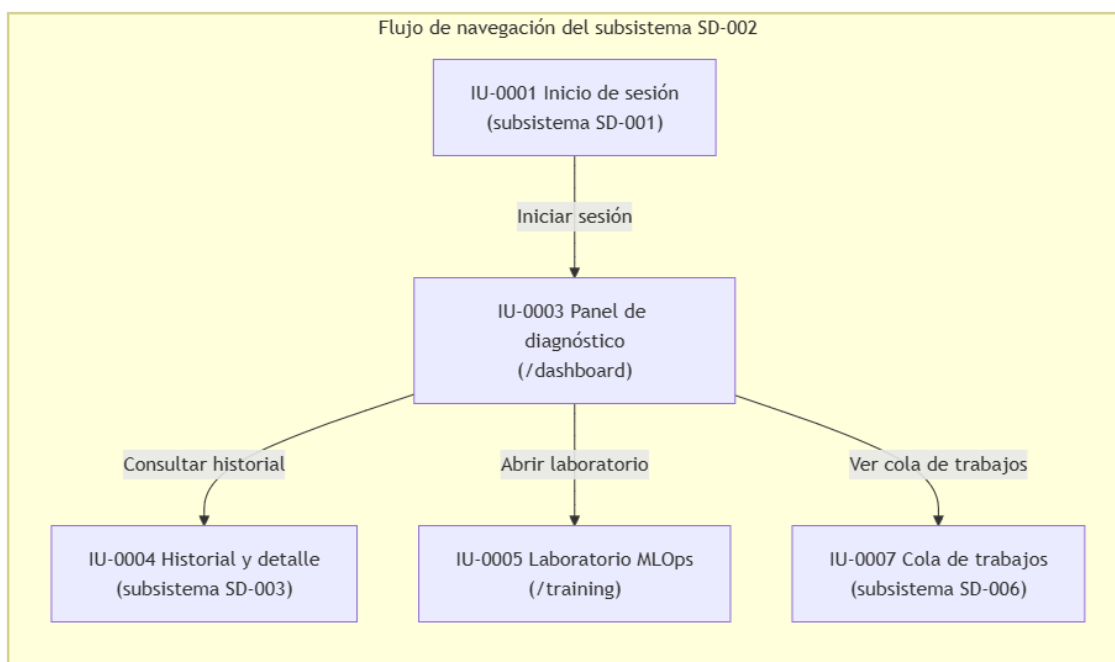


Ilustración x - Flujo de navegación del subsistema SD-002

El flujo refleja la naturaleza central del panel de diagnóstico: el ciclo clínico se resuelve en una única ventana, sin transiciones intermedias, y el resto de los subsistemas se alcanzan desde el panel mediante la barra lateral. La navegación hacia el historial y el detalle, hacia el laboratorio y hacia la cola de trabajos materializa las relaciones de colaboración entre SD-002 y los subsistemas SD-003, SD-004 y SD-006 descritas en el capítulo 18.

6.2.2 Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-002, centrada en las funciones de diagnóstico; las vistas integradas del historial, la cola y la administración se especifican en los apartados de sus subsistemas.

IU-0003: Panel de diagnóstico					
Descripción	Ventana central de la aplicación, correspondiente a la plantilla dashboard.html (ruta /dashboard). Presenta el entorno de diagnóstico rápido: la zona de carga de la radiografía, el selector del modelo de inteligencia artificial, el área de conversación donde se muestra el resultado del análisis y los mapas de explicabilidad, y el visor de imágenes. Integra en su barra lateral la navegación al laboratorio, el historial de consultas, el panel de la cola de trabajos, el acceso a la administración (solo para el rol administrador) y el cierre de sesión. Solo es accesible para usuarios autenticados.				
Campos	Nombre	Tipo Datos	Editable/ Consulta	Oblig.	Descripción
	model-selector	Lista de selección	Editable	Sí	Arquitectura de inteligencia artificial para el diagnóstico, entre las CNN Clásicas disponibles (DenseNet121, ResNet50, MobileNetV2, EfficientNetB0 y ConvNeXtTiny).
	file-input	Archivo	Editable	Sí	Radiografía de tórax en formato JPEG o PNG, de hasta 10 MB, que se adjunta mediante la zona de carga.
	Área de resultado	Contenido	Consulta	-	Zona de la ventana que muestra la imagen enviada, el resultado del diagnóstico (etiqueta y confianza), el modelo empleado y el mapa de explicabilidad generado.
Botones/Enlaces	Nombre	Acción			
	Zona de carga	Permite seleccionar o arrastrar la radiografía de tórax; al adjuntarla habilita el envío.			
	Enviar	Envía la solicitud de diagnóstico al endpoint POST /predict, muestra la posición del trabajo en la cola y sondea su estado hasta que se completa.			
	Diagnóstico Rápido	Recarga el panel de diagnóstico (GET /dashboard).			
	Laboratorio de Entrenamiento	Navega al laboratorio de experimentación MLOps (GET /training), del subsistema SD-004.			
	Historial	Muestra el listado de consultas del usuario en la barra lateral, correspondiente a SD-003.			
	Cerrar Sesión	Cierra la sesión del usuario (GET /logout), del subsistema SD-001.			

	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.

Tabla x - Especificación de interfaz IU-0003

La ventana constituye el entorno de trabajo del profesional sanitario: el diagnóstico se solicita y se visualiza sin abandonar la vista, y la interfaz permanece operativa durante el procesamiento asíncrono mediante el sondeo del estado de la cola. La respuesta del sistema localiza las etiquetas según el idioma de la sesión. La ventana integra las vistas del historial, la cola y la administración, que se especifican en los apartados de SD-003, SD-006 y SD-005 respectivamente.

6.2.3 Informes del subsistema

A continuación se especifica el informe propio de SD-002.

IF- 0001: Informe del diagnóstico				
Descripción	Informe PDF de una consulta de diagnóstico, generado durante el procesamiento de la consulta mediante <code>services/pdf_generator.py</code> . Recoge la identificación de la consulta con la fecha y el modelo empleado, el diagnóstico con su nivel de confianza, la radiografía original y el mapa de explicabilidad, y se entrega como documento descargable desde el detalle de la consulta.			
Módulo de Interfaz	IU-0003 Panel de diagnóstico.			
Datos	Campo	Ordenación	Tipo Datos	Descripción
	Fecha	—	Fecha/hora	Fecha y hora de la consulta.
	Modelo	—	Texto	Arquitectura de inteligencia artificial empleada en el diagnóstico.
	Diagnóstico	—	Texto	Etiqueta de la predicción (Neumonía o Normal), con el color de diagnóstico acorde con el resultado.
	Confianza	—	Numérico	Nivel de confianza de la predicción.
	Radiografía original	—	Imagen	Radiografía de tórax enviada por el usuario.
	Mapa de explicabilidad	—	Imagen	Mapa de calor XAI que justifica la predicción.
Resumen/Acumulado	Resumen			Campos del Resumen
	-			N/A

Tabla x - Especificación de informe IF-0001

El informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial. La generación del documento se adelanta al momento del diagnóstico, de modo que la descarga posterior no depende de una nueva operación del sistema.

6.3 Subsistema SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa la interfaz del historial y el detalle de las consultas, integrada en el panel de diagnóstico. La vista del historial ocupa la barra lateral del panel, donde se muestran las consultas del usuario agrupadas por modelo, y la vista del detalle ocupa el área principal al abrir una consulta, donde se presentan los artefactos y los metadatos de la consulta junto con las acciones de renombrado y eliminación. Ambas vistas constituyen la interfaz IU-0004, correspondiente a las secciones del historial y del detalle de la plantilla dashboard.html, y materializan los casos de uso CU-011 a CU-014.

6.3.1 Flujo de navegación

El flujo de navegación del subsistema SD-003, representado en la figura, refleja la cadena de navegación del análisis: desde el panel de diagnóstico se accede al historial, desde el listado se abre el detalle de una consulta, y desde el detalle se ejecutan, de forma opcional, el renombrado o la eliminación, además de la vuelta al listado. La navegación entre el listado, el detalle y las operaciones de gestión reproduce las relaciones de extensión de los casos de uso del subsistema.

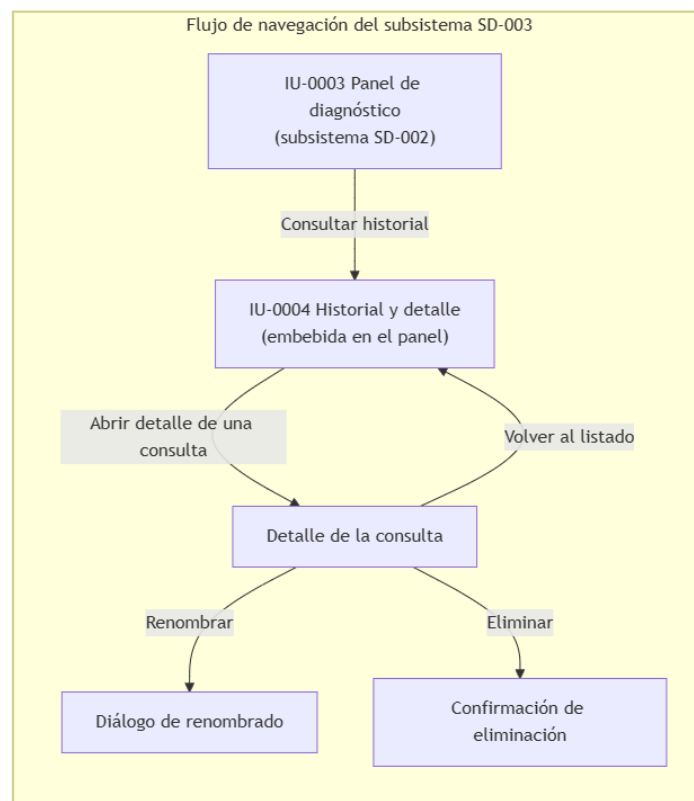


Ilustración x - Flujo de navegación del subsistema SD-003

El flujo refleja la progresión del usuario por el historial: el listado conduce al detalle de una consulta, y desde el detalle se alcanzan las operaciones opcionales de gestión. La navegación hacia el detalle y hacia las operaciones es de ida y vuelta: el usuario puede volver al listado o al panel en cualquier momento, sin que las acciones de gestión interrumpan la navegación principal.

6.3.2 Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-003.

IU-0004: Historial y detalle					
Descripción	Vista integrada del historial y el detalle de las consultas, correspondiente a las secciones del historial y del detalle de la plantilla dashboard.html. La vista del historial ocupa la barra lateral del panel de diagnóstico y muestra las consultas del usuario agrupadas por modelo, con la imagen, la fecha, la etiqueta y la confianza de cada una. La vista del detalle ocupa el área principal al abrir una consulta y muestra la radiografía original, el mapa de explicabilidad, el diagnóstico, la confianza, el modelo, el nombre visible, la fecha y las acciones de renombrado y eliminación.				
Campos	Nombre	Tipo Datos	Editable/Consulta	Oblig.	Descripción
	Listado de consultas	Contenido	Consulta	—	Listado de las consultas del usuario, agrupado por modelo, con la imagen, la fecha, la etiqueta y la confianza de cada consulta.
	cd-original	Imagen	Consulta	—	Radiografía original de la consulta seleccionada.
	cd-xai	Imagen	Consulta	—	Mapa de explicabilidad XAI de la consulta seleccionada.
	cd-label	Texto	Consulta	—	Etiqueta de la predicción (Neumonía o Normal) de la consulta.
	cd-confianza	N Numérico	Consulta	—	Nivel de confianza de la predicción.
	cd-model	Texto	Consulta	—	Arquitectura del modelo empleado en la consulta.
	cd-patient	Texto	Consulta	—	Nombre visible de la consulta, que funciona como etiqueta de organización y no como identificación clínica.
	cd-timestamp	Fecha/hora	Consulta	—	Fecha de la consulta.
Botones/Enlaces	Nombre		Acción		
	Tarjeta del historial		Abre el detalle de la consulta seleccionada, materializando el CU-012.		

	Volver	Cierra el detalle y regresa al listado del historial.
	Renombrar	Abre el diálogo para indicar el nuevo nombre visible de la consulta y lo actualiza, materializando el CU-013.
	Eliminar	Solicita la confirmación de la eliminación y, si se confirma, elimina la consulta y sus artefactos, materializando el CU-014.

Tabla x - Especificación de interfaz IU-0004

La vista del historial solo muestra las consultas del usuario propietario, de modo que el acceso al detalle y a sus artefactos queda restringido al propietario mediante la cadena de datos del listado. La vista del detalle no requiere una petición adicional al servidor: se construye con los datos recibidos en el listado y las imágenes se sirven desde el almacenamiento estático. El informe del diagnóstico se descarga desde el detalle, aunque la generación del documento pertenece a SD-002.

6.3.3 Informes del subsistema

El subsistema SD-003 no genera informes propios: su responsabilidad es la recuperación y la gestión de las consultas ya realizadas, y no produce documentos descargables. El informe del diagnóstico de cada consulta (IF-0001) pertenece al subsistema SD-002, que lo genera durante el procesamiento, de modo que el historial únicamente conserva y sirve la ruta del documento ya generado. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

6.4 Subsistema SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 materializa la interfaz del laboratorio de experimentación, correspondiente a la plantilla `training.html` (ruta `/training`). La ventana integra el asistente de configuración conversacional, la exploración de la carpeta del dataset, el listado de los experimentos guardados, la consola de entrenamiento, la vista de resultados de la sesión —con el ranking, la matriz de Wilcoxon y la validación externa— y la vista de resultados de un modelo, con las métricas de validación cruzada, la calibración y las métricas XAI. La ventana integra además la cola de trabajos y el acceso a la administración, pertenecientes a SD-006 y SD-005. El subsistema genera el informe consolidado de la sesión (IF-0002).

6.4.1 Flujo de navegación

El flujo de navegación del subsistema SD-004, representado en la figura, se organiza en torno al laboratorio como ventana principal. El usuario accede al laboratorio desde el panel de diagnóstico, y dentro de la ventana navega entre el asistente de configuración, la vista de resultados de la sesión y la vista de resultados de un modelo, además de la consola de entrenamiento que se muestra durante la ejecución. Desde el laboratorio se alcanzan la cola de trabajos y, en sentido inverso, el panel de diagnóstico.

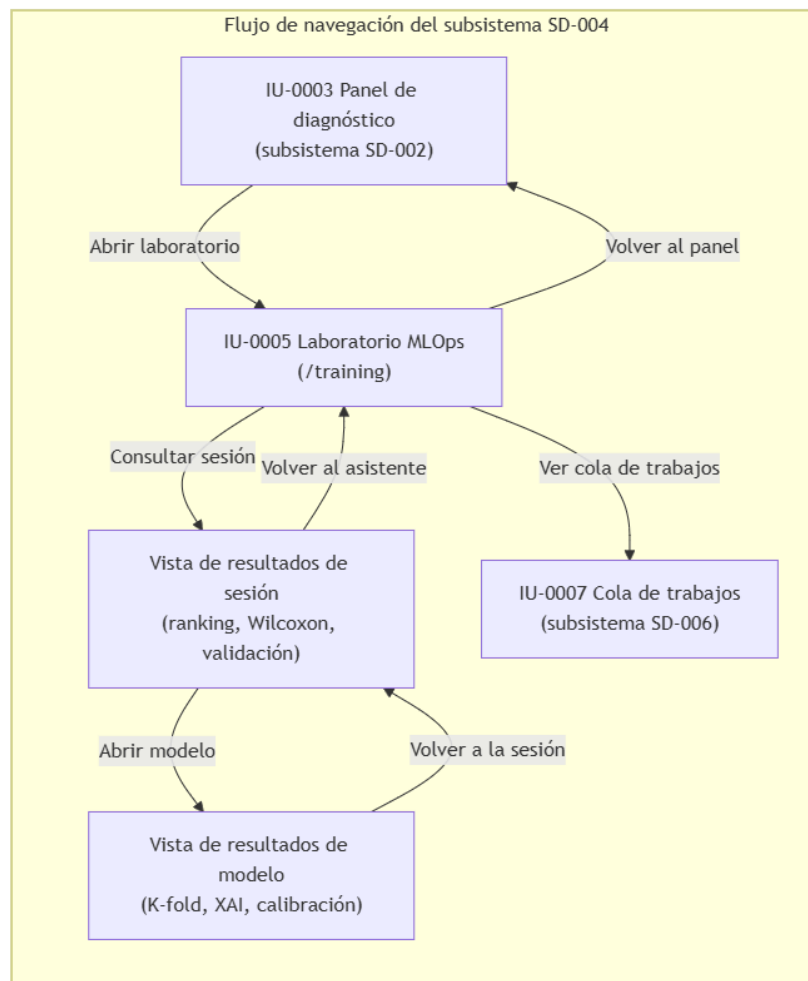


Ilustración x - Flujo de navegación del subsistema SD-004

El flujo refleja la progresión del investigador por el laboratorio: desde el asistente de configuración se lanzan los experimentos, desde el listado de experimentos se consulta la sesión, y desde la vista de la sesión se desciende a los resultados de un modelo concreto. La navegación entre las vistas es de ida y vuelta, y el usuario puede volver al asistente o al panel en cualquier momento sin interrumpir el estado del laboratorio.

6.4.2 Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-004.

IU-0005: Laboratorio de entrenamiento					
Descripción	Ventana del laboratorio de experimentación MLOps, correspondiente a la plantilla training.html (ruta /training). Presenta el asistente de configuración conversacional, la exploración de la carpeta del dataset, el listado de los experimentos guardados, la consola del servidor de entrenamiento, la vista de resultados de la sesión (ranking por AUC, matriz de Wilcoxon, validación externa con curvas ROC y matriz de DeLong, y exploración de modelos) y la vista de resultados de un modelo (validación cruzada K-fold, calibración y métricas XAI). Solo es accesible para usuarios autenticados.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	chat-input	Texto	Editab le	Sí	Mensaje en lenguaje natural dirigido al asistente de configuración del experimento.
	Listado de experime ntos	Conteni do	Consu lta	—	Listado de las sesiones de entrenamiento del usuario, con sus modelos, en la barra lateral.
	session-r anking-ta ble	Tabla	Consu lta	—	Ranking de los modelos de la sesión por la media del AUC de la validación cruzada, con su desviación.
	session-heatmap -img	Imagen	Consu lta	—	Matriz de significancia estadística del test de Wilcoxon.
	external-ranking-t able	Tabla	Consu lta	—	Rendimiento de la validación externa por modelo: exactitud, F1 y AUC.
	external-roc-img	Imagen	Consu lta	—	Curvas ROC de la validación externa.
	external-delong-i mg	Imagen	Consu lta	—	Matriz de significancia de DeLong.
	res-table -body	Tabla	Consu lta	—	Resultados de la validación cruzada K-fold de un modelo (exactitud, precisión, sensibilidad, F1 y AUC por pliegue).

	res-xai-table	Tabla	Consulta	—	Métricas cuantitativas de explicabilidad del modelo (deletion, insertion, sparsity, entropy y stability por método).
	console-logs	Contenido	Consulta	—	Registro de la ejecución del entrenamiento, actualizado durante el procesamiento.
Botones/Enlaces	Nombre		Acción		
	Explorar Carpeta		Abre el explorador del dataset y inserta la ruta seleccionada en la conversación, materializando el CU-017.		
	Enviar		Envía el mensaje al asistente de configuración, materializando el CU-016; si la configuración está completa, lanza el entrenamiento (CU-018).		
	Actualizar		Refresca el listado de experimentos guardados.		
	Volver al Asistente		Oculta la vista de resultados de la sesión y regresa al asistente de configuración.		
	Renombrar		Cambia el nombre visible de la sesión, materializando el CU-029.		
	Reutilizar Configuración		Reutiliza la configuración de la sesión actual en una nueva conversación.		
	Validación Externa		Solicita la validación externa de la sesión, materializando el CU-026.		
	Generar Reporte PDF		Descarga el informe consolidado de la sesión, materializando el CU-028.		
	Recalcular Wilcoxon		Solicita el recálculo de la comparativa estadística de la sesión, materializando el CU-024.		
	Volver a la Sesión		Regresa desde los resultados de un modelo a la vista de la sesión.		
	Generar XAI y Métricas		Ejecuta el análisis de explicabilidad de un modelo, materializando el CU-025.		
	Diagnóstico Rápido		Navega al panel de diagnóstico (GET /dashboard), del subsistema SD-002.		
	Cerrar Sesión		Cierra la sesión del usuario (GET /logout), del subsistema SD-001.		
	Selector de idioma		Cambia el idioma de la interfaz, materializando el CU-004.		
	Botón de tema		Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.		

Tabla x - Especificación de interfaz IU-0005

El informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios, y separa el conocimiento del formato del router: el generador recibe la información preparada por el motor y compone el documento. El informe consolida el análisis estadístico de la sesión y se considera parte de los artefactos de experimentación del laboratorio.

6.4.3 Informes del subsistema

A continuación se especifica el informe propio de SD-004.

IF- 0002: Informe de la sesión de entrenamiento				
Descripción	Informe PDF consolidado de una sesión de entrenamiento, generado bajo demanda mediante <code>services/pdf_generator_mlops.py</code> . Recoge la configuración del experimento, el ranking de modelos con la matriz de Wilcoxon, los resultados de la validación externa con las curvas ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas de explicabilidad y sus mapas de calor. Se entrega como documento descargable desde la vista de la sesión.			
Módulo de Interfaz	IU-0005 Laboratorio de entrenamiento.			
Datos	Campo	Ordenación	Tipo Datos	Descripción
	ID de sesión	—	Texto	Identificador de la sesión de entrenamiento.
	Fecha	—	Fecha/hora	Fecha de generación del informe.
	Dataset	—	Texto	Ruta del dataset de entrenamiento de la sesión.
	Modelos	—	Texto	Arquitecturas entrenadas en la sesión.
	Hiperparámetros	—	Texto	Épocas, tamaño de lote y tasa de aprendizaje de la sesión.
	Ranking	1	Tabla	Modelos ordenados por la media del AUC de la validación cruzada, con su desviación típica.
	Matriz de Wilcoxon	—	Imagen	Matriz de significancia estadística entre los modelos.
	Validación externa	—	Tabla	Exactitud, F1 y AUC de cada modelo sobre la cohorte externa.
	Curvas ROC	—	Imagen	Curvas ROC de la validación externa.
	Matriz de DeLong	—	Imagen	Matriz de comparación estadística de las curvas ROC.

	Métricas XAI	—	Tabla	Métricas de fidelidad de la explicabilidad por modelo y método.
	Mapas de calor XAI	—	Imagen	Mapas de explicabilidad de cada modelo sobre imágenes de ejemplo.
Resumen/Acumulado	Resumen			Campos del Resumen
	Ranking de modelos			Modelo, Media AUC, Desviación

Tabla x - Especificación de informe IF-0002

El informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial. La generación del documento se adelanta al momento del diagnóstico, de modo que la descarga posterior no depende de una nueva operación del sistema.

6.5 Subsistema SD-005: Supervisión y administración

El subsistema SD-005 materializa la interfaz de administración, integrada en las ventanas del panel de diagnóstico y del laboratorio de entrenamiento mediante los diálogos de administración de las plantillas dashboard.html y training.html. La vista de administración es accesible únicamente para el usuario con rol de administrador y presenta el listado de usuarios con sus recuentos de actividad, las consultas de un usuario concreto junto con sus sesiones del laboratorio, y el detalle de una consulta seleccionada. Constituye la interfaz IU-0006 y materializa los casos de uso CU-031 a CU-033.

6.5.1 Flujo de navegación

El flujo de navegación del subsistema SD-005, representado en la figura 95, refleja la progresión del administrador desde el listado general hasta el caso concreto, en coherencia con las relaciones de extensión del análisis. Desde el panel de diagnóstico o el laboratorio, el administrador abre el diálogo de administración con el listado de usuarios; al seleccionar un usuario se muestran sus consultas y sus sesiones del laboratorio, y desde el listado de consultas se abre el detalle completo de una consulta, con la posibilidad de volver en cada paso.

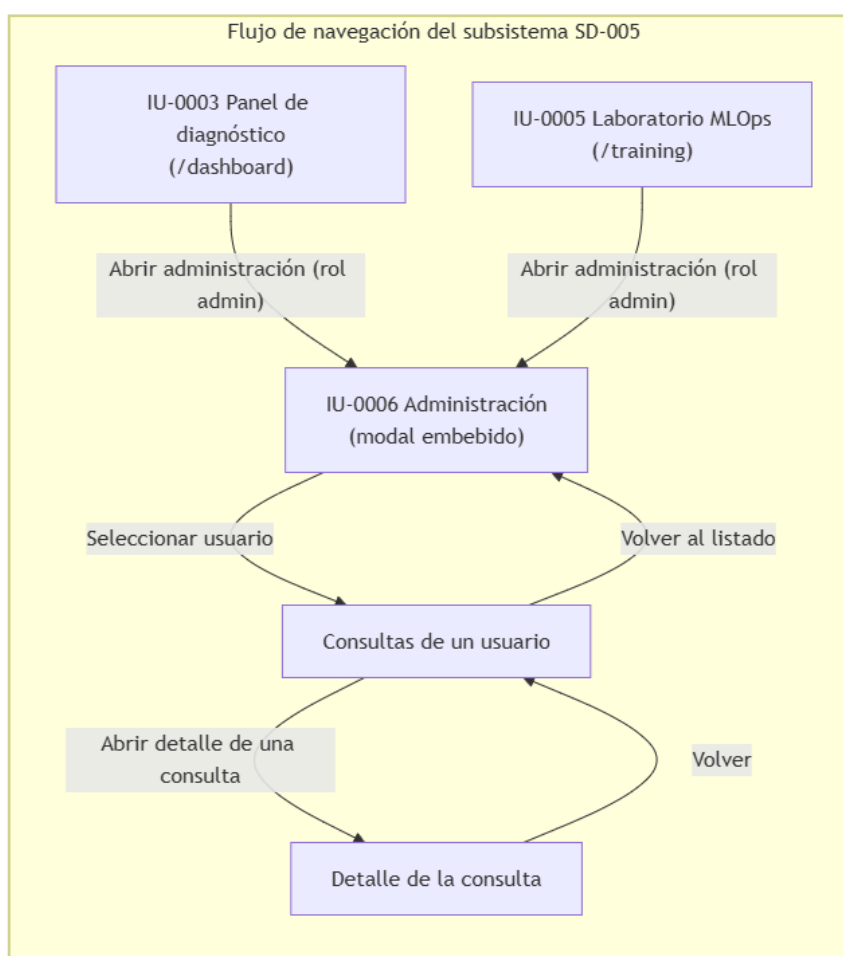


Ilustración x - Flujo de navegación del subsistema SD-005

El flujo refleja la cadena de navegación de la supervisión: el listado de usuarios conduce a la actividad de una cuenta, y desde la actividad se desciende al detalle de una consulta individual. La navegación es de ida y vuelta en cada nivel, de modo que el administrador puede profundizar progresivamente en la información y regresar al nivel anterior sin perder el contexto de la supervisión. El acceso al diálogo de administración queda restringido al rol administrador, y su presencia en la barra lateral depende de la comprobación de rol que realiza el servidor.

6.5.2 Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-005.

IU-0006: Laboratorio de entrenamiento					
Descripción	Vista de supervisión y administración, integrada mediante los diálogos de administración de las plantillas dashboard.html y training.html. Presenta el listado de usuarios con el número de diagnósticos y de sesiones del laboratorio de cada uno, las consultas de un usuario concreto junto con sus sesiones de entrenamiento, y el detalle de una consulta con su imagen, su mapa de explicabilidad, su resultado, su confianza y sus metadatos. Solo es accesible para el usuario con rol de administrador.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	admin-us ers-list	Conteni do	Consu lta	—	Listado de usuarios registrados con el número de diagnósticos y de sesiones del laboratorio de cada uno.
	admin-us er-consul tations-li st	Conteni do	Consu lta	—	Consultas de diagnóstico del usuario seleccionado, con sus sesiones de entrenamiento del laboratorio.
	Detalle de consulta	Conteni do	Consu lta	—	Detalle completo de la consulta seleccionada: radiografía original, mapa de explicabilidad, diagnóstico, confianza, modelo, nombre y fecha.
Botones/Enla ces	Nombre	Acción			
	Panel de Administración	Abre el diálogo de administración con el listado de usuarios; solo se muestra al usuario con rol administrador.			
	Usuario del listado	Abre las consultas y las sesiones del laboratorio del usuario seleccionado, materializando el CU-032.			
	Consulta del listado	Abre el detalle completo de la consulta seleccionada, materializando el CU-033.			
	Cerrar	Cierra el diálogo de administración y regresa a la ventana desde la que se abrió.			

Tabla x - Especificación de interfaz IU-0006

La vista de administración reutiliza el detalle de la consulta del panel de diagnóstico para presentar la consulta supervisada, de modo que la interfaz administrativa no duplica la vista del historial. La autorización administrativa se comprueba en el servidor antes de servir los datos, y no se infiere de la interfaz. El acceso del administrador a los datos de otros usuarios constituye la excepción al aislamiento de datos, acotada a la función de supervisión.

6.5.3 Informes del subsistema

El subsistema SD-005 no genera informes: su responsabilidad es la consulta y la supervisión de la actividad de la plataforma, y no produce documentos descargables. Los informes de la plataforma corresponden a los subsistemas que generan resultados, SD-002 y SD-004, y el administrador puede consultarlos a través de las consultas y las sesiones que supervisa, sin generar informes propios. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

6.6 Subsistema SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa la interfaz de la cola de trabajos, integrada como panel lateral en las ventanas del panel de diagnóstico y del laboratorio de entrenamiento. La vista de la cola muestra los trabajos asíncronos del usuario —diagnósticos, entrenamientos y validaciones externas— con su tipo, su estado y su posición, y permite cancelar los trabajos pendientes, materializando los casos de uso CU-034 y CU-035. Además del panel de la cola, el subsistema comprende los mecanismos transversales de presentación de la interfaz: el selector de idioma y el cambio del tema visual, que ya se describieron como acciones comunes en las fichas de interfaz de los subsistemas anteriores.

6.6.1 Flujo de navegación

El flujo de navegación del subsistema SD-006, representado en la figura, refleja el carácter transversal de la cola de trabajos: el panel se integra en el panel de diagnóstico y en el laboratorio de entrenamiento, de modo que el usuario puede consultar el estado de sus trabajos desde cualquiera de los dos núcleos funcionales sin abandonar su ventana. Desde el panel de la cola se ejecuta la cancelación de un trabajo pendiente, que elimina el trabajo del estado encolado.

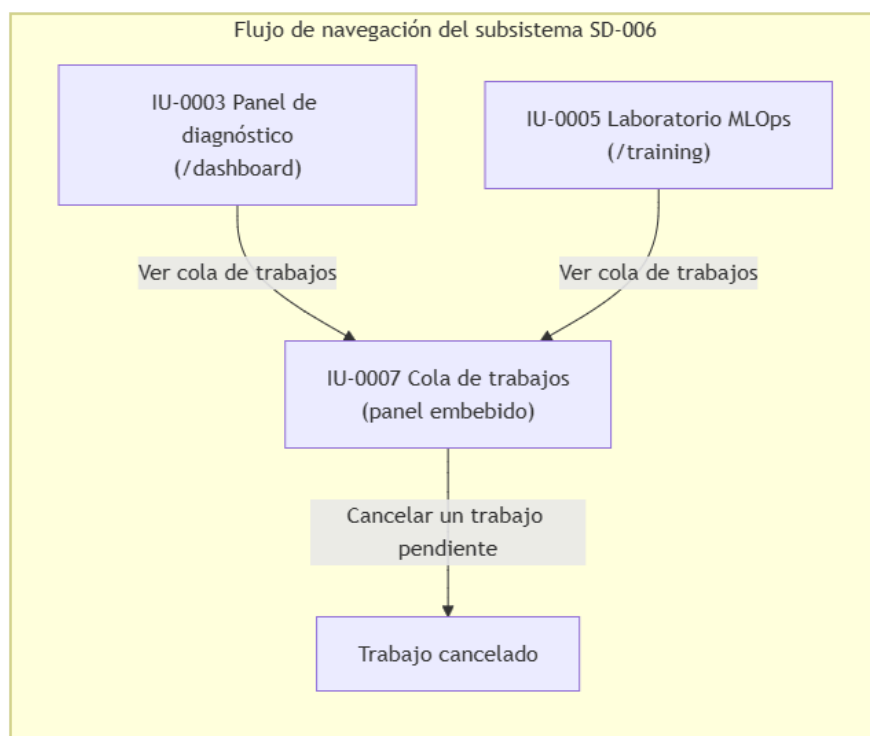


Ilustración x - Flujo de navegación del subsistema SD-006

El flujo refleja la integración transversal del panel de la cola: no constituye una ventana independiente, sino una sección que se muestra en las ventanas de los subsistemas que generan trabajos. La actualización del panel es periódica, de modo que el estado de los trabajos se refleja sin necesidad de navegación adicional, y la cancelación de un trabajo pendiente se resuelve desde el propio panel, sin transiciones intermedias.

6.6.2 Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-006.

IU-0007: Cola de trabajos					
Descripción	Panel lateral de la cola de trabajos, integrado en las ventanas del panel de diagnóstico (dashboard.html) y del laboratorio de entrenamiento (training.html). Muestra los trabajos asíncronos del usuario con su tipo (diagnóstico, entrenamiento o validación externa), su estado (encolado, en ejecución, completado o fallido) y la posición de los trabajos encolados, y permite cancelar los trabajos pendientes. El panel se actualiza de forma periódica, de modo que refleja la progresión de los trabajos sin recargar la ventana.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	queue-items	Contenido	Consulta	—	Listado de los trabajos del usuario con el tipo del trabajo, el estado (encolado, en ejecución, completado o fallido) y la posición de los trabajos encolados.
	Estado del trabajo	Texto	Consulta	—	Indicación del estado del trabajo: en ejecución o la posición en la cola para los encolados.
Botones/Enlaces	Nombre	Acción			
	Cancelar	Cancela un trabajo pendiente de la cola, tras la confirmación del usuario, mediante el endpoint DELETE /api/queue/cancel/{job_id}; solo se muestra para los trabajos en estado encolado, materializando el CU-035.			

Tabla x - Especificación de interfaz IU-0006

El panel de la cola refleja la máquina de estados de los trabajos y solo ofrece la cancelación para los trabajos pendientes: un trabajo en ejecución no se interrumpe, en coherencia con el diseño de SD-006. La actualización periódica del panel permite al usuario conocer en qué momento estará disponible un resultado o si un trabajo ha fallado, materializando el CU-034. Los mecanismos transversales de idioma y de tema visual se integran en las cabeceras de las ventanas, tal y como se describió en las fichas de interfaz de los subsistemas anteriores.

6.6.3 Informes del subsistema

El subsistema SD-006 no genera informes: su responsabilidad es la ejecución asíncrona de los trabajos y las capacidades transversales de presentación, y no produce documentos descargables. Los informes de la plataforma corresponden a los subsistemas que generan resultados, SD-002 y SD-004, y la cola de trabajos únicamente refleja su estado de procesamiento. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

7 Especificación de la construcción del sistema

La especificación de la construcción constituye la etapa final del diseño que determina cómo se materializa el sistema diseñado en un software ejecutable. Los capítulos anteriores definieron la arquitectura, los casos de uso, las clases y las interfaces; este capítulo concreta el entorno tecnológico en el que se construye la aplicación, los paquetes y componentes que la forman, el procedimiento de construcción y puesta en marcha, y la generación del esquema de la base de datos. La especificación cierra la parte de diseño de la memoria y establece la base sobre la que se definen posteriormente las pruebas y la implantación del sistema (Sharp, Rogers, & Preece, 2019).

El capítulo se organiza en cuatro apartados, conforme a la guía de diseño de la memoria (punto 7): el entorno tecnológico de construcción, que describe las herramientas, las dependencias y las restricciones del entorno; los paquetes y componentes de construcción, que representan la estructura del software y sus dependencias; el proceso de construcción, que especifica cómo se prepara el entorno y se lanza la aplicación; y la generación del esquema de base de datos, que describe cómo se materializa el modelo físico definido en el capítulo 20. El contenido se apoya en la arquitectura de soporte del capítulo 19, que establece los subsistemas de soporte sobre los que se asienta la construcción.

La construcción de vitalXAI no produce ejecutables compilados: la aplicación es un sistema Python interpretado, servido por un proceso ASGI y apoyado en un conjunto de dependencias declaradas. Por esta razón, el proceso de construcción consiste en preparar el entorno virtual, instalar las dependencias, configurar las variables de entorno y lanzar el servidor y el worker de la cola, sin una fase de compilación ni de generación de paquetes instalables. Esta naturaleza condiciona las especificaciones de los apartados siguientes, que se centran en la reproducibilidad del entorno y en la configuración necesaria para que la aplicación funcione.

7.1 Entorno tecnológico de construcción

El entorno de construcción de vitalXAI reúne las herramientas necesarias para ejecutar y mantener la aplicación. El sistema se construye sobre Python 3.11, que constituye el lenguaje principal y alberga tanto el servidor web como el motor de aprendizaje profundo, y emplea el framework FastAPI para la capa HTTP, servido por el servidor ASGI Uvicorn (FastAPI, 2024; Uvicorn, 2024). La presentación se compone con el motor de plantillas Jinja2 y los recursos estáticos HTML, CSS y JavaScript; el almacenamiento relacional se apoya en MySQL, accedido a través del conector oficial de Python, y en el entorno de desarrollo se ejecuta mediante MariaDB a través de XAMPP (Oracle, 2024). El aprendizaje profundo se apoya en TensorFlow y en la librería Transformers de Hugging Face para las arquitecturas Transformer (TensorFlow, 2024; Hugging Face, 2024), junto con OpenCV para el preprocesamiento de las imágenes y matplotlib para la generación de los mapas de explicabilidad. Las credenciales de sesión se gestionan con bcrypt y python-jose, la limitación de peticiones con slowapi, los informes PDF con FPDF (FPDF2, 2024) y la configuración conversacional con el cliente del proveedor Groq.

La gestión de las dependencias se resuelve mediante el archivo requirements.txt, que fija las versiones de las librerías de la aplicación, de modo que el entorno de construcción se reproduce de forma determinista. La instalación se realiza sobre un entorno virtual de Python, y las dependencias se organizan en tres grupos: las del servidor web (FastAPI, Uvicorn, Jinja2, slowapi, python-multipart, bcrypt, python-jose), las del aprendizaje automático y el análisis (TensorFlow, Keras, Transformers, OpenCV, matplotlib, pandas, scikit-learn, seaborn) y las de calidad y verificación (pytest, pytest-cov, ruff, mypy), tal y como se describen en el capítulo 19. El peso de estas dependencias condiciona el entorno: la carga de TensorFlow y de los modelos de aprendizaje requiere recursos de memoria y, en su caso, de cómputo acelerado por GPU.

La configuración sensible y operativa permanece fuera del código y se suministra mediante variables de entorno, que se cargan desde un archivo .env en el arranque de la aplicación. La tabla siguiente resume las variables de entorno del sistema, agrupadas por ámbito.

Variable	Variable
JWT_SECRET_KEY	Clave secreta de firma de los tokens de acceso; en producción debe configurarse y no usar la clave de desarrollo.
JWT_ACCESS_EXPIRE_MINUTES	Duración en minutos del token de acceso.
JWT_REFRESH_EXPIRE_DAYS	Duración en días del token de refresco.
REFRESH_ROTATION_GRACE_SECONDS	Periodo de gracia de la rotación del token de refresco.
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	Parámetros de conexión a MySQL (por defecto localhost, root, sin contraseña y base tfg_pneumonia).
DB_POOL_SIZE	Tamaño del pool de conexiones a la base de datos.
GROQ_API_KEY	Clave de la API del proveedor del asistente conversacional.
TFG_DEMO_DATASET, TFG_DEMO_EXTERNAL_DATASET	Rutas por defecto de los datasets de entrenamiento y de validación externa.
TFG_SESSION_ID, TFG_MODEL_NAME, TFG_DATASET_DIR, TFG_EXTERNAL_DATASET_DIR, TFG_EPOCHS, TFG_BATCH_SIZE, TFG_LEARNING_RATE	Variables de entorno que el pipeline MLOps transmite a los scripts de entrenamiento y de validación externa.

Tabla x - Entorno tecnológico de construcción

El entorno impone además una serie de restricciones que condicionan la construcción y la ejecución. La aplicación requiere una instancia de MySQL accesible y con el esquema inicializable; en el entorno de desarrollo se utiliza XAMPP con el servicio de MariaDB activo, y el arranque de la aplicación crea las tablas si no existen. El motor de diagnóstico necesita los pesos de los modelos entrenados, que se conservan en el directorio pneumoniaccn-main/results y se cargan bajo demanda por la primera consulta con cada arquitectura; sin esos pesos, la solicitud de un diagnóstico falla en el procesamiento. El pipeline de experimentación MLOps requiere el directorio pneumoniaccn-main/code con los scripts de entrenamiento, análisis XAI, estadística y validación externa, que se invocan como procesos externos y escriben sus resultados en el directorio training_results. La generación de los mapas de explicabilidad emplea el backend no interactivo de matplotlib (Agg), que evita la dependencia de un entorno gráfico en el servidor.

El diagrama de despliegue representa la disposición física de los componentes del sistema en su entorno de ejecución. El navegador del cliente accede al servidor de aplicación mediante HTTP; el servidor Uvicorn aloja la aplicación FastAPI, que orquesta los routers, los servicios y el middleware, atiende las consultas a la base de datos MySQL y sirve los recursos estáticos; el worker asíncrono comparte la persistencia con la aplicación y ejecuta los scripts del pipeline MLOps, que leen los modelos de Hugging Face y escriben los artefactos en el sistema de ficheros; la aplicación invoca además al proveedor externo Groq para el asistente conversacional.

El diagrama de despliegue refleja la separación entre el ciclo de petición y la ejecución asíncrona: la aplicación atiende las peticiones del navegador y encola los trabajos, mientras que el worker reclama y procesa los diagnósticos y los entrenamientos fuera del ciclo de petición. El almacenamiento de ficheros alberga las plantillas, los recursos estáticos, las imágenes subidas, los informes y los resultados de los entrenamientos, y se expone parcialmente al navegador mediante los montajes de recursos estáticos. Los proveedores externos —el asistente conversacional y el repositorio de modelos— se integran como fronteras del sistema, de modo que los fallos de esos servicios no se confunden con los de la aplicación. Los requisitos no funcionales con impacto sobre el entorno de construcción, como el tiempo de respuesta de la inferencia (RNF-019) o la ejecución sin bloqueo de la interfaz (RNF-020), se satisfacen mediante la reutilización de los modelos en memoria y la cola de trabajos, que no requieren componentes adicionales de construcción.

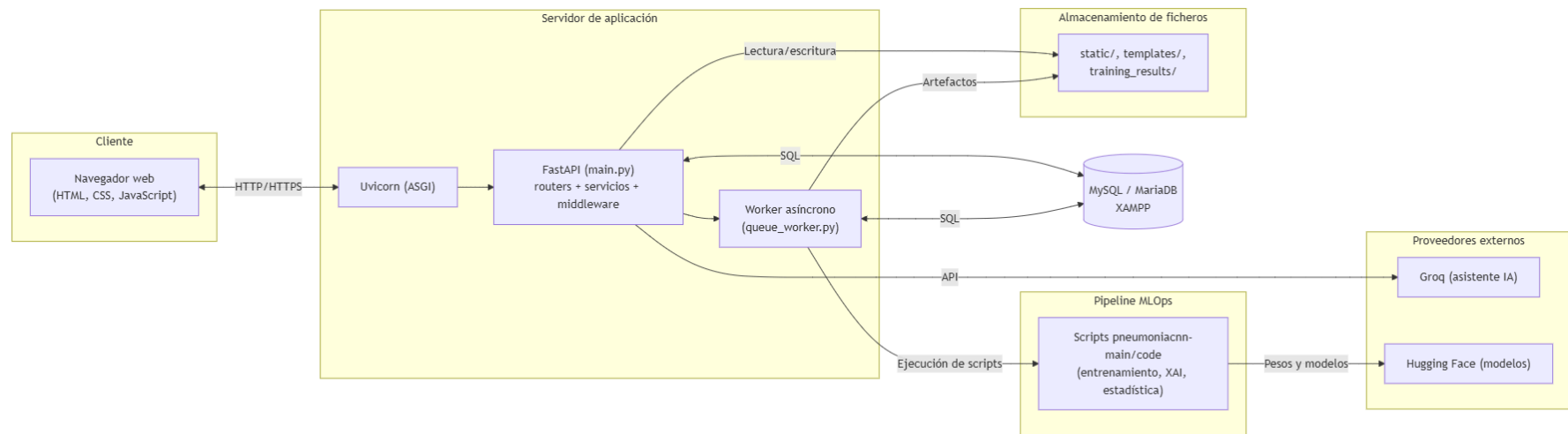


Ilustración x - Diagrama de despliegue del sistema

7.2 Paquetes y componentes de construcción

Los paquetes de construcción de vitalXAI se corresponden con las agrupaciones funcionales del código fuente, representadas en la figura. La estructura distingue la capa HTTP (routers/), la capa de servicios (services/), el núcleo de la aplicación (core), la presentación (templates/ y static/), el pipeline de experimentación (pneumoniacnn-main/) y las pruebas (tests/). El diagrama de paquetes representa estas agrupaciones y las dependencias entre ellas, de modo que la construcción respeta la jerarquía de acoplamiento del sistema.

El diagrama refleja la dependencia principal del sistema: el núcleo de la aplicación compone la aplicación FastAPI a partir de los routers y los servicios, y monta los paquetes de presentación; los routers dependen de los servicios, que a su vez acceden a la persistencia relacional y al pipeline de experimentación. El paquete de pruebas depende de los routers, los servicios y el núcleo, de modo que su construcción no forma parte de la aplicación ejecutable. El paquete pneumoniacnn-main/ constituye un componente externo al código de la aplicación: contiene los scripts del pipeline MLOps y los pesos de los modelos entrenados, y los servicios lo invocan como procesos externos, sin acoplamiento de importación.

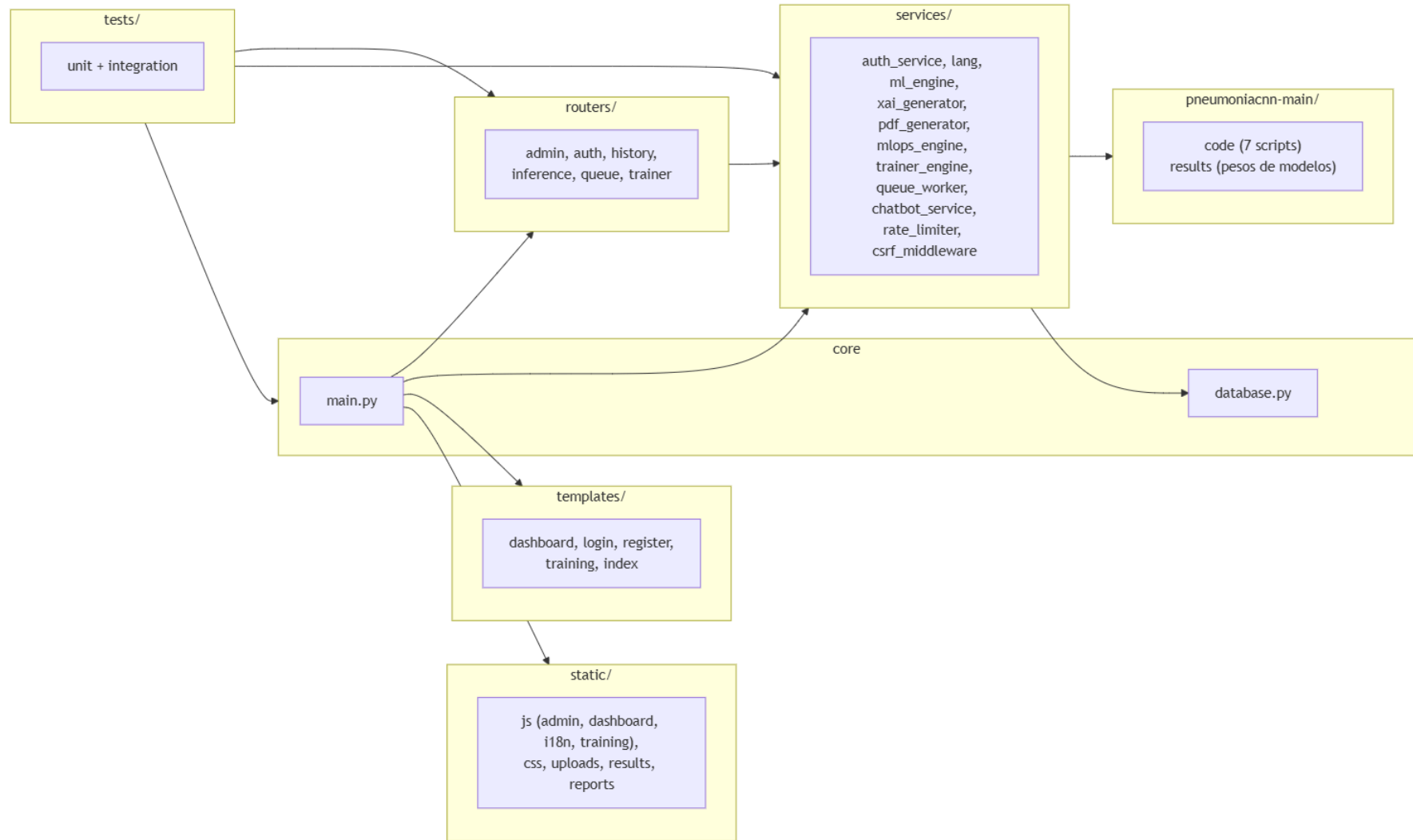


Ilustración x - Diagrama de paquetes de construcción

La tabla siguiente especifica los componentes que forman cada paquete de construcción y su responsabilidad.

Paquete	Componentes	Responsabilidad
core	main.py, database.py	Composición de la aplicación FastAPI, configuración de middleware y montajes, inicialización de la base de datos y pool de conexiones.
routers/	admin.py, auth.py, history.py, inference.py, queue.py, trainer.py	Capa HTTP: definición de los endpoints de la aplicación, autenticación, diagnóstico, historial, cola, laboratorio y administración.
services/	auth_service.py, lang.py, ml_engine.py, xai_generator.py, pdf_generator.py, mlops_engine.py, trainer_engine.py, queue_worker.py, chatbot_service.py, rate_limiter.py, csrf_middleware.py	Lógica de aplicación: criptografía y sesiones, internacionalización, predicción, explicabilidad, informes, laboratorio MLOps, worker de la cola, asistente conversacional, limitación y seguridad.
templates/	login.html, register.html, dashboard.html, training.html, index.html	Plantillas Jinja2 de las páginas de la aplicación.
static/	js/admin.js, js/dashboard.js, js/i18n.js, js/training.js, css, uploads/, results/, reports/	Recursos de presentación: scripts de la interfaz, estilos y directorios de artefactos generados.
pneumoniacnn-main/	code/ (siete scripts), results/ (pesos de modelos)	Pipeline MLOps: entrenamiento K-fold, entrenamiento Transformer, estadística, validación externa, test de DeLong y análisis XAI.
tests/	unit/, integration/, conftest.py	Pruebas unitarias e integración de la aplicación, con sus accesorios y mocks.

Tabla x - Paquetes y componentes de construcción

La estructura de paquetes se corresponde con los subsistemas de soporte definidos en el capítulo 19. El paquete core, con la persistencia relacional, materializa el subsistema de soporte SSOP-001; los paquetes templates/, static/ y el directorio training_results/ materializan el almacenamiento de ficheros y recursos (SSOP-002); el paquete core con la integración de los proveedores externos y el entorno de ejecución materializa el subsistema SSOP-003; y el paquete tests/ materializa la calidad y verificación (SSOP-004). Esta correspondencia mantiene la coherencia entre los paquetes de construcción y los subsistemas de soporte del diseño.

7.3 Proceso de construcción del software

El proceso de construcción de vitalXAI establece los pasos necesarios para preparar el entorno y lanzar la aplicación. Como se señaló en la introducción del capítulo, la construcción no incluye una fase de compilación ni de generación de ejecutables: consiste en preparar el entorno virtual, instalar las dependencias declaradas, configurar las variables de entorno, disponer de los artefactos externos y lanzar el servidor. El proceso se describe en la tabla siguiente, con el orden de ejecución y la finalidad de cada paso.

Orden	Paso	Procedimiento
1	Preparar el entorno	Instalar Python 3.11 y crear un entorno virtual para el proyecto.
2	Instalar dependencias	Ejecutar <code>pip install -r requirements.txt</code> dentro del entorno virtual, lo que instala las versiones fijadas de las librerías de la aplicación.
3	Configurar el entorno	Crear el archivo <code>.env</code> con las variables de entorno del sistema (credenciales de MySQL, claves de sesión y del asistente, y rutas de los datasets).
4	Preparar la base de datos	Iniciar el servicio de MySQL (en el entorno de desarrollo, mediante XAMPP); la aplicación crea automáticamente las tablas del esquema en el arranque.
5	Verificar los artefactos	Comprobar que el directorio <code>pneumoniaccn-main/code</code> contiene los scripts del pipeline y que <code>pneumoniaccn-main/results</code> contiene los pesos de los modelos entrenados.
6	Lanzar la aplicación	Ejecutar <code>python main.py</code> , que inicia el servidor Uvicorn, inicializa la base de datos y arranca el worker de la cola.

Tabla x - Proceso de construcción del software

El proceso de construcción depende de dos familias de requisitos adicionales. En primer lugar, los artefactos del aprendizaje automático: los pesos de los modelos entrenados, conservados en `pneumoniaccn-main/results`, son necesarios para el diagnóstico, de modo que la primera consulta con cada arquitectura carga los pesos y las siguientes reutilizan el modelo en memoria. En segundo lugar, las variables de entorno del pipeline MLOps: los scripts de entrenamiento y de validación externa leen la configuración de cada trabajo desde las variables `TFG_*`, que el motor de experimentación establece al invocarlos, y escriben sus resultados en el directorio `training_results`. Sin estas condiciones, el entorno no dispone de la funcionalidad de diagnóstico ni de experimentación completa.

La figura 99 representa el diagrama de componentes del sistema, que detalla la colaboración entre los componentes construidos. La aplicación compone los routers, que delegan en los servicios; los servicios acceden a la persistencia, invocan al asistente externo y orquestan el worker; el worker procesa los diagnósticos con el motor de predicción, el generador de mapas y el generador de informes, y los entrenamientos con el motor MLOps, que ejecuta los scripts del pipeline; la aplicación sirve la presentación desde las plantillas y los recursos estáticos.

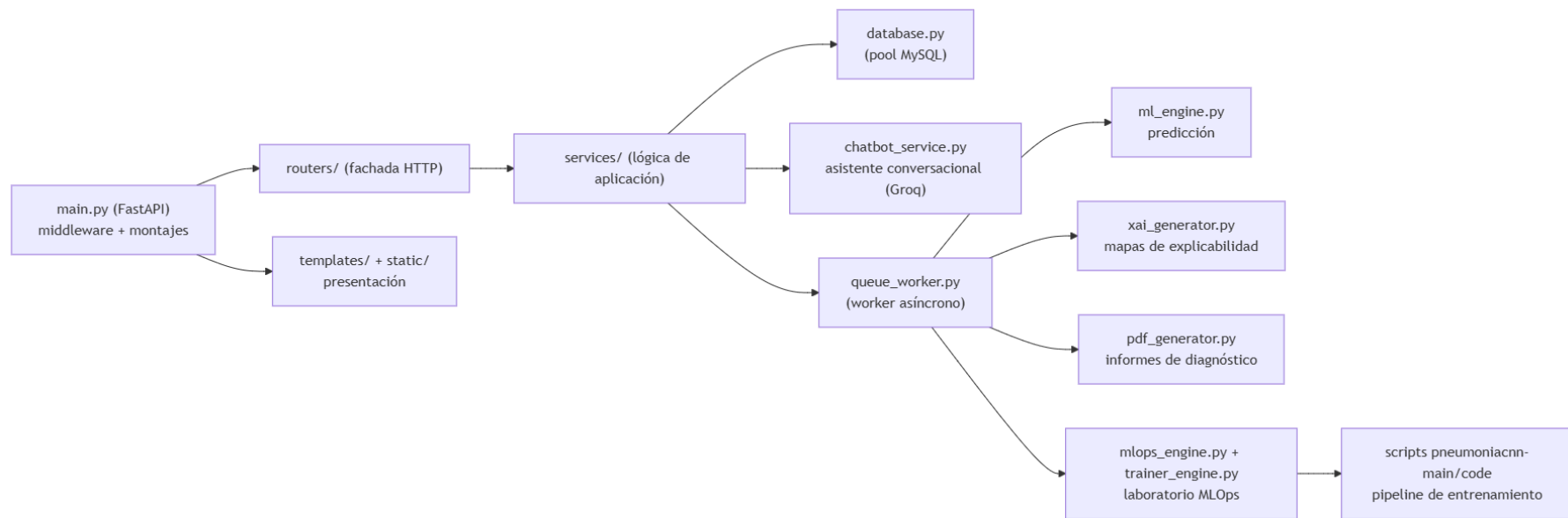


Ilustración x - Diagrama de componentes del sistema

La especificación detallada de los componentes del sistema se resume en la tabla siguiente, indicando qué requiere cada componente para su construcción y cómo se procede.

Componente	Dependencias y requisitos	Procedimiento de construcción
Aplicación web (main.py)	FastAPI, Uvicorn y las dependencias del servidor	Se construye al instalar las dependencias; compone la aplicación, los routers, el middleware y los montajes.
Routers (routers/)	Los módulos de los routers y los servicios	Se incluyen en la aplicación mediante la composición de main.py; no requieren construcción adicional.
Servicios (services/)	Las librerías de seguridad, aprendizaje y generación	Se instalan con requirements.txt; los servicios se importan desde los routers y el worker.
Persistencia (database.py)	MySQL y el conector	Se requiere una instancia de MySQL accesible; el esquema se crea automáticamente en el arranque.
Worker (queue_worker.py)	Las librerías de la aplicación	Se arranca automáticamente en el ciclo de vida de la aplicación.
Motor de diagnóstico (ml_engine.py, xai_generator.py, pdf_generator.py)	TensorFlow, Transformers, OpenCV, matplotlib y FPDF	Se instalan con requirements.txt; requiere los pesos de los modelos para la predicción.
Laboratorio MLOps (ml_ops_engine.py, trainer_engine.py)	Los scripts del pipeline y los datasets	Requiere pneumoniacnn-main/code y las variables TFG_*; los scripts se invocan como procesos.
Presentación (templates/, static/)	Jinja2 y el navegador	Se sirven mediante los montajes de main.py; no requieren compilación.
Asistente conversacional (chatbot_service.py)	La librería del proveedor y la clave de la API	Se instala con requirements.txt; requiere GROQ_API_KEY configurada.

Tabla x - Especificación detallada de los componentes del sistema

La construcción de la aplicación queda así especificada de forma reproducible: con las dependencias instaladas, la configuración suministrada y los artefactos del aprendizaje automático disponibles, la ejecución de main.py produce el sistema en funcionamiento, con la base de datos inicializada y el worker de la cola activo.

7.4 Elaboración de Especificaciones del Modelo Físico de Datos

La generación del esquema de la base de datos materializa el modelo físico de datos definido en el capítulo 20 en las tablas del sistema. vitalXAI no conserva un script SQL externo de instalación: el esquema se define en el módulo `database.py` y se ejecuta en el arranque de la aplicación mediante la función `init_db()`, que se invoca en el ciclo de vida de la aplicación antes de iniciar el worker. La función utiliza sentencias `CREATE TABLE IF NOT EXISTS`, de modo que la base de datos se crea y se verifica automáticamente en cada arranque, sin necesidad de un procedimiento manual de instalación ni de un script de migración adicional.

La tabla siguiente especifica las tablas del esquema, generadas por `init_db()` a partir del modelo físico, con su propósito y sus columnas principales.

Tabla	Propósito	Columnas principales
users	Cuentas de usuario de la plataforma y credenciales de acceso.	id (clave primaria), username (única), password_hash, first_name, last_name, role.
consultations	Consultas de diagnóstico realizadas por los usuarios, con sus rutas de artefactos y su resultado.	id, user_id (clave ajena a users), model_name, original_image_path, xai_image_path, prediction_label, confidence_score, patient_name, pdf_path, timestamp.
training_jobs	Registro histórico de los trabajos de entrenamiento, con su estado y su progreso.	id, user_id (clave ajena), dataset_path, model_name, status, progress, metrics_json, started_at, finished_at.
job_queue	Cola persistente de trabajos asíncronos: diagnósticos, entrenamientos y validaciones externas.	id, user_id (clave ajena), job_type, status, payload (JSON), result (JSON), error_message, created_at, started_at, finished_at.
refresh_tokens	Tokens de refresco de sesión, conservados mediante su hash y su estado de revocación.	id, user_id (clave ajena), token_hash, expires_at, revoked, created_at.

Tabla x - Especificación de las tablas del esquema

Las relaciones entre las tablas se establecen mediante las claves ajenas definidas en las sentencias de creación. Las tablas `consultations`, `training_jobs`, `job_queue` y `refresh_tokens` referencian la tabla `users` a través de la columna `user_id`, de modo que cada registro de actividad pertenece a una cuenta. Las claves ajenas de `training_jobs`, `job_queue` y `refresh_tokens` se declaran con la política `ON DELETE CASCADE`, de modo que la eliminación de un usuario elimina sus trabajos y sus credenciales de sesión; la clave ajena de `consultations` se declara sin esa política, de modo que el borrado de una cuenta deja los registros de diagnóstico conservados, en coherencia con la gestión del historial descrita en el capítulo 18.

La generación del esquema mantiene la trazabilidad con el modelo físico del capítulo 20: las columnas y los tipos de datos de las sentencias de creación corresponden a las entidades y los atributos definidos en el modelo, con las adaptaciones necesarias para el motor MySQL. La sentencia `CREATE TABLE IF NOT EXISTS` garantiza la idempotencia de la inicialización: si la tabla ya existe, la ejecución no la altera ni elimina datos, de modo que el arranque de la aplicación puede repetirse sin riesgo para los datos persistidos. Cuando el modelo físico evolucione con nuevas tablas o columnas, la generación del esquema deberá actualizarse en `database.py` y registrarse en el registro de cambios de la base de datos, conforme a las prácticas de persistencia del proyecto.

8 Preparación inicial de los datos del sistema

La preparación inicial de los datos constituye la etapa del diseño que determina cómo se disponen los datos necesarios para que el sistema funcione por primera vez y cómo evoluciona su esquema de persistencia. El capítulo 24 definió el entorno y el proceso de construcción de la aplicación; este capítulo especifica la carga inicial de datos: qué información debe estar disponible antes del primer uso, en qué entorno se prepara y en qué orden se ejecutan los procedimientos que la materializan (Elmasri & Navathe, 2016). La especificación cierra el diseño de la persistencia y establece la base sobre la que se definen las pruebas y la implantación.

El capítulo se organiza en dos apartados, conforme a la guía de diseño de la memoria (punto 8): el entorno de preparación de los datos, que describe el entorno tecnológico en el que se dispone la carga inicial, y los procedimientos de preparación y evolución del esquema, que definen el proceso, el orden de lanzamiento de los procedimientos y el diseño detallado de cada uno. El contenido se apoya en el modelo físico de datos del capítulo 20 y en la guía de despliegue del sistema, que fija los requisitos operativos de la puesta en marcha.

La naturaleza de la carga inicial de vitalXAI condiciona su especificación. El sistema no dispone de un mecanismo de carga de datos semilla ni de un sistema de migraciones automáticas: el esquema relacional se crea y se verifica automáticamente en el arranque de la aplicación, y las cuentas de usuario se crean mediante el registro, sin datos de prueba precargados en la base de datos. La carga inicial de datos consiste, por tanto, en disponer del esquema inicializado, de los datos de demostración —los conjuntos de imágenes y los pesos de los modelos entrenados— y de la configuración del entorno que permite acceder a ellos. Los cambios futuros del esquema se gestionan de forma manual y se registran en el registro de cambios de la base de datos, sin un proceso de migración formal.

8.1 Entorno de Carga Inicial o Migración

El entorno de preparación de los datos de vitalXAI reúne los elementos necesarios para la puesta en marcha del sistema. La preparación se realiza sobre el mismo equipo que ejecuta la aplicación: el entorno de desarrollo y de demostración del proyecto, que aloja el servidor, la base de datos, los conjuntos de imágenes y los pesos de los modelos. La tabla siguiente resume los elementos del entorno y su papel en la carga inicial.

Elemento de carga	Función en la preparación inicial
MySQL (MariaDB en XAMPP)	Almacén relacional donde la aplicación crea y verifica el esquema en el arranque.
Datasets de demostración	Conjuntos de imágenes de entrenamiento y de validación externa, necesarios para el diagnóstico y la experimentación.
Pesos de los modelos	Pesos de las arquitecturas entrenadas, necesarios para ejecutar la inferencia del diagnóstico.
Configuración del entorno	Variables de entorno con las credenciales y las rutas de los datos, suministradas mediante el archivo .env.

Tabla x - Especificación de entorno de Carga Inicial

La base de datos relacional se prepara mediante el servicio de MySQL del entorno, que en el desarrollo se ejecuta a través de XAMPP. La aplicación no requiere un script de instalación del esquema: la función `init_db()` de `database.py`, invocada en el arranque, ejecuta las sentencias `CREATE TABLE IF NOT EXISTS` que materializan el modelo físico de datos definido en el capítulo 20, creando las tablas `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens` cuando no existen (Oracle, 2024). La verificación de la conexión se realiza al iniciar la aplicación, que informa de la imposibilidad de conectar si el servicio no está activo.

Los datos de demostración comprenden los conjuntos de imágenes y los pesos de los modelos. Los datasets se conservan en el directorio `pneumoniacnn-main/Images` (conjunto de entrenamiento y diagnóstico) y en `pneumoniacnn-main/ExternalDataset` (conjunto independiente para la validación externa), organizados en subcarpetas `NORMAL/` y `PNEUMONIA/`. Los pesos de los modelos entrenados se conservan en el directorio `pneumoniacnn-main/results`, con los archivos de pesos de cada arquitectura, y se cargan bajo demanda en la primera consulta de diagnóstico. La disponibilidad de estos artefactos es condición necesaria para el funcionamiento del diagnóstico y del laboratorio de experimentación.

La configuración del entorno se suministra mediante el archivo `.env`, que la aplicación carga en el arranque. La configuración incluye las credenciales de la base de datos (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_POOL_SIZE`), las claves de sesión (`JWT_SECRET_KEY` y los parámetros de expiración), la clave del asistente conversacional (`GROQ_API_KEY`) y las rutas de los datasets de demostración (`TFG_DEMO_DATASET` y `TFG_DEMO_EXTERNAL_DATASET`), que permiten lanzar los entrenamientos y la validación externa sin recurrir al selector de carpetas. Las cuentas de usuario no se precargan: se crean mediante el registro en la aplicación, de modo que la carga inicial de cuentas se realiza operativamente, conforme al flujo del subsistema de acceso.

El diagrama de despliegue de la figura representa el entorno de preparación de los datos. La aplicación, en el arranque, inicializa el esquema en la base de datos MySQL, carga los pesos de los modelos bajo demanda, accede a los datasets de demostración para el diagnóstico y la experimentación, y lee la configuración del entorno; el asistente conversacional se integra como proveedor externo.

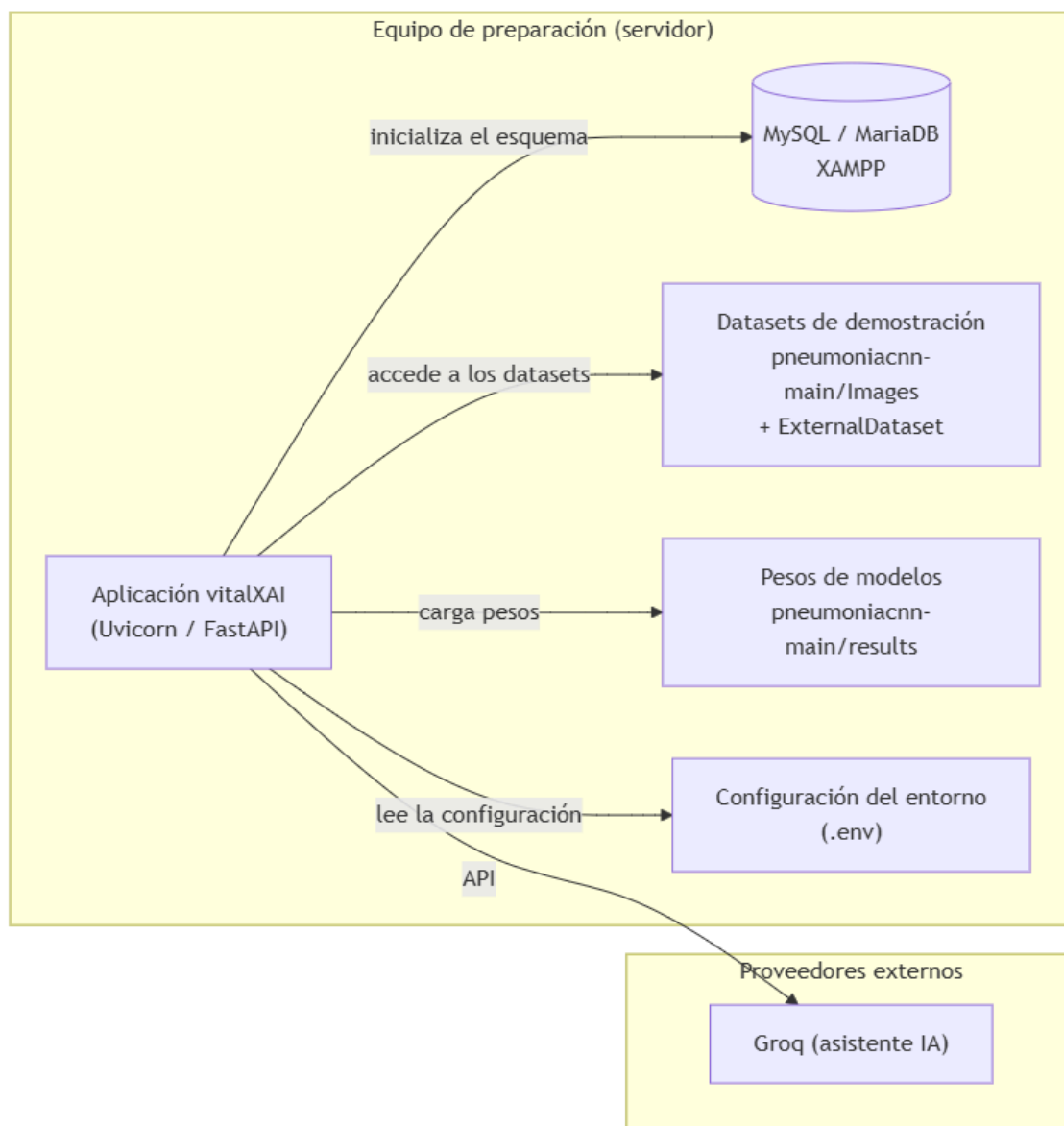


Ilustración x - Diagrama de despliegue del entorno de preparación de los datos

El diagrama refleja que la preparación inicial se resuelve en el equipo que aloja la aplicación, sin componentes de carga adicionales: la base de datos se inicializa desde la propia aplicación, y los datos de demostración y la configuración se disponen como artefactos del sistema de archivos y variables de entorno. La verificación de los requisitos del entorno —el servicio de MySQL activo, la presencia de los datasets y de los pesos, y la configuración correcta— constituye el punto de partida de los procedimientos de preparación descritos en el apartado siguiente.

8.2 Procedimientos de Carga Inicial o Migración

Los procedimientos de preparación de los datos definen el proceso por el que el sistema queda disponible para su primer uso, el orden o la jerarquía de lanzamiento de cada procedimiento y el diseño detallado de cada uno. El proceso se representa mediante el diagrama de actividad de la figura, que refleja la secuencia de preparación: la verificación de los requisitos del entorno, la inicialización del esquema, la disposición de los datos de demostración y de la configuración, y la creación de las cuentas de acceso.

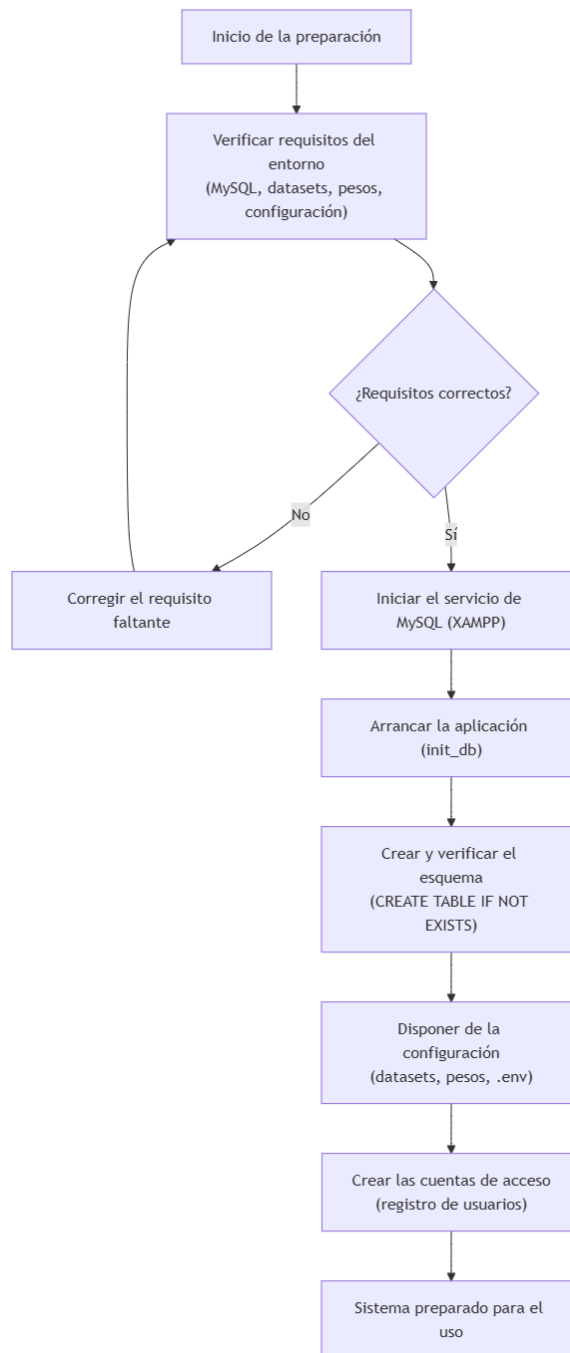


Ilustración x - Diagrama de actividad del proceso de preparación inicial de los datos

El diagrama de actividad refleja la jerarquía de lanzamiento de los procedimientos: la preparación comienza por la comprobación de los requisitos del entorno, que condiciona la corrección de los elementos faltantes; una vez verificados, se inicia el servicio de la base de datos y se arranca la aplicación, que crea y verifica el esquema; a continuación se dispone de los datos de demostración y de la configuración, y finalmente se crean las cuentas de acceso mediante el registro. Esta jerarquía garantiza que cada procedimiento cuenta con las condiciones del anterior antes de ejecutarse.

El diseño detallado de cada procedimiento que participa en la preparación inicial se especifica en la tabla siguiente, con su orden de lanzamiento, su descripción y la verificación de su resultado.

Procedimiento	Orden	Descripción	Verificación
Verificación de requisitos	1	Comprobar que el servicio de MySQL está disponible, que existen los datasets y los pesos de los modelos y que la configuración del entorno es correcta.	El entorno informa de los requisitos faltantes y permite corregirlos antes de continuar.
Inicialización del esquema	2	Ejecutar la aplicación, que invoca <code>init_db()</code> y crea las tablas del esquema con <code>CREATE TABLE IF NOT EXISTS</code> cuando no existen.	El arranque informa de la conexión a la base de datos y de la verificación de las tablas.
Preparación de los datos de demostración	3	Disponer de los datasets de entrenamiento y de validación externa y de los pesos de los modelos en sus directorios correspondientes.	La solicitud de un diagnóstico carga los pesos del modelo; el laboratorio accede a las rutas de los datasets.
Configuración del entorno	3	Suministrar las variables de entorno del sistema mediante el archivo <code>.env</code> , con las credenciales, las claves y las rutas de los datasets.	La aplicación carga la configuración en el arranque y la utiliza en las operaciones de sesión, diagnóstico y experimentación.
Creación de cuentas	4	Crear las cuentas de acceso mediante el registro de usuarios en la aplicación, sin datos de prueba precargados.	El nuevo usuario inicia sesión y accede al panel de diagnóstico.

Tabla x - Especificación del Procedimientos de Carga Inicial

Los procedimientos de preparación de los datos de demostración y de configuración se ejecutan en el mismo orden, ya que ambos deben estar completos antes del uso operativo del sistema; la tabla los ordena de forma conjunta en el paso tercero. La creación de cuentas constituye un procedimiento operativo y continuo, que no forma parte de una carga única sino del funcionamiento normal del subsistema de acceso.

La evolución del esquema de la base de datos se gestiona de forma manual, sin un sistema de migraciones automáticas. Cuando el modelo físico de datos cambia —por la incorporación de una tabla, una columna o una restricción—, la modificación se aplica actualizando las sentencias de creación de `database.py`, que en el siguiente arranque adapta el esquema a las tablas existentes mediante la política `CREATE TABLE IF NOT EXISTS`, y se registra en el registro de cambios de la base de datos del proyecto, con las sentencias SQL del cambio y las notas de reversión. Esta decisión, coherente con la simplicidad del esquema y con el estado del proyecto, evita la complejidad de un sistema de migraciones mientras la base de datos se encuentra en una fase estable; si el esquema evolucionara hacia cambios destructivos o una base de datos con datos persistidos críticos, la incorporación de un mecanismo de migraciones formal debería evaluarse conforme al registro de decisiones del proyecto.

9 Estrategia de verificación del sistema

La estrategia de verificación del sistema constituye la etapa del diseño que determina cómo se comprueba que el sistema construido se comporta conforme a lo especificado. El capítulo 17 definió el plan de pruebas de la plataforma desde la perspectiva del análisis, con las cinco categorías de verificación y sus criterios de éxito; este capítulo, conforme a la guía de diseño de la memoria (punto 9), especifica el entorno en el que se ejecutan las pruebas y la definición de los niveles de verificación, sin reproducir el detalle de cada prueba que ya documenta el capítulo 17. La estrategia distingue así el qué se verifica, que pertenece al plan de pruebas, del dónde y del cómo se verifica, que pertenece a este capítulo (Myers, Sandler, & Badgett, 2011).

El capítulo se organiza en dos apartados: los entornos de verificación, que describen el entorno tecnológico en el que se ejecutan las pruebas, las herramientas utilizadas, el origen de los datos de prueba y las restricciones operativas; y los niveles de verificación y criterios de aceptación, que definen los niveles de prueba, la naturaleza de las pruebas de integración y de sistema con una carga parecida a la de explotación, las validaciones funcionales y no funcionales que cubren las excepciones y los criterios que deben satisfacerse para aceptar cada nivel. El contenido se apoya en la guía de pruebas del proyecto, que fija los comandos, los niveles y el umbral de cobertura, y en el plan de pruebas del capítulo 17.

La verificación de vitalXAI combina la automatización con la verificación sobre el entorno real. Las pruebas unitarias y de integración se ejecutan de forma automatizada, integradas en el flujo de integración continua, y proporcionan una verificación repetible de la lógica de negocio y de los flujos críticos; las pruebas de rendimiento y de sistema se ejecutan sobre el despliegue de la plataforma, de modo que sus criterios se miden sobre el sistema real. Esta combinación, coherente con la metodología de desarrollo dirigida por pruebas del proyecto, garantiza que cualquier cambio posterior detecte las regresiones de forma inmediata y que la verificación final se realice en condiciones próximas a las de explotación.

9.1 Entornos de verificación

El entorno de verificación de vitalXAI se organiza en tres entornos que atienden a distintos niveles de aislamiento y de fidelidad con respecto al sistema real: el entorno unitario, el entorno de integración y el entorno de sistema. Cada entorno emplea sus propios dobles de prueba y su propio origen de datos, de modo que la verificación progresa desde el aislamiento total de los componentes hasta la ejecución sobre el despliegue real. La tabla siguiente resume los tres entornos y sus características.

Entorno	Alcance	Aislamiento	Origen de los datos de prueba
Unitario	Verificación de componentes individuales (lógica de negocio, funciones puras, componentes aislados).	Sin red, sin base de datos y sin operaciones de entrada y salida; todos los servicios externos se sustituyen por dobles.	Fixtures de tests/conftest.py: base de datos simulada, modelo TensorFlow simulado y cliente externo simulado.
Integración	Verificación de la interacción entre capas (API y persistencia) y entre subsistemas.	Base de datos en memoria (SQLite) y servicios externos simulados.	Datos de siembra del directorio de integración y mocks de las APIs externas (Groq y Hugging Face).
Sistema	Verificación integral sobre el despliegue real de la plataforma.	Sin aislamiento: entorno real con la base de datos MySQL, los pesos de los modelos y los datasets.	Datos de demostración del entorno de despliegue (imágenes, pesos y cuentas).

Tabla x - Especificación de Entornos de Pruebas

Las herramientas de verificación del proyecto se especifican conforme a la guía de pruebas. La batería automatizada se ejecuta con el marco de pruebas pytest, con la extensión de cobertura pytest-cov para medir la cobertura de los módulos de la aplicación y la extensión pytest-mock para la simulación de los servicios externos (pytest, 2024). La calidad estática se comprueba con ruff, que verifica el estilo y las reglas del código, y con mypy, que verifica los tipos de los módulos configurados (Ruff, 2024; Mypy, 2024). La verificación se integra en el flujo de integración continua, que ejecuta la batería de pruebas y las comprobaciones estáticas en cada integración.

El origen de los datos de prueba se resuelve mediante las fixtures y los datos de siembra del directorio de pruebas. Las fixtures del módulo conftest.py proporcionan la base de datos simulada, el modelo de TensorFlow simulado y el cliente externo simulado para las pruebas unitarias, de modo que la ejecución no depende de la red, de la base de datos ni de los pesos reales de los modelos. Las pruebas de integración utilizan una base de datos SQLite en memoria con datos de siembra y simulan las APIs externas, de modo que la verificación de los flujos entre subsistemas se centra en la colaboración de las capas internas. Los datos del entorno de sistema proceden del despliegue real: las imágenes de demostración, los pesos de los modelos y las cuentas de acceso, en coherencia con la preparación inicial de los datos del capítulo 25.

Las restricciones operativas del entorno de verificación son las siguientes. Los servicios externos —el proveedor del asistente conversacional, los modelos de Hugging Face y los modelos de TensorFlow— se simulan siempre en las pruebas unitarias y de integración, de modo que la batería automatizada es determinista y no depende de la disponibilidad de esos servicios. El umbral de cobertura del código está fijado en el setenta por ciento, con un valor actual superior a ese umbral, y la batería falla si no se alcanza. Las pruebas unitarias y de integración se ejecutan con los comandos definidos en la guía de pruebas del proyecto; las pruebas de rendimiento y de sistema se ejecutan manual o semiautomatizada sobre el despliegue, con la carga de trabajo parecida a la de explotación que se detalla en el apartado siguiente.

El diagrama de despliegue de la figura representa el entorno de verificación del sistema. Las pruebas unitarias se ejecutan sobre las fixtures aisladas, las pruebas de integración sobre la base de datos en memoria con los servicios externos simulados, y las pruebas de sistema sobre el despliegue real con la base de datos MySQL y los artefactos del aprendizaje automático; las comprobaciones de calidad estática acompañan a la batería automatizada.

El diagrama refleja la progresión de la verificación por entornos: la batería automatizada parte del aislamiento total del entorno unitario, avanza a la interacción entre capas en el entorno de integración y culmina en la verificación sobre el despliegue real en el entorno de sistema. La calidad estática se aplica sobre la batería automatizada, de modo que el estilo y los tipos del código se verifican en cada integración. Los criterios de aceptación de cada nivel se definen en el apartado siguiente, completando la estrategia de verificación del sistema.

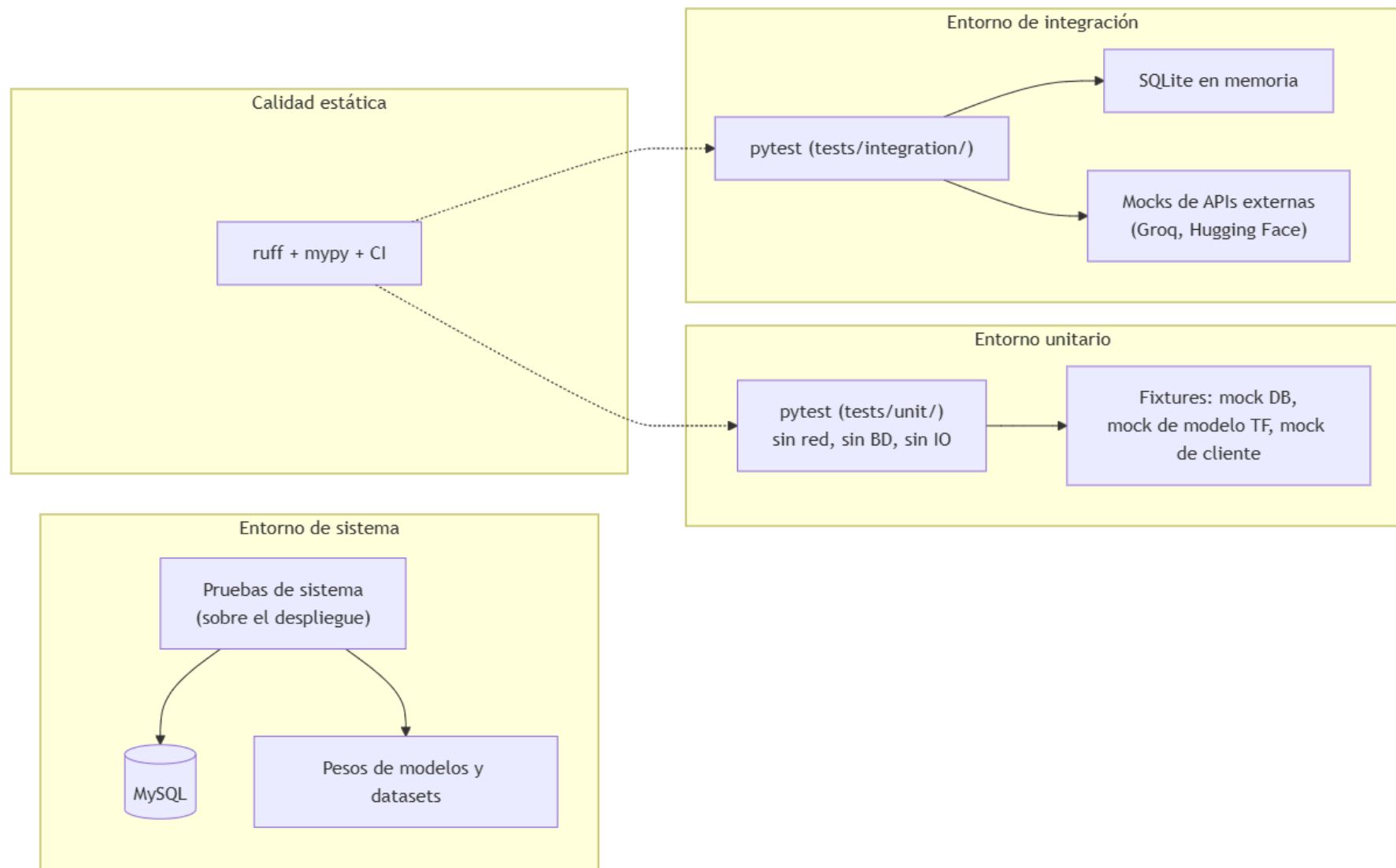


Ilustración x - Diagrama de despliegue del entorno de verificación

9.2 Niveles de verificación y criterios de aceptación

Los niveles de verificación definen la profundidad de la comprobación del sistema, en correspondencia con los entornos descritos en el apartado anterior. Cada nivel agrupa las verificaciones que se ejecutan en su entorno y establece los criterios que deben satisfacerse para aceptarlo, de modo que la superación de un nivel es condición para considerar verificada la parte del sistema que abarca. La definición de los niveles se mantiene coherente con el plan de pruebas del capítulo 17, que detalla las pruebas concretas de cada categoría; en este apartado se especifican los niveles, su alcance y sus criterios de aceptación, sin reproducir el detalle de cada prueba. Los tres niveles se presentan a continuación, con la tabla de criterios de aceptación de cada uno.

9.2.1 Nivel unitario

El nivel unitario verifica el comportamiento correcto de los componentes del sistema de manera aislada, en el entorno unitario, sin dependencia de la red, de la base de datos ni de los servicios externos, que se sustituyen por dobles de prueba. Cubre la lógica de negocio de mayor criticidad funcional: la gestión de cuentas y sesiones, la validación de las entradas, el motor de inferencia con modelos simulados, la generación de las explicaciones, la gestión del historial, el laboratorio de entrenamiento, la cola de trabajos, la internacionalización y el acceso a datos. La verificación unitaria recorre tanto los caminos normales como las excepciones —entradas inválidas, credenciales incorrectas, usuarios duplicados, tokens revocados, modelos ausentes, sesiones ajenas y trabajos no interrumpibles—, de modo que cada componente se comprueba en las condiciones esperadas y en sus desviaciones. Este nivel se corresponde con la categoría de verificación de componentes del capítulo 17.

Criterio	Descripción
Batería unitaria superada	Todos los tests unitarios del directorio tests/unit/ se ejecutan y superan sin fallos.
Cobertura de código	La cobertura de los módulos de la aplicación alcanza al menos el umbral del setenta por ciento.
Calidad estática	La verificación con ruff no reporta errores de estilo y la verificación con mypy no reporta errores de tipos en los módulos configurados.
Determinismo	La batería unitaria se ejecuta sin dependencia de la red, de la base de datos ni de los servicios externos.
Cobertura de excepciones	Los caminos alternativos y de error de los componentes verificados están cubiertos por las pruebas.

Tabla x - Especificación de Nivel unitario

La superación del nivel unitario se verifica en cada ciclo de desarrollo mediante los comandos de la guía de pruebas del proyecto, de modo que la calidad de los componentes se comprueba de forma continua y las regresiones se detectan de inmediato.

9.2.2 Nivel de integración

El nivel de integración verifica la colaboración entre las capas del sistema y entre los subsistemas, en el entorno de integración, superando el aislamiento del nivel unitario. Las pruebas ejercitan la interacción entre la API, la capa de negocio y la persistencia, sustituyendo la base de datos real por una instancia en memoria y los servicios externos por simulaciones, de modo que la verificación se centra en la colaboración interna sin depender del entorno de producción. Los flujos críticos se ejercitan con una carga de trabajo parecida a la de explotación: el flujo de acceso a la plataforma, el flujo de un diagnóstico desde la subida hasta el resultado, el aislamiento de datos entre usuarios, la supervisión administrativa y el lanzamiento de un experimento desde el asistente hasta la cola. La verificación de integración confirma que los subsistemas colaboran correctamente y que las condiciones de autorización y de aislamiento se respetan en los flujos combinados. Este nivel se corresponde con la categoría de verificación de flujos entre subsistemas del capítulo 17.

Criterio	Descripción
Batería de integración superada	Todos los tests de integración del directorio tests/integration/ se ejecutan y superan sin fallos.
Flujos críticos	Los flujos de extremo a extremo entre la API, la capa de negocio y la persistencia se verifican correctamente.
Aislamiento y autorización	Las condiciones de aislamiento de datos entre usuarios y de autorización administrativa se respetan en los flujos combinados.
Carga parecida a la de explotación	Los flujos se ejercitan con una carga de trabajo representativa del uso real de la plataforma.
Servicios externos simulados	La batería de integración no depende de la disponibilidad de los servicios externos, que se simulan en las pruebas.

Tabla x - Especificación de Nivel de integración

La superación del nivel de integración se verifica antes de la integración de los cambios en el flujo de trabajo del proyecto, de modo que la colaboración entre subsistemas queda confirmada antes de la verificación sobre el entorno real.

9.2.3 Nivel de sistema

El nivel de sistema verifica integralmente la plataforma sobre el despliegue real, en el entorno de sistema, sin aislamiento de la base de datos ni de los artefactos del aprendizaje automático. Las pruebas ejercitan los flujos completos de uso con la base de datos MySQL, los pesos de los modelos y los datasets del entorno de despliegue, con una carga de trabajo parecida a la de explotación. La verificación comprende las validaciones funcionales de los flujos completos —el diagnóstico asistido, la gestión del historial, el laboratorio de entrenamiento, la ejecución asíncrona, la supervisión administrativa y las capacidades transversales— y las validaciones no funcionales de rendimiento y capacidad de respuesta, de protección y control de acceso y de usabilidad, cubriendo las excepciones en ambos casos. Este nivel se corresponde con las categorías de protección, de rendimiento y de verificación integral de los flujos completos de uso del capítulo 17.

Criterio	Descripción
Flujos completos	Los flujos de uso de la plataforma se realizan correctamente sobre el despliegue real: diagnóstico, historial, laboratorio, cola, administración y capacidades transversales.
Validaciones funcionales	El comportamiento de los flujos coincide con el especificado en los casos de uso, incluidos los escenarios alternativos y de error.
Seguridad	Los mecanismos de protección (CSRF, cabeceras de seguridad, limitación de peticiones y gestión de tokens) se verifican sobre el sistema real.
Rendimiento	Los tiempos de respuesta de la inferencia y la ejecución sin bloqueo de la interfaz satisfacen los umbrales definidos para el entorno de despliegue.
Artefactos	Los informes PDF del diagnóstico y de la sesión, y los mapas de explicabilidad, se generan correctamente y se entregan al usuario.
Carga de explotación	La verificación se realiza con una carga de trabajo parecida a la de explotación y cubre las excepciones previstas.

Tabla x - Especificación de Nivel de sistema

La superación del nivel de sistema se verifica sobre el despliegue de la plataforma antes de su presentación, y constituye la confirmación final de que el sistema construido satisface los requisitos funcionales y no funcionales del análisis. Con la superación de los tres niveles, la estrategia de verificación del sistema queda completa y garantiza que las evoluciones futuras de la plataforma podrán incorporarse con un riesgo de regresión controlado y medible.

10 Implantación del sistema

La implantación del sistema constituye la etapa final del diseño que determina cómo se pone en operación la plataforma y qué documentación se necesita para operar con ella. Los capítulos 24 y 25 especificaron la construcción del sistema y la preparación inicial de sus datos; este capítulo, conforme a la guía de diseño de la memoria (punto 10), define los requisitos de documentación necesarios para la operación y los requisitos de implantación en el entorno de trabajo, incluidas las necesidades de formación y la infraestructura de instalación. La especificación establece así las condiciones bajo las que el sistema pasa a estar disponible para sus usuarios finales (Sharp, Rogers, & Preece, 2019).

El capítulo se organiza en dos apartados: la documentación necesaria para la operación, que especifica los manuales requeridos y sus características; y la implantación en el entorno de operación, que define las necesidades de formación, los requisitos de infraestructura e instalación y el proceso de puesta en marcha. El contenido se apoya en la guía de despliegue del proyecto y en los capítulos precedentes de diseño, de modo que la implantación mantiene la coherencia con la construcción y con la preparación de los datos.

La implantación de vitalXAI se orienta a dos escenarios de operación. El escenario principal es el despliegue de la plataforma en un equipo que aloja el servidor, la base de datos y los artefactos del aprendizaje automático, al que acceden los usuarios finales a través de la red; en la demostración del proyecto, el acceso se realiza mediante un túnel HTTPS que expone el servidor a través de una URL pública. La documentación y los requisitos de implantación se especifican para ambos escenarios, de modo que la operación del sistema queda cubierta tanto en el uso local como en el acceso remoto.

10.1 Documentación necesaria para la operación

La operación de vitalXAI requiere tres manuales, dirigidos a los distintos roles que intervienen en el uso y el mantenimiento de la plataforma: el manual de usuario, el manual de explotación y administración, y el manual de despliegue e implantación. Cada manual define su destinatario, su contenido y sus características, de modo que toda la documentación necesaria para operar con el sistema queda especificada. La tabla siguiente resume los manuales requeridos.

Documento técnico	Perfil destinatario	Alcance del contenido
Manual de usuario	Profesionales de la plataforma: facultativos e investigadores.	Acceso y registro; diagnóstico asistido (subida de la radiografía, selección del modelo, solicitud del diagnóstico, visualización del resultado y de los mapas de explicabilidad, e informe PDF); gestión del historial (listado, detalle, renombrado y eliminación); laboratorio de entrenamiento (configuración con el asistente, lanzamiento de experimentos, consulta de sesiones y resultados e informe de la sesión); y capacidades transversales (idioma, tema visual y cola de trabajos).
Manual de explotación y administración	Administrador de la plataforma y responsable de la operación.	Supervisión de la plataforma (listado de usuarios, consultas de un usuario y detalle de una consulta); gestión operativa de la actividad; monitorización de la cola de trabajos; y resolución de incidencias operativas.
Manual de despliegue e implantación	Equipo técnico responsable de la puesta en marcha.	Entorno y construcción del sistema; preparación inicial de los datos; despliegue en el entorno de operación; configuración del acceso remoto mediante el túnel HTTPS; y procedimientos de verificación. Se apoya en la guía de despliegue del proyecto y en los capítulos 24 y 25.

Tabla x - Especificación de documentación necesaria para la operación

Las características de los manuales se especifican conforme a la guía de diseño de la memoria. Los documentos se elaboran en formato digital, en el mismo formato de documentación del proyecto —Markdown, convertible a PDF—, de modo que puedan revisarse, distribuirse y consultarse en cualquier entorno. La estructura de cada manual se organiza por rol y por flujo funcional, con secciones dedicadas a cada operación del sistema y a sus condiciones de error, de modo que el lector localiza con rapidez el procedimiento que necesita. El contenido combina la descripción de los pasos de cada operación, las condiciones que deben cumplirse y los resultados esperados, junto con los escenarios alternativos y de error. El control de versiones de los manuales se alinea con la gestión de versiones del proyecto, de modo que cada versión de la documentación corresponde a una versión del sistema y refleja sus cambios. Finalmente, la distribución se dirige a los destinatarios indicados en la tabla: el manual de usuario a los profesionales, el de explotación y administración al administrador y al responsable de operación, y el de despliegue e implantación al equipo técnico.

La documentación del sistema se completa con la guía de despliegue del proyecto, que recoge los requisitos operativos y el procedimiento de la demostración, y con la documentación de diseño de la memoria, que constituye la referencia técnica de la arquitectura, el diseño de casos de uso, de clases y de interfaces, y de la construcción y preparación de los datos. Estos documentos, junto con los tres manuales, conforman el conjunto de documentación necesario para operar con el sistema en su entorno de trabajo.

10.2 Requisitos de Implantación

La implantación en el entorno de operación define las necesidades de formación de los usuarios, los requisitos de infraestructura e instalación y el proceso de puesta en marcha de la plataforma. Los requisitos se apoyan en los capítulos de construcción y de preparación de los datos: la implantación dispone el sistema sobre el entorno descrito en el capítulo 24 y aplica la preparación inicial definida en el capítulo 25, y añade las condiciones de formación y de acceso propias de la operación.

Las necesidades de formación se orientan a los tres roles que operan con la plataforma. La formación de los facultativos se centra en la operación clínica: el diagnóstico asistido, desde la subida de la radiografía hasta el informe PDF, y la gestión del historial de consultas. La formación de los investigadores se centra en el laboratorio de entrenamiento: la configuración conversacional de los experimentos, el lanzamiento de los entrenamientos, la consulta de los resultados y la interpretación de las comparativas estadísticas. La formación del administrador se centra en la administración del sistema: la supervisión de los usuarios y de su actividad, la monitorización de la cola de trabajos y la resolución de las incidencias operativas. La formación se realiza mediante sesiones guiadas sobre la plataforma, apoyadas en el manual de usuario y en el manual de explotación y administración, de modo que cada rol aprende sobre el flujo real que va a utilizar.

Los requisitos de infraestructura e instalación se especifican para el entorno de operación, atendiendo al software, al hardware y a las comunicaciones. En cuanto al software, el sistema requiere Python en su versión 3.11 o superior, el servicio de MySQL (en el entorno de desarrollo, MariaDB mediante XAMPP) y las dependencias declaradas en el archivo requirements.txt (FastAPI, 2024; Oracle, 2024). En cuanto al hardware, la plataforma requiere recursos de memoria y de cómputo suficientes para la carga de los modelos de TensorFlow y la ejecución del pipeline MLOps, además de espacio de almacenamiento para los datasets, los pesos de los modelos y los resultados de los entrenamientos; la disponibilidad de una unidad de cómputo acelerado por GPU reduce de forma significativa el tiempo de los entrenamientos. En cuanto a las comunicaciones, la operación local accede al servidor mediante la dirección local del equipo, mientras que el acceso remoto de la demostración se expone mediante un túnel HTTPS que publica el servidor a través de una URL pública; en un despliegue de producción, la comunicación debe cifrarse con HTTPS mediante un certificado válido.

El diagrama de implantación de la figura representa el sistema en su entorno de operación. Los usuarios finales —el facultativo, el investigador y el administrador— acceden mediante su navegador a través de las comunicaciones, que en el escenario de demostración se establecen mediante el túnel HTTPS; el servidor de operación aloja la aplicación, la base de datos MySQL, los pesos de los modelos y los datasets, y el worker asíncrono.

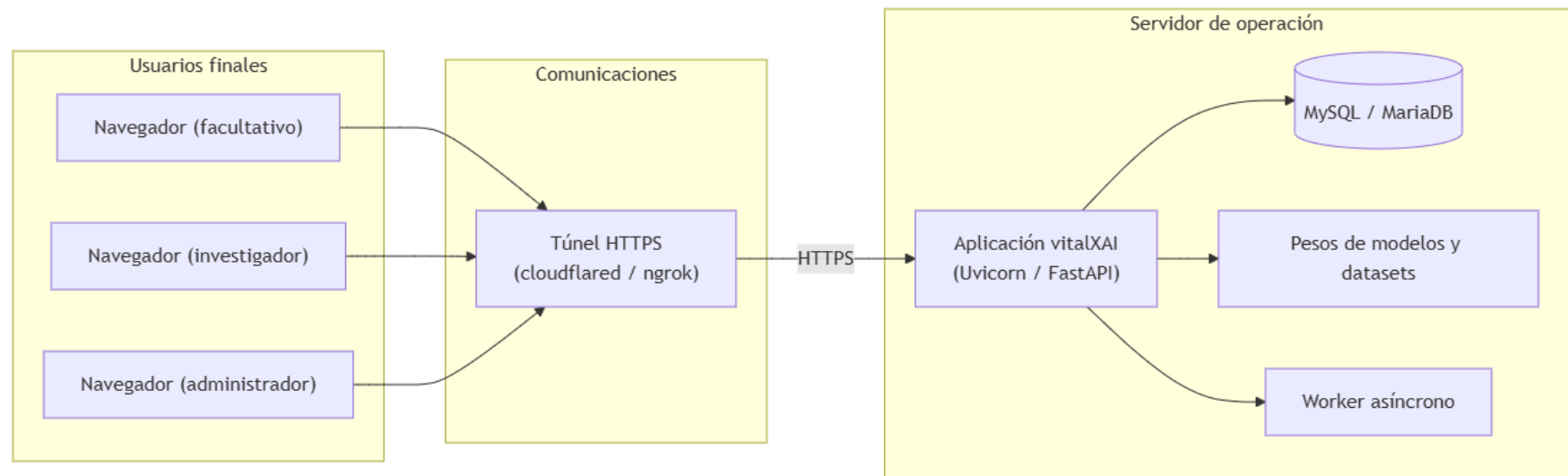


Ilustración x - Diagrama de implantación del sistema en el entorno de operación

El diagrama refleja la disposición de la implantación: los usuarios finales no acceden directamente al equipo que aloja el servidor, sino a través de la comunicación establecida, de modo que los datos de la plataforma permanecen en el servidor de operación. El servidor agrupa la aplicación, la persistencia y los artefactos del aprendizaje automático, y el worker atiende las tareas asíncronas de la cola.

El proceso de implantación se resuelve aplicando los procedimientos definidos en los capítulos precedentes. En primer lugar, se construye el sistema conforme al capítulo 24: se prepara el entorno virtual, se instalan las dependencias y se lanza la aplicación, que inicializa la base de datos y arranca el worker. A continuación, se dispone la preparación inicial de los datos conforme al capítulo 25: se verifican los requisitos del entorno, se inicializa el esquema, se disponen los datasets y los pesos de los modelos, se configura el entorno y se crean las cuentas de acceso. Finalmente, se establece el acceso a los usuarios finales, local o mediante el túnel, y se verifica el sistema conforme a la estrategia de verificación del capítulo 26. Con la formación de los roles completada y la infraestructura dispuesta, la plataforma queda implantada y operativa para sus usuarios.